

University of Nevada, Reno

Deep Generative and Explainable Learning Frameworks for Intent Recognition in Naval Domain

A dissertation submitted in partial fulfillment of the
requirements for the degree of Doctor of Philosophy in
Computer Science and Engineering

by
Md Abu Sayed

Dr. Monica Nicolescu/Dissertation Advisor
Dr. Mircea Nicolescu/Dissertation Co-Advisor

May 2026



THE GRADUATE SCHOOL

We recommend that the dissertation
prepared under our supervision by

Md Abu Sayed

entitled

**Deep Generative and Explainable Learning Frameworks for Intent
Recognition in Naval Domain**

be accepted in partial fulfillment of the
requirements for the degree of

Doctor of Philosophy

Monica Nicolescu, Ph.D.

Advisor

Mircea Nicolescu, Ph.D.

Co-advisor

Alireza Tavakkoli, Ph.D.

Committee Member

Sergiu Dascalu, Ph.D.

Committee Member

Birant Ramazan, Ph.D.

Graduate School Representative

Markus Kemmelmeier, Ph.D., Dean

Graduate School

May, 2026

Abstract

by Md Abu Sayed

Accurate recognition of vessel intent supports maritime situational awareness, autonomous navigation, and tactical decision support. Three obstacles limit progress: the scarcity of labeled hostile-encounter data, noisy and intermittent sensor streams, and the inability of data-driven models to encode symbolic navigation rules and expert heuristics. This dissertation addresses these obstacles through five contributions evaluated on a unified multi-vessel dataset.

First, NavySim is a scriptable multi-agent maritime simulator that generates benign and adversarial encounters with per-timestep intent labels, an intent recognition pipeline integrated, COLREGs-compliant navigation planners, and threat-visualization tools, supplying labeled encounter data unavailable in public AIS corpora. Second, Categorical Permutation Feature Importance (CPFI) is a model-agnostic feature-attribution method that ranks features by importance under class imbalance and multi-level categorical predictors. Third, two intent-classification studies apply the CPFI/Transformer Feature Importance Score (TFIS, developed by a collaborator) pipeline: an intent classifier that detects vessel behavior from long observation windows, and a feature-optimized pipeline that uses attribution scores within the training loop to prune, weight, and re-rank features.

Fourth, we evaluate classical Bayesian trajectory predictors, including the Unscented Kalman Filter (UKF), Extended Kalman Filter (EKF), and UKF with Constant Turn Rate and Velocity (CTRV) dynamics, are evaluated on the same dataset. This assessment aims to highlight the structural limitations of intent-agnostic filters in hostile scenarios. Additionally, it establishes the matched-configuration baselines for comparing learned and generative models. Fifth, MTITP (Multi-Task Intent and Trajectory Prediction) is designed as a unified architecture for joint scenario-intent classification, intent prediction, and intent-conditioned trajectory generation over a short observation window of vessel data. It comprises a three-stream encoder that fuses per-vessel kinematics, pairwise geometry and symbolic COLREGs-derived interaction features through alternating temporal and cross-vessel attention, intent-classification and intent-prediction heads, and a trajectory generator trained under a two-phase protocol. Three backbone variants, named MTITP-WGAN, MTITP-L2, and MarITGAN with an IntentAwareCritic, differ only in the adversarial trajectory loss applied during the joint phase. Across its three variants, MTITP outperforms the strongest classical Kalman trajectory baselines in displacement error and exceeds published deep maritime intent classifiers in past- and future-intent accuracy, with gains concentrated in hostile intent scenarios.

Acknowledgements

I am deeply grateful to my advisor, Dr. Monica Nicolescu, and my co-advisor, Dr. Mircea Nicolescu, for their guidance, patience, and high standards. Their willingness to let me pursue difficult problems, while pulling me back when the scope grew unwieldy, shaped how I now think about research.

I thank my committee, Dr. Alireza Tavakkoli, Dr. Sergiu Dascalu, and Dr. Birant Ramazan, for their careful reading and sharp questions, which improved this work.

This research was supported by the Office of Naval Research under grants N00014-21-1-2234, N00014-22-1-2176, and N00014-22-1-2122.

I am grateful to my friends and collaborators at the University of Nevada, Reno: Tyler J. Becker, Md. Azizul Hakim, Ayesh Meepaganithage, Parvaneh Aliniya, Mayamin Hamid Raha, Casey Brenner, Korben DiArchangel, and Dr. Sushil Louis, for the joint papers and the conversations that turned ideas into experiments.

To my friends and family, thank you for the years of support, patience and perspective.

Above all, I express my deepest gratitude to my wife, Nasrin Akter. Her unwavering love, steadfast presence, and the serene moments she safeguarded have made every chapter of this work and our shared life possible.

Contents

Abstract	i
Acknowledgements	iii
List of Tables	xiii
List of Figures	xviii
1 Introduction	1
Overarching Research Questions	4
1.1 Vessel Intent and Behavior	5
1.2 Problem Statements	6
1.2.1 Problem 1: High-Fidelity Maritime Simulation Architecture for Complex Scenario Generation, Intent Recognition, and Security Strategy Evaluation	7
1.2.2 Problem 2: Explainable Maritime Classification via Feature Selection	10
1.2.3 Problem 3: Feature-Optimized Intent Classification on Maritime Vessel Behaviors	12
1.2.4 Problem 4: Classical Trajectory Prediction Baselines and Their Structural Limitations	13
1.2.5 Problem 5: Multi-Task Intent and Trajectory Prediction (MTITP)	15
2 Literature Review	19
2.1 Maritime Simulation and Scenario Generation	20
2.1.1 Role of Simulation in Maritime Research	20
2.1.2 Multi-Agent Maritime Simulators	21
2.2 Intent and Behavior Recognition in Maritime Domains	22
2.2.1 CPA-Based and Rule-Based Threat Assessment	22
2.2.2 Statistical and Probabilistic Models	23
2.2.3 Deep Learning for Intent Recognition	23
2.2.4 Surveys and Benchmarks	26

2.3	Feature Selection and Feature Importance	27
2.3.1	Classical Feature Selection Methods	27
2.3.2	Tree-Based Feature Importance	28
2.3.3	Neural and Attention-Based Feature Attribution	29
2.4	Trajectory Prediction	31
2.4.1	Classical Filters and Motion Models	31
2.4.2	Deep Learning and Transformer Architectures for Trajectory Prediction	32
2.4.3	Multi-Task Learning for Trajectory and Intent	36
2.5	Generative Modeling and Domain-Conditioned Architectures	38
2.5.1	Foundations and Training Objectives	38
2.5.2	Conditional and Class-Aware Variants	39
2.5.3	Trajectory Applications and Interpretability of Generative Models	39
2.5.4	Embedding Expert Domain Knowledge in Learned Models . .	41
2.6	Summary and Research Gaps	42
3	NavySim: Enhanced Multi-Vessel Simulation and Analysis Engine for Advanced Naval Research	45
3.1	Introduction	45
3.2	Naval Simulator: General Architecture	48
3.3	Naval Scenario Generation	49
3.4	Velocity Obstacles for COLREGS Compliant Ship Driving	53
3.5	Vulnerability and Threat Heatmaps	55
3.5.1	Vulnerability Heatmap	55
	Distance Influenced Efficiency Function	56
	Heading Influenced Efficiency Function	57
3.5.2	Predictive Action Heatmap	59
3.5.3	Combined Threatmap	61
3.6	Intent Recognition	62
3.7	Real-Time, Online Mobile Resource Allocation for Defensive Responses	66
3.8	Conclusion	67
4	Early Classification of Intentions for Maritime Domains Using Deep Learning Models	69
4.1	Introduction	69
4.2	Data Engineering	70
4.2.1	Data Collection	70
4.2.2	Data Preprocessing	71
4.2.3	Feature Development	72
4.3	Research Methodology	74
4.3.1	Feature Selection Using Decision Trees	74

4.3.1.1	Gini Index-Based Decision Trees	75
4.3.1.2	Entropy-Based Decision Trees	76
4.3.1.3	Feature Importance Score	76
4.3.2	Deep Learning Models	77
4.3.2.1	Fully Connected Deep Learning Model (FCDL) . . .	77
4.3.2.2	Long-Short Term Memory Network Model (LSTM) .	79
4.4	Experimental Results	80
4.4.1	Feature Selection	80
4.4.1.1	Implications of Performance Metrics in Decision Trees	81
4.4.2	Analysis of Training and Validation	83
4.4.3	Multi-class Classification for Intent Detection	85
4.4.4	Binary Classification for Hostile and Non-hostile Behaviors . .	88
4.4.5	Comparison with Other Work	89
4.5	Conclusion and Future Work	90
5	Optimizing Features for Maritime Intent Recognition	91
5.1	Introduction	91
5.2	Methodologies	94
5.2.1	Data Engineering	95
5.2.2	Feature Selection	95
5.2.2.1	Categorical Permutation Feature Importance, CPFI .	95
5.2.2.2	Transformer Feature Importance Score, TFIS	96
5.2.3	Intent Recognition	98
5.2.3.1	Long-Short Term Memory Network Model (LSTM) and Fully Connected Deep Learning (FCDL)	98
5.3	Experiments and Results	99
5.3.1	Comparison of Models with CPFI	99
5.3.2	Comparison of Models with TFIS	104
5.4	One-vs-All Classification	105
5.4.1	One-vs-All Classification Performance using LSTM	105
5.4.2	Comparison and Insights	107
5.4.3	Class-Specific Feature Importance and Comparative Feature Relevance	108
5.4.3.1	Insights from Class-Specific Tables	111
	Table 5.5: Class 0 to Class 3	111
	Table 5.6: Class 4 to Class 6	111
5.4.3.2	Comparison with Global Feature Importance Models	112
	LSTM Combined CPFI (Figure 5.2)	112
	Transformer-Based Feature Importance (Figure 5.5) . .	113
5.4.3.3	Synthesis and Comparative Understanding	113
5.5	Conclusion and Future Work	114

6	Classical Bayesian Filtering for Maritime Vessel Trajectory Prediction	116
6.1	Introduction	116
	Research questions.	121
6.2	Methods	122
6.2.1	Constant Velocity Baseline	122
6.2.2	Standard UKF (4D)	123
	State vector.	123
	Process model (linear CV).	123
	Sigma-point generation.	123
	Predict step.	124
	Update step.	124
6.2.3	UKF-5D-CV: UKF with 5D State and CV Dynamics	125
6.2.4	Asymmetric UKF (6D)	125
	State vector.	126
	Process model (constant acceleration).	126
	Asymmetric noise design.	126
6.2.5	EKF with CTRV (5D)	127
	State vector.	127
	Turn-rate initialization from heading.	127
	Nonlinear state transition.	128
	EKF linearization: Jacobian.	128
	Measurement model.	129
	Predict-update cycle.	129
	Default noise parameters.	130
6.2.6	UKF with CTRV (5D)	130
	Sigma-point generation (Merwe scaled).	131
	Omega threshold difference.	131
	EKF vs. UKF-CTRV comparison.	131
6.3	Evaluation Setup	133
6.3.1	Dataset	133
6.3.2	Experiment Configurations	133
6.3.3	Feature Preparation	134
6.3.4	Evaluation Metrics	135
6.4	Results	136
6.4.1	Experiment A: Long-Horizon (obs=40, pred=20)	137
6.4.2	Experiment B: Short-Horizon (obs=20, pred=5)	138
6.4.3	Experiment C: Medium-Horizon (obs=20, pred=10)	140
6.4.4	Experiment D: Long-Prediction, Short-Obs (obs=20, pred=20)	142
6.4.5	Cross-Horizon Summary	143
6.4.6	Horizon-Wise Error Growth	144

6.4.7	Per-Vessel Analysis (Experiment A)	146
6.4.8	Qualitative Trajectory Analysis	146
	Benign case.	147
	Cross, headon, overtake.	147
	Ram: two failure geometries.	147
	Herd worst case.	148
	UKF-CTRV divergence.	149
6.5	Discussion	150
6.5.1	The Linear Ceiling: Why Sigma Points Do Not Help Here . .	150
6.5.2	UKF-CTRV Divergence: Observation and Prediction Windows	152
6.5.3	EKF-CTRV: Short-Horizon Advantage and Arc Compounding	153
6.5.4	Asymmetric UKF: Acceleration Channel	153
6.5.5	Horizon Length and Filter Choice	154
6.5.6	Two Regimes by Intent Type	154
6.5.7	Limitations and Future Directions	155
6.6	Conclusion	156
7	MTITP: Multi-Task Intent and Trajectory Prediction for Complex Maritime Scenarios	158
7.1	Introduction	158
	Research questions.	164
7.1.1	Contributions	165
7.2	Problem Formulation	167
7.2.1	Notation	167
7.2.2	Inputs and Outputs	167
7.2.3	Objective	168
7.3	Dataset and Data Pipeline	169
7.3.1	Dataset	169
7.3.2	Feature Extraction	169
	7.3.2.1 Stream A: Per-Vessel Kinematics	169
	7.3.2.2 Stream B: Continuous Pairwise Geometry	170
	7.3.2.3 Stream C: Symbolic Interaction Features	171
	7.3.2.4 Labels	171
7.3.3	Sliding Window Segmentation	172
7.3.4	Coordinate Conventions and Normalization	172
7.3.5	Dataset Split	173
7.4	Model Architecture: Shared Backbone	173
7.4.1	Three-Stream Spatial-Temporal Encoder	175
	7.4.1.1 Stream A Input Projection	175
	7.4.1.2 Interaction Fusion	177
	Stream B projection.	177

	Stream C embedding.	177
	Fusion.	178
	7.4.1.3 Temporal Block	178
	7.4.1.4 Spatial Block	179
	7.4.1.5 Encoder Outputs	181
7.4.2	Past Intent Classifier	181
7.4.3	Future Intent Predictor	182
7.4.4	Trajectory Generator	182
	7.4.4.1 Conditioning	185
	7.4.4.2 Trajectory Queries	185
	7.4.4.3 Decoder Layers ($N_{\text{dec}} = 3$)	185
	7.4.4.4 Displacement Prediction and Cumulative Summation	186
7.5	Training Variants	187
	7.5.1 Shared Components	188
	Past intent loss.	188
	Future intent loss.	188
	Trajectory smoothness loss.	188
	Two-phase protocol.	188
	Optimizer configuration.	189
	7.5.2 Variant 1: MTITP-WGAN (Baseline)	190
	7.5.2.1 WGAN Critic	190
	Input encoding.	190
	Architecture.	191
	7.5.2.2 Losses	192
	7.5.2.3 Per-Iteration Pipeline	192
	7.5.3 Variant 2: MTITP-L2	195
	7.5.3.1 Loss	195
	7.5.3.2 Per-Iteration Pipeline	195
	7.5.4 Variant 3: MarITGAN, Intent-Aware Interaction-Conditioned Critic	196
	7.5.4.1 IntentAwareCritic: Three Parallel Branches	197
	Branch 1: Trajectory with intent conditioning.	197
	Branch 2: Inter-vessel relative motion geometry.	199
	Branch 3: Intent-scene consistency.	199
	7.5.4.2 Losses	200
	7.5.4.3 Per-Iteration Pipeline	201
7.6	Evaluation Metrics	202
	7.6.1 Trajectory Metrics	202
	7.6.2 Intent Metrics: Notation	202
	7.6.3 Intent Accuracy	203
	7.6.4 Per-Class Precision, Recall, and F1	204

7.6.5	Aggregations across Classes	204
7.6.6	Mean Average Precision	205
7.7	Results	205
7.7.1	Overall Performance	205
7.7.2	Per-Scenario Results	207
7.7.3	Hostile vs. Non-Hostile Operational View	208
	Binary intent quality is not the bottleneck.	210
7.8	Comparison with External Baselines	211
7.8.1	Classical Trajectory Baselines	212
7.8.2	Published Deep-Learning Baselines	214
7.9	Ablation Study	217
7.9.1	Overall Ablation Results	224
7.9.2	Per-Scenario Ablation Breakdown	225
7.9.3	Analysis	226
	A3: The interaction representation carries the intent signal, not the trajectory signal alone.	226
	A11: Intent conditioning is already implicit in the encoder.	227
	A12: Multi-task supervision builds a useful encoder.	227
	A13/A15/A14/A16/A17: L2 training outperforms WGAN- GP at every horizon.	228
	A20: Domain-specific critic (MarITGAN) closes the ADE gap on the three upweighted classes.	229
	A21: Refined MarITGAN training schedule closes the gap to L2.	231
7.9.4	Robustness to Sensor Noise (A18, A19, and A22)	232
7.9.4.1	Dataset Noise Characterisation	233
	Visual signature of noise across datasets.	233
	Non-hostile behaviors.	234
	Hostile behaviors.	236
	Implications for the quantitative results.	238
7.9.4.2	Quantitative Ablation Results	239
	A18: MTITP retains intent discrimination under moder- ate sensor noise.	241
	A19: High noise: trajectory quality degrades proportion- ally, intent is remarkably stable.	243
	A22: MarITGANv2 under sensor noise: trajectory parity with L2, intent advantage retained.	244
7.10	Robustness and Sanity Checks	246
7.10.1	Split Integrity	247
7.10.2	Majority-Class Baselines	248
7.10.3	t-SNE of Encoder Representations	249

	Past-intent input ($\mathbf{e}_{\text{scene}}$)	250
	Future-intent and trajectory input ($\mathbf{E}_{\text{seq}}$)	251
7.10.4	Cross-Dataset Generalization	253
7.10.5	McNemar’s Test for Statistical Significance	254
	Past intent.	256
	Future intent.	256
	Cross-dataset pairs.	256
7.11	Discussion	257
7.11.1	Intent and Trajectory as Separable Pathways	257
7.11.2	Where Trajectory Quality Comes From	258
7.11.3	Scenario-Specific Analysis	259
	Overtake (ADE 1.14 m).	260
	Head-on (ADE 1.99 m).	260
	Ram (ADE 4.33 m).	260
	Herd (ADE 10.63 m).	261
7.11.4	Qualitative Trajectory Comparison	261
	Cross-variant comparison.	265
	7.11.4.1 Best, Median, and Worst Cases on Close Encounters	265
7.11.5	Robustness and Operational Implications	269
7.11.6	Validation Summary and Caveats	271
7.12	Conclusion	273
8	Conclusion	276
8.1	Summary of Contributions	277
	Chapter 3: NavySim.	277
	Chapter 4: Early Intent Classification.	277
	Chapter 5: CPFI.	277
	Chapter 6: Classical Trajectory Baselines.	278
	Chapter 7: MTITP.	278
8.2	Answers to Dissertation Research Questions	279
	RQ1.	279
	RQ2.	280
8.3	Limitations	280
8.4	Future Work	281
8.5	Closing Remarks	281
A	Supplementary Material for Chapter 7: MTITP	283
A.1	Implementation Notes	283
	Coordinate scale consistency.	283
	Training stability.	284
A.2	Hyperparameter Summary	285

A.3 Feature Dimensions	288
Bibliography	290

List of Tables

1.1	Detail Description of Intent and Behaviors	6
2.1	Recent deep maritime trajectory predictors (2022–2025), grouped by architectural family. Each row reports the design lineage and the primary contribution as a comparator to MTITP.	35
2.2	Summary of literature gaps and corresponding dissertation contributions.	43
3.1	Overview of basic functionality blocks.	50
3.2	Overview of the functionality of different managers	51
3.3	Overview of the scenario generation module.	52
3.4	Parameters of resources and their efficiency function to build the <i>vulnerability heatmap</i> of DDG51	58
3.5	Definition of hostile and non-hostile behaviors.	64
3.6	Definition features computed from raw observations, with the meaning of the values that could be assigned to them.	65
4.1	Performance Metrics Using Decision Trees (DT)	81
4.2	Feature Importance Using Gini Index and Entropy	82
4.3	Multi-class Performance Comparison Between Fully Connected Deep Learning (FCDL) and LSTM Models Using All and Top-15 Selected Features	85
4.4	Binary Classification Performance of FCDL and LSTM Models Using All and Top 15 Features	89
4.5	Normalized Confusion Matrices for FCDL and LSTM Models (Hostile vs Non-Hostile)	89
5.1	Feature Importance Using PFI and CPFI on LSTM	100
5.2	Multi-class Performance Comparison Between FCDL and LSTM Models Using Different Selected Features from CPFI	102
5.3	Multi-class Performance Comparison Between Fully Connected Deep Learning (FCDL) and LSTM Models Using All and Top-13 Selected Features from Transformer Feature Importance	105
5.4	Classification Results for One-vs-All Models	106

5.5	Top-23 Permutation Feature Importance (MAE) for One-vs-All LSTM Classifiers — Class 0 (Benign) to Class 3 (Cross)	109
5.6	Top-23 Permutation Feature Importance (MAE) for One-vs-All LSTM Classifiers — Class 4 (Headon) to Class 6 (Overtake)	110
5.7	Summary of Feature Importance Perspectives	113
6.1	EKF/CTRV default noise parameters.	130
6.2	Comparison of EKF-CTRV and UKF-CTRV for the 5D CTRV model.	132
6.3	Overall trajectory prediction performance: Experiment A ($T_{\text{obs}} = 40$, $T_{\text{pred}} = 20$). † numerical divergence, not model mismatch (see Section 6.5.2).	137
6.4	Per-scenario ADE (meters): Experiment A. Bold = best non-divergent model per scenario. UKF-CTRV values affected by divergence (see Section 6.5.2). UKF-5D-CV is omitted from per-scenario tables (statistically indistinguishable from CV at every scenario, $\Delta\text{ADE} < 0.05$ m); summary Table 6.11 reports its overall metrics.	138
6.5	Overall trajectory prediction performance: Experiment B ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$). Bold = best per column.	139
6.6	Per-scenario ADE (meters): Experiment B. Bold = best model per scenario. UKF-5D-CV and Asym-UKF-6D omitted (UKF-5D-CV is indistinguishable from CV; Asym-6D is consistently worst across all scenarios; both are fully tabulated in summary Table 6.11).	140
6.7	Overall trajectory prediction performance: Experiment C ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 10$). † partial divergence (see text).	141
6.8	Per-scenario ADE (meters): Experiment C. Bold = best model per scenario. UKF-5D-CV and Asym-UKF-6D omitted (see Table 6.6 caption); summary Table 6.11 tabulates all six models.	141
6.9	Overall trajectory prediction performance: Experiment D ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 20$). † numerical divergence triggered by long prediction rollout.	142
6.10	Per-scenario ADE (meters): Experiment D. Bold = best non-divergent model per scenario. UKF-5D-CV and Asym-UKF-6D omitted (see Table 6.6 caption); summary Table 6.11 tabulates all six models.	143
6.11	Cross-model comparison of classical Kalman filter baselines at $T_{\text{obs}} = 20$ across three prediction horizons. ADE columns correspond to Experiments B, C, and D; $t=1$ MAE is reported as a representative single-step value. † = numerical divergence; Bold = best per column among non-divergent models.	144
6.12	Horizon-wise MAE (meters): Experiment A ($T_{\text{pred}} = 20$).	145
6.13	Horizon-wise MAE (meters): Experiment B ($T_{\text{pred}} = 5$).	145
6.14	ADE decomposed by vessel role (Experiment A, aggregated over all scenarios).	146

7.1	Optimizer settings. All parameter groups use Adam with the same β values across all three training variants. The Critic group is absent in MTITP-L2.	190
7.2	MTITP variants: overall test-set performance on the canonical configuration (noiseless dataset; $T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$; $K = 20$ samples; 7 active scenarios). ADE is the best-of- K minADE@20 by construction; FDE differs from minFDE@20 only at sub-decimeter precision and is reported alone. Bold = best per column.	206
7.3	Per-scenario test ADE (m) across MTITP variants on the canonical configuration (noiseless dataset; $T_{\text{pred}} = 5$; $K = 20$). Past/Future intent accuracy and future mAP are shown for the WGAN baseline; the L2 and MarITGAN v2 variants match within 0.1 percentage point at the overall level. Bold = lowest ADE per scenario.	207
7.5	Hostile vs. non-hostile binary classification for past intent (per-window). Per-class accuracy equals recall in the binary case. NH = benign/cross/-headon/overtake; H = block/herd/ram. Bold = best per column across rows.	209
7.6	Hostile vs. non-hostile for future intent (per-step <code>beh_label</code>). Columns as in Table 7.5. Bold = best per column across rows.	210
7.7	Trajectory error sliced by ground-truth hostile flag (best-of- K , native test split). H = hostile, NH = non-hostile. ADE/FDE in meters. Bold = lowest (best) per column.	210
7.8	MTITP vs. classical Kalman filter baselines at matched configuration ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$). KF metrics from Chapter 6, Experiment B. Bold = best per column.	212
7.9	Per-scenario ADE (meters): MTITP vs. classical Kalman filters at matched configuration ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$). KF metrics from Chapter 6, Experiment B. Bold = best per scenario. Stationary excluded from all evaluations.	213
7.10	External-baseline comparison on Noisy_Dataset_1 (past-intent scenario classification). Meepaganithage et al. [67] use a class-balanced split; MTITP-L2 (A18) uses the natural distribution. Acc/Prec/Rec/F1 are weighted averages; binary-collapse rows use per-class entries from Table 7.5.	215
7.11	Per-class precision / recall / F1 for seven-way past intent on Noisy_Dataset_1. Bi-LSTM [67] (class-balanced split); MTITP-L2 (A18) (natural distribution, Table 7.23). Bold = higher per class per metric.	216
7.12	Architecture ablations: structural components removed from MTITP-WGAN.	220
7.14	Trajectory loss ablations: L2 versus WGAN-GP across prediction horizons.	221
7.16	Robustness ablations: training and evaluation on sensor-noisy datasets.	222

7.18	MarITGAN ablations: intent-aware, interaction-conditioned critic and its noisy-data variant.	223
7.20	Architecture ablations: overall test-set metrics at $T_{\text{pred}} = 5$, noiseless dataset. ΔADE is relative to MTITP-WGAN. Bold marks best per column.	224
7.21	Trajectory loss comparison: L2 vs. WGAN-GP at $T_{\text{pred}} \in \{5, 10, 20\}$. Same generator architecture; only training loss and critic activation differ. Intent accuracy ($\sim 99\%$) is identical across configurations and omitted. Bold = lower ADE per horizon.	225
7.22	Per-scenario ADE (m) across ablations. Bold marks best (lowest) ADE per scenario on Noiseless Dataset and $T_{\text{pred}} = 5$	226
7.23	Robustness ablation: MTITP-L2 on moderate noise (A18, Noisy_Dataset_1) vs. clean baseline (A13). ADE/FDE in meters; Past Acc. and Future Acc. are scenario and per-step intent accuracy. Bold = better per metric per row.	240
7.24	Robustness ablation: MTITP-L2 on high noise (A19, Noisy_Dataset_2) vs. clean baseline (A13). ADE/FDE in meters; Past Acc. and Future Acc. are scenario and per-step intent accuracy. Bold = better per metric per row.	241
7.25	Adversarial vs. L2 under moderate noise (Noisy_Dataset_1): A22 (MarITGAN v2) vs. A18 (L2). ADE/FDE in meters; Past Acc. and Future Acc. as fractions. Bold = better per metric per row; ties unbolded.	245
7.26	Dataset split after excluding stationary (stride $s = 10$, seed 42). 70/15/15 split at the <i>encounter</i> level, so window counts deviate from these ratios.	247
7.27	Intent accuracy: trivial baselines vs. trained models (noiseless test set). Majority-class always predicts the most frequent training class; random samples uniformly over 7 active classes.	249
7.28	Cross-dataset generalization: each model evaluated on all three datasets using its own normalization. Bold = native (in-distribution) result. ADE in meters; Past-Acc and Future-Acc as fractions. A22 omitted (not cross-evaluated). Diagonal cells are produced by the validation-notebook pipeline, which differs from the headline-results pipeline (Table 7.20) in its normalization file and in explicit filtering of stationary-vessel windows; native values may therefore differ from the corresponding rows of Table 7.20 by a small amount (e.g., A21 native ADE 2.18 vs. headline 2.23). Both pipelines load the same <code>ckpt_best_ade.pt</code> checkpoint per model.	253
7.29	McNemar’s test for noiseless-trained models (MTITP-WGAN, A13, A21) on the shared 136,614-window noiseless test set. ns = not significant; *** = $p < 0.001$	255
A.1	MTITP data and windowing hyperparameters.	285

A.2	MTITP model architecture hyperparameters.	286
A.3	MTITP training and optimisation hyperparameters.	287
A.4	Stream A per-vessel kinematic features (8 per vessel per timestep). . .	288
A.5	Stream B pairwise geometry features (12 per timestep).	289

List of Figures

1.1	Advanced Maritime Vessel Intent Recognition Framework Using NavySim, Generative Adversarial Networks, and Expert Knowledge in the Loop	7
3.1	The system architecture diagram of the naval simulator (NavySim)	48
3.2	Main menu layout for scenario generation.	53
3.3	The own ship and target ship positions in three different situations. (a) The head-on situation. (b) The overtaking situation. (c) The crossing situation.	54
3.4	Resource efficiency at certain location X.	56
3.5	DDG51's Resources and <i>Vulnerability Heatmap</i> adapted in the Unity simulator with the shader; the snapshot covering only a 4.5 km distance around the vessel.	58
3.6	<i>Predictive Action Heatmap</i> over time as future possible hotspots, adapted in Unity simulator.	61
3.7	<i>Threatmap</i> illustrating potential high-danger hotspots around the selected DDG51 ship (blue team) caused by the three pilot vessels (red team). Two pilot vessels show RAM (red flags), and one shows HERD (yellow flag) intent toward the DDG51 ship. Group behavior, including ship names and intents, is flagged in the top panel.	63
4.1	The three regions around a vessel: Head-on (H) - directly in front; Overtake (O) - directly behind; Cross (C) - either side of the ship is the cross region. In this case, the blue and red vessels are within the cross region of each other.	72
4.2	Proposed Research Methodology.	75
4.3	Fully Connected Model Architecture	78
4.4	LSTM Model Architecture	79
4.5	Training and validation accuracy and loss curves for the FCDL model using the top 15 features.	84
4.6	Training and validation accuracy and loss curves for the LSTM model using the top 15 features.	84

4.7	Sample scenarios for each type of behavior. Top: Position graph of vessels in latitude/longitude coordinates. Middle: Intent predicted over time. Bottom: Ground truth over time. In (b), (c), (d), and (f), the ground truth starts as the labeled intent and then ends with a benign intention.	85
4.8	Confusion matrices for the FCDL and LSTM models using the top 15 features.	87
4.9	ROC Curves Using Top 15 Features	87
5.1	A Deep-Learning Framework for Maritime Intent Recognition	94
5.2	LSTM Feature Importance Score by CPFI	101
5.3	(Left) Accuracy over training epochs. (Right) Loss over training epochs. Both plots include training and validation data for comparison. . . .	102
5.4	(Left) Accuracy over training epochs. (Right) Loss over training epochs. Both plots include training and validation data for comparison with the selected subset of features.	103
5.5	Transformer Based Feature Importance Score	103
6.1	Benign-scenario forecasts. Both filters track the ground truth to within a meter over twenty steps; the EKF's arc model provides no measurable advantage when the underlying motion is straight-line.	147
6.2	Median-difficulty CV forecasts on three encounter geometries. Persistent heading and speed produce sub-meter to low single-digit ADE despite the encounter classification; linear extrapolation is sufficient when neither vessel maneuvers mid-window.	148
6.3	Two ways to be wrong on the same scenario class (ram). The geometry of each predicted trajectory exposes the underlying motion model: CV produces straight lines that overshoot, the EKF-CTRV produces arcs that curl away from the true future.	148
6.4	Herd, CV (worst case, 245.7 m ADE). Both vessels execute looping ground-truth trajectories (dashed); the predicted future (dotted) is forced to extrapolate linearly because a single deterministic motion model cannot represent the multi-modal maneuver distribution. . . .	149
6.5	EKF-CTRV vs. UKF-CTRV forecast comparison on a long-horizon encounter. The UKF-CTRV's covariance diverges; the EKF-CTRV remains stable despite its own arc-extrapolation error on hostile scenarios.	150
6.6	Horizon-wise MAE curves under both experiment configurations. EKF-CTRV's $t = 1$ advantage erodes by $t \approx 3-4$; UKF-CTRV divergence in Experiment A appears as a steep upward curve after $t = 5$	150

6.7	Per-horizon RMSE (Experiment A, $T_{\text{pred}} = 20$) for all six classical filters. The three linear-dynamics methods (CV, UKF-4D, UKF-5D-CV) trace a single curve; the EKF-CTRV peels above the linear band after $t \approx 4$; the Asym-UKF-6D and the divergent UKF-CTRV separate as distinct upper envelopes. RMSE is shown alongside MAE (Figure 6.6) because RMSE penalizes the worst-case scenarios that drive aggregate ADE. .	151
6.8	Scenario ADE heatmap (Experiment B). Near-zero error on easy scenarios but 12–13 m ADE on herd marks the ceiling of intent-agnostic filters.	151
7.1	MTITP system architecture. The three-stream encoder (kinematics, pairwise geometry, symbolic interactions) feeds the intent heads and trajectory generator. The training-only critic is variant-specific: MTITP-WGAN (domain-agnostic), MarITGAN (three-branch IntentAware-Critic shown), MTITP-L2 (no critic).	174
7.2	Three-stream Spatial-Temporal Encoder. Stream A (per-vessel kinematics) is projected and concatenated with Streams B and C through Interaction Fusion, then passed through alternating Temporal and Spatial Transformer blocks to produce <code>enc_seq</code> and <code>scene_emb</code>	176
7.3	Future Intent Predictor. Temporal mean-pooling over the encoder sequence <code>enc_seq</code> produces $\mathbf{e}_{\text{pool}}$, which a two-layer MLP maps to per-step intent embeddings $\mathbf{E}_{\text{int},t}$ and class logits $\ell_{\text{future},t}$ over the T_{pred} -step horizon. The intent embeddings are reused as conditioning input by the Trajectory Generator and the IntentAwareCritic.	183
7.4	Trajectory Generator. Gaussian noise $\mathbf{z}$ and the per-step intent embedding $\mathbf{E}_{\text{int},t}$ are concatenated and projected to form the conditioning vector $\mathbf{c}_t$, which is added to learned trajectory queries. A three-layer Transformer decoder cross-attends to <code>enc_seq</code> , predicts per-step displacements, and accumulates them into the future trajectory $\hat{\mathbf{X}}$	184
7.5	Per-iteration training pipeline for the GAN variants of MTITP. [A] Intent step updates the encoder and intent heads via $\mathcal{L}_{\text{intent}}$. After warmup, <code>enc_seq</code> and <code>intent_emb</code> are detached and reused by [B] critic step (N_c updates with WGAN-GP) and [C] generator step ($\mathcal{L}_G$ adds $\lambda_{\text{phys}}\mathcal{L}_{\text{phys}} + \mathcal{L}_{\text{aux}}$ for MarITGAN). MTITP-L2 omits step B and replaces step C with a direct L2 regression on the future trajectory. Detaching the encoder outputs prevents trajectory-side gradients from rewriting the intent representation.	193
7.6	IntentAwareCritic used by MarITGAN. Three parallel branches reason over (1) the future trajectory with intent conditioning, (2) inter-vessel relative motion geometry computed inside the forward pass, and (3) intent-scene consistency via cross-attention between the future intent embeddings and <code>enc_seq</code> . Branch outputs are concatenated and fused by an MLP into a scalar Wasserstein score.	198

- 7.7 Benign encounter across the three datasets. The two vessels travel in roughly parallel directions with no engagement. The minimum vessel-pair distance is large (about 5,300 m), and the heading offset that separates benign from cross is small. The Noisy_Dataset_1 column shows how the simulated AIS/GPS jitter perturbs the path while preserving the start and end positions. The marker $\bullet$ denotes the start position and $\blacktriangle$ the end position. 235
- 7.8 Cross encounter across the three datasets. The two trajectories meet at an oblique angle. The Noisy_Dataset_1 column retains the macro crossing geometry, but the precise crossing point and timing, which the model has to regress, are blurred by the mid-track jitter. The Noisy_Dataset_2 column shows a structurally similar but independently sampled encounter at higher noise. 235
- 7.9 Headon encounter across the three datasets. The two vessels approach on near-reciprocal headings, and the discriminative signal is a small lateral offset that determines the passing side. The Noisy_Dataset_1 jitter is comparable in magnitude to that offset, which is consistent with the increase in headon ADE from 0.88 m on the clean baseline to 7.14 m on Noisy_Dataset_1. 236
- 7.10 Overtake encounter across the three datasets. A faster vessel passes a slower one travelling in the same general direction. The defining feature is a small relative-velocity vector, which is the kind of low-magnitude kinematic cue that does not survive the Noisy_Dataset_1 perturbation and is replaced by a different sample geometry under Noisy_Dataset_2. 236
- 7.11 Block encounter across the three datasets. One vessel cuts ahead of another to obstruct its path. The cut-in is a sharp geometric feature visible in all three columns, and the Noisy_Dataset_1 column perturbs the mid-track without erasing the L-shape of the maneuver. Block ADE increases from 1.19 m on the clean baseline to 6.09 m on Noisy_Dataset_1, which is much less than the order-of-magnitude jumps reported for the non-hostile behaviors. 237
- 7.12 Herd encounter across the three datasets. One vessel repeatedly maneuvers to corral or steer the other, producing a high-frequency oscillating geometry that is large compared to the noise envelope. Herd is the only behavior for which Noisy_Dataset_1 actually improves ADE (7.66 m on the clean baseline, 5.96 m on Noisy_Dataset_1). We attribute this to the noise acting as a regularizer on a genuinely multi-modal future. . 238

7.13	Ram encounter across the three datasets. A sharp adversarial closure brings the two vessels to within about 15 m, often with a heading reversal at the terminal phase. The displacement of the maneuver is much larger than the noise envelope, which is consistent with ram ADE only doubling on Noisy_Dataset_1 (2.87 m on the clean baseline, 6.26 m on Noisy_Dataset_1) where the smoother behaviors increase by an order of magnitude.	238
7.14	t-SNE projections of scene embeddings $\mathbf{e}_{\text{scene}} \in \mathbb{R}^{2D_{\text{model}}}$ (4,000 test samples per model; one point per window, colored by scenario). MarITGAN v2 (A21, top-left) shows tighter clustering of the three upweighted classes (headon/herd/ram) from scenario-weighted training. Noisy-data models (A18, A19) retain seven-way separability despite corrupted inputs.	251
7.15	t-SNE projections of the encoder sequence $\mathbf{E}_{\text{seq}}$ (mean-pooled over time per vessel and concatenated; 4,000 test samples per model, colored by scenario). $\mathbf{E}_{\text{seq}}$ is the input the future-intent predictor and the trajectory generator cross-attend to. All five models produce well-separated clusters with class-internal compactness comparable to or tighter than $\mathbf{e}_{\text{scene}}$, indicating that the discriminative signal is present at every timestep the decoders consume.	252
7.16	McNemar’s test $-\log_{10}(p)$ for past (left) and future (right) intent. Rows/cols: MTITP-WGAN, A13, A18, A19, A21. Annotations: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, ns not significant. Cross-dataset pairs (A18/A19 vs. noiseless models) violate the paired-sample assumption and are shown for reference.	255
7.17	Per-scenario trajectory forecasts on <i>non-hostile</i> encounters. Columns: MTITP-WGAN, MTITP-L2, MarITGAN v2. Same encounter shown across columns for direct comparison; per-panel <i>minADE</i> is reported in each subtitle.	263
7.18	Per-scenario trajectory forecasts on <i>hostile</i> encounters. Columns: MTITP-WGAN, MTITP-L2, MarITGAN v2. Same encounter shown across columns. The wider spread in the sample reflects genuine multimodal uncertainty about the future rather than poor calibration.	264
7.19	Best, median, and worst cases on the <i>head-on</i> test slice (rows) across the three MTITP variants (columns). Same encounter shown across each row.	267
7.20	Best, median, and worst cases on the <i>herd</i> test slice (rows) across the three MTITP variants (columns). Same encounter shown across each row.	268
7.21	Best, median, and worst cases on the <i>ram</i> test slice (rows) across the three MTITP variants (columns). Same encounter shown across each row.	269

Chapter 1

Introduction

Maritime situational awareness is a critical component of global security, commercial logistics, and autonomous navigation systems. Accurate recognition of vessel intent enables safer navigation, improved tactical planning, and proactive risk mitigation. The growing complexity of maritime environments, defined by multi-agent dynamics, overlapping operational goals, and diverse vessel behaviors, poses significant challenges for automation and intelligent decision-making. Recent surveys of AI-driven maritime security document a pronounced shift from rule-based monitoring toward learned anomaly detection, trajectory prediction, dark-vessel tracking, and threat classification, while emphasizing that the field still lacks the labeled adversarial encounter data needed to train modern deep models [1], and comprehensive reviews of Automatic Identification System (AIS) analytics and transformer applications observe that the

field is now bottlenecked by labeled benchmark scarcity rather than by architectural innovation [2–4].

High-profile incidents such as the USS Fitzgerald collision in 2017 [5] underscore the urgent need for enhanced predictive maritime awareness technologies. In response, maritime simulation platforms, often implemented as serious games, have emerged as indispensable tools for both research and training. These systems provide risk-free, controlled environments that simulate real-world vessel interactions and enable the development of core competencies in navigation, maneuvering, and behavioral interpretation [6–8]. In parallel, recent surveys of multimodal maritime perception argue that fusing radar, electro-optical/infrared imagery, AIS, and synthetic-aperture radar streams is necessary to overcome the limitations of AIS-only baselines, such as non-cooperative targets and signal gaps, in detection, classification, and tracking [9], sharpening the demand for simulators that can deliver synchronized labeled multi-sensor data rather than position traces alone.

Despite progress in this domain, several persistent challenges hinder the advancement of intelligent maritime systems. These include: (1) the limited availability of diverse and high-quality domain-specific datasets [2, 10]; (2) noise and uncertainty in sensor readings; and (3) the inability of purely data-driven models to capture maritime contextual nuances encoded in international navigation rules guided by International Regulations for Preventing Collisions at Sea (COLREGs) and expert tactical heuristics. Recent vessel trajectory prediction surveys catalog an explosion of architectural variants

since 2020 but report few systematic comparisons under noisy conditions [4], reinforcing that data quality and labeled adversarial coverage, rather than model capacity, now drive the relevant accuracy gaps.

To address these limitations, this dissertation introduces an integrated framework that combines simulation-based data generation, feature selection, and behavior-specific representation learning, generative modeling, expert knowledge infusion, and multi-task learning. The overarching goal is to enhance maritime intent recognition by enabling the generation, classification, prediction, and interpretation of complex vessel behaviors and trajectories, especially in scenarios where real-world data is sparse, noisy, or incomplete.

This research addresses five interrelated problems:

1. **Simulation-based scenario generation and maritime behavior modeling**
using a high-fidelity, multi-agent platform (NavySim). Also, used as a test-bed for validating algorithms in real-time scenarios.
2. **Explainable maritime classification via feature selection:** Categorical Permutation Feature Importance (CPFI), a model-agnostic attribution method for robust feature ranking under class imbalance and many-level categorical predictors; paired in downstream chapters with the Transformer Feature Importance Score (TFIS), a complementary attention-based attribution method developed by a collaborator.

3. **Feature-optimized intent classification on maritime vessel behaviors:** deep-learning classifiers for detection of vessel intent and feature-optimized intent-recognition pipelines that operationalize explainability methods like CPFI/TFIS.
4. **Classical trajectory prediction baselines and their structural limitations:** a systematic evaluation of physics-based Bayesian filters that establishes the performance ceiling of intent-agnostic methods and exposes the failure modes that motivate the joint deep model in Problem 5.
5. **Multi-task intent and trajectory prediction (MTITP)** using a unified architecture (MTITP) that predicts vessel scenario intent and future trajectories from a shared three-stream encoder conditioned on vessel kinematics, pair-wise geometric and symbolic COLREGs-derived interaction features, studied through three training variants of a common backbone (MTITP-WGAN, MTITP-L2, MarITGAN).

These problem areas collectively contribute to developing a robust, interpretable, and operationally deployable framework for understanding maritime vessel behavior. Figure 1.1 offers a high-level overview of the proposed architecture and its constituent modules.

Overarching Research Questions. Two dissertation-level questions cut across Problems 4 and 5 and are answered jointly by Chapters 6 and 7:

RQ1 *What are the structural limitations of intent-agnostic classical trajectory predictors on adversarial maritime vessel scenarios, and what properties must a successor model have to overcome them?*

RQ2 *Can a joint intent-trajectory prediction model outperform both independent intent classifiers and classical trajectory predictors in maritime vessel encounters?*

The remainder of this chapter presents each problem in detail, outlining its motivation, associated challenges, and the corresponding methodology developed to address it.

1.1 Vessel Intent and Behavior

Two key terms are frequently used throughout this dissertation and require clear definitions. **Behavior** refers to the kinematic actions exhibited by a vessel during an encounter. These actions include ramming, blocking, herding, crossing, overtaking, head-on approach, or benign navigation, and are categorized into one of the seven scenario classes summarized in Table 1.1. **Intent** refers to the higher-level adversarial posture that can be inferred from these behaviors. The three hostile behaviors (RAM, BLOCK, and HERD) indicate adversarial intent, while the four remaining behaviors (CROSS, OVERTAKE, HEADON, and BENIGN) suggest non-hostile intent. The intent-recognition and trajectory-prediction models developed in this dissertation aim to predict both layers: the fine-grained behavior labels at each encounter and

timestep, as well as the binary classification of hostile versus non-hostile intent, based on observed vessel motion and pairwise geometry.

TABLE 1.1: Detail Description of Intent and Behaviors

Behaviors	Intent	Detail Definition
RAM	Hostile	One ship intentionally attempts to collide with another ship, showing an aggressive maneuver.
BLOCK	Hostile	One ship attempts to stop another ship from reaching a particular location.
HERD	Hostile	One ship tries to force another ship to direct in a specific direction.
CROSS	Non-Hostile	A ship’s trajectory intersects with another ship’s, creating a potential risk of collision while crossing.
OVERTAKE	Non-Hostile	A ship approaches another from behind at an angle of at least 22.5 degrees relative to the center from the end, in order to overtake.
HEADON	Non-Hostile	A ship approaches the path of another ship within 20 degrees of its bow, posing a risk of collision.
BENIGN	Non-Hostile	Behaviors that do not show any of the hostile actions defined in the preceding rows.

1.2 Problem Statements

The following five problems represent key challenges in advancing maritime vessel intent recognition, spanning simulation-driven data generation, feature-importance methods, intent classification, physics-based baseline evaluation, and multi-task generative modeling. Each problem addresses a specific gap in the current state-of-the-art, and

together they contribute to a comprehensive framework for understanding, predicting, and reacting to complex maritime behaviors. Figure 1.1 provides a visual summary of the proposed system architecture and its major components.

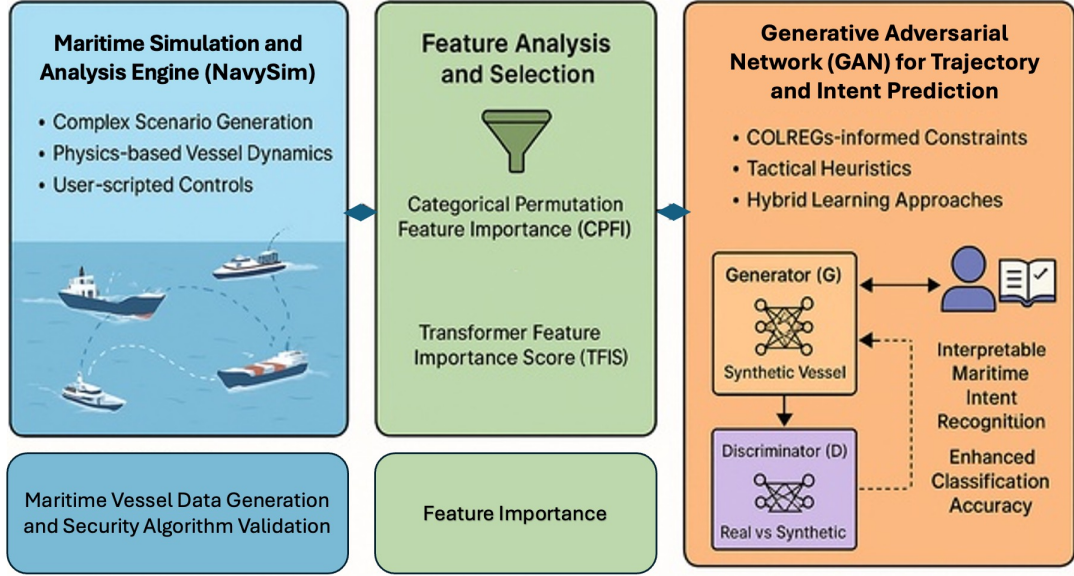


FIGURE 1.1: Advanced Maritime Vessel Intent Recognition Framework Using NavySim, Generative Adversarial Networks, and Expert Knowledge in the Loop

1.2.1 Problem 1: High-Fidelity Maritime Simulation Architecture for Complex Scenario Generation, Intent Recognition, and Security Strategy Evaluation

Real-world maritime datasets, such as those derived from the Automatic Identification System (AIS), are widely used to train intent recognition and navigation models. However, these datasets often suffer from critical limitations. They tend to be sparse, contain noisy or missing values, lack behavioral diversity, and rarely capture adversarial

or rare interactions such as ramming, blocking, or coordinated herding maneuvers [10]. These shortcomings hinder the development of robust machine learning models for maritime safety and behavior understanding in multi-agent settings.

In addition, the maritime domain presents unique simulation challenges. Vessel motion is governed by complex dynamics and influenced by environmental factors such as currents and wind. Navigation must also comply with international protocols such as the COLREGs. The resulting behavior space, particularly in multi-vessel scenarios, makes it difficult to simulate realistic interactions using rule-based or purely data-driven methods alone.

The contemporary maritime simulation landscape remains heterogeneous and partially overlapping with NavySim’s design objectives. Industrial big-data and digital-twin analytics platforms such as VesselAI [11] prioritize fleet-scale AIS and sensor analytics with federated learning over single-vessel hydrodynamic fidelity, while engineering-grade autonomous-ship twins [12] emphasize asset-level analytics and six-degree-of-freedom hydrodynamic modeling for an individual vessel; recent maritime digital-twin reviews [13] catalog these efforts but observe that scriptable multi-vessel encounter generation remains outside the scope of most existing platforms. Open-source robotics simulators such as USVSim [14] dominate the unmanned surface vehicle community but lack COLREGs-aware adversarial scripting. Operational situational-awareness platforms (GMSA [15]) emit anomaly flags rather than intent labels, and recent

synthetic-traffic generators [16] and LLM-driven AIS expansions [17] produce trajectory volume but stop short of attaching the per-encounter behavioral annotations required for supervised intent learning. Public AIS-derived navigation databases constructed from VTS streams [18] aggregate real navigation traces at coarse behavioral granularity. NavySim is positioned against this landscape as a labeled-encounter simulator: scriptable adversarial behaviors, COLREGs-compliant planners, and per-timestep intent labels are first-class outputs rather than incidental byproducts.

To overcome these limitations, we developed **NavySim**, a high-fidelity, extensible simulation platform designed to generate complex, multi-vessel maritime scenarios [6, 19]. NavySim supports both research and operational applications through the following core capabilities:

- **Scenario Generation:** Fully scriptable control over dynamic scenarios, enabling adversarial and cooperative behaviors within a physics-based vessel motion engine.
- **Threat and Vulnerability Visualization:** Real-time heatmaps indicating proximity-based risk, exposure, and threat levels between agents.
- **COLREGs-Compliant Navigation:** Velocity Obstacle-based planning models ensure all navigation remains consistent with maritime collision avoidance protocols.

- **Behavior Modeling and Intent Recognition:** Real-time inference using Hidden Markov Models (HMMs) and Deep learning models to recognize both individual and coordinated behaviors.
- **Decision-Support Integration:** On-the-fly evaluation of autonomous navigation and defensive resource allocation strategies, supported by live visualization tools.

NavySim, elaborated in Chapter 3, serves as the foundational simulation infrastructure for this dissertation. It facilitates the generation of maritime trajectory data, the annotation of vessel intents, and the validation of complex behavioral scenarios under controlled yet realistic conditions.

1.2.2 Problem 2: Explainable Maritime Classification via Feature Selection

Operational deployment of maritime classifiers depends not only on accuracy but on *explainability*: regulators, operators, and class societies increasingly require that intent and threat decisions be traceable to identifiable input features, with explainable-AI surveys aimed at autonomous surface ships arguing that feature-level transparency is now both a regulatory and scientific requirement under emerging the International Maritime Organization (IMO) and class-society guidance [20]. High-dimensional navigational datasets, however, contain redundant or irrelevant features that erode

interpretability while inviting overfitting and inefficiency [21–23], and recent surveys of maritime ML observe that feature pipelines in this domain remain largely hand-engineered, with few dedicated studies of categorical or symbolic vessel attributes [24]. Closing this gap requires feature-attribution machinery that can both rank features for principled selection and surface the cues a model actually relies on for intent decisions.

To address this, we develop **Categorical Permutation Feature Importance (CPFI)**, a model-agnostic attribution method designed for robust feature ranking under class imbalance and the presence of constant-valued or non-informative features. CPFI is grounded in recent critiques of out-of-bag and impurity-based importance scoring that document systematic misranking under correlated predictors and many-level categorical variables [25], and it forms the core feature-attribution contribution of this dissertation. To capture dynamic, nonlinear relationships that permutation-based scoring cannot recover, CPFI is paired in downstream chapters with the **Transformer Feature Importance Score (TFIS)**, an attention-based attribution method developed by a collaborator and applied here as a complementary tool rather than a core contribution; TFIS is motivated by sensitivity analyses showing that SHAP attributions can flip feature rankings under baseline-distribution changes [26, 27] and by recent maritime applications that surface operator-interpretable cues through attention-weight inspection [28] and SHAP-based attribution [29]. Region-specific ship-classification studies further demonstrate that the most informative features shift markedly with operating environment [30], reinforcing the case for context-aware attribution rather than fixed importance lists. CPFI and the paired CPFI/TFIS pipeline

together support both principled feature pruning and operator-facing explanations of classifier decisions; they are developed and validated in Chapters 4 and 5, where they also drive the intent classifiers studied in Problem 3.

1.2.3 Problem 3: Feature-Optimized Intent Classification on Maritime Vessel Behaviors

Intent recognition is the bridge between raw vessel motion and operational decision-making: tactical alerting, route advisories, and threat triage all depend on identifying *what* a vessel is doing well before its trajectory completes. The maritime intent-recognition literature has matured rapidly, with symbolic-feature transformers [31], dual-channel attention models [32], LSTM-with-cross-attention baselines [33], and Bayesian extensions of HMM/DBN traditions [34] all reaching strong benchmark numbers, while domain-adapted classifiers for fishing-vessel behavior [35, 36] and harbor-berthing intent [37] demonstrate that explicit context encoding consistently outperforms purely data-driven baselines.

Building on Problem 2’s feature-importance machinery, this dissertation contributes two intent classification studies. First, an **intent classifier** detects vessel behavior over a long observation window (the first 500 frames of each scenario), using deep sequence models trained on the CPFI/TFIS-selected feature subsets to maximize discrimination at each observation step. Second, a **feature-optimized intent recognition** pipeline applies the same attribution methods inside the training loop to prune, weight,

and re-rank features specifically for the intent-classification objective, improving both accuracy and interpretability over fixed-feature baselines. These studies are presented in Chapters 4 and 5 respectively. Together, they establish the standalone intent-recognition results against which the joint MTITP model of Problem 5 is later compared.

1.2.4 Problem 4: Classical Trajectory Prediction Baselines and Their Structural Limitations

Before deploying complex data-driven models, it is essential to establish rigorous physics-based baselines to quantify the ceiling for trajectory prediction for intent-agnostic methods. Without this foundation, claims about the value of learned representations and adversarial generation are difficult to contextualize. Classical filter design for maritime tracking remains an active research area: recent work has produced geodetic-coordinate Unscented Kalman Filters (UKFs) that eliminate projection error on long-range tracks [38], improved sigma-point parameterizations [39], continuous-discrete formulations for non-uniformly sampled AIS [40], square-root cubature variants of the Interacting Multiple Model (IMM) estimator that stabilize the update step under aggressive maneuvering [41], adaptive IMM schemes that grow the model bank online [42], and production-grade IMM implementations tuned for cooperative AIS

targets [43]. Each of these refinements is algorithmic rather than structural; the underlying filter family remains bounded by a finite library of analytic motion templates, with no representation of tactical scenarios or vessel-pair interactions.

To address this, we conduct a systematic evaluation of six classical Bayesian trajectory predictors on the same two-vessel maritime scenario dataset used throughout this dissertation: a Constant Velocity (CV) baseline, a Standard Unscented Kalman Filter (UKF-4D), a UKF with 5D constant-velocity state (UKF-5D-CV), an Asymmetric UKF with acceleration state (UKF-CA-6D), an Extended Kalman Filter with the Coordinated Turn Rate and Velocity model (EKF-CTRV), and a UKF with CTRV dynamics (UKF-CTRV). Four experiment configurations vary the observation window and prediction horizon to expose how filter stability and accuracy depend jointly on both parameters. The comparison is intentionally restricted to the canonical KF/EKF/UKF building blocks that the recent refinements share, so that the structural gap, rather than implementation tuning, is what is being measured.

This evaluation is presented in full in Chapter 6, where it characterizes the structural limitations of intent-agnostic filters on adversarial maritime scenarios and supplies the matched-configuration classical baselines (ADE, FDE) against which MTITP is compared in Chapter 7.

1.2.5 Problem 5: Multi-Task Intent and Trajectory Prediction (MTITP)

Robust trajectory prediction and intent recognition are both essential to maritime situational awareness. The standalone intent classifiers of Problem 3 produce scenario labels but no intent predictions or trajectory forecasts, whereas the classical baselines of Problem 4 produce trajectories but no intent context, thereby hitting a structural accuracy ceiling on adversarial scenarios for that reason. Addressing both gaps simultaneously requires a unified model in which intent supervision regularizes trajectory prediction and trajectory geometry constrains intent recognition.

The maritime intent recognition and trajectory prediction literature has anticipated each ingredient of this design without combining them. SEMINT [31] couples symbolic vessel intent descriptors with a Large Language Model (LLM) backbone for trajectory prediction; however, intent is not formally defined and is only inferred through semantic reasoning by the LLM. Multi-vessel interaction modeling has been explored through hybrid GNNs [44], dual-channel spatio-temporal attention models such as DUCTS-TMA [32], and LSTM-with-cross-attention baselines [33]; and generative–discriminative coupling appears in transformer-VAE trackers [45], cooperative conditional VAEs [46], and region-aware GANs [47, 48], with statistical alternatives including Bayesian formulations [34] and multi-factor scoring [49]. On the trajectory side, transformer-based AIS forecasters have proliferated since TrAISformer [50] into

graph-augmented [51], multi-graph-fusion [52, 53], and physics-augmented [54] variants, while the generative-trajectory landscape has fragmented into adversarial [55, 56], variational [57, 58], and diffusion families [59–61]. The closest published joint-learning analogs are PTINet [62], IntentFormer [63], VesNet [64], and ship-specific MTL [65, 66], which validate the joint-objective hypothesis but do not produce both a tactical scenario label and intent-conditioned multimodal trajectories from a single maritime backbone.

To address this, we developed **MTITP** (Multi-Task Intent and Trajectory Prediction), a unified architecture for joint scenario-intent classification, intent prediction, and future-trajectory generation. MTITP defines a shared backbone consisting of a three-stream encoder, past- and future-intent heads, and an intent-conditioned trajectory generator, and is evaluated across three training variants distinguished by the trajectory loss used during joint training. Key contributions include:

- **Three-Stream Encoder:** Per-vessel kinematics (Stream A), continuous pairwise geometry (Stream B), and symbolic COLREGs-derived interaction features (Stream C) are fused through alternating temporal self-attention and cross-vessel attention layers. Stream C embeds maritime domain expertise (port/starboard relationship, CPA classification, heading-change class) directly into the encoder rather than as external conditioning.
- **Joint Intent Classification, Prediction and Trajectory Prediction:** Past scenario class and future per-step behavioral labels are predicted simultaneously

from the shared encoder, allowing intent supervision to regularize the encoder toward scene-discriminative features that also improve trajectory generation.

- **Three Training Variants of a Shared Backbone:** The encoder, intent heads, and trajectory generator are held fixed across three variants distinguished only by the trajectory loss applied during the joint phase. **MTITP-WGAN** (baseline) employs a domain-agnostic Wasserstein critic with gradient penalty; **MTITP-L2** removes the critic and supervises trajectories with direct L2 regression; and **MarITGAN** replaces the Wasserstein critic with a purpose-built IntentAware-Critic that scores trajectory–intent agreement, relative-motion geometry, and intent-conditioned scene cross-attention.
- **Two-Phase Training Protocol:** A 15-epoch encoder warmup builds a stable intent-aware representation before the joint phase begins, preventing the encoder collapse observed when classification and trajectory objectives are applied simultaneously from the start.

The three variants are evaluated against the matched-configuration classical baselines of Problem 4 and against published external maritime-intent baselines, and a structured ablation study isolates the contributions of each architectural component: multi-task intent supervision, symbolic interaction features, and the choice of trajectory loss.

This problem is explored in full in Chapter 7, which covers architecture design, the two-phase training strategy, complete experimental evaluation, and the structured

ablation study. The classical baseline comparisons underpinning the motivation for MTITP are presented in Chapter 6.

In summary, this dissertation advances maritime situational awareness through a pipeline spanning simulation, feature selection as an explainability tool, intent classification, intent prediction, classical baseline evaluation, and multi-task generative modeling. Chapter 3 describes the NavySim simulation platform (Problem 1); Chapters 4 and 5 develop the CPFI and TFIS attribution methods (Problem 2) and apply them to early and feature-optimized intent classification on maritime vessel behaviors (Problem 3); Chapter 6 establishes the classical trajectory-prediction baselines and their structural failure modes (Problem 4); and Chapter 7 presents MTITP, the joint intent-trajectory prediction model that addresses the gaps surfaced by Problems 3 and 4 simultaneously (Problem 5), also introduces intent prediction. Each component is described in detail, supported by experiments on the “HII-Mission Technologies” generated dataset [67].

Chapter 2

Literature Review

Maritime intent recognition and trajectory prediction have largely been studied in isolation: classifiers produce no forecasts, and kinematic filters carry no intent context. We survey five areas of the literature that bear directly on this dissertation: maritime simulation, intent recognition, feature selection, trajectory prediction, and generative modeling with domain priors. Each section closes with a precise gap statement, and Table 2.2 consolidates these gaps into the unified research agenda this work addresses.

2.1 Maritime Simulation and Scenario Generation

2.1.1 Role of Simulation in Maritime Research

Real-world maritime data, primarily sourced from the Automatic Identification System (AIS), suffers from spoofing, signal loss, and irregular sampling rates [2], and severely under-represents adversarial encounters such as ramming, blocking, and coordinated herding [1]. These deficiencies make AIS alone insufficient for training adversarial intent classifiers and motivate simulation-based data generation. Tsoukalas et al. [68] built a Norcontrol PPT 2000-based engine room simulator for training marine engineers. Hjelmervik et al. [69] showed that progressive scenario complexity improves trainee docking performance. Both works demonstrate the value of synthetic environments but are confined to single-vessel or small-scale settings, and neither provides labeled intent annotations.

The digital-twin and analytics literature provides industrial framing. Ilias et al. [11] present VesselAI, an HPC-driven analytics framework for fleet-scale AIS and sensor streams. Madusanka et al. [13] review the maritime digital-twin landscape and report that most deployed twins target asset-health monitoring rather than fleet-level behavioral simulation. Menges et al. [12] reach a similar conclusion for autonomous-ship digital twins, where high-fidelity dynamics and perception dominate the research agenda over scenario authoring.

2.1.2 Multi-Agent Maritime Simulators

Multi-agent simulation frameworks have been developed in adjacent domains. Petrović [70] explored virtual environments for training AI agents, and Petrović et al. [71] proposed AViLab as a gamified agent-based learning tool. Jakob et al. [72] applied multi-agent planning to anti-piracy missions, emphasizing adaptive coordination under dynamic threats. None of these platforms generates scriptable adversarial naval encounters or couples simulation output to an intent recognition pipeline.

A second cluster of recent platforms targets multi-vessel simulation more directly. Wang et al. [14] present USV-Sim with realistic hydrodynamics but no built-in intent labeling. Cao et al. [73] use multi-agent reinforcement learning with UAV critics and USV actors for cooperative target tracking. Grigoropoulos et al. [15] introduce GMSA, a digital-twin platform that scripts encounter sequences atop live AIS streams. Lee et al. [16] synthesize multi-vessel traffic for downstream learning, and Kim et al. [18] construct a hierarchical navigation database from VTS AIS streams. None of these systems combines scriptable adversarial scenario generation, COLREGs-compliant agent control, and labeled intent ground truth in a single pipeline.

Perception and security strands round out the simulation literature. Wang et al. [9] survey multimodal maritime perception and argue that fusing radar, EO/IR imagery, AIS, and SAR is necessary to overcome AIS-only limitations. Talpur et al. [1] survey AI applications in maritime security and identify the shortage of labeled threat and adversarial-behavior data as a recurring obstacle.

NavySim [6, 19] fills this gap with a Unity-based multi-vessel simulator. It provides scriptable adversarial and benign maritime scenario generation, International Regulations for Preventing Collisions at Sea (COLREGs)-compliant velocity obstacle navigation, Hidden Markov Model (HMM) and deep-learning-based intent recognition, and real-time threat heatmaps. NavySim forms the primary data generation and algorithm testing infrastructure of this dissertation (Chapter 3).

Gap addressed by this dissertation: *Existing simulators focus on single-vessel education or multi-agent coordination without adversarial intent labeling, and no platform ties scriptable encounter generation to an integrated intent recognition and threat visualization pipeline. NavySim addresses this by providing a unified environment that produces labeled multi-vessel encounter data for downstream machine learning.*

2.2 Intent and Behavior Recognition in Maritime Domains

2.2.1 CPA-Based and Rule-Based Threat Assessment

Closest Point of Approach (CPA) and time-to-CPA (tCPA) are the foundational metrics for maritime collision risk. Hilgert et al. [74] established a CPA-based risk standard aligned with COLREGs. Mou et al. [75] confirmed that decreasing CPA values correlate strongly with elevated collision probability. Rule-based systems built

on COLREGs encode domain knowledge explicitly but cannot generalize to adversarial encounters in which vessels deliberately violate standard rules, which is the regime this dissertation targets.

2.2.2 Statistical and Probabilistic Models

Carlson et al. [76] introduced a multinomial HMM that encodes rates of change of navigational parameters rather than absolute values, enabling early hostile behavior classification. Zhang et al. [77] surveyed thirty years of vessel behavior recognition and identified the absence of deep generative models applied to maritime scenarios as a critical open problem. Chen et al. [34] fuse vision and radar measurements through a Bayesian framework that combines a cooperative target, RANSAC-based radar-image correspondence, and YOLOv5 ship tracking into a posterior over intersection intents. None of these probabilistic methods outputs trajectory forecasts alongside intent labels.

2.2.3 Deep Learning for Intent Recognition

Wang et al. [78] proposed a two-stage framework that uses Random Forest and Bi-LSTM for coarse classification of head-on, overtaking, and crossing behaviors, then refines crossing scenarios with LightGBM and attention-enhanced LSTMs.

Meepaganithage et al. [79] compared HMMs, Recurrent Neural Networks (RNNs), Gated Recurrent Units (GRUs), and Long Short-Term Memory (LSTM) networks across six encounter categories, finding LSTMs superior for early hostile intent prediction. Meepaganithage et al. [67] follow up with bidirectional LSTM and Transformer backbones under a sliding-window protocol on noisy NavySim AIS streams, providing external classifier baselines used in Chapter 7.

Sayed et al. [80] demonstrated that LSTM-based classifiers paired with behavior-specific feature selection generalize well across vessel types and maneuvering strategies.

Attention-based LSTMs [81] provide partial interpretability by highlighting which timesteps drive classification, but the attention scores do not translate into spatial trajectory forecasts. Graph-based approaches represent vessels as nodes and encode spatial relationships as edges, capturing group-level tactical patterns invisible to per-vessel models, but they still produce only intent labels rather than predicted positions. Wu et al. [33] interleave cross-attention layers that fuse self-vessel features with neighborhood context from AIS and remote sensing in their LSTM-DCross model. Huang et al. [32] use a dual-channel spatial-temporal attention network with parallel kinematic and multi-attribute streams. Ma et al. [44] represent each scene as a graph and combine node-level kinematic encoders with learned edge features.

A second cluster moves intent recognition closer to the generative-trajectory regime. Wang et al. [45] pair a Transformer encoder with a Social Variational Autoencoder decoder that treats latent intent as a stochastic variable conditioned on past kinematics.

Jiang et al. [46] extend the idea to cooperating vessels through a shared latent prior. Chen et al. [31] build SEMINT, an LLM-empowered framework that infers voyage-level intent over a vessel spatio-temporal knowledge base and uses the inferred intent as conditioning context for long-term trajectory prediction; intent inference and trajectory generation are sequential rather than jointly optimized.

Specialized and cross-modal architectures complete the recent landscape. Kim et al. [82] rasterize AIS trajectories into 2D heatmaps and fine-tune a DenseNet-121 backbone for behavior classification. Han et al. [35, 36] exploit fishing-specific motion priors such as low-speed loitering and periodic transit-and-shoot patterns. Shin et al. [37] report that vessel trajectory prediction in inner-harbor approaches improves across Bi-LSTM, Bi-GRU, and Transformer backbones when berthing-side information (port or starboard) is supplied as a model input. On the adversarial-supervision side, Jia et al. [47] train a risk-aware GAN (RAGAN) whose discriminator scores generated trajectories against multi-ship encounter risk patterns, and Chen et al. [48] extend the social-GAN family to regional ship behavior and trajectory prediction; both motivate the MTITP critic head that scores trajectory–intent consistency directly (Chapter 7).

A parallel line of work approaches behavior monitoring as unsupervised anomaly detection rather than supervised classification. Xie et al. [57] train a Gaussian-mixture variational autoencoder on AIS streams and flag trajectories whose reconstruction error exceeds a Dynamic Time Warping threshold. Guo et al. [83] add a memory module and skip connections to a VAE reconstruction backbone, and Hou et al. [58]

replace the recurrent encoder with a multi-head self-attention transformer for cross-sea bridge waters. These models recover deviations from learned normal behavior but do not produce intent labels or named scenario classes; the threat-vs-routine distinction emerges only as an outlier flag.

2.2.4 Surveys and Benchmarks

Reviews of the field point to recurring bottlenecks. Li et al. [4] review machine-learning methods for ship trajectory prediction and find that data quality, not model capacity, drives much of the reported accuracy gap. Xue et al. [24] catalogue recent maritime ML works by feature pipeline and observe that dedicated importance studies for categorical or symbolic vessel attributes are uncommon. Yang et al. [2] reach a similar conclusion in their AIS review and note that systematic noise-stress evaluation is rarely reported. Xie et al. [3] narrow the lens to Transformer-based AIS monitoring, surveying trajectory prediction, behavior detection, and behavior prediction as separate research tracks.

Gap addressed by this dissertation: *Across statistical, recurrent, attention-based, graph-based, and adversarial intent classifiers, trajectory prediction and behavior classification are treated as separate tasks: SEMINT [31] and ShipTrack-TVAE [45] couple symbolic features or stochastic decoders to intent inference but stop short of jointly optimized intent and trajectory objectives, and adversarial trajectory models [47, 48] demonstrate that critic-based supervision sharpens maritime behavior structure*

but stop short of scoring trajectory–intent consistency. Recent surveys [2, 4, 24] confirm that data quality and noise robustness dominate accuracy gaps, and that no published model produces both intent labels and trajectory forecasts simultaneously. The Multi-Task Intent and Trajectory Prediction (MTITP) framework (Chapter 7) closes this gap by jointly optimizing intent and trajectory objectives through a shared encoder, demonstrating that intent supervision improves trajectory accuracy and vice versa.

2.3 Feature Selection and Feature Importance

2.3.1 Classical Feature Selection Methods

Feature selection improves generalization, efficiency, and interpretability in high-dimensional domains. Filter methods rank features by statistical criteria such as mutual information or chi-squared scores; wrapper methods evaluate subsets with a downstream classifier but scale poorly; embedded methods such as Least Absolute Shrinkage and Selection Operator (LASSO) regularization perform selection and fitting simultaneously.

Quinzan et al. [84] developed causal feature selection that outperforms correlation filters under distribution shift. Covert et al. [85] proposed dynamic selection that reduces acquisition cost by observing only the features necessary for a given input.

Wang et al. [86] addressed selection under non-stationary distributions via deep reinforcement learning. These methods operate on numeric features and do not account for the categorical structure common in maritime datasets.

The maritime literature illustrates this gap in practice. Huang et al. [87] present a cargo-ship classifier built on four hand-designed kinematic and geometric AIS features, devoting substantial attention to manual selection criteria such as correlation thresholds and predictive importance. Katz Nielsen [30] trains a strait-specific classifier and reports that the most informative features differ from those identified in open-sea or harbor studies. Both works motivate the importance-based selection methods developed in Chapter 4.

2.3.2 Tree-Based Feature Importance

Breiman [88] introduced Permutation Feature Importance (PFI) for Random Forests. Fisher et al. [89] generalized PFI to be model-agnostic. Disha et al. [90] applied Gini-impurity-weighted Random Forests for intrusion detection. Kamalov et al. [91] optimized feature sets using F_1 -score from one-dimensional decision trees, and Hakim et al. [92] applied Extra Trees for early stroke prediction. Standard PFI is biased toward high-cardinality and numeric features and systematically under-ranks categorical and minority-class features. In maritime intent recognition, where behaviorally decisive features such as side of approach and relative sector are categorical, this bias directly distorts importance rankings and degrades feature selection quality.

Smith et al. [25] show that out-of-bag-based variable importance measures (MDA, Holdout, AIR) become biased when categorical predictors are encoded with target-based schemes, and propose an unbiased encoding to recover correct rankings. This supports the bias arguments that motivate CPFI (Chapter 4).

2.3.3 Neural and Attention-Based Feature Attribution

SHAP [93] provides theoretically grounded, model-agnostic attribution based on cooperative game theory. Integrated Gradients and saliency maps offer gradient-based alternatives [94]. Margeloiu et al. [95] embedded feature selection within deep architectures for biomedical data. Hassanieh et al. [96] developed selective deep autoencoders for unsupervised feature selection, and Zhang et al. [97] applied sparse $\ell_{2,p}$ norm regularization for high-dimensional data. None of these methods corrects the categorical bias inherent to permutation-based importance on mixed-type maritime feature sets.

Applied maritime work increasingly relies on post-hoc attribution to justify model decisions. Wu et al. [29] apply SHAP to a ship-risk classifier and find that decisive features vary between coastal and open-sea contexts. Zhang et al. [20] survey explainable AI for autonomous surface ships and discuss emerging regulatory and class-society guidance on AI assurance. Cheng et al. [28] integrate attention modules into a multi-feature deep classifier for five fishing-vessel types, supporting the use of attention as a per-feature signal in TFIS (Chapter 4).

The reliability of these attribution methods is itself an active concern. Hwang et al. [26] report that SHAP attributions are sensitive to feature representation, with binning or encoding choices able to shift feature rankings on the same trained model. Huang and Marques-Silva [27] prove that SHAP scores can yield misleading information about feature importance and provide empirical evidence that the predicted failure modes appear in standard ML classifiers. These critiques motivate attribution methods designed to be less sensitive to such choices.

Gap addressed by this dissertation: *Standard permutation importance is biased against categorical and minority-class features [25], which makes it ill-suited to maritime datasets where behaviorally decisive features are often categorical and where region-conditional importance shifts are routinely observed [29, 30]. SHAP-based attribution is itself fragile under feature-representation choices and correlated features [26, 27], motivating attribution methods that are robust to such choices. This dissertation introduces CPFI (Categorical Permutation Feature Importance) and TFIS (Transformer Feature Importance Score) to correct these biases directly (Chapter 4).*

2.4 Trajectory Prediction

2.4.1 Classical Filters and Motion Models

The Kalman Filter (KF) [98] is the optimal estimator for linear Gaussian systems and maintains a state distribution via recursive predict–update cycles. The Extended Kalman Filter (EKF) [99] linearizes nonlinear dynamics via a first-order Taylor expansion, and the Unscented Kalman Filter (UKF) [100] propagates deterministic sigma points through the exact nonlinear function. The Coordinated Turn Rate and Velocity (CTRV) motion model augments the state with a turn rate ω , enabling the EKF and UKF to trace circular arcs and model sustained maneuvers more accurately than constant-velocity assumptions. Perera et al. [101] applied KF-based tracking to AIS data for vessel state estimation. The Interacting Multiple Model (IMM) filter [102] mixes estimates from a bank of sub-filters operating under different motion hypotheses, achieving better coverage of mode-switching behaviors but still operating without access to intent labels.

Recent work refines these classical filters along three lines: numerical stability, sensor-coordinate handling, and adaptive model banks. He et al. [39] sharpen short-horizon UKF accuracy through refined sigma-point selection and adaptive scaling. Cole and Schamberg [38] formulate the UKF directly in geodetic coordinates to avoid projection error on long-range tracks. Yang et al. [41] build the IMM on square-root cubature

primitives in IMM-SCKF to improve covariance conditioning during high-acceleration transients.

The IMM family has also moved toward online adaptation. Li et al. [42] adapt IMM transition probabilities online for underwater maneuvering targets and accommodate one-step randomly delayed measurements. Liu et al. [40] develop a Center Difference Kalman Filter (CDKF) for unmanned surface vessel navigation under GNSS occlusion, avoiding the analytical Jacobians required by the EKF. None of these refinements addresses the structural limitation: no parametric motion model can represent the deliberate course changes that define adversarial intent, so all intent-unaware filters share a non-trivial accuracy floor on scenarios such as herding and ramming.

A systematic evaluation of these filter families on adversarial naval encounters is reported in Chapter 6. This motivates the joint intent–trajectory formulation of MTITP.

2.4.2 Deep Learning and Transformer Architectures for Trajectory Prediction

Deep sequence models supplant the parametric motion assumption with learned representations. Social Force Models [103] and Social LSTM [104] introduced interaction-aware trajectory prediction for pedestrians, modeling repulsive and attractive forces between agents. Social GAN [105] replaced deterministic prediction with a generative

model that produces diverse, socially acceptable trajectory samples. These pedestrian-focused methods establish the multimodality principle but do not transfer to maritime domains, where kinematic constraints and intent semantics differ substantially: vessels cannot stop or steer arbitrarily, adversarial encounters involve deliberate rule violations, and COLREGs geometry structures the interaction space.

Trajectron++ [106] incorporates heterogeneous agent types and map context through a CVAE with a recurrent encoder. In the maritime domain, LSTMs [79] and encoder-decoder sequence models have been applied to vessel trajectory prediction. The dominant limitation across all these approaches is that predictions are purely kinematic and do not condition on intent labels.

Transformer architectures provide a complementary mechanism for sequential prediction. Vaswani et al. [107] replaced recurrent processing with multi-head self-attention, enabling parallel sequence processing and long-range dependency modeling without vanishing gradients. AgentFormer[108] replaced the recurrent backbone with a Transformer that attends jointly over time and agents, demonstrating that full-window context substantially improves long-horizon accuracy over Kalman-style single-step updates. Cross-attention has emerged as the standard mechanism for inter-agent interaction modeling: one sequence queries information from another, allowing agent-level dependencies to be captured without dedicated graph structures. Sinusoidal positional encoding [107] injects sequence order into token representations, and learnable CLS tokens, introduced by BERT [109], aggregate sequence-level information for

classification. These mechanisms together provide the building blocks for multi-agent sequence models, but existing maritime applications process only continuous numeric features and do not exploit symbolic interaction semantics such as COLREGs sector membership or relative bearing class.

The maritime literature has absorbed these mechanisms quickly. Nguyen and Fablet [50] introduced TrAISformer, which quantizes spatial coordinates into a discrete vocabulary and frames forecasting as next-token generation with a cross-entropy loss. Wang et al. [54] inject physics-derived motion priors as additive biases on attention logits in TPTrans to reduce long-horizon drift. Yuan et al. [51] apply GAT-style edge-conditional attention inside each Transformer layer in GATransformer.

Graph-structured forecasters form a parallel design line. Liu et al. [110] formulate joint multi-vessel forecasting in ST-MGT through a spatio-temporal multi-graph transformer that attends over time and over multiple scene graphs in a single backbone. Jiang et al. [52] fuse multiple complementary scene graphs in STMGF-Net through attention-weighted aggregation. Wang et al. [111] target complex and crowded sea areas with a spatio-temporal graph convolutional network. Yang et al. [53] encode the scene with multiple parallel graphs in MGAED, decoding futures with a teacher-forced sequence head.

A diffusion-based design line has emerged in parallel. Wenzhe et al. [61] cast vessel forecasting as iterative denoising over a spatio-temporal interaction graph in STGDPM. Li et al. [60] adapt classifier-free guidance to AIS forecasting in DiffuTraj, producing

stochastic forecasts over multiple plausible futures. Zhang et al. [112] augment the denoising score with a physics-based motion-model term to preserve dynamical realism. Across these architectures the common limitation persists: predictions remain purely kinematic, and intent labels, when present, are produced by separate downstream heads rather than co-optimized with the trajectory loss. Table 2.1 summarizes the recent maritime-trajectory landscape.

TABLE 2.1: Recent deep maritime trajectory predictors (2022–2025), grouped by architectural family. Each row reports the design lineage and the primary contribution as a comparator to MTITP.

Method	Family	Primary contribution
TrAISformer [50]	Transformer	Discrete AIS tokenization with cross-entropy loss; long-horizon multimodal forecasting
TPTrans [54]	Transformer	Physics-prior bias on self-attention logits for long-horizon stability
GATransformer [51]	Graph + Transformer	GAT-style edge-conditional attention inside each Transformer layer
STMGF-Net [52]	Multi-graph fusion	Attention-weighted fusion of multiple complementary scene graphs
DAA-SGCN [111]	Spatio-temporal GCN	Spatio-temporal graph convolution for interaction extraction in crowded sea areas
MGAED [53]	Graph encoder–decoder	Parallel multi-graph attention with teacher-forced decoding
STGDPM [61]	Diffusion graph	Iterative denoising over spatio-temporal interaction graph
DiffuTraj [60]	Diffusion	Classifier-free guidance for stochastic AIS forecasting
Physics-Diff [112]	Physics-guided diff.	Motion-model score term for kinematic plausibility
MTITP (this work)	Transformer + GAN, multi-task	Three-stream encoder fusing kinematic, geometric, and symbolic COLREGs features; jointly optimizes intent and trajectory; intent-aware critic

2.4.3 Multi-Task Learning for Trajectory and Intent

Multi-task learning (MTL) shares model capacity across related objectives, improving data efficiency and representation quality. Caruana [113] established that related auxiliary tasks regularize the shared representation and reduce overfitting. Kendall et al. [114] demonstrated principled task weighting via homoscedastic uncertainty, enabling stable joint training across heterogeneous loss scales. Wu et al. [65] proposed the Multi-Task Ship Graph Convolutional Network (MTSGCN), treating trajectory prediction and motion planning as mutually reinforcing objectives via a learned adjacency matrix and reporting gains over single-task baselines.

Cross-domain MTL work parallels the MTITP design. Munir et al. [62] introduced PTINet, which jointly predicts pedestrian crossing intent and trajectory from a shared scene encoder. Sharma et al. [63] condition trajectory decoding on a latent intent token produced by a separate intent classifier head in IntentFormer.

The maritime literature has also begun to adopt joint MTL. Jiang et al. [64] present VesNet, jointly learning route patterns and future trajectories within a single backbone. Shin et al. [66] pair a convolutional trajectory backbone with auxiliary heads on related kinematic targets.

Two recent papers analyze loss balancing in MTL. Kirchdorfer et al. [115] characterize regimes in which Kendall-style uncertainty weighting works well, and Kirchdorfer et al.

[116] note that it can collapse onto a single task when one task is disproportionately easier, a failure mode MTITP monitors for in Chapter 7.

We extend the MTL principle to joint intent classification and trajectory generation in MTITP (Chapter 7). A shared Transformer encoder is trained under three variants of a common backbone (MTITP-WGAN, MTITP-L2, and MarITGAN) so that the contribution of multi-task intent supervision can be separated from the contribution of the adversarial training objective. Detailed accuracy and ablation results are reported in Chapter 7.

Gap addressed by this dissertation: *Deep maritime trajectory models are intent-agnostic, classical filters cannot represent the multimodal future distributions induced by adversarial intent, and existing Transformer- and graph-based predictors (TrAISformer [50], TPTrans [54], GATransformer [51], STMGF-Net [52], MGAED [53] and the diffusion line [60, 61, 112]) process only continuous numeric features and do not condition on intent labels (Table 2.1). Chapter 6 quantifies the resulting accuracy ceiling for intent-unaware methods, and MTITP (Chapter 7) addresses it through joint intent–trajectory prediction with a generative decoder and a three-stream encoder that fuses continuous kinematic, continuous geometric, and symbolic interaction features.*

2.5 Generative Modeling and Domain-Conditioned Architectures

2.5.1 Foundations and Training Objectives

Goodfellow et al. [117] framed data generation as a minimax game between a generator G and a discriminator D . The generator G maps latent noise $z \sim p_z$ to synthetic samples while D learns to distinguish real from generated data, and at equilibrium G implicitly learns the data distribution without explicit density estimation. Training instability, including mode collapse, vanishing gradients, and non-convergence, is the central practical challenge. Arjovsky et al. [118] replaced the Jensen–Shannon divergence with the Wasserstein-1 distance and provided smoother gradients throughout training. Gulrajani et al. [119] stabilized training further via a gradient penalty term (WGAN-GP), enforcing the 1-Lipschitz constraint without weight clipping. Miyato et al. [120] proposed spectral normalization of discriminator weights as a complementary mechanism. The WGAN-GP objective and its training stability properties motivate the adversarial training variants used in MTITP (Chapter 7).

Recent applied work continues to use the WGAN-GP recipe. Liu et al. [55] combine multi-head attention with WGAN-GP for trajectory prediction of ships crossing under a bridge. Mtetwa et al. [56] propose adaptive gradient-penalty schemes that adjust the penalty coefficient from observed training dynamics.

2.5.2 Conditional and Class-Aware Variants

Mirza and Osindero [121] introduced conditional GANs (cGANs), conditioning both G and D on auxiliary class label vectors to enable targeted generation. Odena et al. [122] extended this with the Auxiliary Classifier GAN (AC-GAN), training the discriminator to simultaneously perform real/fake discrimination and class prediction. Radford et al. [123] introduced DCGANs, stabilizing image generation through convolutional architectures and batch normalization. These conditional and class-aware mechanisms inform the intent-conditioned generation strategy adopted in MTITP, where conditioning is supplied through learned intent representations from the shared encoder rather than discrete one-hot label vectors (Chapter 7).

MTITP supplies its conditioning signal through learned intent embeddings produced by the shared encoder rather than discrete one-hot label vectors or predefined latent clusters.

2.5.3 Trajectory Applications and Interpretability of Generative Models

Esteban et al. [124] proposed a recurrent cGAN for medical time-series synthesis. Yoon et al. [125] introduced TimeGAN, combining adversarial, autoencoding, and stepwise supervised losses to preserve temporal dynamics. Social GAN [105] demonstrated that GAN-generated trajectory samples outperform deterministic and CVAE-based

predictions on pedestrian benchmarks. Frid-Adar et al. [126] showed that GAN-synthesized medical images improve classifier performance under class imbalance, motivating the use of GANs to augment rare adversarial maritime encounter data. These works collectively validate the WGAN-GP trajectory generation approach but operate without intent conditioning.

For interpretability, Bau et al. [127] introduced GAN Dissection, visualizing the role of individual units in generating specific semantic concepts. SHAP-based attribution and integrated gradients [93] have been applied for post-hoc explanation of generator outputs. These tools remain relevant for analyzing MTITP’s generative behavior but do not address the underlying limitation: existing GAN-based trajectory models are either domain-agnostic or intent-unaware, and their critics score sample realism in isolation from the predicted scenario.

Beyond the GAN family, Jiang et al. [59] apply denoising diffusion to multi-agent motion prediction in autonomous driving, offering an alternative stochastic decoder to the WGAN-GP backbone used in MTITP.

As a discriminative counterpart, Anuja and Annrose [128] present an AE-CNN-LSTM architecture for maritime threat detection in which synthesized rare-class behaviors substitute for scarce real anomalies during training.

2.5.4 Embedding Expert Domain Knowledge in Learned Models

Expert knowledge has long entered machine learning through hand-crafted features. CPA-derived features such as tCPA, distance at CPA, and side of approach encode encounter geometry in terms meaningful to navigators. Carlson et al. [76] showed that parameterizing HMM observations as rates of change, an encoding derived from expert understanding of threat development, substantially improves early classification. The CPFI and TFIS methods (Chapter 4) extend this principle by producing behavior-specific importance scores that identify which expert-derived features carry the most discriminative weight. Feature engineering alone, however, embeds knowledge as fixed priors rather than allowing the model to learn feature relevance jointly with the prediction objective.

Hybrid models combining data-driven learning with explicit rules or ontologies have been explored in robotics, autonomous driving, and medical diagnosis. In autonomous navigation, COLREGs-compliant planning [74] constrains learned planners to legally admissible actions. In NLP, knowledge graphs fused with language models improve factual grounding. These works demonstrate that structural integration of domain knowledge outperforms post-hoc feature engineering, but no existing maritime trajectory model integrates COLREGs semantics directly into the encoder architecture.

Conditioning generative models on domain knowledge is also an active research

direction. TimeGAN [125] allows control over generated sequence properties through explicit conditioning. These approaches treat the conditioning signal as a fixed external input, and the domain knowledge does not participate in representation learning.

MTITP (Chapter 7) takes a structurally different approach by representing COLREGs-derived symbolic interaction features as learned embeddings inside the encoder, jointly trained with the trajectory and intent objectives rather than supplied as fixed priors. The encoder architecture and the specific symbolic features used are described in Chapter 7.

Gap addressed by this dissertation: *Existing GAN-based trajectory models are either domain-agnostic or intent-unaware; no critic explicitly scores trajectory–intent consistency, and no maritime model learns the relevance of COLREGs symbolic interaction states jointly with trajectory and intent objectives. MTITP (Chapter 7) addresses both limitations through a shared encoder that learns intent-conditioned trajectory representations, three training variants that isolate the contribution of the adversarial objective, and a maritime-specific critic that scores trajectory–intent consistency directly.*

2.6 Summary and Research Gaps

Table 2.2 maps each identified gap to the dissertation contribution that resolves it.

TABLE 2.2: Summary of literature gaps and corresponding dissertation contributions.

Gap in Literature	Dissertation Contribution	Chapter
AIS data lacks adversarial encounter diversity; no unified simulation for multi-vessel labeled data generation	NavySim: scriptable multi-vessel simulator with HMM intent recognition and threat heatmaps	Chapter 3
Standard PFI is biased against categorical and minority-class features; no maritime-specific feature attribution method	CPFI (Categorical Permutation Feature Importance) and TFIS (Transformer Feature Importance Score)	Chapter 4
Classical Kalman filters fail on adversarial maneuver scenarios; no quantified ceiling for intent-unaware prediction	Systematic evaluation of CV, UKF, Asymmetric UKF, and EKF/CTRV on 8 encounter classes	Chapter 6
Deep maritime trajectory models are intent-agnostic; trajectory prediction and intent recognition treated as separate tasks; GAN critics are domain-agnostic	MTITP: three-stream Transformer encoder shared across three training variants (MTITP-WGAN baseline, MTITP-L2, MarITGAN with maritime-specific IntentAwareCritic), jointly trained on intent and trajectory objectives	Chapter 7
Maritime trajectory models treat COLREGs rules, CPA geometry, and relative bearing sectors as passive inputs or ignore them entirely	MTITP Stream C embeds COLREGs symbolic interaction features directly into the encoder as learned representations jointly trained with intent and trajectory objectives	Chapter 7

The preceding survey identifies five limitations that the present work addresses jointly. The literature lacks adversarial encounter data, and NavySim generates it. Standard importance methods misrank categorical maritime features, and CPFI and TFIS correct this bias. Intent-unaware classical filters hit a structural accuracy ceiling on adversarial scenarios, and the KF baseline chapter quantifies it. Deep trajectory models produce no intent output and existing GAN critics are domain-agnostic; MTITP closes both gaps with joint intent–trajectory optimization studied under three training variants of a shared backbone. Symbolic COLREGs knowledge has historically been treated as a passive prior, and MTITP integrates it as a learned structural component of the encoder.

Chapter 3

NavySim: Enhanced Multi-Vessel Simulation and Analysis Engine for Advanced Naval Research

3.1 Introduction

Maritime simulation games and serious simulators are vital for naval education, research, and operational training. These platforms provide a controlled environment to replicate real-world situations, helping in the development of critical skills such as collision avoidance, maneuvering strategies, and intent recognition of surrounding vessels. This chapter presents NavySim 2.0, an enhanced Unity-based naval simulator designed to create complex maritime scenarios involving multiple autonomous

agents. Key capabilities include complex scenario generation, COLREGs-compliant collision avoidance using velocity obstacles, vulnerability and threat heatmap visualizations, a generalized intent recognition pipeline that supports both Hidden Markov Models and deep-learning classifiers, and real-time mobile resource allocation for defensive responses. The system architecture, scenario generation tools, and simulation methodologies are described, with illustrative examples highlighting the simulator’s capabilities.

The use of game engines for simulating real-world scenarios has become a prominent approach for research and training in domains where real-world data is scarce, expensive, or hazardous to collect, such as medicine, maritime navigation, and military security. In naval operations, understanding the surrounding environment and predicting the intentions of nearby vessels are critical for both navigation safety and security. However, the training of machine learning models in this domain is often constrained by limited datasets.

NavySim 2.0 addresses these challenges by offering a flexible, extensible simulation environment capable of generating dynamic maritime scenarios, modeling agent behaviors, and providing real-time visual and analytical tools for navigation, threat assessment, and defense strategy evaluation. Building upon earlier versions of the simulator [6] and a companion user study on navigation controls and subconscious learning in the same environment [129], NavySim 2.0 introduces significant improvements across system architecture, intent recognition, heatmap-based visualization,

and defensive resource allocation.

The development of NavySim [6, 19] represents a collaborative effort between the Robotics Research Lab, the Computer Vision Lab, and the Evolutionary Computing Systems Lab (ECSL) at the University of Nevada, Reno. For completeness, this chapter presents a detailed overview of this simulation environment, which is tailored to the complexities of the naval domain. NavySim supports both scripted and manual control of vessel agents, enabling the exploration of a wide range of navigational and security scenarios. It incorporates COLREGs-compliant navigation protocols, developed by Korben DiArchangel [130], to ensure realistic and lawful maritime behavior. Visual analytic capabilities include vulnerability heatmaps and Closest Point of Approach (CPA)-based predictive action heatmaps, which provide intuitive insights into navigational risk and threat anticipation. An integrated intent recognition pipeline interprets surrounding naval agents' behavioral intentions in real time. The pipeline is model-agnostic: it exposes a common interface for swapping in different recognizers, and has been instantiated both with the Hidden Markov Model (HMM) backbone [6] developed in collaboration with Parvaneh Aliniya, Mayamin Hamid Raha, and Tyler J. Becker, and with the deep-learning classifiers developed elsewhere in this dissertation (Chapters 4, 5, and 7). Additionally, a real-time autonomous defensive response framework, implemented by Casey Brenner [131], allows vessel agents to respond adaptively to perceived threats, enhancing the security dimension of the simulation.

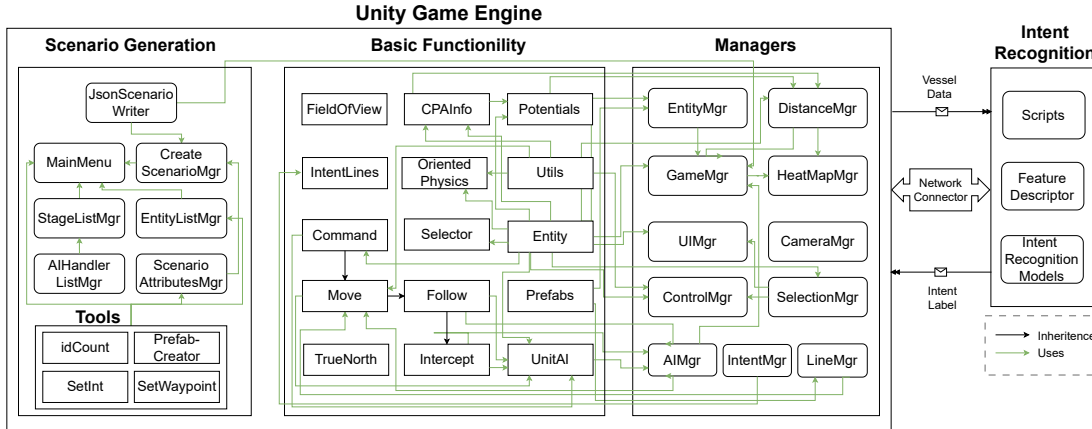


FIGURE 3.1: The system architecture diagram of the naval simulator (NavySim)

3.2 Naval Simulator: General Architecture

The general architecture of the simulator, shown in Figure 3.1, contains four different components. The *basic functionality blocks* implement supporting class objects for defining entities and their physics models, navigation behaviors, computation of CPA, and other utilities (Table 3.1). The *managers* are blocks of code that are continuously running to enable desired functionality related to the entities' behavior, camera positioning, autonomous navigation and avoidance, information display, manual vessel control, heatmap computation and visualization, and running of the navigation scenarios (Table 3.2). The *scenario generation module* enables the development of complex multi-vessel scenarios (Table 3.3, Section 3.3). The *intent recognition module* provides functionality for predicting vessels' intentions through a pluggable backend, with a default HMM implementation that uses navigation-related features as observations and an extension path to deep-learning classifiers via the same data-exchange interface (Section 3.6).

We use Unity version 2021.38.8f1 long-term support (LTS) as the game engine. We build the simulator based on the initial version available on GitHub [132].

3.3 Naval Scenario Generation

A scenario is defined to include a list of vessels and a list of stages in which each vessel is assigned a navigation behavior. Our simulator provides the individual navigation behaviors for *Follow*, *Move*, and *Intercept*, as well as group behaviors, in which multiple vessels are assigned a similar navigation goal. The AI manager (*AIMgr*) provides functionality for assigning a behavior to a vessel's *UnitAI*, and the *UnitAI* itself updates the vessel's state according to the assigned behavior at each timestep. These behaviors can be used to replicate various real-life naval scenarios with multiple autonomous vessels.

Our simulator has a main menu UI for scenario generation (Figure 3.2). The menu guides users through three main steps: first, choosing the desired number of ships and assigning their associated attributes such as type (*e.g.*, Pilot Vessel), alignment (blue or red team), and initialization coordinates; second, choosing the stage progression conditions; and third, choosing the desired number of stages and assigning desired navigation behaviors per stage. The menu is built out of Unity-provided UI GameOb-jects, such as text boxes, drop down menus, and toggles, and their event systems

Basic Block	Functionality
Entity	Defines a vessel entity with position, speed, desired speed, maximum speed, heading, desired heading, acceleration, turn rate, mass, force type (0: blue, 1:red, 2:neutral), and ship name. Attaches is a camera rig, selection circle, and heat map for the entity. Entity types are DDG51, Container, Minesweeper, Oil Service Vessel, Orient Explorer, Pilot Vessel, Smit Houston, Tanker, and Tug Boat.
Oriented Physics	Updates entity position in the world, updates its velocity according to desired velocity and acceleration, clamp speed according to minimum and maximum speed of the entity, change heading according to desired heading and turn rate.
Utils	Contains tools to manage angle conversion, heading calculation, clamping min-max of any property of entities, simple mathematical conversions, or checks.
IntentLines	Displays color-coded intent lines between pairs of vessels.
Selector	Links a selected entity in the simulated world to its heatmap, and switches between particular heatmaps with key press.
Command	Connects entity to a command template, contains DHDS class with the desired speed and desired heading property.
Move	Extends Command, moves an entity from point A to B using AIMgr and Utils, draws a line from the current position to the desired move position with LineMgr.
Follow	Extends Move, contains tools to follow the target by an offset in addition to the inherited tools from Move.
Intercept	Extends Follow to follow the target entity by an offset of zero.
UnitAI	Contains tools to add, remove, or set commands, destroy single or all commands, and decorate between prior and current commands.
TrueNorth	Contains tools to calculate the True North in the simulated world.
FieldOfView	Displays the environment from the perspective of a particular vessel during scenario play.
Prefabs	Contains the template for the entities with their 3D model, properties, and scripts, similar templates for the IntentLines.
CPAInfo	Handles CPA-related calculation (CPA time, CPA distance, target relative bearing, target absolute bearing) between a set of two entities.
Potentials	Handles obstacle avoidance between two entities using potential fields.

TABLE 3.1: Overview of basic functionality blocks.

Managers	Functionality
CameraMgr	Controls camera movement, switching between different views.
EntityMgr	Generates instances of entities with the Entity properties and Prefabs.
LineMgr	Draws Move, Follow, and Intercept lines between selected entities.
AIMgr	Provides functionality for individual and group navigation behaviors.
UIMgr	Displays scenario or selected entity information (Entity Name, Speed, Desired Speed, Heading, Desired Heading) in the control panel.
SelectionMgr	Selects a particular entity in the simulated world with a Mouse or Keyboard, contains functions to select the entity closest to the mouse click, update, and delete selection.
ControlMgr	Manages manual movement of selected entities with the mouse or keyboard.
DistanceMgr	Contains CPAInfo and Potential classes to calculate and control CPA-related information between a set of entities.
GameMgr	Instantiates entities in the simulated world, creates scenarios, uses AIMgr to create specific vessel movements, logs vessel data, controls data flow between the simulator and the intent recognition module, sends data to DistanceMgr and the intent recognition system, and calculates CPA-based action heatmap's preliminary calculation such as the relative location of the potential threat in CPA time along with DistanceMgr.
HeatMapMgr	Controls necessary data flow between GameMgr, DistanceMgr for Heatmaps, and Action Heatmap computation and visualization.
IntentMgr	Controls data flow between GameMgr, IntentLines, and Python scripts for computation and visualization of Intent and Group Intents.
NavMgr	Produces COLREGs compliant, waypoint navigation trajectories

TABLE 3.2: Overview of the functionality of different managers

assign the user-given scenario attributes to a *SceneInitializer* object. A scenario is saved by serializing the *SceneInitializer* object into a JSON file.

The game manager (*GameMgr*) is responsible for deserializing a JSON scenario file into a new *SceneInitializer* object, which provides all the data for instantiating entities and running the stages of the scenario. The *GameMgr* uses the entity manager (*EntityMgr*) to instantiate new entities at their given initialization coordinates. During

Scenario Generation	Functionality
JsonScenarioWriter	Contains the serializable scenario classes.
MainMenu	Plays the scene and sets the scenario object attributes.
CreateScenarioMgr	Serializes the scenario object, saves file.
StageListMgr	Gets the user-given stage attributes.
EntityListMgr	Gets the user-given entity attributes.
AIHandlerListMgr	Handles instantiation and removal of various AI Handler lists.
ScenarioAttributesMgr	Handles instantiation, removal, and toggling of scenario attributes.
Tools	Utilities used by the other scripts for object tracking, prefab instantiation, and attribute assignment.

TABLE 3.3: Overview of the scenario generation module.

GameMgr's update loop, the *SceneInitializer* object provides the conditions for stage transitions and assigns given navigation behaviors to applicable vessels within the active stage. This multi-stage system enables the creation of complex scenarios, with each vessel being set to complex sequences of navigation goals, following the control of autonomous behaviors. Another way to control vessels is by attaching navigation behaviors to mouse clicks: selecting an entity brings up options for available navigation behaviors that can be assigned. Lastly, during a scenario, the *GameMgr* records the complete information about each vessel (position, speed, heading, type (red or blue), CPA distance, and CPA time) for further post-processing.

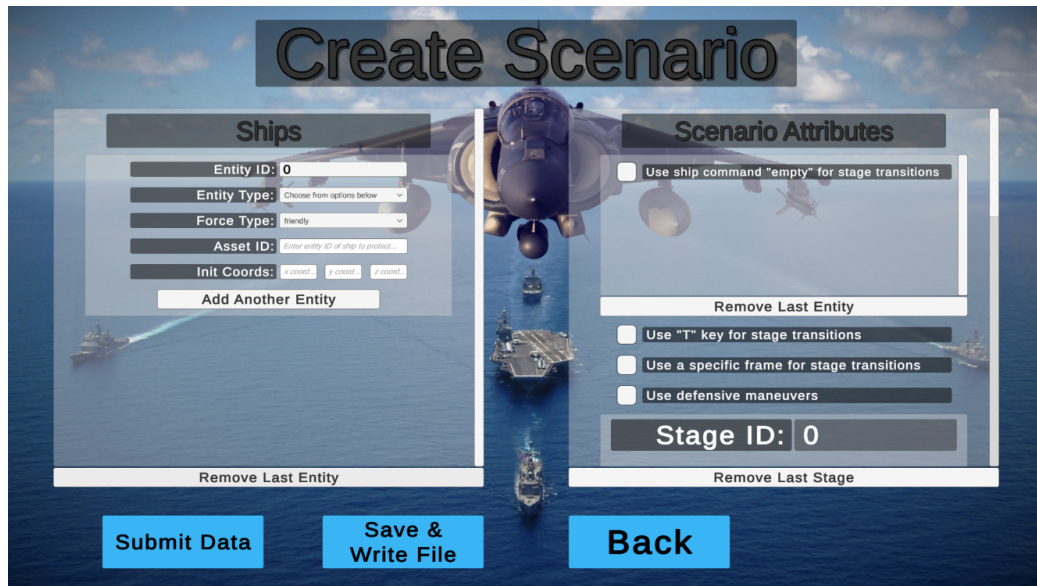


FIGURE 3.2: Main menu layout for scenario generation.

3.4 Velocity Obstacles for COLREGS Compliant Ship Driving

Korben [130] developed a Velocity Obstacle COLREGs Collision Avoidance Algorithm (VOCCAA) to drive simulated vessels following the nautical rules of the road in our naval simulator. Velocity obstacles (VOs) have been developed to assist agents in determining whether or not they will collide with another moving object [133]. This algorithm does not require information held by other agents, and requires relatively few computations. We use velocity obstacles along with the CPA in order to detect whether or not any two vessels will collide with each other and to provide vessels in a risk-of-collision with a new trajectory that is compliant with COLREGs [134] and moves the vessel toward its assigned waypoint. The trajectory found by VOCCAA moves the ownship in a way that gets within a specified distance of the target vessel.

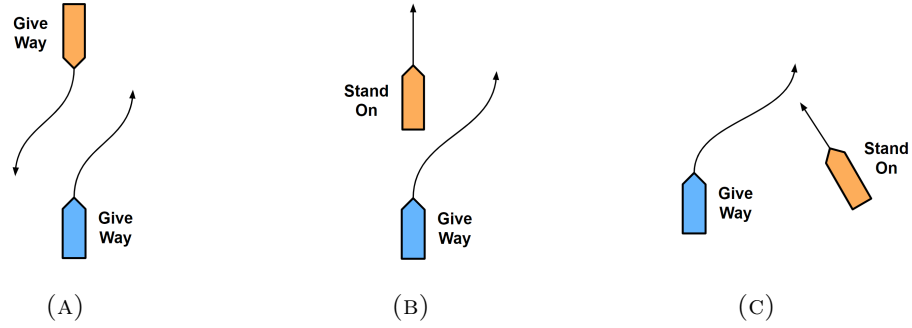


FIGURE 3.3: The own ship and target ship positions in three different situations. (a) The head-on situation. (b) The overtaking situation. (c) The crossing situation.

The following COLREGs is considered in this work: 1) rules 13, 14, and 15, which specify what to do when there is a risk of collision and the vessels are in an overtaking, head-on, crossing situation (Figure 3.3) and 2) rules 16 and 17, which specify the give-way and stand on vessel roles and their respective actions in the above situations [134]. In a head-on situation, both vessels are heading towards each other and are both expected to give way. In an overtaking situation, the vessel behind the other vessel is expected to turn to avoid a collision while attempting to pass the vessel. In a crossing situation, the give way vessel must turn towards the stern (behind) of the stand on vessel. In all three of these situations, the give way vessel is expected to turn to its starboard (to its right). Furthermore, the give way vessel is required to maneuver to reduce risk of collision, while the stand on vessel must maintain its current course and speed, unless it is required to give way in another risk-of-collision situation. This section of the work was completed in collaboration with other lab members, and details can be found in this journal article [19].

3.5 Vulnerability and Threat Heatmaps

As a part of a comprehensive understanding of the naval agent’s inherent capabilities and vulnerabilities, as well as the potential threats from the surrounding environment, we propose several heatmap-based visualization techniques supported by novel mathematical methods and Unity Shader [6, 135]. Heatmaps are a data visualization technique that employs color variation (hue or intensity) to communicate information rapidly. We use a grid heatmap representing weak and strong zones around a ship with colors in a two-dimensional matrix: red encodes threats and green represents safe zones.

3.5.1 Vulnerability Heatmap

A *vulnerability heatmap* (*VHM*) is a visual solution for vessel operators to identify critical vulnerable zones and levels. To represent a vessel’s capabilities, we propose models representing the coverage of its dynamic assets. For a naval agent’s resources, denoted as $R_{A_i} = Sensors_{A_i} \cup Defenses_{A_i}$, we define resource efficiency at a location X with the function $f_r(P_r, X)$, which considers the distance (d) from the center and the angle (α) with its orientation vector ($\vec{v}_r$). While uniform efficiency suggests equal coverage, real resources vary in effectiveness. We integrate these models representing each asset to visualize coverage in a *vulnerability heatmap*, with efficiency functions that may depend on distance ($f_r(d)$), heading ($f_r(\alpha)$), or both ($f_r(d, \alpha)$).

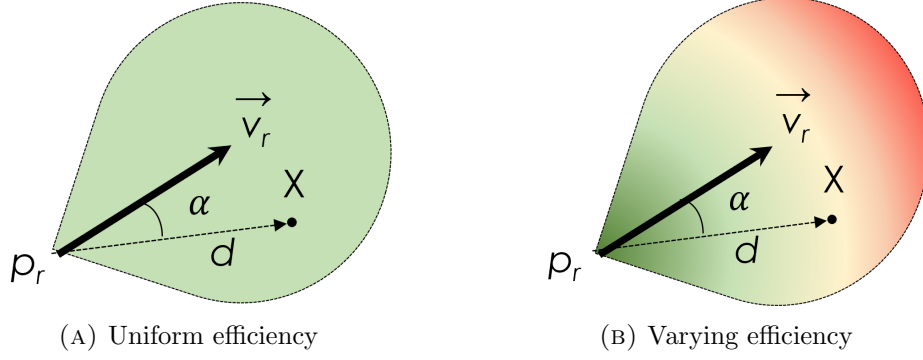


FIGURE 3.4: Resource efficiency at certain location X.

Distance Influenced Efficiency Function A resource with a 360° field of view can be modeled with a distance-based efficiency function, $f_r(d)$. We define an *exponential decay efficiency function* that models assets with a larger coverage area, but its accuracy drops exponentially with distance:

$$f_r(d) = \frac{1}{e^{c(d-d_{min})}}, d_{min} \leq d \leq d_{max} \quad (3.1)$$

where, c is a drop constant, d is the distance between the asset and location X , d_{min} , and d_{max} are the minimum and maximum range of the asset, respectively.

According to the resource's behavior, the higher the value of c , the faster the drop in coverage efficiency. For the entire range of assets available to agents in our simulation engine, we developed additional efficiency functions that are influenced by the same parameters (d , d_{min} , d_{max} , and c), including *piece-wise decay*, *radial exponential decay*, and *inverse normalized* functions. These efficiency functions are detailed in [135].

Heading Influenced Efficiency Function As discussed before, the pose of a resource affects its coverage around a ship. For example, ship guns mounted on turrets cover a conical region but can only aim in one direction, while a camera has a 360-degree field of view but loses efficiency over distance. Different resources require different efficiency functions. In the DDG51 ship, guns provide fixed coverage, whereas visual sensors encompass a full circle. We propose an efficiency function $f_r(\alpha)$ based on the asset's heading ($\vec{v}_r$) and the angle (α) with location X (see Figure 3.4).

$$f_r(\alpha) = 1 - \frac{|\alpha|}{\theta_{max}} \quad (3.2)$$

where θ_{max} is the maximum angular coverage by the resource.

A resource with conical or angular coverage combines both types of functions. To integrate the effect of both distance and angular distance on the asset efficiency, we propose to compute a weighted average of both effects as follows:

$$f_r(P_r, X) = w_d \cdot f_r(d) + w_\alpha \cdot f(\alpha) \quad (3.3)$$

where, $w_d + w_\alpha = 1$

The efficiency function takes values between $[0, 1]$, with 0 representing no coverage and 1 representing maximum coverage. These values are converted to the RGB range with a mathematical linear interpolation function where 0 represents red and 1 represents green. Finally, we get a red and green gradient-based *VHM* that illustrates the

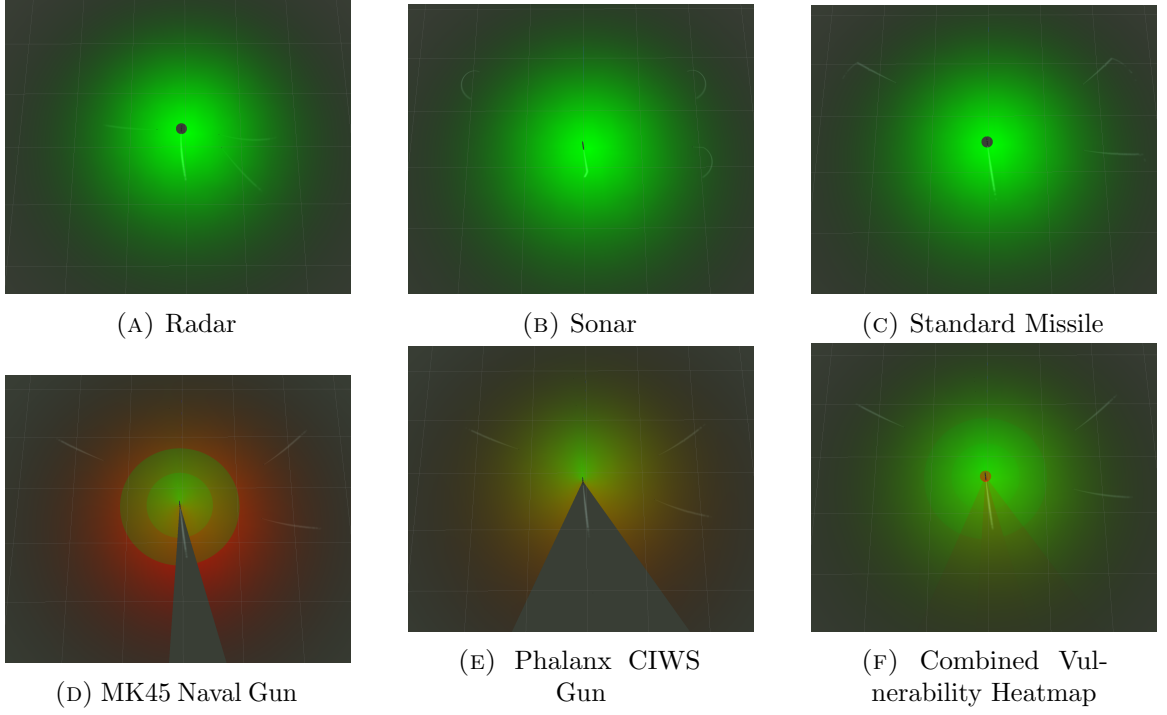


FIGURE 3.5: DDG51's Resources and *Vulnerability Heatmap* adapted in the Unity simulator with the shader; the snapshot covering only a 4.5 km distance around the vessel.

Asset	Min/Max Distance (km)	FOV	Efficiency Function	C
Radar	0.1 / 104	360	Radial Exponential	0.2
Sonar	0.0 / 23.1	360	Radial Exponential	0.2
MK45 Gun	0.0 / 24.1	85	Piece-wise Decay +Heading Influenced	1.0
Phalanx CIWS Gun	0.0 / 5.5	75	Exponential Decay +Heading Influenced	1.0
Standard Missile	0.1 / 104	360	Inverse Normalized	1.0

TABLE 3.4: Parameters of resources and their efficiency function to build the *vulnerability heatmap* of DDG51

coverage of assets in a comprehensive way. Figure 3.5 shows the *VHM* for the DDG51 ship with the following resources: radar, sonar, standard missile, MK45 naval gun, and Phalanx CIWS gun. Some efficiency functions referenced in this example are detailed in [135]. Table 3.4 presents the parameters for these resources.

3.5.2 Predictive Action Heatmap

Understanding the behavior of nearby vessels is essential for safe navigation. We propose a *predictive action heatmap* that encodes the relative approach of other vessels (in red) to a specific ship (in blue) by identifying the CPA, TCPA, and relative bearing. A CPA within a threshold would trigger VOCCAA to avoid potential collisions.

The CPA-based *PAHM* encodes potential future threats based on current CPA information at time t_c . It displays red circles at predicted threat hotspots, indicating where the other vessel will be relative to the blue ship at TCPA, with hotspot intensity reflecting the perceived threat level.

PAHM considers the impact of both the CPA distance and TCPA. The safe maximum range for CPA and TCPA values is considered based on the distance and time allowing the blue ship to take defensive or evasive action to tackle any threat. According to priorities, the system designer can emphasize the distance or time of CPA with respective weights. We calculate the CPA_{effect} for each red ship with Equation (3.4). Then, we define a circular area instead of a single point to show the CPA impact of a red vessel w.r.t. the blue ship in its relative location in CPA time. The center of this circle has an intensity value calculated from CPA_{effect} of Equation (3.4), while the value drops to zero for the furthest side of the circle.

$$CPA_{effect} = \frac{w_T \cdot \left(1 - \frac{TCPA}{TCPA_{max}}\right) + w_C \cdot \left(1 - \frac{CPA}{CPA_{max}}\right)}{w_T + w_C} \quad (3.4)$$

where, w_T : weight of CPA time, w_C : weight of CPA distance,

The function $g_a(CPA_a, X)$ defined in the Equation (3.5), computes the contribution of agent a 's future behavior, discounted by time and distance (*i.e.*, closer CPAs in time/distance have a higher threat level than farther CPAs in time/distance). This function takes a radius as an input to control the size of the circle. We can select a fixed radius that is a desired distance for safe navigation (e.g., 100m.), or we can change it according to the value of CPA_{effect} and map it from the range of $[0, 1]$ to a range of $[0, 200]$ meters. We adopted the second approach.

$$g_a(CPA_a, X) = CPA_{effect} - \frac{d}{d_{radius}} \quad (3.5)$$

Finally, the $PAHM$, $A(X)$, combines threat hotspots from all red agents a , with a weight to control its impact at a location X w.r.t. the blue vessel using the function in Equation (3.6):

$$A(X) = \sum_{a=1}^{|A_R|} \left(w_a \cdot g_a(CPA_a, X) \right) \quad (3.6)$$

The $PAHM$'s underlying 2D grid and coordinate system are the same as the core of the *vulnerability heatmap*. Thus, we have to measure the CPA impact of the red ships on the blue ship and update the $PAHM$ grid. However, the CPA effect recorded in the $PAHM$ represents different information than that from the *vulnerability heatmap*: the higher the value in the grid coordinate, the higher the risk the blue ship faces in

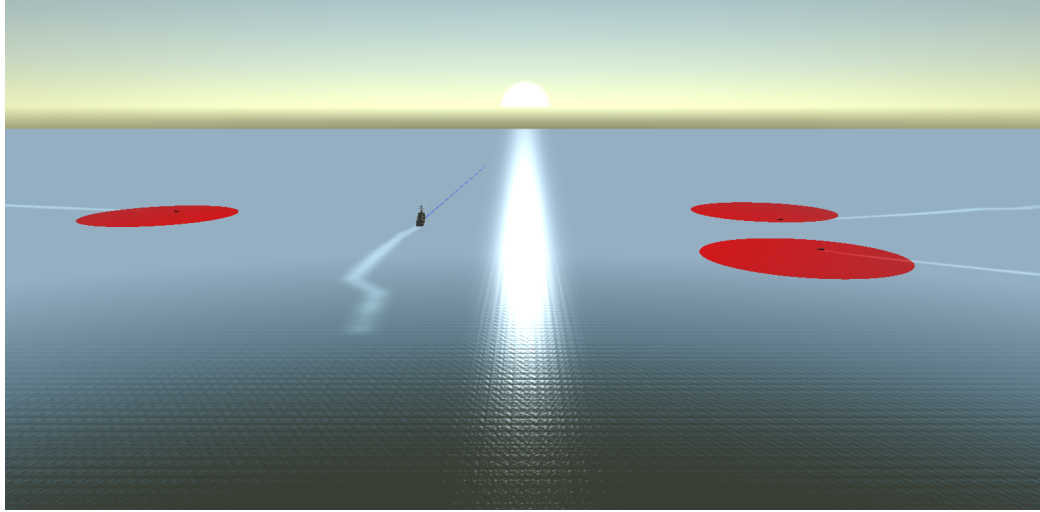


FIGURE 3.6: *Predictive Action Heatmap* over time as future possible hotspots, adapted in Unity simulator.

that location, and thus the higher intensity of red is displayed. Outside the threat regions, the heatmap does not display any color to ease the visualization.

Figure 3.6 illustrates an example of the CPA effects (with different TCPAs) from three potential red ships. The hotspots are located at the future relative positions w.r.t. the DDG51 ship, where they would be at their closest point of approach (after the corresponding TCPAs have elapsed). This shows, in the present time, the future possible impacts providing an essential tool for threat assessment.

3.5.3 Combined Threatmap

The combined threat heatmap, *threatmap* in short, can be defined as a union of the *vulnerability heatmap* and the *predictive action heatmap*. However, a challenge in combining these heatmaps comes from the difference in representation. A higher value in the vulnerability heatmap means more coverage from resources, resulting in a green

and red color for high and low coverage respectively. On the other hand, a higher value in *PAHM* means more potential threat from a red ship, encoded with a red color. Our solution is to use the *vulnerability heatmap* as the basis of the *threatmap* and then subtract the risk value of *PAHM* from it.

For example, a location with a combined coverage of 0.8 (80% covered) and a hotspot with a risk of 0.7 will have a value of 0.1 in the resulting *threatmap*, illustrated with a higher intensity of red color, meaning a lower coverage. Thus, the *threatmap* employs the same color palette and definition as the *vulnerability heatmap*, allowing the naval officers to observe potential threats in an intuitive visualization. Figure 3.7 shows an example of a resulting *threatmap* from a three red vessel and two blue vessel scenario. The three red vessels are in a direct collision path with the selected DDG51, and the *threatmap* shows the maximum threat around the blue ship. The heatmaps are generated with Shader and integrated into the NavySim by the process explained in [6].

3.6 Intent Recognition

Our naval simulator is equipped with an early intent recognition pipeline that detects red vessels' intentions and categorizes them as one of the seven navigation behaviors described in Table 3.5. Furthermore, each behavior is classified as either hostile or non-hostile.

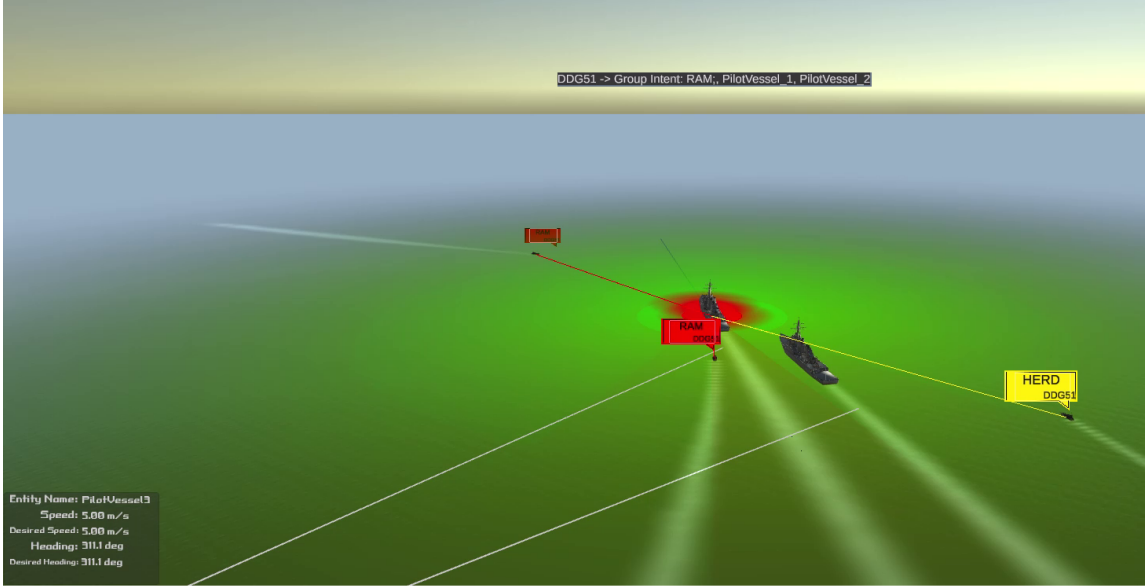


FIGURE 3.7: *Threatmap* illustrating potential high-danger hotspots around the selected DDG51 ship (blue team) caused by the three pilot vessels (red team). Two pilot vessels show RAM (red flags), and one shows HERD (yellow flag) intent toward the DDG51 ship. Group behavior, including ship names and intents, is flagged in the top panel.

The pipeline is model-agnostic by design. It is implemented in Python and integrated with the Unity simulation through a socket-based protocol for real-time data exchange: *GameMgr* streams per-timestep vessel observations to the Python backend, the backend writes intent predictions back, and the Unity-side *IntentLine* module renders them as color-coded lines between vessels. Any classifier that consumes a fixed-length observation window and returns a per-class likelihood can be plugged into this interface, so the simulator decouples the recognizer choice from the rest of NavySim. The deep-learning models developed elsewhere in this dissertation, namely the Hakim et al. [136] classifier (Chapter 4), the CPFI/TFIS-driven LSTM and FCDL networks of Chapter 5 [80], and the MTITP encoder of Chapter 7, all consume the same observation interface and serve as drop-in alternatives to the HMM baseline.

Behavior	Definition
Block	Hostile behavior in which a vessel tries to prevent another vessel from accessing a specific point or area.
Ram	Hostile behavior in which a vessel attempts to execute an intercept with a near zero Closest Point of Approach (CPA) with respect to another vessel.
Herd	Hostile behavior in which a vessel attempts to influence the direction of another vessel into a set location.
Cross	Benign behavior in which a vessel's path crosses the path of another from some angle. Due to the fact that the paths cross, there is a risk of collision.
Headon	Benign behavior in which a vessel approaches another vessel's path from the bow (Figure 4.1), but is obeying COLREGs. The ships have near reciprocal courses and risk collision.
Overtake	Benign behavior in which a first vessel approaches another one from behind (Figure 4.1) from the end of the second vessel, in order to pass.
Benign	Other behaviors that do not exhibit any of the hostile behaviors defined in previous rows.

TABLE 3.5: Definition of hostile and non-hostile behaviors.

The default backbone is a bank of Hidden Markov Models (HMMs). HMMs incorporate the dependencies between observed variables through hidden variables over time, and we train one HMM per behavior class using scenarios representative of that class, following the strategy of Carlson et al. [76] and using the *hmmlearn* library. This HMM implementation was developed by Parvaneh Aliniya, Tyler J. Becker, and Mayamin Hamid Raha [6].

For both training of the HMMs and for classification, at each time step, the raw vessel data (velocity, location, and heading) is converted into a set of features that capture patterns of change in the relative movements between the two vessels, as well as navigation-relevant position features (e.g., if vessels are within each other's

Feature	Meaning	Symbol: Meaning	Symbol: Meaning	Symbol: Meaning
Delta Distance	Represents the change in location for a vessel across two time steps.	10: Moving closer	11: Moving farther	12: Stationary
Delta Speed	Represents the change in speed for a vessel across two time steps.	20: Accelerating	21: Decelerating	22: Constant
Delta Relative Bearing	Represents the change in relative bearing (with values in intervals $[0, 180]$, $[0, -180]$) between two vessels across two time steps.	30: Increasing	31: Decreasing	32: Constant
Delta Absolute Difference Heading	Represents the change in absolute heading difference between two vessels across two time steps.	40: Increasing	41: Decreasing	42: Constant
CPA Time	Represents CPA time information.	50: Increasing	51: Decreasing	52: Constant
CPA Distance Threshold	Indicates if the CPA distance is below/above the danger threshold.	60: above	61: Below	-
CPA Delta Distance	Represents change in CPA distance across two time steps.	70: Increasing	71: Decreasing	72: Constant
Side	Represents the location of vessel B relative to the forward direction of vessel A at current time step.	81: Left of A	82: Right of A	-
RegionAB	Represents the location of vessel A with respect to B, from the perspective of headon/cross/overtake situations.	90: Headon region ($\pm 22.5^\circ$)	91: Cross region ($\pm [22.5, 157.5]^\circ$)	92: Overtake region ($\pm [157.5, 180]^\circ$)
RegionBA	Represents the location of vessel B with respect to A, from the perspective of headon/cross/overtake situations.	100: Headon region ($\pm 22.5^\circ$)	101: Cross region ($\pm [22.5, 157.5]^\circ$)	102: Overtake region ($\pm [157.5, 180]^\circ$)
Beam	Represents the location of vessel B relative to the beam of vessel A at current time step.	110: Front of A	111: Behind A	-

TABLE 3.6: Definition features computed from raw observations, with the meaning of the values that could be assigned to them.

cross/overtake/headon regions). The complete set of features is presented in Table 3.6. From the actual feature values, we generate an index corresponding to those features from the list of all possible combinations of the feature values. The index thus represents the observation from a red vessel in each time step.

In the prediction stage, to capture history of changes inherent in each behavior, we use a rolling time window of 20 frames (thus 20 feature vectors) as input to the HMMs. The outputs of the HMMs indicate how likely the sequence and the transitions between the frames are in a given HMM. The model with the highest probability of the input is considered as the predicted intent. This process is performed continuously and on-line, computing the intent of all red vessels with respect to all blue vessels of interest (could be all or just selected few).

To integrate the intent recognition system with the NavySim, we use shared files and a messaging approach between the Python module and Unity, using Socket and Net.Sockets. For each time step, *GameMgr* writes the information about all the

vessels to a "log data" file and sends a message to the intent module to read from the file. The intent module computes and writes the results of intent recognition to a "prediction" file and sends a signal to Unity to read the result.

Coordinated hostile groups can pose a greater threat and flagging these events is vital for effective defensive actions. When two or more vessels exhibit the same non-benign behavior toward a blue vessel, we identify that event as group behavior toward the blue vessel. With the non-blue ships trying to occupy the vulnerable regions around the boat and group intents toward it, we can identify hostile intentions clearly. Figure 3.7 illustrates the system flagging group behaviors; two pilot vessels eliciting RAM behavior toward a DDG51 ship in the scenario. In addition, we can also see threat hotspots around the DDG51 vessel from the *Threatmap*. Combining the group hostile behaviors and threats visible from the *Threatmap*, we can see the ship's urgent potential threat in this scenario.

3.7 Real-Time, Online Mobile Resource Allocation for Defensive Responses

We now consider the problem of responding in real-time to detected threats, in scenarios with multiple blue and red vessels, with the goal of minimizing damage and increasing protection for high-value assets in the blue team. We assume that the blue team includes vessels that can act as mobile defensive resources, thereby enabling us to

implement a mobile resource allocation strategy against predicted threats. With this approach a high-value asset (*i.e.*, an asset under protection) now has dynamic options for further reducing its vulnerability. Herein, we discuss our own real-time strategy, however, our framework allows for the any alternate real-time implementation to replace our own. This section of the work was completed in collaboration with other lab members; full details of the underlying allocation algorithms are reported in our prior work [137, 138] and the consolidated journal version [19].

3.8 Conclusion

The naval simulator introduced in this paper represents a significant leap forward in maritime simulation technology, offering a powerful, flexible platform for maritime research. By integrating structures for complex scenario generation, COLREGs-compliant navigation and obstacle avoidance, vulnerability and predictive threat visualization with heatmaps, a generalized intent recognition pipeline that supports HMM and deep-learning classifiers, and real-time resource allocation for vulnerable or under-threat assets, this simulator offers a comprehensive and interactive environment for maritime security and navigational studies. The collision avoidance and COLREG-compliant protocols introduce the platform for further autonomous ship driving research. Furthermore, the simulator provides intuitive solutions for visualizing a naval agent’s inherent vulnerability and the potential intent or threat it faces from other agents or groups of agents, which makes understanding threats faster and simpler. In

addition, its real-time computation capabilities for vulnerability attributes allow users to employ effective defense strategies to protect high-value assets with mobile resources, simulating realistic responses to dynamic threats. The simulator's capabilities are demonstrated in various complex naval scenarios, suggesting its potential as a safe and secure platform for formulating and solving real-world maritime problems. Future work could enhance NavySim to handle more complex threat dynamics and multi-agent interactions, improving its impact on maritime research and education.

Chapter 4

Early Classification of Intentions for Maritime Domains Using Deep Learning Models

4.1 Introduction

Effective detection of intentions in a maritime domain has become more crucial for the enhancement of the maritime security systems. Predicting intentions accurately based on a set of predictive attributes in the naval domain can guide strategic decision-making capabilities, which reduces the likelihood of relative incidents. In this paper, we focus on developing a deep learning-based framework for the immediate classification of both hostile and non-hostile behaviors in the maritime domain to minimize the

potential threats. For this process, first, we collected maritime datasets for different ship navigation behaviors. Second, we developed a set of features from the raw vessel data. Third, we applied different feature selection methods to identify the optimal subset of features, and finally we developed and evaluated the performance of two deep learning models for these different feature subsets. The experimental results show that by using the optimal feature subset our proposed deep learning models can effectively identify seven distinct navigation behaviors, with an overall accuracy of 97% in a maritime domain. On the other hand, our proposed model can distinguish between Hostile and Non-Hostile behaviors with an overall accuracy of 99%.

Publication note. The work in this chapter was reported by Hakim et al. [136].

4.2 Data Engineering

4.2.1 Data Collection

The data used in this research has been generated using a simulation environment developed by Huntington Ingalls Industries Unmanned Systems (HII). Between 2000 and 2500 scenarios per behavior were generated, in which one vessel (considered to be the own ship) is given a specific navigation waypoint, while another vessel (neutral/hostile) is driven by autonomous controllers that implement the navigation behaviors described in Table 1.1. In each scenario the initial locations of the two

vessels were randomly generated, thus ensuring that the potentially hostile vessel made different navigation maneuvers in order to achieve its goals. For each scenario, the simulation records the *position* (given in latitude/longitude coordinates), the *heading* (in degrees), and the *speed* (in meters/second) for both the own ship and the other vessel. All data is applied a level of noise consistent with that expected from the on-board sensors. Depending on the behavior navigation type, the scenarios transition through several stages: *wait* (vessel waiting for a condition to be met), *transit* (vessel gets into position before beginning execution), *execute* (vessel is executing maneuvers defined by the behavior), *complete* (vessel has completed maneuver defined by the behavior). During each scenario the own ship is using a controller for goal seeking, while obeying the International Regulations for Preventing Collisions at Sea (COLREGs). Thus, the own ship may take evasive maneuvers if the neutral/hostile ship interferes with its navigation.

4.2.2 Data Preprocessing

The different behaviors in the original dataset had varying numbers of scenario files, and those files contained different numbers of frames. To balance the training data for the classes, we selected the longest 1,000 files per class and then chose the first 500 frames from each file. If any file had fewer than 500 frames, we padded them by adding 0s to make the file length 500 and then used a mask during model training to ignore them. After data preprocessing, we divided the dataset into 70%-15%-15%

splits for training, validation, and testing sets, respectively. As a result, the resulting training set includes 700 files with 500 frames per file for each of the 7 behaviors, while the validation and testing sets each contain 150 files with 500 frames per behavior.

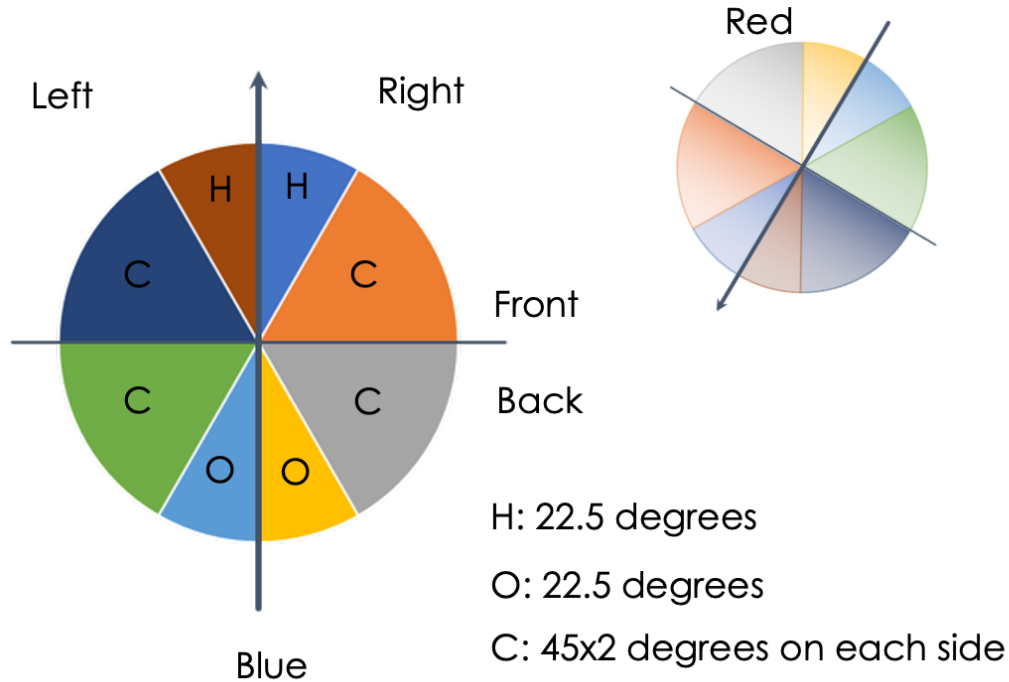


FIGURE 4.1: The three regions around a vessel: Head-on (H) - directly in front; Overtake (O) - directly behind; Cross (C) - either side of the ship is the cross region. In this case, the blue and red vessels are within the cross region of each other.

4.2.3 Feature Development

Based on the position, heading and speed information for each vessel, we compute a set of naval-specific features that can be used for behavior classification (we refer to our ship as *OWN* and the other ship as *OTHER*): **b_sp**: *OWN* speed (*m/s*), **b_head**: *OWN* heading (*degrees*), **r_speed**: *OTHER* speed (*m/s*), **r_accel**: *OTHER*

acceleration (m/s^2), **r_head**: *OTHER* heading (*degrees*), **abs_dheading**: difference in the heading between *OWN* and *OTHER* with respect to their absolute headings (*degrees*), **dist**: current distance between *OWN* and *OTHER* (*m*), **rel_b_r**: bearing of *OTHER* relative to *OWN* (*degrees*), **rel_r_b**: bearing of *OWN* relative to *OTHER* (*degrees*), **abs_b_r**: absolute bearing between *OTHER* and *OWN* (*degrees*), **abs_r_b**: absolute bearing between *OWN* and *OTHER* (*degrees*), **cpa_time**: estimated time at which (assuming constant velocity and heading) *OWN* and *OTHER* will be closest (*s*), **cpa_dist**: estimated closest point of approach (CPA) distance that *OWN* and *OTHER* will reach in the future (*m*), **S_side**: symbol representing the location of *OTHER* with respect to *OWN* as either left or right side (Figure 4.1), **S_beam**: symbol representing the location of *OTHER* as either in front of, or behind *OWN*, **S_reg_b_r**: symbol representing the region around *OWN* with *OTHER* in it (Figure 4.1), **S_reg_r_b**: symbol representing the region around *OTHER* with *OWN* in it (Figure 4.1), **S_ddist**: symbol representing the change in distance between the ships over time as increasing, decreasing, or constant, **S_accel**: symbol representing the change in velocity of *OTHER* over time as increasing, decreasing, or constant, **S_drel_bear**: symbol representing the change in relative bearing between *OWN* and *OTHER* over time as increasing, decreasing, or constant, **S_dabs_heading**: symbol representing the change in absolute heading of *OTHER* as increasing, decreasing, or constant, **S_cpa_ddist**: symbol representing the change in CPA distance over time as increasing, decreasing, or constant, **S_cpa_t**: symbol representing the change in CPA time over time as increasing, decreasing, or constant. [6] explains the generation

process of the symbols from other vessel data.

The feature computation and the intent predictions can both be performed in real time. The deep learning models are embedded in an online classifier that takes as input data measured through a vessels network of sensors and can display, at each frame, the predicted intent of other nearby vessels.

4.3 Research Methodology

This paper is focused on classifying intentions of vessels in the maritime domain based on raw data that can be collected solely from a ship’s on-board sensors. In particular, the aim is to disambiguate 7 different navigation behaviors, as described in Table 1.1. These behaviors are further divided between hostile and non-hostile, classified by Naval domain experts. The overall proposed research methodology is shown in Figure 4.2.

4.3.1 Feature Selection Using Decision Trees

Given the large set of naval-specific features previously developed, we aim to identify if all or a subset of them would best enable the classification of naval behaviors. Toward this end, we use decision trees (DTs), which, by leveraging criteria such as the Gini index and entropy, provide an effective method for evaluating and selecting important features.

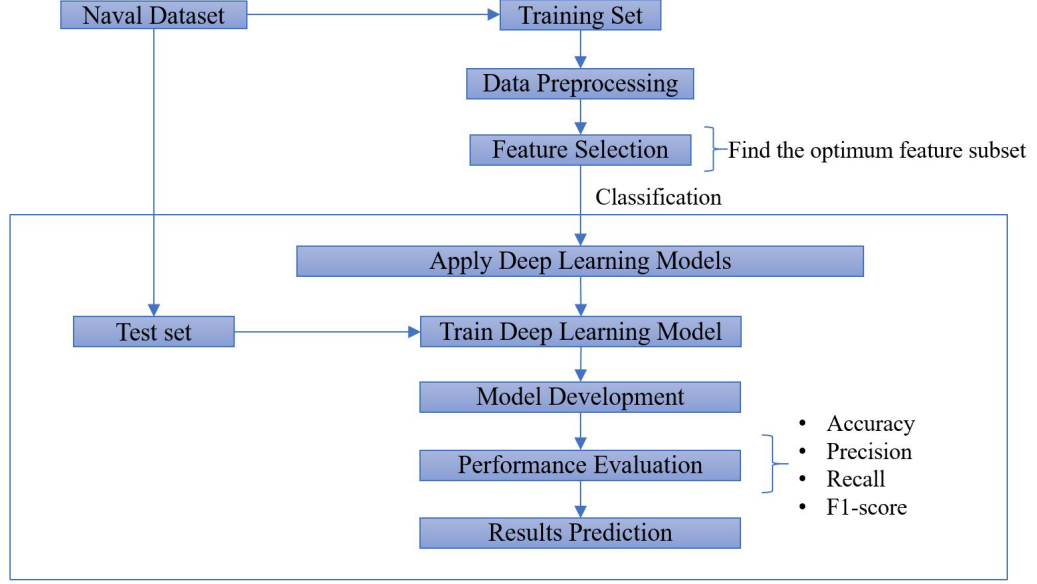


FIGURE 4.2: Proposed Research Methodology.

4.3.1.1 Gini Index-Based Decision Trees

The Gini index $G(D)$ measures the impurity of a dataset D , by assessing the likelihood of a randomly selected element being misclassified if it were labeled according to the class distribution, defined as follows:

$$G(D) = 1 - \sum_{i=1}^m p_i^2 \quad (1)$$

where p_i is the probability of class i in the dataset D , and m is the number of classes.

During the construction of a decision tree, each potential feature split is evaluated based on its Gini gain, which measures the reduction in impurity. The feature with the highest Gini gain is selected, enabling for feature selection by ranking the features according to their effectiveness in separating the classes.

4.3.1.2 Entropy-Based Decision Trees

Entropy is a measure of uncertainty or disorder within a dataset. In the context of decision trees, it is used to calculate information gain, which reflects the reduction in entropy when a feature is used for splitting. The entropy $E(D)$ for a dataset D is given by:

$$E(D) = - \sum_{i=1}^m p_i \log_2(p_i) \quad (2)$$

where p_i is the probability of class i in the dataset D , and m is the number of classes.

The information gain for a feature A is calculated as:

$$Gain(D, A) = E(D) - \sum_{v \in Values(A)} \frac{|D_v|}{|D|} E(D_v) \quad (3)$$

where D_v is the subset of D for which feature A has value v .

Entropy-based decision trees prioritize the most informative features by selecting the feature that maximizes information gain, thus enhancing the feature selection process.

4.3.1.3 Feature Importance Score

As decision trees are constructed, the importance of each feature is evaluated based on its contribution to reducing impurity or entropy. The importance score of a feature

is derived from the cumulative reductions in impurity or entropy it achieves across all the splits in the tree. This score quantitatively measures the feature’s relevance to the predictive model. In this research, we employ decision trees with both Gini index and entropy criteria, to select features that contribute most significantly to the model’s predictive power.

4.3.2 Deep Learning Models

The impact of the proposed feature selection strategy on intent recognition is evaluated using two deep learning architectures, both developed by collaborators within our research team. The Long Short-Term Memory (LSTM) module was implemented by Ayesh Meepaganithage, and the Fully Connected Deep Learning (FCDL) module was developed by Md Azizul Hakim. These models serve as the foundation for intent classification tasks and are described in detail below.

4.3.2.1 Fully Connected Deep Learning Model (FCDL)

The multi-layer architecture is shown in Figure 5.1. The *input layer* takes input features from the training data. The *first/second/third hidden layers* are dense layers with 256, 128 and 64 neurons, respectively. The neurons are activated using the ReLU activation function, followed by batch normalization and a dropout layer with a 40% dropout rate to prevent over-fitting. The *output layer* consists of 7 neurons, each

of which outputs the probability for one of the 7 behavior classes. The neurons are activated using the softmax activation function.

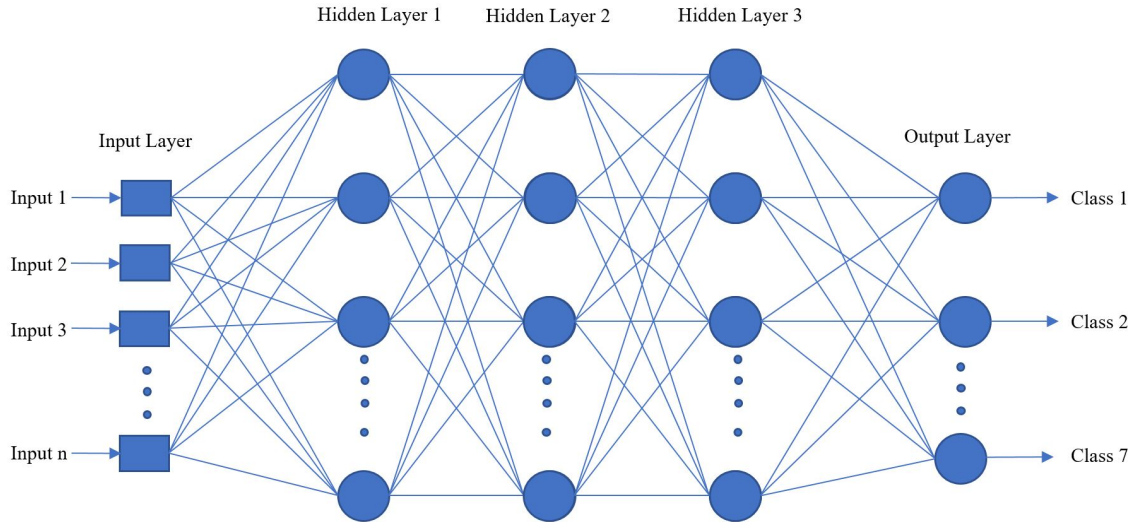


FIGURE 4.3: Fully Connected Model Architecture

To improve our model accuracy by avoiding over-fitting [139], we utilized i) *L2 regularization*, which reduces the squared sum of the differences between the target and predicted classes, ii) *batch normalization* to accelerate the training process by normalizing the inputs to each layer of deep learning model, and iii) *early stopping technique* to monitor the validation loss with a patience of 5 epochs. We experimented with different optimizers such as AdaGrad, AdamW, and Adam for training our fully connected DL model and obtained the best performance using the Adam optimizer with a learning rate of 0.001 and 40% of dropout. To address the challenge of handling temporal data for a fully connected deep learning model, we employed a sliding-window approach [140] with a window size of 20 which helps to extract temporal dependencies by dividing the dataset into a fixed-length window.

4.3.2.2 Long-Short Term Memory Network Model (LSTM)

The architecture of the proposed LSTM model is shown in Figure 4.4. LSTM models are specialized to handle long-term dependencies in sequential data. Each LSTM unit consists of a cell state and three gates: the input gate, forget gate, and output gate. The cell state works as a memory unit that gets updated throughout the sequence. The forget gate discards irrelevant data from the cell state, the input gate decides which new information should be stored, and the output gate determines which data is passed to the next hidden state, which will control the final output. This three-gate mechanism helps LSTM models retain relevant information from previous steps and discard useless information, enhancing their capability for long-term predictions.

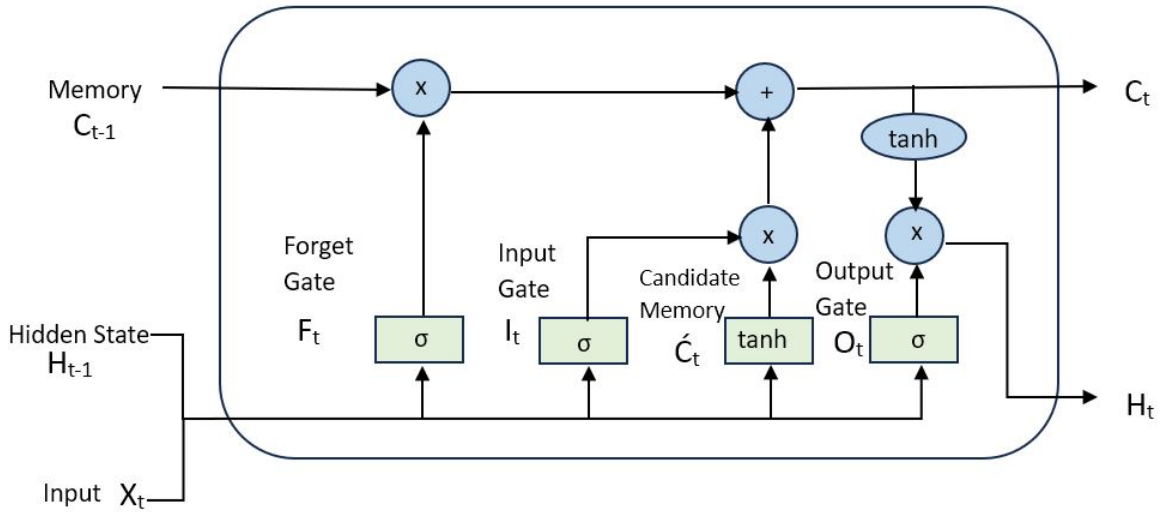


FIGURE 4.4: LSTM Model Architecture

Our proposed LSTM model consists of 4 layers: a masking layer to ignore the padding in the input files, an LSTM layer with 64 units, followed by another LSTM layer

with 64 units. Finally, we use a dense layer with softmax activation for multiclass classification or sigmoid activation for binary classification.

For hyperparameters, we used a learning rate of 0.001, the Adam optimizer, sparse categorical cross-entropy loss for multi-class classification, and binary cross-entropy loss for binary classification. We employed early stopping with a patience of 5. Although we trained the model for 100 epochs, if the validation loss did not improve for 5 consecutive epochs, training was stopped early.

When using an LSTM model, we evaluate performance based on the entire file, meaning that one prediction will be made to classify the entire file, rather than making predictions on a per-frame basis. In this way, the LSTM model can capture the sequential relationships within the behaviors, leading to more accurate predictions. However, as mentioned before, the LSTM model only uses the first 500 frames from each file. Thus, the LSTM model predicts the behavior for the entire file based on the first 500 frames.

4.4 Experimental Results

4.4.1 Feature Selection

Using the dataset described in Section 4.2.2 we trained a Gini index-based and an Entropy-based decision tree to select the top features based on the importance score

TABLE 4.1: Performance Metrics Using Decision Trees (DT)

Behaviors	Model	Acc	Pr	Re	F1
Benign	Gini DT	0.99	0.99	0.99	0.99
	Entropy DT	0.98	0.99	0.99	0.99
Block	Gini DT	0.96	0.96	0.96	0.96
	Entropy DT	0.96	0.95	0.96	0.96
Cross	Gini DT	0.41	0.56	0.41	0.47
	Entropy DT	0.50	0.70	0.50	0.58
Headon	Gini DT	0.92	0.89	0.92	0.91
	Entropy DT	0.95	0.87	0.95	0.91
Herd	Gini DT	0.99	0.99	0.99	0.99
	Entropy DT	0.99	0.99	0.99	0.99
Overtake	Gini DT	0.91	0.79	0.91	0.85
	Entropy DT	0.91	0.84	0.91	0.88
Ram	Gini DT	0.78	0.83	0.79	0.79
	Entropy DT	0.84	0.84	0.84	0.84
Overall	Gini DT	0.95	-	-	-
	Entropy DT	0.96	-	-	-

(IS), to use in the training of the deep learning models. In each data instance, a timestep of a naval scenario for a particular behavior is represented by the 23 features described in Section 4.2.3.

4.4.1.1 Implications of Performance Metrics in Decision Trees

Table 4.1 provides a comparative analysis of the performance metrics for Gini-index-based and entropy-based DTs across various behaviors. The overall performance metrics suggest a high efficacy for both decision tree models. The entropy-based DT exhibits a marginally higher overall accuracy (0.96) compared to the Gini-index-based DT (0.95). This slight difference underscores the robustness and reliability

TABLE 4.2: Feature Importance Using Gini Index and Entropy

Rank	Gini Index		Entropy	
	Features	IS	Features	IS
1	dist (m)	0.5348	dist (m)	0.5027
2	S_ddist	0.1524	S_ddist	0.1680
3	r_speed (mps)	0.1428	r_speed (mps)	0.1090
4	abs_dheading	0.0607	S_beam	0.0758
5	S_reg_r_b	0.0404	S_reg_r_b	0.0462
6	S_reg_b_r	0.0134	abs_dheading	0.0409
7	cpa_time (s)	0.0125	S_reg_b_r	0.0115
8	rel_r_b	0.0104	cpa_time (s)	0.0100
9	cpa_dist (m)	0.0092	cpa_dist (m)	0.0091
10	S_beam	0.0082	rel_r_b	0.0089
11	rel_b_r	0.0049	b_sp (mps)	0.0073
12	b_sp (mps)	0.0038	r_accel	0.0037
13	r_accel	0.0028	rel_b_r	0.0027
14	abs_b_r	0.0022	abs_b_r	0.0014
15	abs_r_b	0.0009	S_drel_bear	0.0013
16	r_head (deg)	0.0002	abs_r_b	0.0008
17	b_head (deg)	0.0002	S_dabs_heading	0.0003
18	S_drel_bear	0.0002	b_head (deg)	0.0002
19	S_dabs_heading	0.0001	r_head (deg)	0.0002
20	S_cpa_ddist	0.0000	S_side	0.0000
21	S_cpa_t	0.0000	S_cpa_ddist	0.0000
22	S_side	0.0000	S_cpa_t	0.0000
23	S_accel	0.0000	S_accel	0.0000

of both models in performing classification tasks, with a slight edge in favor of the entropy-based method.

The comparative analysis of performance metrics reveals that while both Gini-index-based and entropy-based DTs are effective for feature selection and predictive modeling, the entropy-based model generally offers better accuracy and reliability in classifying behaviors such as *CROSS*, *HEADON*, and *RAM*. These findings indicate that entropy-based decision trees may be the preferred method for applications requiring

high precision and recall.

Table 4.2 ranks the features based on their importance scores derived from Gini index-based and entropy-based decision trees. The first outcome is that the distance feature **dist** has the highest importance score in both criteria. Subsequent features like **S_ddist**, **r_speed** and **abs_dheading** also show significant importance in both methods, albeit with some variation in their individual scores. After the 8-th feature, the importance score of features drops below 1%. Even though both models show different performance matrices, they have similar top 15 features; only rank 15 and 16 switched for Entropy DT. Previous experiments on similar datasets produced the same top 15 features as the Gini index-based DT. Thus, along with aiming to maintain symmetry (to keep both **abs_b_r** and **abs_r_b** in the feature set, we selected the top 15 features from the Gini DT.

4.4.2 Analysis of Training and Validation

Using the training data set, we trained the FCDL and LSTM models, using both the full set of 23 features as well as the top 15 features identified in Section 4.4.1. The size of the input layers is determined by the number of input features in the training dataset. In this case, the size of the input layers is 23 while using all 23 features and it is 15 while using the top 15 features.

We tuned the hyperparameters heuristically to ensure a smooth convergence of both accuracy and loss function during the training period of the models. We then analyzed the performance on classifying individual behaviors, as well as the performance of identifying the correct hostile/non-hostile classes, as a binary classification problem.

Figures 4.5, 4.6 show the accuracy and loss curves during training and validation using the top 15 features for the FCDL and LSTM models respectively, indicating that both models are well generalized with closely aligned training and validation accuracy.

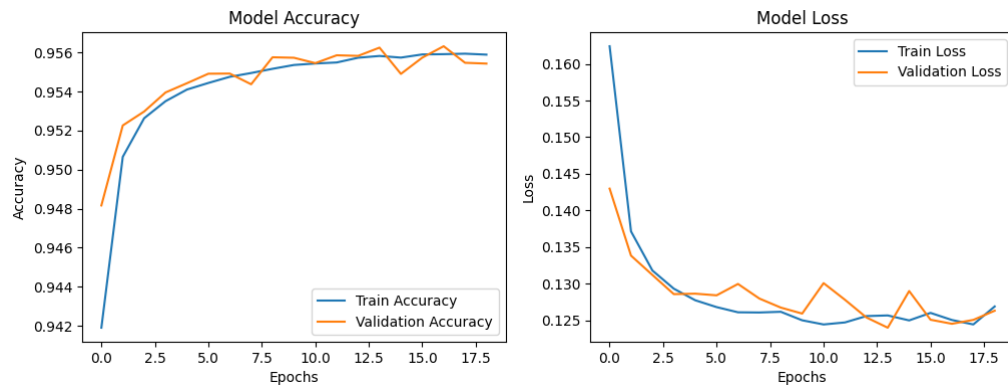


FIGURE 4.5: Training and validation accuracy and loss curves for the FCDL model using the top 15 features.

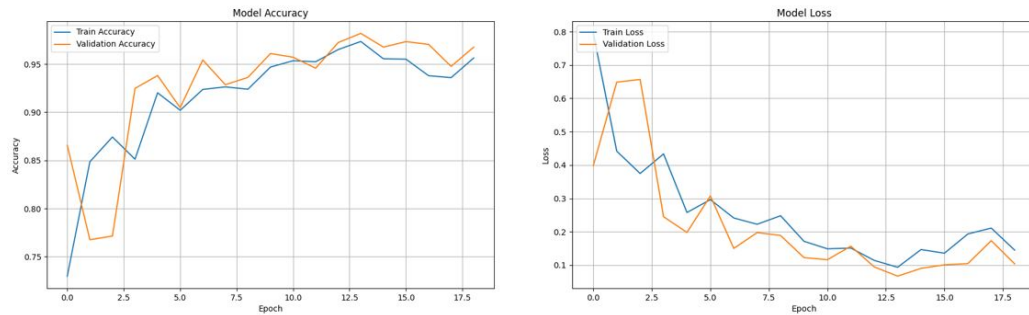


FIGURE 4.6: Training and validation accuracy and loss curves for the LSTM model using the top 15 features.

TABLE 4.3: Multi-class Performance Comparison Between Fully Connected Deep Learning (FCDL) and LSTM Models Using All and Top-15 Selected Features

Behavior	Fully Connected DL Model								LSTM Model							
	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
Benign	0.97	0.96	0.97	0.97	0.98	1.00	0.98	0.99	0.99	0.99	0.97	0.98	1.00	1.00	0.99	0.99
Block	0.85	0.76	0.87	0.81	0.99	0.94	1.00	0.97	0.99	0.96	1.00	0.98	0.99	0.96	0.94	0.95
Ram	0.81	0.84	0.81	0.83	0.99	0.96	0.99	0.98	0.99	0.97	0.95	0.96	0.99	0.95	0.97	0.96
Cross	0.70	0.67	0.70	0.69	0.98	0.94	0.99	0.96	0.99	0.98	0.93	0.95	0.98	0.98	0.91	0.94
Headon	0.47	0.53	0.47	0.50	0.89	0.84	0.89	0.87	1.00	0.99	1.00	0.99	1.00	1.00	0.99	1.00
Herd	0.74	0.74	0.75	0.74	0.89	0.92	0.89	0.90	1.00	0.99	1.00	0.99	1.00	1.00	1.00	1.00
Overtake	0.89	0.94	0.89	0.92	0.98	1.00	0.99	0.99	0.89	0.98	0.96	0.95	0.98	0.88	0.97	0.92
Overall Accuracy	0.88	-	-	-	0.97	-	-	-	0.97	-	-	-	0.97	-	-	-

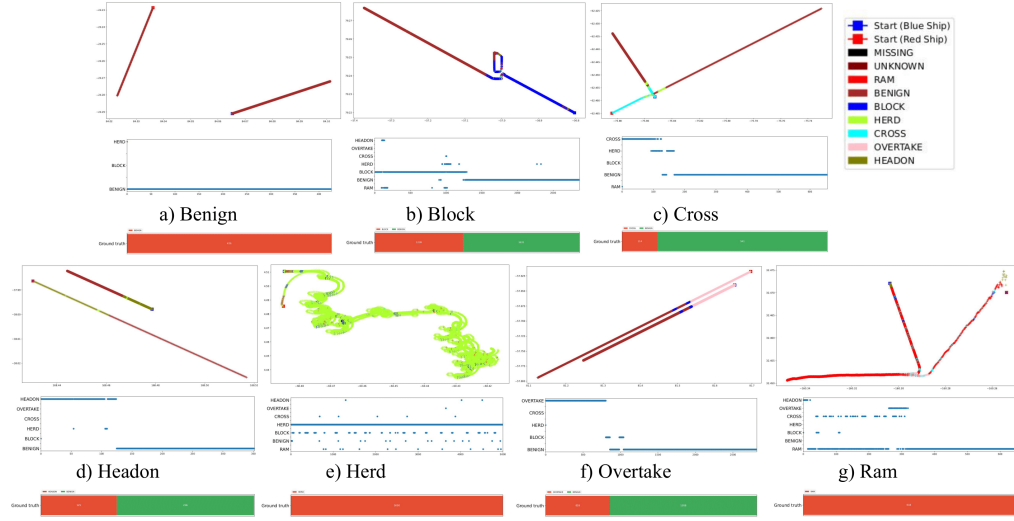


FIGURE 4.7: Sample scenarios for each type of behavior. **Top:** Position graph of vessels in latitude/longitude coordinates. **Middle:** Intent predicted over time. **Bottom:** Ground truth over time. In (b), (c), (d), and (f), the ground truth starts as the labeled intent and then ends with a benign intention.

4.4.3 Multi-class Classification for Intent Detection

We evaluated the performance of our FCDL and LSTM models using accuracy, precision, recall, and F1-score. The results for individual class prediction and overall accuracy are shown in Table 5.3. A key note is that the FCDL model performs classification per frame, while, as mentioned in Section 4.3.2.2, the LSTM model gives

one classification label for an entire scenario exhibiting a particular behavior.

The results indicate that the FCDL model, using top 15 features led to significantly better performance compared to using all features. In contrast, for the LSTM model, the use of selected features did not have a notable improvement. This is likely because the LSTM models inherently perform feature selection within their architecture.

Figure 4.8 shows the confusion matrices for both models (FCDL and LSTM) based on the top 15 features. To make it easier to interpret the model performance due to the different number of samples for each behaviors, we normalized the confusion matrices, which allows us a better comparison between different behaviors.

For the FCDL model, we obtain a high degree of accuracy (above 98%) for all models except *HEADON* and *HERD*. For *HEADON*, the most incorrect predictions are made to *HERD* and *CROSS*. This is explicable, as the feature values computed during a head-on scenario have a high-degree of overlap with the observations from herding behaviors (the other vessel changes direction and comes head-on toward the own ship) and from crossing scenarios (as some crossing scenarios have the other vessel coming at a sharp angle toward the own ship). In turn, for *HERD*, as shown in the position plot in Figure 4.7 e), the trajectory of the other vessel during a herding maneuver, resembles situations from other types of behaviors: coming from an angle from behind (consistent with crossing), coming from an angle from the front (consistent with head-on), or moving ahead of the own vessel (consistent with blocking). In general, the misclassifications are due to the inherent similarities between navigation maneuvers,

as it can be seen from the position plots in Figure 4.7. The LSTM model, since it takes into account the entire scenario, has the ability to rely on more data, containing temporal information of past observation. Thus, the classification results for the LSTM model are better than FCDL.

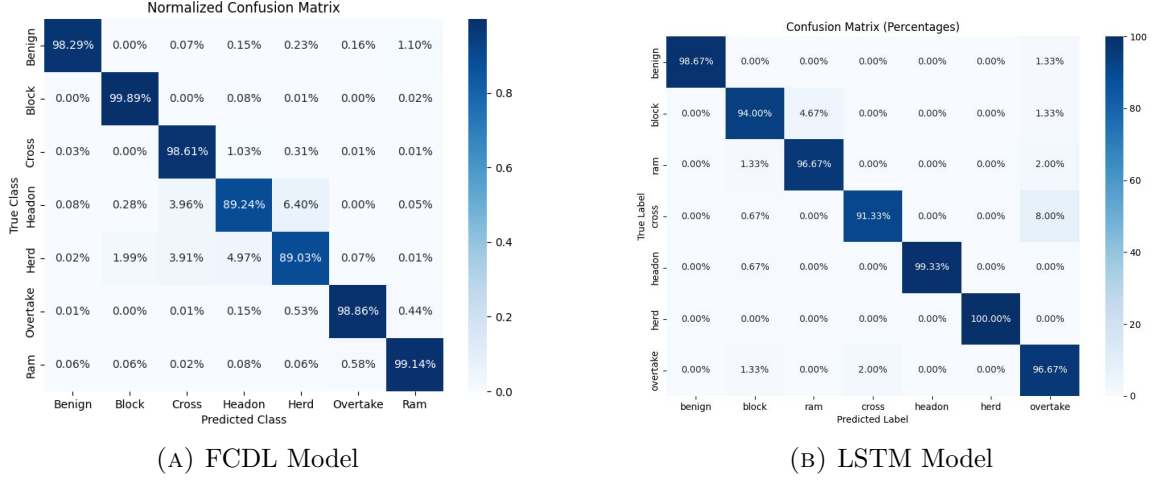


FIGURE 4.8: Confusion matrices for the FCDL and LSTM models using the top 15 features.

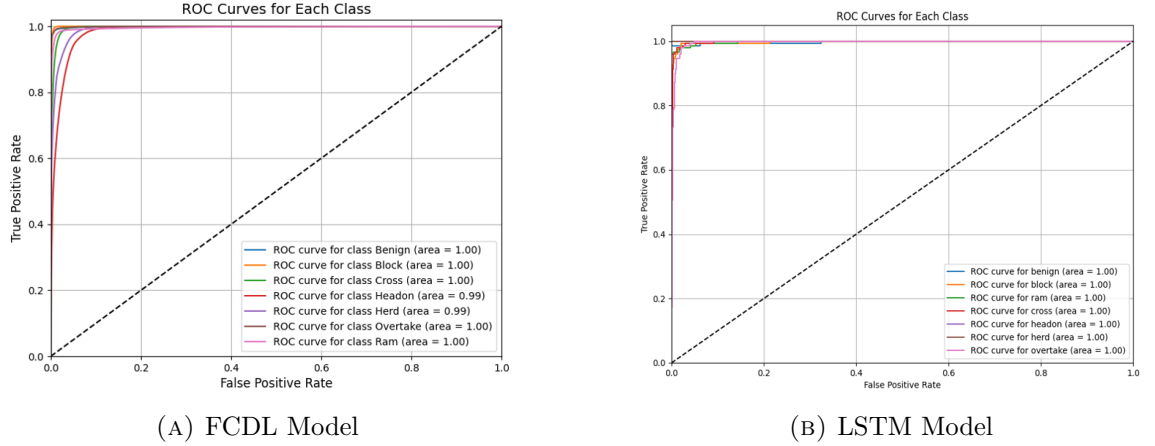


FIGURE 4.9: ROC Curves Using Top 15 Features

The Receiver Operating Characteristic (ROC) curves for both the FCDL and LSTM models using the top 15 features are shown in Figure 4.9. ROC curve [141] evaluates a model's performance by displaying the trade-off between True positive rate and False positive rate at different classification thresholds. Our ROC curves indicate that the

models using top 15 features are performing with a nearly perfect Area Under the ROC Curve (AUC) scores for all the classes. The position of the curves show that both the models achieve a high True positive rates while keeping the False positive rates low.

4.4.4 Binary Classification for Hostile and Non-hostile Behaviors

To perform a binary classification between hostile (*RAM*, *BLOCK*, and *HERD*) and non-hostile (*CROSS*, *OVERTAKE*, *HEADON*, and *BENIGN*) behaviors, we used the top 15 features and retrained our FCDL and LSTM models. We modified the output layer of the FCDL model to perform binary classification, leaving the rest of the network unchanged. Our models perform very well in distinguishing between hostile and non-hostile behaviors, achieving 99% accuracy with the LSTM classifier with the top 15 features. Table 4.4 shows the binary classification report as well as the individual accuracy for both hostile and non-hostile classes. As for the multi-class predictions, for both the FCDL and LSTM models, for the binary classification task using the top 15 features led to better performance compared to using the entire feature set. We show the normalized confusion matrices for the binary classification in Table 4.5. An important observation is that the LSTM performs very well with a lower False Negative and False Positive Rate.

TABLE 4.4: Binary Classification Performance of FCDL and LSTM Models Using All and Top 15 Features

Behavior	Fully Connected DL Model								LSTM Model							
	All Features				Selected Features				All Features				Selected Features			
	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
Hostile	0.79	0.95	0.96	0.95	0.97	0.95	0.96	0.96	0.96	0.94	0.97	0.95	0.99	1.00	0.99	0.99
Non-Hostile	0.91	0.99	0.99	0.99	0.96	0.99	0.99	0.99	0.96	0.97	0.95	0.96	0.99	0.99	1.00	1.00
Overall Accuracy	0.88	-	-	-	0.97	-	-	-	0.96	-	-	-	0.99	-	-	-

TABLE 4.5: Normalized Confusion Matrices for FCDL and LSTM Models (Hostile vs Non-Hostile)

Actual	Predicted (FCDL)		Predicted (LSTM)	
	Non-Hostile	Hostile	Non-Hostile	Hostile
Non-Hostile	97.43%	2.57%	99.67%	0.33%
Hostile	4.43%	95.57%	0.89%	99.11%

4.4.5 Comparison with Other Work

We performed a comparative analysis between our work and prior results on intent classification in the maritime domain. A first approach uses Hidden Markov Model (HMMs) [76] to classify four different intentions (*BLOCK*, *BENIGN*, *HERD* and *RAM*) in the maritime domain, with 67% accuracy. A second approach, used a Bidirectional Long Short-Term Memory (Bi-LSTM) model [79] to predict vessel's behaviors (*BENIGN*, *RAM* and *BLOCK*) in the naval domain, with 91.84% of accuracy. Neither of these methods used a feature selection method to identify the most relevant features. In addition, in this work the classification task contains more than double the number of classes, containing behaviors that have a higher degree of similarity and thus being harder to disambiguate. In this work we achieve 97% of total accuracy with both FCDL and LSTM models, while using the top 15 relevant features.

4.5 Conclusion and Future Work

In this research, we proposed a deep learning based framework to classify different intentions as well as to detect potential threats in a maritime domain. Selection of optimal subset of features are done by applying decision tree based feature selection methods and irrelevant features are excluded according to the feature importance score. Our proposed framework performs very well in identifying intentions with a prediction accuracy of 97% and detecting threats with a prediction accuracy of 99% in a naval domain, from noisy data. This work can be extended with finding more optimal subset of features and apply other neural network approaches such as transfer learning techniques, and transformer model with a much bigger dataset.

Chapter 5

Optimizing Features for Maritime Intent Recognition

5.1 Introduction

This chapter presents advancements in maritime intent recognition through novel feature selection methodologies and model optimization. A modified categorical permutation feature importance method is introduced to address the challenges of categorical/constant features and imbalanced datasets, avoiding bias towards normal features. Leveraging this improved feature selection approach, significant performance enhancements are demonstrated using Long Short-Term Memory (LSTM) and Fully Connected Deep Learning (FCDL) networks. In addition, attention-based importance scores of transformers are used to select optimal features, further refining the training

of the LSTM and FCDL models. The proposed methods achieve superior accuracy in recognizing the intent of naval agents in proximity to own ship, highlighting their potential for maritime situational awareness.

The collision of the USS Fitzgerald in 2017 [5] underscored the critical need for enhanced predictive maritime awareness technologies. In current complex and dynamic maritime environments, discerning the intent of surrounding naval agents early is crucial to vessel security and safety [76, 135]. Naval intent refers to the mission or purpose behind a naval agent’s actions or positioning. These behaviors fall into hostile and non-hostile categories, such as RAM, BLOCK, HERD (hostile) and CROSS, OVERTAKE, HEADON (non-hostile) (see Table 1.1). For intent recognition tasks, we observe techniques ranging from traditional probabilistic [142] and machine learning [76] to deep learning techniques [143–145] in the literature. In this chapter, we use a Long Short-Term Memory (LSTM) and a Fully Connected Deep Learning (FCDL) model (developed by lab collaborators) to predict the hostile/non-hostile intent of the ships or boats based on real-time data. The motivation behind using these models is to capture the temporal history of data to analyze hidden intent and employ these models physically on on-water boats/ships. Keeping the models simpler and lightweight helps deploy them on boats with limited resources.

These early intent detection algorithms rely on vast data from vessel sensor systems (e.g., Radar, Sonar, Visual Sensors). While large datasets benefit deep learning models, such as large language models (LLMs), the saying “Garbage in - Garbage out” remains

true. Excessive features can lead to overfitting, especially with small samples [21], and increase the problem space’s dimensionality, consuming more memory, processing time, and introducing biases from noisy features [22, 23].

Traditional feature importance methods, such as permutation feature importance, are model-agnostic and assess contributions by changing feature values [146]. However, these methods struggle with categorical or constant features, often overemphasizing normal features. To address these challenges, we propose the Categorical Permutation Feature Importance (CPFI) method, which ensures a balanced treatment of all feature types, particularly relevant for real-world maritime datasets.

The main contributions of this dissertation are as follows:

1. **Development of a Categorical Permutation Feature Importance (CPFI):**

A novel feature selection mechanism tailored for maritime datasets, effectively addressing the challenges posed by categorical and constant features, as well as class imbalance. This ensures an optimal subset of classification features for intent recognition.

2. **Comprehensive Validation and Performance Evaluation:** Leveraging the

Transformer’s attention mechanism to dynamically assign importance scores to individual features, enabling robust feature selection that captures complex interdependencies and reduces high dimensionality. This contribution, developed by our lab member Md Azizul Hakim, was used for comparative experiments

alongside the CPFI-based feature selection method. Results showcase high-accuracy classification of navigation behaviors, even in the presence of noisy maritime data, validating the robustness of the feature selection mechanisms.

Publication note. The work in this chapter was reported by Sayed et al. [80].

5.2 Methodologies

This dissertation focuses on classifying the intentions of vessels in the maritime domain using raw data collected exclusively from a ship's on-board sensors. Specifically, the objective is to distinguish between seven distinct navigation behaviors. Figure 5.1 illustrates the overall research methodology.

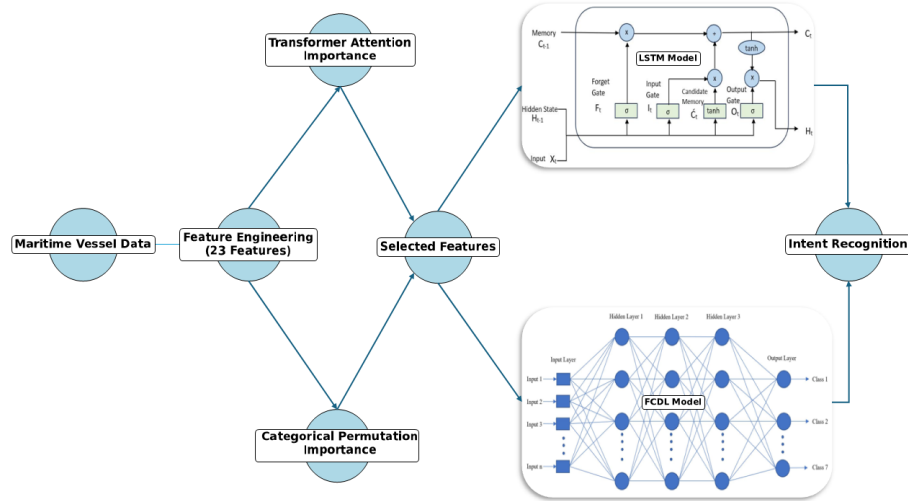


FIGURE 5.1: A Deep-Learning Framework for Maritime Intent Recognition

5.2.1 Data Engineering

The data engineering processes, which encompass data collection, data pre-processing, and feature development, are identical to those outlined in section 4.2 of chapter 3. Feature computation and intent predictions occur in real time using deep learning models within an online framework. This system processes data from a vessel's sensors and displays the predicted intent of nearby vessels in each frame.

5.2.2 Feature Selection

5.2.2.1 Categorical Permutation Feature Importance, CPFI

Permutation feature importance (PFI) by Fisher et al. [89] is a model-agnostic technique for inspecting the contribution of individual features to the performance of a trained model, providing valuable insights by quantifying the impact of each feature on predictive accuracy or other performance metrics. However, this technique does not work well for categorical or constant features. Imagine two features **S_cpa_ddis**: have three possible values 0 - vessels moving away from each other, 1 - vessels moving toward each other, 2 - constant (vessels keeping course), **dist**: real number. For a particular naval scenario containing 100 samples, **S_cpa_ddis** have 0 - 5 instances, 1 - 5 instances, 2 - 90 instances, **dist**: real number. With PFI, even with multiple shuffling **S_cpa_ddis** does not see any changes that would lower the performance of the model while for **dist**, we will see a lot more changes to impact the model adversely

after the permutation. Therefore, PFI will always put more importance score for **dist** than **S_cpa_ddis**. This is also true for the categorical or constant features in other problems. To tackle this issue, we take random value from the possible values of the feature and repeat them for all instances of categorical features while permuting the normal features among all instances like PFI. This keeps the purpose of losing feature-specific information relation that results in showing the importance of that feature in the overall performance measure of the model.

Let $X = \{x_1, x_2, \dots, x_n\}$ represent the set of features. Define a permutation dictionary $PD(x_i)$ to keep track of the type of feature:

$$PD(x_i) = \begin{cases} \{c_1, c_2, \dots, c_n\}, & \text{if } x_i \text{ is categorical/constants,} \\ 0, & \text{if } x_i \text{ is normal feature.} \end{cases}$$

The CPFPI algorithm is defined in Algorithm 1, and for experiments in this dissertation, we use $mean_absolute_error(M(X_{val}^{shuffled}))$ as a performance metric.

5.2.2.2 Transformer Feature Importance Score, TFIS

Our lab member Md Azizul Hakim proposed the use of the TFIS algorithm, which was employed in this dissertation for comparative experiment with CPFPI. Based on the self-attention mechanism, transformer based architecture capture the complex relationships and dependencies among the features which makes it a powerful feature

Algorithm 1: Categorical Permutation Feature Importance (CPFI)

Require: Trained model M , validation dataset (X_{val}, y_{val}) , performance metric $metric$, number of repetitions n , permutation dictionary $PD(x_i)$

Ensure: Importance scores for each feature

- 1: Define categorical/constant feature options and put zero for normal features
 $P_{baseline} = metric(M(X_{val}), y_{val})$
 - 2: Compute the baseline performance $P_{baseline} = metric(M(X_{val}), y_{val})$
 - 3: **for** each feature f_i in X_{val} **do**
 - 4: Initialize an empty list of performance drops S_i
 - 5: **for** $r = 1$ to n **do**
 - 6: **if** $PD(f_i) = 0$ **then**
 - 7: Shuffle the values of feature f_i in X_{val} , creating $X_{val}^{shuffled}$
 - 8: **else**
 - 9: Replace f_i in X_{val} with randomized samples from $PD(f_i)$ to generate $X_{val}^{shuffled}$.
 - 10: **end if**
 - 11: Compute the performance $P_{shuffled} = metric(M(X_{val}^{shuffled}), y_{val})$
 - 12: Calculate the performance drop $D = P_{baseline} - P_{shuffled}$
 - 13: Append D to S_i
 - 14: **end for**
 - 15: Compute the average importance score for f_i : $Importance(f_i) = \text{mean}(S_i)$
 - 16: **end for**
 - 17: Rank features by their importance scores in descending order
-

selection tool. Traditional feature selection techniques depends on the statistical correlations, but the transformer method assigns attention scores to each individual feature dynamically. The self-attention function [147] calculates the relevance of each input features to others using the following equation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1)$$

where Q , K , and V are the query, key, and value matrices respectively which are derived from the input features. The softmax functions calculate the attention weights that measures the importance of each feature which enable the model to create a rank

and select the most important features to improve the performance by reducing the high dimensionality.

Algorithm 2: Transformer-Based Feature Selection (TBFS)

Require: Trained transformer model M , validation dataset (X_{val}, y_{val}) , performance metric $metric$, number of shuffling repetitions n

Ensure: Importance scores for each feature

- 1: Compute the baseline performance: $P_{\text{baseline}} = metric(M(X_{val}), y_{val})$
 - 2: **for** each feature f_i in X_{val} **do**
 - 3: Initialize an empty list of performance drops S_i
 - 4: **for** $r = 1$ to n **do**
 - 5: Save the original values of feature f_i : $\text{save_col} \leftarrow X_{val}[:, i]$
 - 6: Shuffle the values of feature f_i : $X_{val}^{\text{shuffled}}[:, i] \leftarrow \text{shuffle}(X_{val}[:, i])$
 - 7: Compute the performance with shuffled feature values:
 $P_{\text{shuffled}} = metric(M(X_{val}^{\text{shuffled}}), y_{val})$
 - 8: Calculate the performance drop: $D = P_{\text{baseline}} - P_{\text{shuffled}}$
 - 9: Append D to S_i
 - 10: Restore original feature values: $X_{val}[:, i] \leftarrow \text{save_col}$
 - 11: **end for**
 - 12: Compute the average importance score for f_i : $Importance(f_i) = \frac{1}{n} \sum_{r=1}^n D_r$
 - 13: **end for**
 - 14: Rank features by their importance scores in descending order
-

5.2.3 Intent Recognition

5.2.3.1 Long-Short Term Memory Network Model (LSTM) and Fully Connected Deep Learning (FCDL)

The LSTM and FCDL models used in this chapter are the same as defined in Chapter ??.

These models have been adapted and applied in a new experimental context specific to the scenarios introduced in this chapter. No changes were made to the architectures; instead, the models were retrained, fine-tuned, and evaluated using the datasets

and behavior labels to assess their generalization performance in maritime intent recognition.

For training, we used the Adam optimizer with a 0.001 learning rate, sparse categorical cross-entropy for multiclass classification, and binary cross-entropy for binary classification. Early stopping (patience: 10) was applied, halting training if validation loss failed to improve for five consecutive epochs, up to a maximum of 100 epochs.

5.3 Experiments and Results

5.3.1 Comparison of Models with CPFI

With Categorical Permutation Feature Importance (CPFI), we hypothesized categorical/constant features to have an equal chance of getting picked for the subset by the selection procedure. As shown in Table 5.1, the likelihood of categorical features (represented by symbols that start with a capital “S” in this dataset) being selected by Categorical Permutation Feature Importance (CPFI) from an LSTM model increased by 20% compared to Permutation Feature Importance (PFI) for the top 10 important features. Notably, all of the top 5 features selected by CPFI are categorical features, which is 60% more than PFI selected top 5 features.

TABLE 5.1: Feature Importance Using PFI and CPFI on LSTM

Rank	PFI	CPFI
1	S_CPA_TIME	S_ACCEL
2	cpa_time	S_CPA_TIME
3	S_DELTA_DIST	S_DELTA_DIST
4	abs_dheading	S_DREL_BEAR
5	dist (m)	S_DABS_HEADING
6	S_reg_r_b	cpa_time
7	b_speed	S_SIDE
8	r_speed	abs_dheading
9	rel_b_r	dist (m)
10	S_DREL_BEAR	b_speed

We train the LSTM and FCDL models with our experimental setup and run the CPFI technique on top of it to find out the feature importance for each model. In Figure 5.2, we can see the features' importance score (IS) against the baseline in the mean absolute error measure calculated on the LSTM model. We selected the top features until a big drop in feature importance was detected or no/small changes in IS were seen for a few consecutive features. In this way, we selected the top 8 features and re-trained and evaluated the LSTM model with all features and the subset of features. Similarly, we could select either the top 4 features or a total of 21 features with CPFI on the FCDL model. We illustrated the results on Table 5.2. We can select two

subsets of features (Top-4 or Top-21) for FCDL with CPFI. The subset with Top-4 features certainly can not perform well and with the top-21 subset option, we get a 1% accuracy boost over the baseline. Based on our selection criteria, we can choose two subsets: the top-6 or top-8 subset of features, and the Top-8 subset of features performs better in all performance measures. The figure 5.3 and 5.4 also indicate that none of the LSTM models overfit on their respective experimental setups.

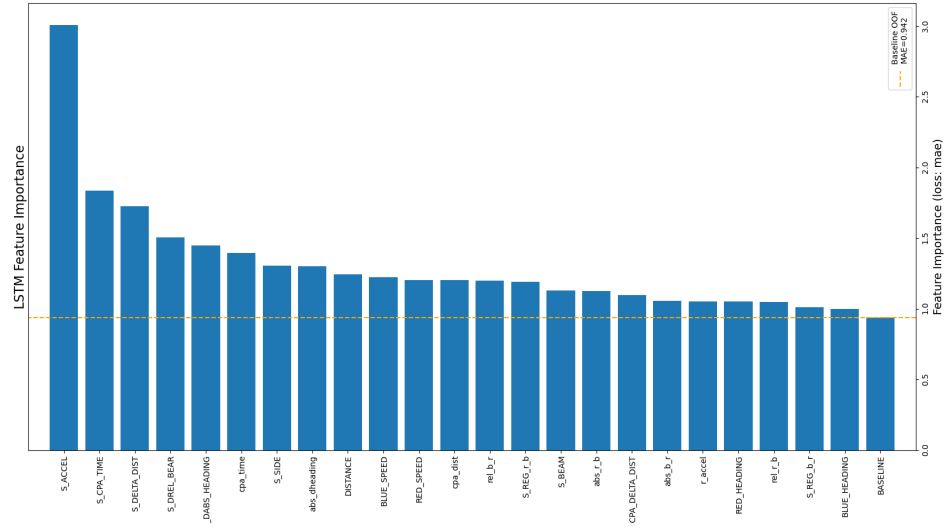
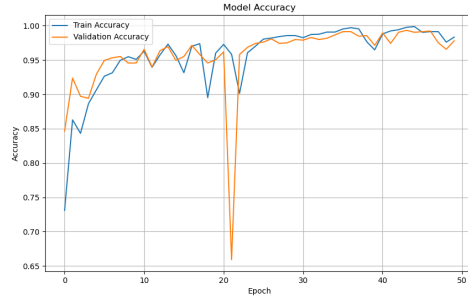


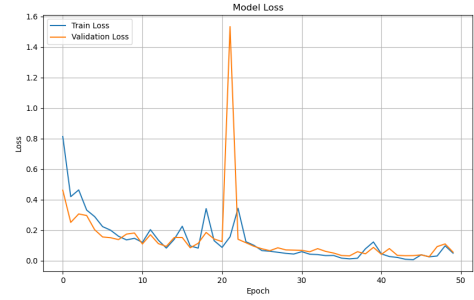
FIGURE 5.2: LSTM Feature Importance Score by CPFI

TABLE 5.2: Multi-class Performance Comparison Between FCDL and LSTM Models Using Different Selected Features from CPFI

Behavior	FCDL Model								LSTM Model							
	Top-4 Features				Top-21 Features				Top-6 Features				Top-8 Features			
	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
Benign	0.87	0.98	0.87	0.92	0.96	1.00	0.96	0.98	0.97	0.96	0.99	0.97	0.99	1.00	0.99	0.99
Block	0.05	0.58	0.05	0.10	0.98	0.82	0.98	0.89	0.84	0.91	0.84	0.87	0.94	0.96	0.94	0.95
Ram	0.75	0.71	0.75	0.73	0.97	0.92	0.97	0.95	0.93	0.97	0.93	0.95	0.94	0.93	0.94	0.94
Cross	0.41	0.63	0.41	0.50	0.89	0.87	0.90	0.88	0.92	0.90	0.92	0.91	0.97	0.95	0.97	0.96
Headon	0.61	0.20	0.61	0.30	0.82	0.56	0.82	0.66	0.92	0.93	0.92	0.93	0.99	0.98	0.99	0.99
Herd	0.66	0.45	0.66	0.54	0.70	0.89	0.70	0.79	1.00	0.97	1.00	0.99	1.00	0.98	1.00	0.99
Overtake	0.92	0.99	0.92	0.95	0.98	0.99	0.98	0.99	0.89	0.85	0.95	0.90	0.92	0.84	0.89	0.86
Overall Accuracy	0.81	-	-	-	0.95	-	-	-	0.93	-	-	-	0.97	-	-	-



(A) Accuracy over training epochs with all features.



(B) Loss over training epochs with all features.

FIGURE 5.3: (Left) Accuracy over training epochs. (Right) Loss over training epochs. Both plots include training and validation data for comparison.

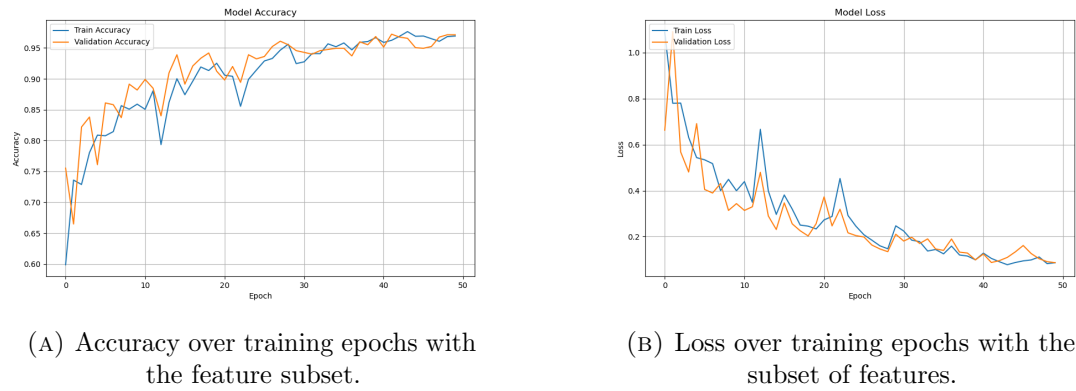


FIGURE 5.4: (Left) Accuracy over training epochs. (Right) Loss over training epochs. Both plots include training and validation data for comparison with the selected subset of features.

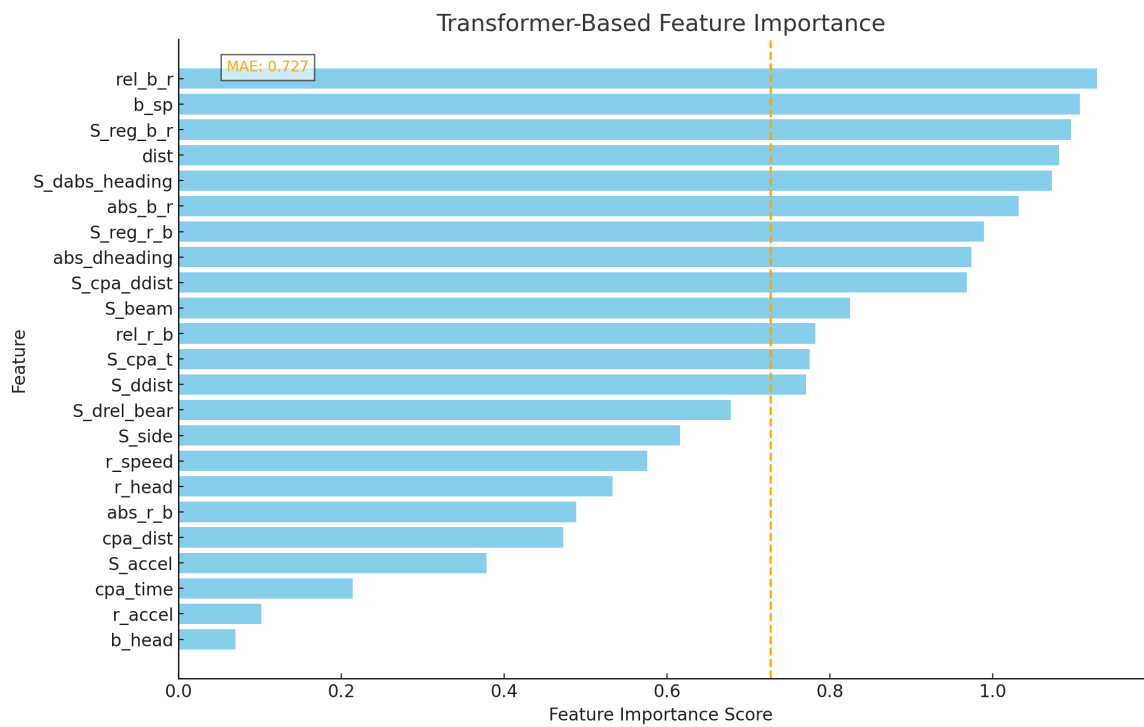


FIGURE 5.5: Transformer Based Feature Importance Score

5.3.2 Comparison of Models with TFIS

The transformer model captures the correlations between different input sequences which helps in better handling of long distance dependencies. We trained both FCDL and LSTM models using all 23 features as well as the top 13 features based on the feature importance set shown in Fig. 5.5. In this experiment, we used a multi-head attention mechanism so that it can help the model extract more features. During the training process, to ensure a smooth convergence on both accuracy and loss function we heuristically tuned the hyperparameter. We analyzed the prediction performance on identifying individual behaviors as well as the overall prediction performance. Fig. 4.5 shows the accuracy and loss curves for the FCDL using the top 13 features during the training and validation steps.

We evaluated the performance of both FCDL and LSTM models using different performance evaluation matrices such as accuracy, precision, recall, and F1-score are shown in Table 5.3. Results indicate that both FCDL and LSTM models perform significantly better while using the top 13 feature set. We also observe the LSTM model perform better than the FCDL model. Even the best performing FCDL model has 2% less accuracy than the base LSTM model and the highest performing LSTM model with features selected by TFIS is 98%.

TABLE 5.3: Multi-class Performance Comparison Between Fully Connected Deep Learning (FCDL) and LSTM Models Using All and Top-13 Selected Features from Transformer Feature Importance

Behavior	FCDL Model								LSTM Model							
	All Features				Top-13 Features				All Features				Top-13 Features			
	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
Benign	0.97	0.96	0.97	0.97	0.97	0.99	0.97	0.99	0.98	0.94	0.98	0.96	0.95	0.97	0.95	0.96
Block	0.85	0.76	0.87	0.81	0.94	0.68	0.94	0.79	0.97	0.88	0.97	0.92	0.99	0.97	0.99	0.98
Ram	0.81	0.84	0.81	0.83	0.90	0.91	0.90	0.90	0.90	0.96	0.90	0.93	0.97	1.00	0.97	0.99
Cross	0.70	0.67	0.70	0.69	0.85	0.78	0.85	0.81	0.84	0.95	0.84	0.89	0.98	0.99	0.98	0.99
Headon	0.47	0.53	0.47	0.50	0.83	0.45	0.83	0.59	0.97	1.00	0.97	0.98	1.00	1.00	1.00	1.00
Herd	0.74	0.74	0.75	0.74	0.58	0.79	0.58	0.67	0.96	0.99	0.96	0.98	1.00	0.97	1.00	0.99
Overtake	0.89	0.94	0.89	0.92	0.96	0.99	0.96	0.98	0.95	0.85	0.95	0.90	0.97	0.95	0.97	0.96
Overall Accuracy	0.88	-	-	-	0.92	-	-	-	0.94	-	-	-	0.98	-	-	-

5.4 One-vs-All Classification

5.4.1 One-vs-All Classification Performance using LSTM

To further investigate class-specific behavior recognition performance, we trained separate *One-vs-All* (*OvA*) binary LSTM classifiers for each behavior class. This approach isolates the learning process for each class, allowing the model to better focus on the nuances of a single behavior type when distinguishing it from all others.

TABLE 5.4: Classification Results for One-vs-All Models

Model Name	Target Class	Accuracy (%)
LSTM Class 0 vs All	Benign (0)	99.90
LSTM Class 1 vs All	Block (1)	99.81
LSTM Class 2 vs All	Ram (2)	99.81
LSTM Class 3 vs All	Cross (3)	99.52
LSTM Class 4 vs All	Headon (4)	99.81
LSTM Class 5 vs All	Herd (5)	99.62
LSTM Class 6 vs All	Overtake (6)	98.67

These results, summarized in Table 5.4, reveal consistently high accuracies (all above 98.5%) across all target behaviors. Notably:

- The **Benign** class achieves the highest accuracy at **99.90%**, suggesting the model can easily differentiate normal vessel patterns from abnormal or aggressive behaviors.
- The **Overtake** class, while still achieving a strong accuracy of **98.67%**, demonstrates relatively more overlap with other behavior types, aligning with earlier multi-class results where slight reductions in precision and recall were observed.
- All aggressive behaviors (**Block**, **Ram**, **Cross**, **Headon**, and **Herd**) are classified with accuracy exceeding **99.5%**, confirming the LSTM’s effectiveness

in capturing subtle patterns that differentiate these behaviors when treated independently.

5.4.2 Comparison and Insights

The OvA results reinforce the conclusions drawn from multi-class evaluations:

- **LSTM consistently outperforms FCDL**, particularly when fewer yet more informative features are used (e.g., Top-13 or CPFI-selected Top-8).
- The LSTM model’s ability to **specialize in individual behavior classes** leads to enhanced detection performance, especially for behaviors that may be misclassified in a multi-class setting due to class imbalance or inter-class similarity.
- Feature selection techniques such as Transformer-based importance and CPFI not only improve multi-class performance but also contribute to robust binary classification in OvA settings.

In summary, the One-vs-All LSTM models serve as an effective supplement to multi-class classifiers, offering an **alternative approach** for highly accurate and behavior-specific intent recognition in maritime environments.

5.4.3 Class-Specific Feature Importance and Comparative Feature Relevance

To better understand how different classes rely on different input features, we performed categorical permutation-based feature importance (CPFI) analysis on each of the one-vs-all LSTM classifiers. Tables 5.5 and 5.6 present the top-23 ranked features for each classifier based on Mean Absolute Error (MAE) degradation upon permutation.

TABLE 5.5: Top-23 Permutation Feature Importance (MAE) for One-vs-All LSTM Classifiers — Class 0 (Benign) to Class 3 (Cross)

Rank	Benign	Block	Ram	Cross
1	S_ACCEL (0.824)	S_DELTA_DIST (0.446)	S_DABS_HEADING (0.384)	S_CPA_DELTA_DIST (0.270)
2	S_CPA_TIME (0.415)	S_ACCEL (0.094)	S_CPA_DELTA_DIST (0.373)	S_REG_r_b (0.260)
3	cpa_time (0.181)	cpa_dist (0.084)	S_SIDE (0.372)	S_CPA_TIME (0.174)
4	S_DELTA_DIST (0.094)	DISTANCE (0.062)	S_REG_b_r (0.357)	S_DELTA_DIST (0.159)
5	S_REG_b_r (0.022)	S_REG_b_r (0.050)	S_BEAM (0.307)	S_REG_b_r (0.145)
6	S_BEAM (0.010)	S_BEAM (0.046)	rel_r_b (0.299)	S_BEAM (0.144)
7	RED_SPEED (0.009)	RED_SPEED (0.034)	RED_SPEED (0.285)	S_SIDE (0.144)
8	DISTANCE (0.006)	S_DABS_HEADING (0.026)	rel_b_r (0.284)	S_DREL_BEAR (0.143)
9	S_REG_r_b (0.005)	S_CPA_TIME (0.024)	abs_b_r (0.281)	S_DABS_HEADING (0.143)
10	S_DREL_BEAR (0.004)	S_DREL_BEAR (0.023)	DISTANCE (0.278)	S_ACCEL (0.143)
11	BLUE_SPEED (0.002)	r_accel (0.023)	cpa_dist (0.277)	abs_dheading (0.132)
12	rel_r_b (0.001)	rel_b_r (0.022)	RED_HEADING (0.276)	cpa_dist (0.128)
13	rel_b_r (0.001)	S_REG_r_b (0.022)	r_accel (0.275)	rel_r_b (0.122)
14	cpa_dist (0.001)	RED_HEADING (0.022)	abs_r_b (0.272)	abs_r_b (0.120)
15	abs_dheading (0.001)	rel_r_b (0.022)	BLUE_HEADING (0.268)	RED_SPEED (0.119)
16	S_SIDE (0.001)	abs_r_b (0.022)	abs_dheading (0.187)	abs_b_r (0.119)
17	S_DABS_HEADING (0.001)	cpa_time (0.021)	cpa_time (0.149)	r_accel (0.118)
18	r_accel (0.000)	BLUE_SPEED (0.020)	S_ACCEL (0.129)	DISTANCE (0.116)
19	abs_b_r (0.000)	BLUE_HEADING (0.019)	S_DELTA_DIST (0.128)	rel_b_r (0.114)
20	abs_r_b (0.000)	abs_b_r (0.017)	BLUE_SPEED (0.116)	BLUE_HEADING (0.113)
21	RED_HEADING (0.000)	abs_dheading (0.016)	S_REG_r_b (0.116)	RED_HEADING (0.112)
22	BLUE_HEADING (0.000)	S_SIDE (0.013)	S_CPA_TIME (0.096)	BLUE_SPEED (0.109)
23	S_CPA_DELTA_DIST (0.000)	S_CPA_DELTA_DIST (0.011)	S_DREL_BEAR (0.087)	cpa_time (0.109)

TABLE 5.6: Top-23 Permutation Feature Importance (MAE) for One-vs-All LSTM Classifiers — Class 4 (Headon) to Class 6 (Overtake)

Rank	Headon	Herd	Overtake
1	S_REG_r_b (0.208)	S_DELTA_DIST (0.154)	S_BEAM (0.390)
2	S_DREL_BEAR (0.143)	S_ACCEL (0.074)	S_REG_r_b (0.370)
3	S_CPA_DELTA_DIST (0.143)	RED_SPEED (0.070)	S_REG_b_r (0.164)
4	S_CPA_TIME (0.143)	cpa_dist (0.033)	S_ACCEL (0.142)
5	S_SIDE (0.143)	S_REG_b_r (0.010)	S_DELTA_DIST (0.110)
6	S_REG_b_r (0.143)	DISTANCE (0.010)	S_CPA_TIME (0.104)
7	S_DELTA_DIST (0.143)	S_DREL_BEAR (0.006)	S_SIDE (0.100)
8	S_ACCEL (0.143)	BLUE_SPEED (0.006)	S_DREL_BEAR (0.094)
9	S_BEAM (0.143)	S_REG_r_b (0.006)	abs_dheading (0.077)
10	abs_dheading (0.142)	abs_dheading (0.005)	S_DABS_HEADING (0.077)
11	cpa_time (0.119)	cpa_time (0.005)	RED_SPEED (0.064)
12	rel_b_r (0.116)	S_CPA_TIME (0.005)	cpa_dist (0.060)
13	RED_SPEED (0.115)	S_DABS_HEADING (0.005)	cpa_time (0.056)
14	rel_r_b (0.115)	rel_r_b (0.004)	abs_r_b (0.052)
15	cpa_dist (0.110)	S_BEAM (0.004)	BLUE_HEADING (0.051)
16	r_accel (0.110)	rel_b_r (0.004)	abs_b_r (0.051)
17	abs_r_b (0.108)	r_accel (0.004)	DISTANCE (0.049)
18	abs_b_r (0.106)	abs_b_r (0.004)	RED_HEADING (0.048)
19	BLUE_HEADING (0.105)	RED_HEADING (0.004)	r_accel (0.047)
20	BLUE_SPEED (0.105)	S_CPA_DELTA_DIST (0.004)	BLUE_SPEED (0.046)
21	DISTANCE (0.102)	BASELINE (0.004)	S_CPA_DELTA_DIST (0.040)
22	RED_HEADING (0.061)	abs_r_b (0.003)	rel_r_b (0.039)
23	S_DABS_HEADING (0.003)	BLUE_HEADING (0.003)	rel_b_r (0.037)

5.4.3.1 Insights from Class-Specific Tables

Table 5.5: Class 0 to Class 3 This table presents the feature importance for the benign and three aggressive vessel behaviors: block, ram, and cross. The following trends are observed:

- **Class 0 (Benign)** shows a strong dependency on `S_ACCEL`, `S_CPA_TIME`, and `cpa_time`, indicating the importance of acceleration and time-to-CPA in identifying non-hostile behavior.
- **Class 1 (Block)** prioritizes spatial separation (`S_DELTA_DIST`) and closing speed (`DISTANCE`, `RED_SPEED`), aligning with the blocking pattern’s reliance on intercept geometry.
- **Class 2 (Ram)** emphasizes heading divergence (`S_DABS_HEADING`, `S_SIDE`), relative positioning (`abs_b_r`, `rel_r_b`), and linear alignment, which are critical for characterizing direct approach behaviors.
- **Class 3 (Cross)** highlights both interaction features (`S_CPA_TIME`, `S_DREL_BEAR`) and directional changes (`abs_dheading`), reflecting the complex geometry involved in vessel crossing scenarios.

Table 5.6: Class 4 to Class 6 This table focuses on the remaining classes: head-on, herd, and overtake.

- **Class 4 (Head-on)** is highly sensitive to orientation features such as `S_REG_r_b`, `S_DREL_BEAR`, and `S_CPA_DELTA_DIST`, which encode angular approach and relative bearing.
- **Class 5 (Herd)** emphasizes speed (`S_ACCEL`, `RED_SPEED`, `BLUE_SPEED`) and acceleration, reflecting the need to track dynamic motion consistency in group-following behaviors.
- **Class 6 (Overtake)** depends on a blend of relational positioning (`S_REG_b_r`, `S_REG_r_b`) and directional vectors (`S_DABS_HEADING`), CPA time increasing / decreasing (`S_CPA_TIME`), consistent with lane-switching or passing behavior.

Observation: While some features such as `S_CPA_TIME`, `S_ACCEL`, and `S_DELTA_DIST` are shared across multiple behaviors, others like `S_REG_r_b` or `S_SIDE` exhibit class-specific relevance.

5.4.3.2 Comparison with Global Feature Importance Models

LSTM Combined CPFI (Figure 5.2) : The global CPFI analysis of the LSTM model aggregates feature influence across all classes. Top-ranked features like `S_ACCEL`, `S_CPA_TIME`, and `S_DELTA_DIST` reappear consistently in the class-specific tables, reaffirming their global significance. However, the global ranking dilutes the role of behavior-specific features such as `S_DREL_BEAR` or `S_SIDE`, which are critical in more specialized classifiers.

Transformer-Based Feature Importance (Figure 5.5) : The Transformer model emphasizes a different set of features—most notably `rel_b_r`, `b_sp`, `S_reg_b_r`, and `dist`—which are centered on spatial interaction rather than motion dynamics. Unlike the LSTM-CPFI, which rewards time-sensitive variables, the Transformer relies on attention to extract positional relationships across sequences. Notably, features like `r_accel` and `b_head`, which have low ranking in both class-specific and global LSTM CPFI, are similarly deprioritized in the Transformer output.

5.4.3.3 Synthesis and Comparative Understanding

TABLE 5.7: Summary of Feature Importance Perspectives

Source	Primary Focus	Key Features
Class-Specific LSTM + CPFI	Behavior-sensitive temporal and relational cues that are uniquely relevant for specific intent classes. For example, <code>S_SIDE</code> and <code>S_DREL_BEAR</code> are particularly influential in recognizing behaviors like ramming or crossing.	<code>S_ACCEL</code> , <code>S_CPA_TIME</code> , <code>S_DREL_BEAR</code> , <code>S_SIDE</code>
Combined LSTM + CPFI	Globally influential kinematic features that contribute consistently across all behavioral categories, with particular emphasis on acceleration and time-to-collision indicators.	<code>S_ACCEL</code> , <code>S_CPA_TIME</code> , <code>S_DELTA_DIST</code>
Transformer Attention Importance	Focuses on spatial and relational geometry between agents using attention scores. Emphasizes spatial proximity and heading alignment through relative position-based features.	<code>rel_b_r</code> , <code>b_sp</code> , <code>S_reg_b_r</code> , <code>dist</code>

The class-specific tables provide a nuanced understanding of behavior-dependent feature salience. While the combined LSTM CPFI emphasizes overall temporal influence, and the Transformer model captures geometric interactions, the per-class

analysis reveals that effective behavior classification requires balancing both temporal dynamics and spatial awareness. These insights support the potential value of hybrid or ensemble models combining LSTM and Transformer paradigms for intent recognition in maritime scenarios. A future task for this is to train the per-class LSTM models with a feature subset selected from both the CPFI and TFIS for further analysis.

5.5 Conclusion and Future Work

In conclusion, this chapter presents a comprehensive framework for maritime intent recognition by combining innovative feature selection strategies with efficient deep learning models tailored for real-time deployment on resource-constrained platforms. The integration of Categorical Permutation Feature Importance (CPFI) and Transformer Feature Importance Score (TFIS) has proven to be highly effective in improving the interpretability and predictive accuracy of the classification models, particularly under challenging conditions such as class imbalance and noisy sensor inputs.

A detailed analysis of feature relevance was conducted using both global and class-specific perspectives. The LSTM-based Categorical Permutation Feature Importance (CPFI) revealed a consistent emphasis on motion dynamics and temporal features, including `S_ACCEL`, `S_CPA_TIME`, and `S_DELTA_DIST`, as critical across multiple behaviors. At the same time, class-specific CPFI analysis uncovered behavior-dependent feature importance, such as `S_SIDE`, `S_DREL_BEAR`, and `abs_dheading`—that do not appear

prominently in global rankings but are crucial for detecting specific hostile maneuvers like ramming, crossing, or overtaking.

In contrast, the Transformer-based attention model identified a different set of influential features, including `rel_b_r`, `b_sp`, `S_REG_B_R`, and `dist`, which emphasize spatial alignment, interaction geometry, and relative vessel configuration. This divergence in model sensitivity indicates that Transformer architectures are well-suited to extract spatial relational patterns, while LSTM models excel in leveraging temporal dynamics and CPA-based cues.

The combined findings suggest that both temporal and spatial representations are complementary in capturing maritime intent, and hybrid architectures that incorporate attention mechanisms with recurrent or convolutional sequence models may offer superior performance. Moreover, by embedding these optimized feature selection methods into Long Short-Term Memory (LSTM) and Fully Convolutional Deep Learning (FCDL) models, the framework demonstrates its robustness and suitability for real-time intent recognition aboard autonomous and semi-autonomous naval systems.

These advancements significantly enhance situational awareness by enabling early detection of a wide range of vessel behaviors—both hostile and benign—thus contributing to maritime safety and operational security. Future work will focus on expanding this framework to larger and more diverse datasets, investigating hybrid and ensemble learning architectures, and exploring domain adaptation strategies to generalize the CPFI and TFIS approaches across other dynamic, multi-agent domains.

Chapter 6

Classical Bayesian Filtering for Maritime Vessel Trajectory Prediction

6.1 Introduction

Maritime vessel trajectory prediction underpins a broad range of operational capabilities: collision avoidance and COLREGs-aware passage planning on autonomous surface vessels, traffic management in congested waterways, anomaly detection in port-approach corridors, and threat assessment for adversarial encounters. Each of these depends on forecasting where vessels will be over horizons of seconds to a few minutes, with accuracy that directly determines how early a warning is issued, how

much margin a planner has to negotiate a safe passing geometry, and how confidently an operator can dismiss a benign track or escalate a suspicious one.

Classical Bayesian filters have served this role for decades and define the baseline against which any new approach must be measured. Kalman filters underpin the embedded state estimators inside production trackers, the front-end smoothers feeding deep classifiers, and the fallback extrapolators that operational systems rely on when learned models cannot be deployed. This chapter establishes that baseline. The objective is to characterize the space of classical predictors precisely enough to identify which parts of the trajectory prediction problem they solve cleanly and which parts they do not, so that the design of the deep model in Chapter 7 is motivated against a measured envelope rather than an assumed one.

The vessel-tracking literature converges on a small family of motion models for this task. The Kalman Filter (KF) [98] is the optimal linear estimator under Gaussian noise; the Extended Kalman Filter (EKF) linearizes nonlinear dynamics via a first-order Taylor expansion [99]; and the Unscented Kalman Filter (UKF) propagates a deterministic set of sigma points through the exact nonlinear function [100]. The dynamics layered on top fall into three canonical categories: Constant Velocity (CV), Constant Acceleration (CA), and Coordinated Turn Rate and Velocity (CTRV), the last of which augments the state with an explicit turn-rate variable ω for circular-arc motion. Interacting Multiple Model (IMM) filters [102] combine several of these under a Markov transition matrix and perform well on non-adversarial maritime traffic [101];

they are not included in this comparison and are flagged as a natural extension in Section 6.5.7. A broader survey of these methods and their use in ship-tracking and surveillance is provided in Chapter 2; only the material required to follow the experimental design is presented here.

Classical Kalman filter design for maritime tracking remains an active area of research, and the contemporary literature provides relevant reference points against which the present comparison is positioned. Recent UKF refinements include geodetic-coordinate formulations that eliminate projection error on long-range tracks [38] and improved sigma-point parameterizations that sharpen short-horizon accuracy on slowly-maneuvering targets [39]. Sigma-point variants such as the Center Difference Kalman Filter retain accuracy under GNSS occlusion for unmanned surface vessel navigation while avoiding the analytical Jacobians required by the EKF [40]. On the multiple-model side, square-root cubature variants of the IMM stabilize the update step under aggressive maneuvering [41], and adaptive IMM schemes update transition probabilities online and accommodate one-step randomly delayed measurements for underwater maneuvering targets [42]. Production-grade IMM implementations tuned for cooperative targets [43] continue to set the practical bar for filter-based vessel tracking when vessels broadcast compliant AIS. Each of these refinements is algorithmic rather than structural: the underlying coverage of the filter family is still bounded by a finite library of analytic motion templates, with no representation of tactical scenario or vessel-pair interaction. The comparison reported in this chapter is restricted to the canonical KF/EKF/UKF building blocks that these refinements share.

Recent surveys document a shift in operational maritime systems from reactive rule-based monitoring toward intent-aware approaches for anomaly detection, dark-vessel tracking, and threat classification, alongside a persistent shortage of labeled threat and adversarial-behavior data at the volume needed to train modern deep classifiers [1]. In parallel, recent surveys of multimodal maritime perception argue that fusing radar, electro-optical/infrared imagery, AIS, and synthetic-aperture radar streams is necessary to overcome the limitations of AIS-only baselines, such as non-cooperative targets and signal gaps [9], and operational situational-awareness platforms such as GMSA [15] embed live tracking inside larger pattern-mining and anomaly-flagging stacks. Kalman-family filters are not displaced by these developments; they remain embedded state estimators within multimodal trackers and the front-end smoothers that feed deep classifiers. This chapter asks where the classical filter envelope ends and which adversarial scenario classes lie outside it; the answer determines what a learned model must add on top.

Generative simulation has begun to fill the labeled-encounter gap that surveys repeatedly identify. Closely related work has shown that synthetic vessel behaviors can substitute for rare real-world anomalies in detector training [128], and synthetic-traffic generators now produce AIS-like trajectories at scale, though typically without the intent annotations required for supervised behavior learning [16]. Public AIS-derived navigation databases constructed from VTS streams provide hierarchical traffic patterns at coarse behavioral granularity [18], and engineering-grade autonomous-ship digital twins prioritize single-vessel 6-DOF hydrodynamic and perception fidelity

over labeled multi-vessel encounter generation [12]. The simulator-driven labeled corpus used in this dissertation, on which both this chapter’s classical baselines and Chapter 7’s deep model are evaluated, addresses this gap by supplying per-encounter intent labels alongside trajectory data.

This chapter evaluates six predictors: a Constant Velocity (CV) baseline, a Standard Unscented Kalman Filter (UKF-4D), a UKF with a 5D constant-velocity state (UKF-5D-CV), an Asymmetric UKF with an acceleration state (UKF-CA-6D), an Extended Kalman Filter with the Constant Turn Rate and Velocity (EKF-CTRV) motion model, and a UKF using the same CTRV dynamics. These predictors are assessed across seven active scenario types and four experiment configurations that vary the observation window and prediction horizon: Experiment A ($T_{\text{obs}} = 40$, $T_{\text{pred}} = 20$), Experiment B ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$), Experiment C ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 10$), and Experiment D ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 20$).

Three properties of the design space prove to be decisive. First, whether the dynamics are genuinely nonlinear: when the dynamics are linear, sigma-point propagation reduces to the analytic Kalman update and the UKF provides no advantage over the closed-form matrix product. Second, whether every state variable is directly observable from the measurement model: when a state is not directly observable, its covariance grows during predict-only steps and the filter eventually becomes numerically unstable. Third, the prediction horizon at which a nonlinear arc model ceases to be advantageous, since the constant- ω assumption accumulates error faster than velocity extrapolation

does. The experimental results surface all three effects empirically and identify which scenarios remain out of reach for any single-motion-model predictor, motivating the intent-aware multi-task design developed in Chapter 7.

Research questions. The chapter is organized around one dissertation-level research question (RQ1) and three chapter-specific subquestions (RQ-KF1–3); each subquestion probes a targeted aspect of classical-filter behavior, and together their answers constitute the evidence for RQ1.

RQ1 (dissertation-level, shared with Chapter 7): *What are the structural limitations of intent-agnostic classical trajectory predictors on adversarial maritime vessel scenarios, and what properties must a successor model have to overcome them?*

1. **RQ-KF1.** *How does prediction accuracy vary across classical filter designs (linear versus nonlinear dynamics, 4D versus 5D versus 6D state) as a function of prediction horizon and scenario difficulty?*
2. **RQ-KF2.** *At what prediction horizon does a nonlinear CTRV motion model provide a genuine accuracy advantage over constant-velocity extrapolation, and where does that advantage reverse?*
3. **RQ-KF3.** *Under what combination of observation window length and prediction horizon does the UKF-CTRV become numerically unstable, and what is the underlying mechanism?*

6.2 Methods

All six methods operate on the same per-vessel state representation and share a common predict–update interface. Let k index discrete time steps with interval $\Delta t = 1$ s, and let $\mathbf{x}_k$ denote the hidden state at step k .

6.2.1 Constant Velocity Baseline

The CV predictor is the simplest trajectory extrapolator. For each vessel:

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \end{bmatrix} = \begin{bmatrix} x_k \\ y_k \end{bmatrix} + \begin{bmatrix} v_x \\ v_y \end{bmatrix} \Delta t \quad (6.1)$$

where (v_x, v_y) are fixed to the last observed velocity. No covariance is maintained. The CV predictor is included as a zero-cost lower bound: any filter that fails to outperform CV on the aggregate metric provides no practical benefit.

Strengths: Computationally trivial; maximally stable under straight or gently varying motion; no parameter tuning required.

Limitations: No uncertainty propagation; cannot model acceleration or turning behavior.

6.2.2 Standard UKF (4D)

The Unscented Kalman Filter propagates a set of deterministically chosen *sigma points* through the exact (possibly nonlinear) process model, then recovers the predicted mean and covariance via a weighted sum. For a state of dimension n , $2n + 1$ sigma points are used.

State vector. The hidden state stacks position and velocity in the plane:

$$\mathbf{x} = [x, y, v_x, v_y]^T \in \mathbb{R}^4 \quad (6.2)$$

Process model (linear CV). The state transition F advances position by velocity over one step and leaves velocity unchanged:

$$\mathbf{x}_{k+1} = F \mathbf{x}_k, \quad F = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6.3)$$

Sigma-point generation. Let P_k be the current state covariance and $\lambda = \alpha^2(n + \kappa) - n$ the composite scaling parameter. The $2n + 1$ sigma points are:

$$\chi_k^{(0)} = \mathbf{x}_k, \quad \chi_k^{(i)} = \mathbf{x}_k \pm \sqrt{(n + \lambda) P_k} \big|_i, \quad i = 1, \dots, n \quad (6.4)$$

Predict step. Each sigma point is propagated through F and the predicted mean and covariance are recovered as weighted sums:

$$\hat{\chi}_{k+1|k}^{(i)} = F \chi_k^{(i)} \quad (6.5)$$

$$\hat{\mathbf{x}}_{k+1|k} = \sum_i W_m^{(i)} \hat{\chi}_{k+1|k}^{(i)} \quad (6.6)$$

$$P_{k+1|k} = \sum_i W_c^{(i)} (\hat{\chi}^{(i)} - \hat{\mathbf{x}})(\hat{\chi}^{(i)} - \hat{\mathbf{x}})^T + Q \quad (6.7)$$

where $W_m^{(i)}$ and $W_c^{(i)}$ are the mean and covariance weights, and Q is the process noise covariance.

Update step. The measurement $\mathbf{z}_k = [x, y, v_x, v_y]^T$ is a direct observation of the full state. The Kalman gain K and posterior state are:

$$K = P_{k+1|k} H^T (H P_{k+1|k} H^T + R)^{-1}, \quad \mathbf{x}_{k+1} = \hat{\mathbf{x}}_{k+1|k} + K(\mathbf{z}_{k+1} - H \hat{\mathbf{x}}_{k+1|k}) \quad (6.8)$$

where $H = I_4$ (identity) and R is the measurement noise covariance.

Why the Standard UKF is not genuinely nonlinear. Because the underlying dynamics are linear (Equation 6.3), the sigma-point weighted sum reduces exactly to the matrix product $F \hat{\mathbf{x}}$, identical to a plain KF. The sigma-point propagation incurs computational overhead without providing accuracy gains over the CV baseline. Its sole practical benefit is the principled covariance estimate P_k , useful for downstream uncertainty-aware decision systems.

6.2.3 UKF-5D-CV: UKF with 5D State and CV Dynamics

The UKF-5D-CV uses the same 5D state as the CTRV filters ($[p_x, p_y, v_x, v_y, \omega]^T$) but replaces the CTRV arc dynamics with linear constant-velocity dynamics. The position and velocity components propagate identically to the 4D UKF; ω advances as a random walk under process noise without influencing position. This method serves as a structural control: it isolates the effect of adding the turn-rate state dimension from the effect of using nonlinear CTRV dynamics.

Because the position–velocity dynamics are linear, sigma-point propagation again reduces to the analytic Kalman update for those components. In practice, UKF-5D-CV is virtually indistinguishable from CV and UKF-4D across all configurations (ADE difference $< 0.3\%$). The ω state does not contribute to prediction accuracy when driven by CV dynamics, confirming that additional state dimensions provide no benefit without matching dynamics.

6.2.4 Asymmetric UKF (6D)

The Asymmetric UKF extends the state to include acceleration and introduces vessel-specific noise parameters to reflect the asymmetric observability inherent in two-vessel encounters.

State vector. The state is augmented with acceleration components in both axes:

$$\mathbf{x} = [x, y, v_x, v_y, a_x, a_y]^T \in \mathbb{R}^6 \quad (6.9)$$

Process model (constant acceleration). The transition matrix integrates acceleration into velocity and position over one step:

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 & \frac{1}{2}\Delta t^2 & 0 \\ 0 & 1 & 0 & \Delta t & 0 & \frac{1}{2}\Delta t^2 \\ 0 & 0 & 1 & 0 & \Delta t & 0 \\ 0 & 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (6.10)$$

Asymmetric noise design. In a two-vessel encounter, one vessel (the red vessel, with measured acceleration available) may exhibit higher kinematic observability than the other (the blue vessel). Vessel-specific noise covariances encode this asymmetry:

- **Red vessel** (higher observability): lower process noise $Q_r = (0.05)^2 I$ and lower measurement noise $R_r = (0.02)^2 I$.
- **Blue vessel** (lower observability): higher process noise $Q_b = (0.15)^2 I$ and higher measurement noise $R_b = (0.08)^2 I$.

Limitations: The added acceleration state amplifies prediction error under the constant-acceleration assumption when vessels undergo aggressive maneuver changes. In hostile scenarios, the red vessel’s measured acceleration creates a high-confidence but wrong arc; this phenomenon is detailed in Section 6.5.4.

6.2.5 EKF with CTRV (5D)

The Extended Kalman Filter with the Coordinated Turn Rate and Velocity model is the first genuinely *nonlinear* filter in this evaluation. It adds a turn-rate state ω and propagates position along a circular arc.

State vector. The state appends a turn-rate component to the planar position and velocity:

$$\mathbf{x} = [p_x, p_y, v_x, v_y, \omega]^T \in \mathbb{R}^5 \quad (6.11)$$

where ω (rad/s) is the instantaneous turn rate.

Turn-rate initialization from heading. The raw dataset contains compass heading ψ (degrees). Turn rate is estimated using central differences:

$$\omega[t] = \frac{\text{wrap}(\psi[t+1] - \psi[t-1])}{2 \Delta t} \quad (6.12)$$

where $\text{wrap}(\cdot)$ maps angle differences to $[-\pi, \pi]$, correctly handling the $0^\circ/360^\circ$ boundary.

Nonlinear state transition. The CTRV dynamics propagate position along a circular arc when the turn rate is non-negligible and degenerate to straight-line CV otherwise. *Turning case* ($|\omega| > 10^{-4}$ rad/s):

$$p'_x = p_x + \frac{v_x \sin(\omega \Delta t) + v_y (1 - \cos(\omega \Delta t))}{\omega} \quad (6.13)$$

$$p'_y = p_y + \frac{v_y \sin(\omega \Delta t) - v_x (1 - \cos(\omega \Delta t))}{\omega} \quad (6.14)$$

$$v'_x = v_x \cos(\omega \Delta t) - v_y \sin(\omega \Delta t) \quad (6.15)$$

$$v'_y = v_x \sin(\omega \Delta t) + v_y \cos(\omega \Delta t) \quad (6.16)$$

$$\omega' = \omega \quad (6.17)$$

Straight-line limit ($|\omega| \leq 10^{-4}$ rad/s):

$$p'_x = p_x + v_x \Delta t, \quad p'_y = p_y + v_y \Delta t, \quad v'_x = v_x, \quad v'_y = v_y, \quad \omega' = \omega \quad (6.18)$$

EKF linearization: Jacobian. Because $f(\mathbf{x})$ is nonlinear, the EKF propagates covariance through the first-order Taylor approximation $F = \partial f / \partial \mathbf{x}$. With $s = \sin(\omega \Delta t)$ and $c = \cos(\omega \Delta t)$:

$$F = \begin{bmatrix} 1 & 0 & \frac{s}{\omega} & \frac{1-c}{\omega} & \frac{v_x(\omega\Delta t c - s) + v_y(\omega\Delta t s - (1-c))}{\omega^2} \\ 0 & 1 & -\frac{1-c}{\omega} & \frac{s}{\omega} & \frac{v_y(\omega\Delta t c - s) - v_x(\omega\Delta t s - (1-c))}{\omega^2} \\ 0 & 0 & c & s & \Delta t (v_y c - v_x s) \\ 0 & 0 & -s & c & -\Delta t (v_x c + v_y s) \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (6.19)$$

This Jacobian is computed analytically at each prediction step.

Measurement model. Measurements $\mathbf{z} = [p_x, p_y, v_x, v_y]^T$ are a linear function of the state:

$$\mathbf{z} = H\mathbf{x}, \quad H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad (6.20)$$

Predict–update cycle. The filter alternates a nonlinear state propagation with a linear measurement-driven correction. *Predict:*

$$\hat{\mathbf{x}}_{k|k-1} = f(\mathbf{x}_{k-1}, \Delta t), \quad P_{k|k-1} = F P_{k-1} F^T + Q \quad (6.21)$$

Update:

$$\mathbf{y} = \mathbf{z}_k - H\hat{\mathbf{x}}_{k|k-1} \quad (6.22)$$

$$S = HP_{k|k-1}H^T + R \quad (6.23)$$

$$K = P_{k|k-1}H^TS^{-1} \quad (6.24)$$

$$\mathbf{x}_k = \hat{\mathbf{x}}_{k|k-1} + K\mathbf{y}, \quad P_k = (I - KH)P_{k|k-1} \quad (6.25)$$

Default noise parameters. Process and measurement noise covariances are fixed to the heuristic defaults listed in Table 6.1 and shared across all scenarios.

TABLE 6.1: EKF/CTRV default noise parameters.

Parameter	Symbol	Value	Rationale
Position process noise	$\sigma_{p,Q}$	0.50 m	Unmodeled positional disturbances
Velocity process noise	$\sigma_{v,Q}$	0.30 m/s	Speed fluctuations
Turn-rate process noise	$\sigma_{\omega,Q}$	0.05 rad/s	Moderate maneuver changes per step
Position meas. noise	$\sigma_{p,R}$	0.20 m	GPS/AIS position noise
Velocity meas. noise	$\sigma_{v,R}$	0.10 m/s	Speed sensor noise

6.2.6 UKF with CTRV (5D)

The UKF-CTRV uses the same 5D state and nonlinear CTRV dynamics as the EKF (Section 6.2.5) but replaces Jacobian-based linearization with Merwe scaled sigma-point

propagation. This eliminates the first-order truncation error in covariance propagation at the cost of evaluating the dynamics at $2n + 1 = 11$ sigma points per step.

Sigma-point generation (Merwe scaled). The scaled sigma-point parameterization follows Wan and van der Merwe [148, 149]. With state dimension $n = 5$ and parameters $\alpha = 10^{-3}$, $\beta = 2$, $\kappa = 0$:

$$\lambda = \alpha^2(n + \kappa) - n, \quad \sigma_0 = \hat{\mathbf{x}}, \quad \sigma_i = \hat{\mathbf{x}} \pm \sqrt{(n + \lambda)P} \mathbf{e}_i, \quad i = 1, \dots, n \quad (6.26)$$

The tight $\alpha = 10^{-3}$ keeps sigma points closely clustered around the mean, giving $\sqrt{n + \lambda} \approx 0.0022$, a deliberate design to limit sigma-point spread in the unobserved ω dimension.

Omega threshold difference. The CTRV dynamics switch between arc and straight-line branches at a threshold ω_{thr} . The EKF uses $\omega_{\text{thr}} = 10^{-4}$ rad/s; the UKF-CTRV uses a wider threshold of 10^{-3} rad/s. This ensures that all 11 sigma points fall on the same branch when the mean ω is near zero, preventing geometrically inconsistent weighted means.

EKF vs. UKF-CTRV comparison. The two CTRV filters differ only in how they propagate covariance through the nonlinear dynamics; Table 6.2 contrasts their structural properties.

TABLE 6.2: Comparison of EKF-CTRV and UKF-CTRV for the 5D CTRV model.

Aspect	EKF-CTRV	UKF-CTRV
Nonlinearity handling	Jacobian (1st-order Taylor)	11 sigma points through f
Linearization error	Yes, $O(\Delta \mathbf{x}^2)$ truncation	None (exact to 2nd order)
Jacobian required	Yes (5×5 , analytical)	No
ω threshold	10^{-4} rad/s	10^{-3} rad/s (wider)
Sigma-point branching	Not applicable	Mitigated by wide threshold
Constant- ω forecast	Yes	Yes (identical limitation)
Long-obs stability	Stable (single-point evaluation)	Diverges at $T_{\text{obs}} \geq 40$ or $T_{\text{pred}} \geq 20$
Computational cost	Lower (one f per step)	Higher (11 f per step)

Why UKF-CTRV diverges and EKF-CTRV does not. ω is never directly measured; P_ω grows by $\sigma_{\omega,Q}^2$ each predict step with no measurement update to shrink it. After ~ 15 – 20 steps the sigma-point spread in the ω dimension crosses the branching threshold: some sigma points propagate via arc dynamics, others via straight-line CV. Their weighted mean is geometrically inconsistent, inflating P in the position and velocity dimensions. Wider P further widens the sigma-point spread, a positive feedback loop that drives the filter to diverge. The EKF evaluates f at exactly one point (the current mean); if that mean has $|\omega| < 10^{-4}$, only the straight-line branch is used and no branch mixing occurs. The EKF is therefore structurally immune to this instability.

6.3 Evaluation Setup

The six filters defined in Section 6.2 are evaluated on a common dataset, observation/prediction split, and metric set so that performance differences reflect filter design rather than data preparation. This section fixes the dataset, the four experiment configurations that vary observation length and prediction horizon, the per-filter feature inputs, and the displacement metrics used throughout Section 6.4.

6.3.1 Dataset

All experiments use the *Noiseless_Dataset_1* two-vessel encounter dataset, the same dataset used in Chapters 4 and 7. The dataset covers seven active encounter scenario types (Table 1.1); the stationary class is excluded from all training and evaluation splits.

6.3.2 Experiment Configurations

Four experiment configurations are evaluated to assess the joint effects of observation window length and forecast depth:

- **Experiment A (long-obs, long-horizon):** $T_{\text{obs}} = 40$, $T_{\text{pred}} = 20$, stride = 10.

Stresses filters with a long observation warm-up and a 20-step forecast rollout.

- **Experiment B (short-horizon):** $T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$, stride = 10. Matches the MTITP evaluation settings (Chapter 7), enabling direct cross-model comparison.
- **Experiment C (medium-horizon):** $T_{\text{obs}} = 20$, $T_{\text{pred}} = 10$, stride = 10. Intermediate configuration to assess prediction error growth with horizon depth at the same observation length as Experiment B.
- **Experiment D (long-horizon, short-obs):** $T_{\text{obs}} = 20$, $T_{\text{pred}} = 20$, stride = 10. Isolates the effect of prediction horizon on filter stability: the observation window is identical to Experiments B and C, but the prediction rollout is extended to 20 steps.

Scenarios are partitioned 70/15/15 (train/validation/test) by encounter ID with seed 42. All results are reported on the natural test-split distribution; no per-scenario re-balancing is applied.

6.3.3 Feature Preparation

The state inputs supplied to each filter family differ in dimension and in which derived kinematic quantities are computed from the raw track:

- **4D methods** (CV, Standard UKF): state (p_x, p_y, v_x, v_y) at each timestep.

- **6D method** (Asymmetric UKF): state $(p_x, p_y, v_x, v_y, a_x, a_y)$ with acceleration from velocity finite differences.
- **5D methods** (EKF/CTRV, UKF-CTRV): state $(p_x, p_y, v_x, v_y, \omega)$ with turn rate from heading central differences (Equation 6.12).

6.3.4 Evaluation Metrics

Let $\hat{\mathbf{p}}_t^{(v)} \in \mathbb{R}^2$ denote the predicted position of vessel $v \in \{1, 2\}$ at forecast step $t \in \{1, \dots, T_{\text{pred}}\}$ for window $i \in \{1, \dots, N\}$, and let $\mathbf{p}_t^{(v)}$ denote the corresponding ground-truth position. All metrics are expressed in meters.

Average Displacement Error (ADE) is the mean Euclidean error across all forecast steps, both vessels, and all test windows:

$$\text{ADE} = \frac{1}{2NT_{\text{pred}}} \sum_{i=1}^N \sum_{v=1}^2 \sum_{t=1}^{T_{\text{pred}}} \left\| \hat{\mathbf{p}}_{i,t}^{(v)} - \mathbf{p}_{i,t}^{(v)} \right\|_2. \quad (6.27)$$

Final Displacement Error (FDE) measures error at the forecast endpoint and is most sensitive to long-horizon drift:

$$\text{FDE} = \frac{1}{2N} \sum_{i=1}^N \sum_{v=1}^2 \left\| \hat{\mathbf{p}}_{i,T_{\text{pred}}}^{(v)} - \mathbf{p}_{i,T_{\text{pred}}}^{(v)} \right\|_2. \quad (6.28)$$

Horizon-wise MAE and RMSE resolve error growth with forecast depth by holding t fixed:

$$\text{MAE}(t) = \frac{1}{2N} \sum_{i=1}^N \sum_{v=1}^2 \left\| \hat{\mathbf{p}}_{i,t}^{(v)} - \mathbf{p}_{i,t}^{(v)} \right\|_2, \quad \text{RMSE}(t) = \sqrt{\frac{1}{2N} \sum_{i=1}^N \sum_{v=1}^2 \left\| \hat{\mathbf{p}}_{i,t}^{(v)} - \mathbf{p}_{i,t}^{(v)} \right\|_2^2}. \quad (6.29)$$

ADE and FDE summarize aggregate accuracy; horizon MAE and RMSE expose the shape of the error curve and identify the step at which one filter family overtakes another. Per-scenario tables apply Equation 6.27 restricted to windows of a single encounter type, and per-vessel tables restrict the sum over v . All metrics are computed in the original metric coordinate frame; no normalization is applied at evaluation time.

6.4 Results

Results are organized by experiment configuration. Section 6.4.1 reports the long-observation, long-horizon setting (Experiment A) that exposes filter behavior under maximum unobserved-propagation budget. Sections 6.4.2–6.4.4 sweep the prediction horizon at fixed $T_{\text{obs}} = 20$ to isolate the effect of forecast depth on each filter family. Aggregate horizon-wise error growth and per-vessel performance are reported jointly across all four configurations in Sections 6.4.6 and 6.4.7; qualitative trajectory analysis follows in Section 6.4.8. Two patterns are flagged in advance because they recur in every configuration: (i) the three linear-dynamics filters (CV, UKF-4D, UKF-5D-CV)

cluster within 0.7% of each other on aggregate ADE, and (ii) the hostile scenarios (block, ram, herd) set an error floor that no filter design moves.

6.4.1 Experiment A: Long-Horizon (obs=40, pred=20)

Table 6.3 summarizes aggregate ADE and FDE for all six methods under Experiment A.

TABLE 6.3: Overall trajectory prediction performance: Experiment A ($T_{\text{obs}} = 40$, $T_{\text{pred}} = 20$). † numerical divergence, not model mismatch (see Section 6.5.2).

Model	State	ADE (m)	FDE (m)	Dynamics
Constant Velocity	4D	34.79	79.82	Linear (CV)
UKF-5D-CV	5D	34.90	79.87	Linear (CV)
Standard UKF	4D	35.01	79.90	Linear (CV)
EKF / CTRV	5D	37.47	80.40	Nonlinear (CTRV)
UKF-CTRV†	5D	53.32	140.43	Nonlinear (CTRV)
Asymmetric UKF	6D	57.50	152.27	Linear (CA)

CV, UKF-5D-CV, and Standard UKF cluster at 34.79–35.01 m ADE ($< 0.7\%$ spread).

EKF-CTRV is at 37.47 m and Asymmetric UKF at 57.5 m. UKF-CTRV reports 53.3 m, but this value is affected by numerical divergence (Section 6.5.2).

Table 6.4 shows the scenario-level breakdown for all six models.

TABLE 6.4: Per-scenario ADE (meters): Experiment A. **Bold** = best non-divergent model per scenario. UKF-CTRV values affected by divergence (see Section 6.5.2). UKF-5D-CV is omitted from per-scenario tables (statistically indistinguishable from CV at every scenario, $\Delta\text{ADE} < 0.05$ m); summary Table 6.11 reports its overall metrics.

Scenario	Type	CV	UKF-4D	EKF-CTRV	UKF-CTRV	Asym-6D
benign	Non-Hostile	0.078	0.106	0.077	46.981	0.148
overtake	Non-Hostile	0.247	0.266	0.276	43.805	0.430
cross	Non-Hostile	1.244	1.272	1.433	43.221	1.879
headon	Non-Hostile	0.523	0.562	0.532	43.399	0.794
block	Hostile	7.298	7.179	7.528	20.456	16.060
ram	Hostile	12.370	11.748	43.909	76.092	30.673
herd	Hostile	84.974	85.666	88.270	79.437	136.862

EKF-CTRV leads on benign (0.077 m); CV/UKF-4D lead on ram (11.7 m) and herd (85.0 m). Asymmetric UKF is worst on herd (136.9 m) and block (16.1 m). UKF-CTRV values reflect divergence rather than capability and are not directly comparable.

6.4.2 Experiment B: Short-Horizon (obs=20, pred=5)

Table 6.5 presents the Experiment B results for all classical filters. All results are on the natural test-split distribution. Comparison with the MTITP deep model is presented in Chapter 7 (Section 7.8.1).

TABLE 6.5: Overall trajectory prediction performance: Experiment B ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$). **Bold** = best per column.

Model	State	ADE (m)	FDE (m)	Dynamics
EKF / CTRV	5D	5.06	10.52	Nonlinear (CTRV)
Constant Velocity	4D	5.20	10.17	Linear (CV)
UKF-5D-CV	5D	5.29	10.27	Linear (CV)
Standard UKF	4D	5.40	10.41	Linear (CV)
UKF-CTRV	5D	5.40	11.43	Nonlinear (CTRV)
Asymmetric UKF	6D	5.88	12.10	Linear (CA)

Table 6.6 shows the per-scenario breakdown for Experiment B.

TABLE 6.6: Per-scenario ADE (meters): Experiment B. **Bold** = best model per scenario. UKF-5D-CV and Asym-UKF-6D omitted (UKF-5D-CV is indistinguishable from CV; Asym-6D is consistently worst across all scenarios; both are fully tabulated in summary Table 6.11).

Scenario	Type	CV	UKF-4D	EKF-CTRV	UKF-CTRV
benign	Non-Hostile	0.054	0.059	0.052	1.140
overtake	Non-Hostile	0.080	0.083	0.082	1.110
cross	Non-Hostile	0.195	0.201	0.208	1.202
headon	Non-Hostile	0.102	0.109	0.103	1.147
block	Hostile	0.988	1.070	0.971	1.248
ram	Hostile	3.971	3.883	6.719	7.501
herd	Hostile	12.687	13.182	11.993	11.670

EKF-CTRV leads on benign (0.052 m) and block (0.971 m); CV leads on cross (0.195 m) and headon (0.102 m). EKF ram error is 6.7 m versus 4.0 m for CV. Herd ADE remains above 11.6 m for every filter. UKF-CTRV is stable at this horizon (5.4 m).

6.4.3 Experiment C: Medium-Horizon (obs=20, pred=10)

Table 6.7 shows aggregate performance at Experiment C ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 10$), the intermediate horizon.

TABLE 6.7: Overall trajectory prediction performance: Experiment C ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 10$). † partial divergence (see text).

Model	State	ADE (m)	FDE (m)	Dynamics
Constant Velocity	4D	12.98	28.79	Linear (CV)
UKF-5D-CV	5D	13.08	28.92	Linear (CV)
Standard UKF	4D	13.23	29.08	Linear (CV)
EKF / CTRV	5D	14.31	33.45	Nonlinear (CTRV)
UKF-CTRV†	5D	16.26	39.72	Nonlinear (CTRV)
Asymmetric UKF	6D	17.10	41.55	Linear (CA)

TABLE 6.8: Per-scenario ADE (meters): Experiment C. **Bold** = best model per scenario. UKF-5D-CV and Asym-UKF-6D omitted (see Table 6.6 caption); summary Table 6.11 tabulates all six models.

Scenario	Type	CV	UKF-4D	EKF-CTRV	UKF-CTRV
benign	Non-Hostile	0.061	0.074	0.060	6.342
overtake	Non-Hostile	0.124	0.132	0.135	6.054
cross	Non-Hostile	0.449	0.463	0.510	6.193
headon	Non-Hostile	0.197	0.213	0.208	6.156
block	Hostile	2.881	2.935	2.887	4.618
ram	Hostile	6.346	6.066	18.190	22.727
herd	Hostile	31.816	32.454	33.906	32.029

UKF-CTRV benign ADE rises to 6.3 m, indicating partial divergence at this horizon. EKF-CTRV ram error grows to 18.2 m versus 6.1 m for UKF-4D. CV herd ADE rises to 31.8 m, roughly $2.5\times$ the value at $T_{\text{pred}} = 5$.

6.4.4 Experiment D: Long-Prediction, Short-Obs (obs=20, pred=20)

Table 6.9 shows aggregate performance at Experiment D ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 20$), which isolates the effect of prediction horizon by holding T_{obs} fixed at 20.

TABLE 6.9: Overall trajectory prediction performance: Experiment D ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 20$). † numerical divergence triggered by long prediction rollout.

Model	State	ADE (m)	FDE (m)	Dynamics
Constant Velocity	4D	34.28	78.64	Linear (CV)
UKF-5D-CV	5D	34.38	78.69	Linear (CV)
Standard UKF	4D	34.49	78.72	Linear (CV)
EKF / CTRV	5D	37.74	80.58	Nonlinear (CTRV)
UKF-CTRV†	5D	53.25	140.87	Nonlinear (CTRV)
Asymmetric UKF	6D	56.67	150.06	Linear (CA)

TABLE 6.10: Per-scenario ADE (meters): Experiment D. **Bold** = best non-divergent model per scenario. UKF-5D-CV and Asym-UKF-6D omitted (see Table 6.6 caption); summary Table 6.11 tabulates all six models.

Scenario	Type	CV	UKF-4D	EKF-CTRV	UKF-CTRV
benign	Non-Hostile	0.077	0.105	0.076	46.856
overtake	Non-Hostile	0.244	0.263	0.291	43.965
cross	Non-Hostile	1.242	1.270	1.489	43.081
headon	Non-Hostile	0.505	0.543	0.561	43.506
block	Hostile	7.239	7.123	7.522	20.622
ram	Hostile	11.783	11.191	46.905	74.913
herd	Hostile	84.488	85.176	89.289	79.348

UKF-CTRV ADE in Experiment D is 53.3 m, comparable to Experiment A and substantially higher than Experiments B (5.4 m) and C (16.3 m), even though T_{obs} is held at 20. The divergence is driven by the 20-step prediction rollout rather than by the observation window; the mechanism is analyzed in Section 6.5.2.

6.4.5 Cross-Horizon Summary

Table 6.11 pulls Experiments B, C, and D together at fixed $T_{\text{obs}} = 20$ so that each model can be read as an ADE curve over $T_{\text{pred}} \in \{5, 10, 20\}$.

TABLE 6.11: Cross-model comparison of classical Kalman filter baselines at $T_{\text{obs}} = 20$ across three prediction horizons. ADE columns correspond to Experiments B, C, and D; $t=1$ MAE is reported as a representative single-step value. $\dagger$ = numerical divergence; **Bold** = best per column among non-divergent models.

Model	$t=1$ MAE	ADE (m), $T_{\text{obs}} = 20$		
	(m)	$T_{\text{pred}}=5$	$T_{\text{pred}}=10$	$T_{\text{pred}}=20$
CV	1.03	5.20	12.98	34.28
UKF-5D-CV	1.08	5.29	13.08	34.38
UKF-4D	1.14	5.40	13.23	34.49
EKF-CTRV	0.89	5.06	14.31	37.74
UKF-CTRV	0.88	5.40	16.26	53.25 [†]
Asym-UKF-6D	1.05	5.88	17.10	56.67

The three linear-dynamics filters (CV, UKF-4D, UKF-5D-CV) stay within 0.7% of each other at every horizon. EKF-CTRV leads at $t=1$ and at $T_{\text{pred}}=5$ and is overtaken by CV at $T_{\text{pred}}=10$. UKF-CTRV transitions from stable at $T_{\text{pred}}=5$ to fully divergent at $T_{\text{pred}}=20$. Asym-UKF-6D is last at every horizon. Two patterns underlie these numbers: a long-horizon divergence specific to UKF-CTRV (Section 6.5.2), and a split between non-hostile and hostile scenarios that limits all six filters (Section 6.5.6).

6.4.6 Horizon-Wise Error Growth

Table 6.12 shows error growth under Experiment A and Table 6.13 under Experiment B.

TABLE 6.12: Horizon-wise MAE (meters): Experiment A ($T_{\text{pred}} = 20$).

Step t	CV	UKF-4D	EKF-CTRV	UKF-CTRV
1	1.050	1.163	0.912	0.898
5	10.374	10.621	10.457	11.559
10	29.346	29.645	33.100	39.981
20	79.823	79.899	80.398	140.433

TABLE 6.13: Horizon-wise MAE (meters): Experiment B ($T_{\text{pred}} = 5$).

Step t	CV	UKF-4D	EKF-CTRV	UKF-CTRV
1	1.031	1.143	0.892	0.884
2	2.654	2.855	2.278	2.335
3	4.795	5.033	4.426	4.650
4	7.349	7.569	7.192	7.694
5	10.172	10.412	10.520	11.428

EKF-CTRV leads at $t = 1$ in both configurations (0.912 m and 0.892 m). CV is ahead by $t = 5$ in Experiment A and by $t = 10$ (29.3 vs. 33.1 m). The EKF/CV crossover occurs at $t \approx 4$ –5 in both configurations. UKF-CTRV in Experiment A grows from 40.0 m at $t = 10$ to 140.4 m at $t = 20$ (Section 6.5.2).

6.4.7 Per-Vessel Analysis (Experiment A)

Table 6.14 shows the error decomposed by vessel role under Experiment A. The breakdown is reported only for Experiment A because the long-horizon configuration produces the largest absolute role gap; the per-vessel ordering is qualitatively the same in Experiments B–D.

TABLE 6.14: ADE decomposed by vessel role (Experiment A, aggregated over all scenarios).

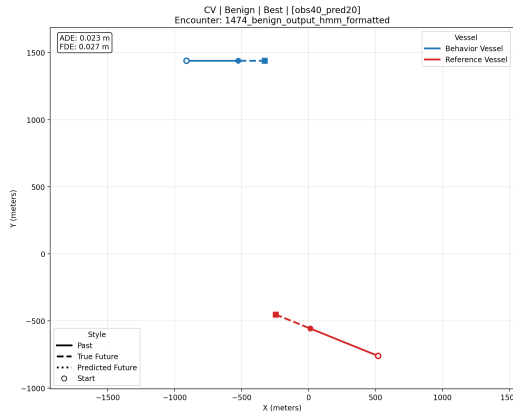
Vessel	CV	UKF-4D	EKF-CTRV	UKF-CTRV
Blue (behavior)	18.991	19.067	24.205	38.922
Red (reference)	50.598	50.958	50.725	67.712

The reference (red) vessel error is roughly $2.7\times$ the behavior (blue) vessel error for CV (50.6 vs. 19.0 m). EKF-CTRV reports the highest blue-vessel error (24.2 m, 27% above UKF-4D).

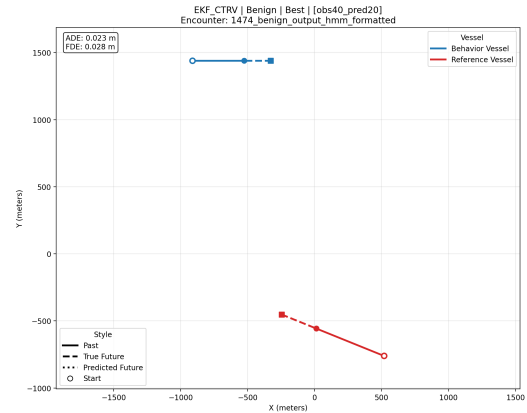
6.4.8 Qualitative Trajectory Analysis

The figures in this section show $T_{\text{obs}} = 40$, $T_{\text{pred}} = 20$ forecasts (Experiment A) on representative encounters drawn from the per-scenario best/median/worst pools generated during evaluation. In every plot, the past observation is solid, the true future is dashed, and the predicted future is dotted; ADE/FDE values are case-specific.

Benign case. Figure 6.1 shows a benign encounter. CV and EKF-CTRV both produce sub-meter ADE: the underlying motion is close to the model’s assumed dynamics, and the predicted (dotted) line tracks the true future (dashed) for both vessels.



(A) Benign, CV (best case).



(B) Benign, EKF/CTRV (best case).

FIGURE 6.1: Benign-scenario forecasts. Both filters track the ground truth to within a meter over twenty steps; the EKF’s arc model provides no measurable advantage when the underlying motion is straight-line.

Cross, headon, overtake. Figure 6.2 shows median-case CV forecasts on three encounter geometries. The geometry imposes a kinematic constraint, but neither vessel deviates from its heading enough to break the constant-velocity assumption within a 20-step window. The predicted trajectory tracks the true future closely in each panel.

Ram: two failure geometries. Figure 6.3 shows worst-case ram forecasts under CV and EKF-CTRV. CV (left, ADE 101.6 m) extrapolates straight past the turn point and continues on the original heading. EKF-CTRV (right, ADE 207.3 m) integrates a

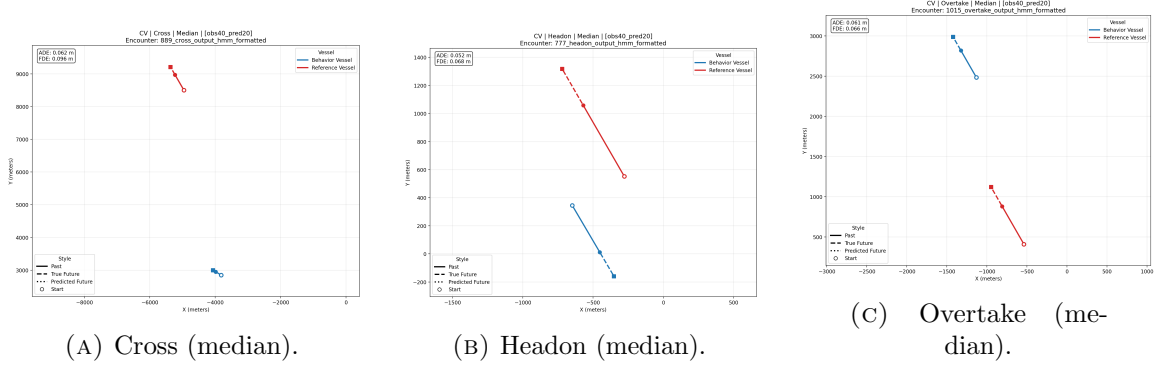


FIGURE 6.2: Median-difficulty CV forecasts on three encounter geometries. Persistent heading and speed produce sub-meter to low single-digit ADE despite the encounter classification; linear extrapolation is sufficient when neither vessel maneuvers mid-window.

small noisy ω into a tight predicted loop over twenty steps. The two filters miss in geometrically opposite directions on the same scenario class.

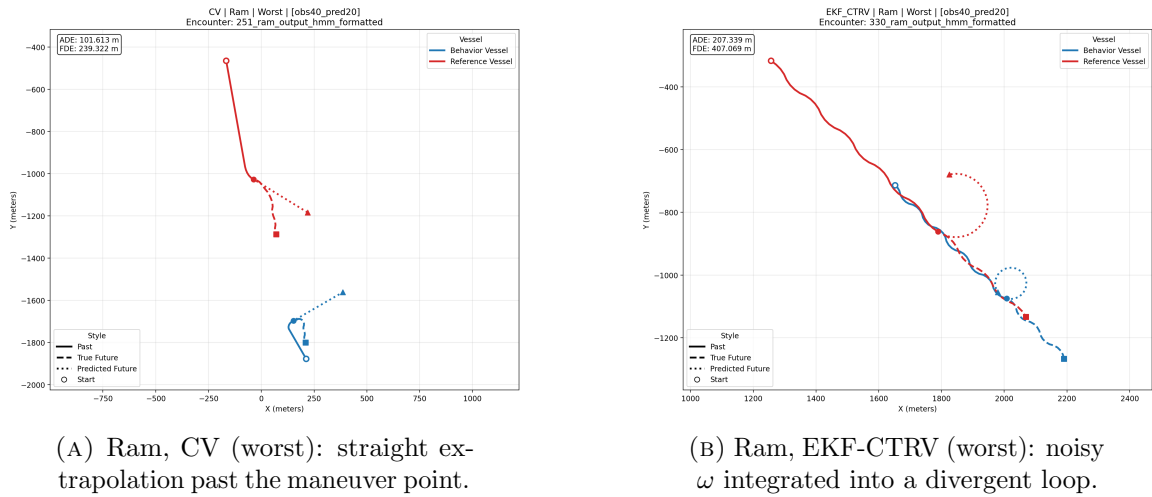


FIGURE 6.3: Two ways to be wrong on the same scenario class (ram). The geometry of each predicted trajectory exposes the underlying motion model: CV produces straight lines that overshoot, the EKF-CTRV produces arcs that curl away from the true future.

Herd worst case. Figure 6.4 shows a worst-case herd forecast under CV: 245.7 m ADE, with both vessels following looping ground-truth paths while the predicted line

extrapolates straight. A single-mode filter produces one point prediction, so when the future distribution is multi-modal a coordinated lateral maneuver shows up as a large displacement error rather than as multiple candidate futures.

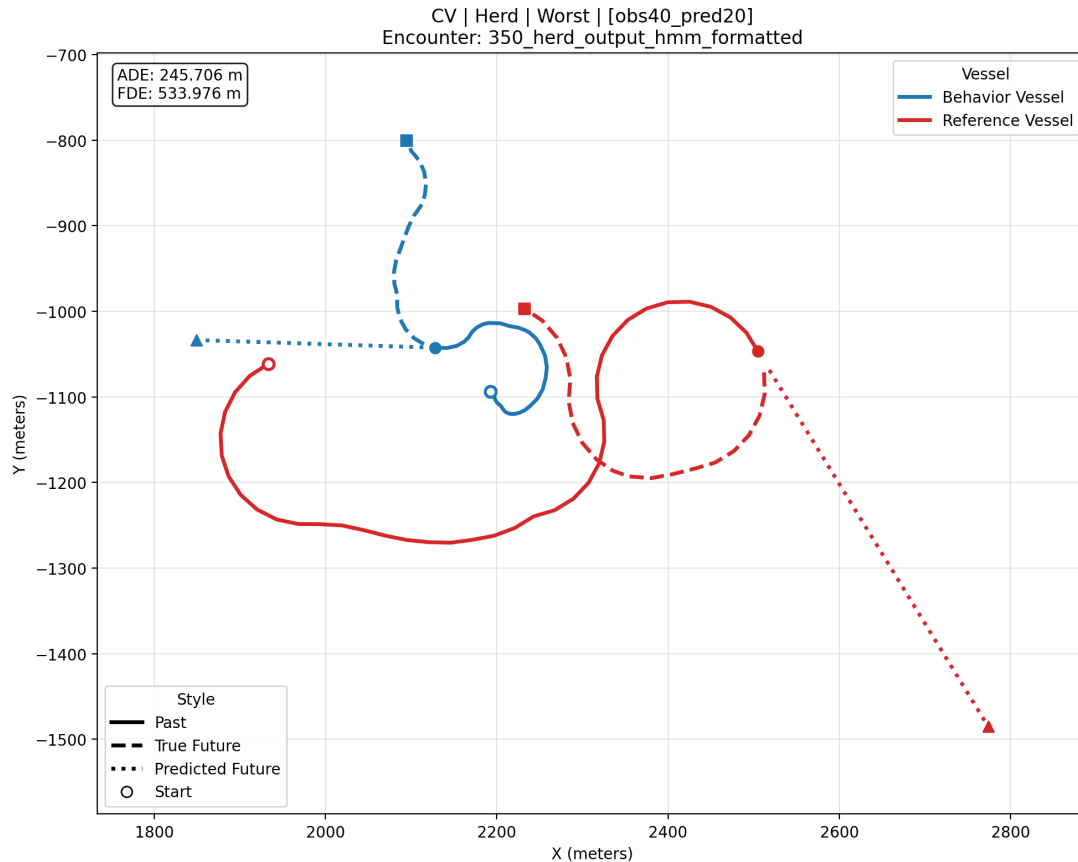


FIGURE 6.4: Herd, CV (worst case, 245.7 m ADE). Both vessels execute looping ground-truth trajectories (dashed); the predicted future (dotted) is forced to extrapolate linearly because a single deterministic motion model cannot represent the multi-modal maneuver distribution.

UKF-CTRV divergence. Figure 6.5 places EKF-CTRV and UKF-CTRV on the same track. The EKF prediction stays close to the true future; the UKF-CTRV prediction has lost coherence by the end of the rollout, consistent with P_ω inflating without a corresponding measurement update.

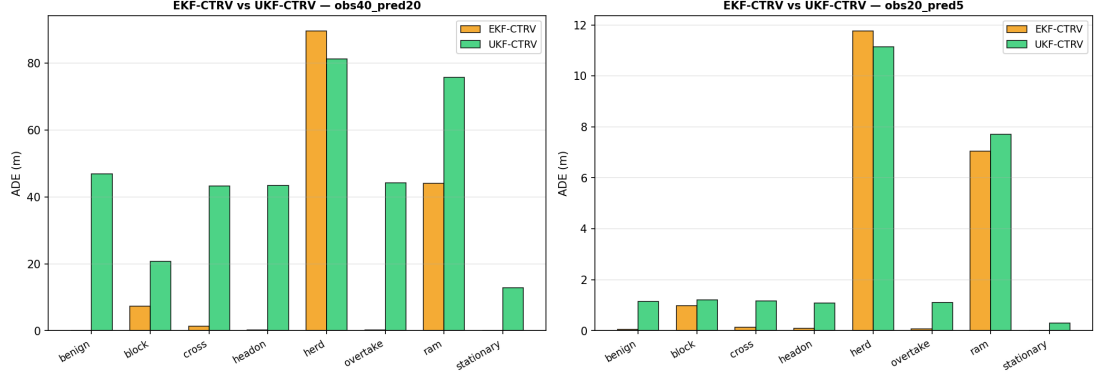


FIGURE 6.5: EKF-CTRV vs. UKF-CTRV forecast comparison on a long-horizon encounter. The UKF-CTRV’s covariance diverges; the EKF-CTRV remains stable despite its own arc-extrapolation error on hostile scenarios.

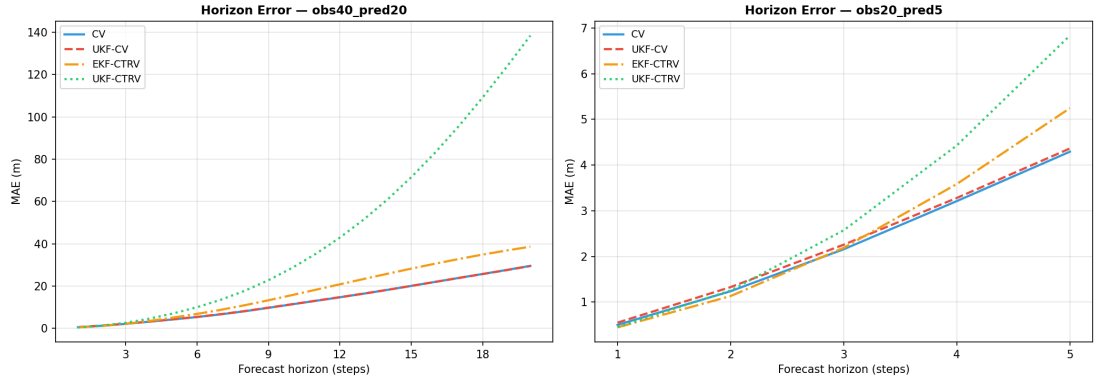


FIGURE 6.6: Horizon-wise MAE curves under both experiment configurations. EKF-CTRV’s $t = 1$ advantage erodes by $t \approx 3$ –4; UKF-CTRV divergence in Experiment A appears as a steep upward curve after $t = 5$.

6.5 Discussion

6.5.1 The Linear Ceiling: Why Sigma Points Do Not Help Here

CV, UKF-4D, and UKF-5D-CV give nearly identical ADE across all four configurations (spread $< 0.7\%$). The reason is structural: when process dynamics f are linear, the

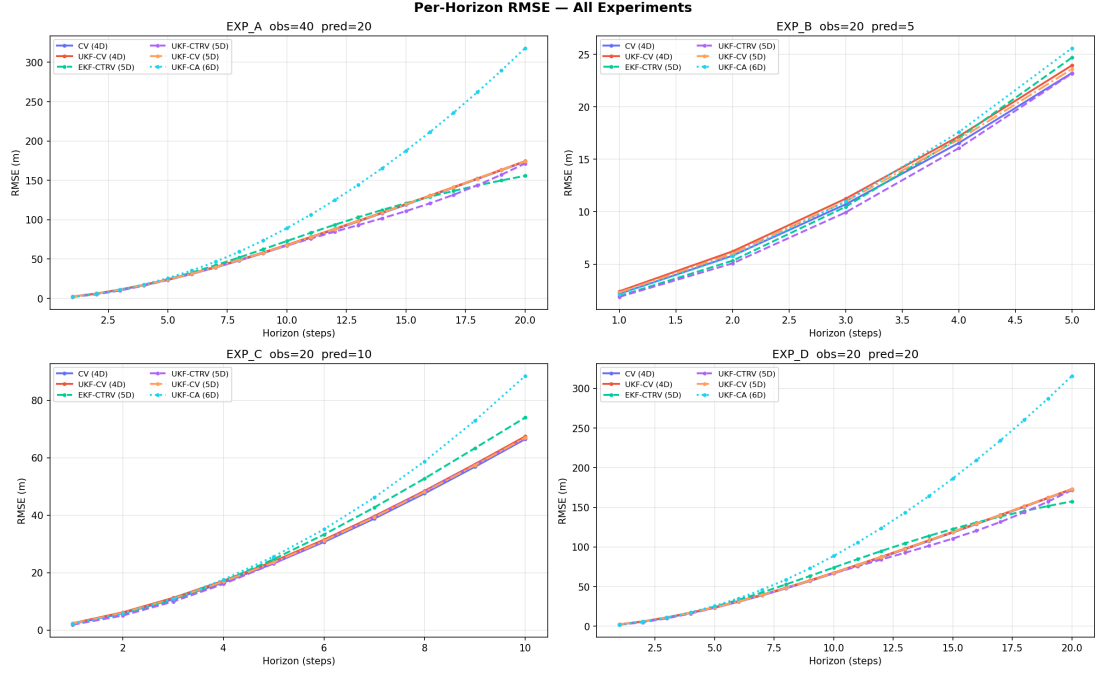


FIGURE 6.7: Per-horizon RMSE (Experiment A, $T_{\text{pred}} = 20$) for all six classical filters. The three linear-dynamics methods (CV, UKF-4D, UKF-5D-CV) trace a single curve; the EKF-CTRV peels above the linear band after $t \approx 4$; the Asym-UKF-6D and the divergent UKF-CTRV separate as distinct upper envelopes. RMSE is shown alongside MAE (Figure 6.6) because RMSE penalizes the worst-case scenarios that drive aggregate ADE.

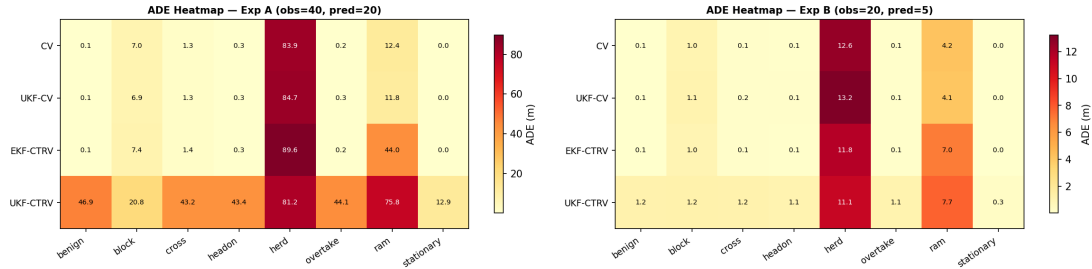


FIGURE 6.8: Scenario ADE heatmap (Experiment B). Near-zero error on easy scenarios but 12–13 m ADE on herd marks the ceiling of intent-agnostic filters.

sigma-point weighted sum reduces to the analytic matrix product $F\hat{\mathbf{x}}$, so the UKF returns the same prediction as CV at higher cost. Extending the state via UKF-5D-CV does not change this, because ω advances as an uncoupled random walk and never feeds back into position. For position prediction accuracy, the linear filters are interchangeable; UKF retains an advantage only when its covariance estimate P_k is consumed by a downstream uncertainty-aware system.

6.5.2 UKF-CTRV Divergence: Observation and Prediction Windows

UKF-CTRV reports 53.3 m ADE in Experiments A and D, with per-scenario errors elevated even on benign (46.9 m vs. an expected ~ 0.08 m). The mechanism is numerical divergence rather than model mismatch. Because ω is not in H , P_ω grows each predict step by $\sigma_{\omega,Q}^2 = 0.0025 \text{ (rad/s)}^2$. After roughly 15–20 unobserved steps the sigma-point spread in ω exceeds the branching threshold $\omega_{\text{thr}} = 10^{-3} \text{ rad/s}$, mixing arc and straight-line propagations within a single update step; the resulting covariance inflation feeds back into the sigma-point spread.

The condition triggers in Experiment A through the 40-step warm-up alone, and in Experiment D through the 20-step prediction rollout. Experiment B ($T_{\text{pred}} = 5$) and C ($T_{\text{pred}} = 10$) accumulate fewer unobserved steps and remain stable or partially stable. EKF-CTRV evaluates f at the current mean only, so no branch mixing occurs. Two

standard mitigations apply: capping P_ω each step or adding ω to H if heading-rate measurements are available; neither was applied here.

6.5.3 EKF-CTRV: Short-Horizon Advantage and Arc Compounding

EKF-CTRV records the lowest $t = 1$ MAE in this evaluation (0.912 m in Experiment A, 0.892 m in Experiment B), and leads on benign and block where the motion is dominated by smooth, persistent heading arcs.

The same constant- ω assumption drives its long-horizon error. Once the filter locks in a turn-rate estimate from noisy heading finite differences, that estimate propagates unchanged for the entire prediction rollout. A ram vessel approaching near-straight ($\omega \approx 0$) produces a small but nonzero ω estimate; over 20 steps this extrapolates into an arc with error scaling as $\frac{1}{2}|\omega|T_{\text{pred}}^2v$, yielding 43.9 m for $|\omega| = 0.05$ rad/s. At $T_{\text{pred}} = 5$ the same error compresses to 6.7 m. EKF-CTRV is the lowest-error choice in this evaluation for $T_{\text{pred}} \leq 4$ and one of the higher-error choices on hostile scenarios beyond that.

6.5.4 Asymmetric UKF: Acceleration Channel

Asymmetric UKF is last in both long-horizon settings (57.5 m at Experiment A, 56.7 m at Experiment D), with the largest gap on herd (136.9 m vs. CV's 85.0 m).

The acceleration state amplifies pre-maneuver kinematics into a specific arc, which is accurate when motion is cooperative (benign, overtake) and inaccurate on hostile maneuvers. With the heuristic noise settings used here, the acceleration channel does not improve over a velocity-only filter; per-scenario noise tuning is a likely prerequisite for it to be useful in this comparison.

6.5.5 Horizon Length and Filter Choice

Within the configurations tested, EKF-CTRV reports the lowest error for $T_{\text{pred}} \leq 4$ and CV/Standard UKF for $T_{\text{pred}} \in [5, 20]$. UKF-CTRV is stable at Experiment B (25 total unobserved steps) and divergent at Experiments A and D (60 and 40 unobserved steps).

6.5.6 Two Regimes by Intent Type

Errors are separated by Intent type across all four configurations. Benign, headon, overtake, and cross fall in the sub-meter to low single-digit ADE range; herd, ram, and block fall one to three orders of magnitude higher. No filter in this evaluation drops below 11.6 m ADE on herd at any tested horizon.

6.5.7 Limitations and Future Directions

Several caveats bound the scope of the comparison and qualify how its conclusions transfer to other settings.

- Turn rate ω is derived from noisy heading finite differences. Measurement noise propagates directly into EKF/CTRV arc predictions.
- Noise parameters were set by heuristic defaults and not tuned per scenario or per behavior. Per-behavior noise adaptation is expected to reduce EKF ram/herd error.
- UKF-CTRV Experiments A and D results are affected by divergence instability and are not valid trajectory predictions.
- Results are point estimates on a single deterministic split; no confidence intervals or variance across random seeds are reported.
- Cross-model comparison between classical filters and MTITP at the matched configuration ($T_{\text{pred}} = 5$) and at longer horizons is presented in Chapter 7.

The failure modes documented above point to concrete future directions, with follow-on filter designs that target each mechanism individually.

1. **P_ω capping for UKF-CTRV:** Clip $P_\omega \leq P_\omega^{\text{max}}$ after each predict step to break the divergent feedback loop, then re-evaluate under Experiments A and D.

2. **IMM-CTRV/CV:** Combine EKF-CTRV and CV sub-filters in an Interacting Multiple Model framework. Automatic mode switching via innovation likelihood is expected to eliminate both the ram failure mode and the long-horizon arc divergence.
3. **Direct ω observation:** Add ω to H if compass heading rate is available, eliminating the unobservable-growth problem for both EKF-CTRV and UKF-CTRV.
4. **Per-scenario noise tuning:** Increase $\sigma_{\omega,Q}$ for ram and herd; decrease for benign to lock in smooth heading evolution.
5. **Hybrid physics + learning:** Use the Kalman filter state as input features to MTITP. The filter provides accurate short-horizon kinematics; the GAN extends to multi-modal long-horizon prediction.

6.6 Conclusion

This chapter evaluated six classical Bayesian filters (CV, Standard UKF (4D), UKF-5D-CV, Asymmetric UKF (6D), EKF-CTRV, and UKF-CTRV) on seven two-vessel scenarios under four observation and prediction configurations. The cross-horizon summary is reported in Section 6.4.5; the qualitative trajectory cases are in Section 6.4.8; the discussion of mechanisms is in Section 6.5.

Three observations summarize the findings. First, when the underlying dynamics are linear, the choice of covariance-propagation scheme makes little difference: CV, UKF-4D, and UKF-5D-CV give similar ADE at every horizon tested. Second, the EKF-CTRV is competitive at very short horizons but loses its advantage as the constant - ω arc compounds; CV is the more reliable choice for $T_{\text{pred}} \geq 10$ in this evaluation. Third, the UKF-CTRV is unstable when the observation warm-up plus the prediction rollout exceeds the convergence budget for the unobservable ω state, which is what produces the divergent ADE in Experiments A and D.

Several caveats apply. Turn-rate is derived from noisy heading finite differences, the noise parameters were set by heuristic defaults rather than tuned per scenario, and the results are point estimates on a single deterministic split. Within those limits, the hostile scenarios (block, ram, herd) consistently produce the largest errors across all six filters, suggesting that a single parametric motion model is not sufficient to represent these encounters. Concrete filter-side extensions are listed in Section 6.5.7; the comparison with an intent-aware deep model is presented in Chapter 7, where the same metrics are reported on the matched short-horizon configuration.

Chapter 7

MTITP: Multi-Task Intent and Trajectory Prediction for Complex Maritime Scenarios

7.1 Introduction

Maritime surveillance and autonomous navigation require two coupled outputs from a vessel encounter: the scenario class that describes *what* is happening, and the future trajectory that predicts *where* each vessel will be a few seconds from now. A kinematic forecast alone does not distinguish a COLREGs-compliant crossing from an attempted ram, and a scenario label without a position forecast does not localize the threat. The previous chapter showed that classical baselines (a deterministic

constant-velocity propagator and several Kalman-filter variants) produce competent kinematic forecasts for routine traffic but lack a representation of tactical scenarios, and they degrade or diverge in the adversarial scenarios that maritime surveillance prioritizes. In this chapter, this dissertation develops a single learned model that produces a behavior class per scenario window, a per-step behavioral prediction, and a multimodal trajectory distribution, all from a shared encoder.

The existing literature handles each part of this problem separately. Maritime intent and scenario classification have been studied extensively in isolation: HMMs [76], handcrafted-feature classifiers [6], and CPA-based threat assessment [75, 135] produce a tactical label but do not forecast future positions. Deep-learning trajectory predictors such as Social LSTM [104], Social GAN [105], Trajectron++ [106], and AgentFormer [108] generate multimodal futures for pedestrians and road vehicles, but were not designed for the symbolic relational features (port/starboard side, CPA trend, heading-change class) provided by AIS data for maritime encounters, and none of them produces a tactical scenario label. Multi-task formulations are emerging only at the boundaries: Wu et al. [65] report a 26% gain from jointly training trajectory prediction and motion planning in a maritime graph network, indicating that the two tasks share an underlying structure that benefits from joint optimization. To our knowledge, no published model integrates intent classification, future behavior prediction, and intent-conditioned multi-modal trajectory generation into a single encoder for maritime encounters. A full review of these threads is provided in Chapter 2; only the positioning needed to motivate the design is presented here.

The maritime intent-recognition literature is the most directly relevant body of work for MTITP, and it organizes into four overlapping clusters that each anticipate one piece of the architecture without combining them. The closest published antecedent is SEMINT [31], which couples symbolic vessel descriptors (role, mission type, regulatory status) with a Large Language Model (LLM) backbone for intent classification; however, intent is not formally defined and is only inferred through semantic reasoning by the LLM. SEMINT validates the symbolic-feature embedding strategy adopted in MTITP’s Stream C (described in Section 7.3.2.3) but treats trajectory forecasting as a downstream rather than a joint task. Multi-vessel interaction modeling is captured by hybrid GNN architectures [44], dual-channel spatio-temporal attention models such as DUCTS-TMA [32], and LSTM-with-cross-attention baselines [33]; each leverages neighborhood context but treats COLREGs and other regulatory structures implicitly, leaving the network to recover interaction rules from data; a strategy that reaches strong benchmark numbers but limits interpretability and degrades on noisy AIS. ShipTrack-TVAEs pairs a transformer encoder with a variational decoder for trajectory tracking [45]; cooperative conditional VAEs jointly model multi-vessel intent [46] but report sharp accuracy degradation as AIS quality drops; the risk-aware GAN (RAGAN) [47] trains a discriminator that scores generated trajectories against multi-ship encounter risk patterns; and the regional ship-behavior social GAN [48] applies adversarial supervision to joint behavior and trajectory prediction in regional traffic. The intent-aware critic in MTITP generalizes this critic-as-supervisor pattern while explicitly decoupling the generator and intent objectives. Bayesian

formulations extend the HMM/DBN tradition with calibrated posterior probabilities for vision–radar intent fusion [34], and DenseNet-style image-recognition over rasterized tracks [82] demonstrates that vision backbones remain a valid baseline class even when sequence structure is discarded. Domain-adapted classifiers for fishing-vessel behavior [35, 36] and harbor-berthing intent [37] reach high accuracy by riding a strong domain prior. Those priors do not transfer to general adversarial threat assessment, and that limitation pushes MTITP toward domain-general symbolic features rather than scenario-specific motion templates. Each of the four design ingredients – symbolic feature encoding, multi-vessel interaction reasoning, critic-as-classifier supervision, and explicit context encoding – has been validated independently in this maritime literature. What is missing is a single backbone that ties them together. The contribution of this dissertation (Chapter 7) is that backbone: one encoder pass yields the tactical scenario label, per-step behavioral predictions, and intent-conditioned multi-modal trajectories.

Three contemporaneous strands of work outside the maritime intent-recognition literature establish the broader machine-learning context for Multi-Task Intent and Trajectory Prediction (MTITP). First, transformer-based Automatic Identification System (AIS) forecasters have proliferated since TrAISformer [50], with graph-augmented variants such as ST-MGT [110] and GATransformer [51] explicitly modeling pairwise vessel interactions, multi-graph fusion designs (STMGF-Net [52], MGAED [53], DAA-SGCN [111]) that decouple spatial and temporal attention, and physics-augmented variants (TPTrans [54]) that inject motion priors into the attention path; recent

surveys characterize this growth and observe that systematic comparisons under noise are rare [2–4]. Second, the generative-trajectory landscape has fragmented into adversarial, variational, and diffusion families, with WGAN-GP refinements [55, 56], Gaussian-mixture and memory-augmented VAEs [57, 58, 83], and diffusion-based predictors (MotionDiffuser [59], DiffuTraj [60], ST-GDPM [61], physics-guided diffusion [112]) all competing on the same forecasting benchmarks. MTITP commits to the WGAN-GP backbone for tractable single-pass inference and pairs it with a purpose-built intent-aware critic; the empirical question this chapter addresses is whether that combination matches direct L2 regression on accuracy while retaining the multi-modal sampling that L2 forfeits. Third, the multi-task-learning literature provides the closest published analogues to the joint intent–trajectory problem MTITP solves: PTINet [62] jointly predicts pedestrian crossing intent and trajectory through a shared scene encoder, IntentFormer [63] architecturally conditions the trajectory decoder on a latent intent token, VesNet [64] jointly estimates vessel state and route pattern from a single backbone, and ship-specific multi-task models [65, 66] report that intent or auxiliary heads regularize trajectory regression, with concurrent analyses of uncertainty-based loss-balancing schemes [115, 116] informing the joint loss design. Yet none of these systems provides both a tactical behavior label and intent-conditioned multimodal generation in the maritime setting.

We propose MTITP (Multi-Task Intent and Trajectory Prediction), a unified model that produces three outputs from a single encoder pass: a scenario-level past-intent classification from the Past Intent Classifier, a per-step future-intent prediction from the

Future Intent Predictor, and an intent-conditioned multi-modal trajectory distribution from the Trajectory Generator. Three architectural commitments shape the design. The encoder processes the scene through three complementary streams: per-vessel kinematics, pairwise geometry, and symbolic COLREGs features, instead of relying on the raw position-velocity history that traditional filters utilize. The trajectory generator is then conditioned on the model’s own predicted intent, so the spatial output is calibrated to the tactical mode identified by the encoder. Training is staged: the intent heads warm up on their own before any trajectory-side loss is applied, which keeps the encoder from having to learn semantic structure and chase a moving generator at the same time.

MTITP defines a shared backbone consisting of the three-stream encoder, the past- and future-intent heads, and the intent-conditioned trajectory generator. We study three variants of this backbone, distinguished only by the trajectory loss applied during the joint training phase:

- **MTITP-WGAN (baseline).** The shared backbone trained against a domain-agnostic Wasserstein GAN [118] with Gradient Penalty critic. This is the baseline model and the version of MTITP referred to by the chapter title and the headline numbers.
- **MTITP-L2.** The shared backbone with the critic removed; the trajectory generator is trained directly with an L2 regression loss against the ground-truth

future. This variant isolates the contribution of the adversarial signal from the contribution of the architecture.

- **MarITGAN (Maritime Intent-Trajectory GAN).** The shared backbone trained against a purpose-built *IntentAwareCritic* with three branches that score trajectories against the model’s own predicted intent and against pairwise vessel geometry, with additional physics and scenario-weighted auxiliary losses on the generator side. Two refinements are evaluated in the ablation studies (A20, A21).

The shared backbone is described once in Section 7.4; the three critics, loss configurations, and training pipelines are then presented side by side in Section 7.5.

Research questions. The chapter is organized around one dissertation-level research question (RQ2) and two chapter-specific subquestions (RQ-MTITP1, RQ-MTITP2) that directly probe it. Internal design choices, which features to use, which loss to optimize, which critic to pair with the generator, are framed and answered as ablation sub-questions in Section 7.9 rather than restated at the introductory level.

RQ2 (dissertation-level, shared with Chapter 6): *Can a joint intent-trajectory prediction model outperform both independent intent classifiers and classical trajectory predictors in maritime vessel encounters?*

RQ-MTITP1 (subquestion of RQ2): *Does multi-task supervision, jointly training intent classification and trajectory generation through a shared encoder, improve encoder representations compared to training either task in isolation?*

RQ-MTITP2 (subquestion of RQ2): *Does the joint model retain its intent-discrimination and trajectory-prediction advantage over independent baselines under realistic sensor noise?*

7.1.1 Contributions

This dissertation advances maritime intent recognition and trajectory prediction in this chapter through the following contributions:

1. **Three-stream encoder.** A fusion of kinematic, geometric, and symbolic streams through alternating temporal self-attention and interaction-gated cross-vessel attention layers, producing both a full sequence encoding for decoder cross-attention and a compact scene summary for intent classification and prediction.
2. **Intent-conditioned trajectory generator.** A Transformer decoder conditioned on future intent embeddings and stochastic noise, producing K diverse trajectory samples calibrated to the predicted intent mode.

3. **Three training variants of the same backbone.** MTITP-WGAN (baseline, Wasserstein GAN with Gradient Penalty), MTITP-L2 (critic removed, L2 regression loss), and MarITGAN (purpose-built IntentAwareCritic with three branches plus physics and scenario-weighted auxiliary losses), allowing the contribution of the adversarial signal to be isolated from the contribution of the architecture.
4. **Two-phase training protocol.** A warmup protocol that builds a stable intent-aware encoder before any trajectory-side loss is allowed to modify it, preventing the known instability of simultaneous classification and adversarial objectives.
5. **Coordinate-consistent training.** A separation of relative training space and absolute inference space that prevents the scale explosion observed when mixing coordinate systems in GAN-based trajectory models.
6. **End-to-end evaluation.** Per-scenario breakdown across seven behaviors (four COLREGs-relevant non-hostile and three hostile) on both trajectory and intent metrics, head-to-head comparison against the classical trajectory baselines of Chapter 6 (constant-velocity propagator and Kalman-filter variants), a five-check validation suite (split integrity, majority-class margins, t-SNE cluster visualization, cross-dataset evaluation, and McNemar’s test) that rules out the obvious alternative explanations, an operationally-motivated hostile vs. non-hostile binary collapse, and a head-to-head comparison against published classifiers on the same noisy dataset.

7.2 Problem Formulation

7.2.1 Notation

Let $\mathcal{V} = \{0, 1\}$ index the two vessels (blue $v = 0$, red $v = 1$) in a scenario. Time is discretised at $\Delta t = 1$ s. We denote by $\mathbf{X}_t^v \in \mathbb{R}^d$ the feature vector of vessel v at time t .

7.2.2 Inputs and Outputs

Given an observation window $[\tau, \tau + T_{\text{obs}})$ with $T_{\text{obs}} = 20$ steps, the model receives three complementary representations of the scenario:

$$\mathbf{A} \in \mathbb{R}^{B \times 2 \times T_{\text{obs}} \times 8} \quad \text{per-vessel kinematics (Stream A)} \quad (7.1)$$

$$\mathbf{B} \in \mathbb{R}^{B \times T_{\text{obs}} \times 12} \quad \text{continuous pairwise geometry (Stream B)} \quad (7.2)$$

$$\mathbf{C} \in \mathbb{Z}^{B \times T_{\text{obs}} \times 11} \quad \text{symbolic interaction indices (Stream C)} \quad (7.3)$$

$$\mathbf{p}_{\text{last}} \in \mathbb{R}^{B \times 2 \times 2} \quad \text{last observed } (x, y) \text{ for each vessel} \quad (7.4)$$

where B is batch size. The model produces three outputs over prediction horizon

$T_{\text{pred}} = 5$ steps:

$$\hat{s} \in \{0, \dots, C-1\}^B \quad \text{past behavior class per window} \quad (7.5)$$

$$\hat{b} \in \{0, \dots, C-1\}^{B \times T_{\text{pred}}} \quad \text{future behavior class per step} \quad (7.6)$$

$$\{\hat{\mathbf{X}}^{(k)}\}_{k=1}^K \in \mathbb{R}^{B \times 2 \times T_{\text{pred}} \times 2} \quad K \text{ absolute trajectory samples} \quad (7.7)$$

Here C denotes the number of behavior classes; $C = 7$ in this work.

7.2.3 Objective

The overall learning objective minimizes:

$$\mathcal{L} = \lambda_{\text{past}} \mathcal{L}_{\text{past}} + \lambda_{\text{future}} \mathcal{L}_{\text{future}} + \mathcal{L}_{\text{GAN}} \quad (7.8)$$

where $\mathcal{L}_{\text{past}}$ and $\mathcal{L}_{\text{future}}$ are cross-entropy losses for the two intent prediction tasks and $\mathcal{L}_{\text{GAN}}$ is the Wasserstein adversarial objective for trajectory generation. The precise forms are given in Section 7.5.

7.3 Dataset and Data Pipeline

7.3.1 Dataset

We use datasets of paired-vessel trajectory sequences generated for maritime vessel scenarios. Each simulation produces a CSV file containing per-timestep observations for two vessels under one of seven active behaviors (Table 3.5). The simulator encodes COLREGs-relevant behavior patterns, yielding sequences with annotated scenario-level labels and per-timestep behavioral labels (`beh_label` $\in \{1, \dots, 8\}$).

The dataset spans approximately 7,998 CSV files yielding $\sim 880,000$ windowed samples per dataset variant at stride 10 and window size of $T_{\text{obs}} + T_{\text{pred}} = 25$ (noiseless and Noisy-1; Noisy-2 yields ~ 1.22 M because of its longer trajectories and more scenarios per behaviors). The scenario-level train/val/test split is reported in Table 7.26.

7.3.2 Feature Extraction

7.3.2.1 Stream A: Per-Vessel Kinematics

We convert raw geographic coordinates (latitude ϕ , longitude λ) to a local Cartesian frame centered at the blue vessel’s position at the first timestep of each scenario using an equirectangular projection:

$$x_t = (\lambda_t - \lambda_0) \frac{\pi}{180} R_{\oplus} \cos(\phi_0 \frac{\pi}{180}), \quad y_t = (\phi_t - \phi_0) \frac{\pi}{180} R_{\oplus} \quad (7.9)$$

where $R_{\oplus} = 6,371,000$ m. Velocity components are derived from heading θ (degrees, clockwise from North) and speed σ (m/s):

$$v_x = \sigma \sin \theta_{\text{rad}}, \quad v_y = \sigma \cos \theta_{\text{rad}} \quad (7.10)$$

The full per-vessel feature vector at timestep t is:

$$\mathbf{A}_t^v = (x, y, v_x, v_y, \sin \theta, \cos \theta, \sigma, \dot{\sigma}) \in \mathbb{R}^8 \quad (7.11)$$

The sine/cosine decomposition of heading avoids the discontinuity at $0^\circ/360^\circ$. Acceleration $\dot{\sigma} = \Delta\sigma/\Delta t$ is the finite-difference approximation. Full feature definitions are given in the appendix (Table A.4).

7.3.2.2 Stream B: Continuous Pairwise Geometry

Stream B captures the time-varying spatial relationship between the two vessels in 12 continuous features per timestep. Defining the Closest Point of Approach (CPA) as the minimum future distance under constant-velocity assumptions:

$$\mathbf{B}_t = \underbrace{(d, \alpha^{br}, \alpha^{rb}, \rho^{br}, \rho^{rb}, \Delta\psi, x_r, y_r, \Delta v_x, \Delta v_y)}_{\text{geometry}}, \underbrace{(t_{\text{CPA}}, d_{\text{CPA}})}_{\text{CPA}} \in \mathbb{R}^{12} \quad (7.12)$$

where d is inter-vessel distance; α^{br}, α^{rb} are absolute bearings; ρ^{br}, ρ^{rb} are relative bearings; $\Delta\psi$ is the heading difference; (x_r, y_r) is the relative position vector; $(\Delta v_x, \Delta v_y)$

is the velocity difference; t_{CPA} and d_{CPA} are the time to and distance at CPA, clipped to $[-300, 500]$ s and $[0, 12,000]$ m respectively. Full definitions are given in the appendix (Table A.5).

7.3.2.3 Stream C: Symbolic Interaction Features

Stream C encodes 11 discrete situational-awareness indicators, developed in Table 3.6 in Chapter 3, derived from COLREGs decision logic and classical maritime situational awareness theory. Each feature $c_i \in \mathbb{Z}$ indexes a small vocabulary and is processed by a dedicated embedding table inside the model, avoiding any implicit ordinal assumption:

$$\mathbf{C}_t = (c_{\text{side}}, c_{\text{beam}}, c_{\text{reg}}^{br}, c_{\text{reg}}^{rb}, c_{\Delta\rho}, c_{\Delta d}, c_{\ddot{\sigma}}, c_{\Delta\psi}, c_{t_{\text{CPA}}}, c_{d_{\text{CPA}}}, c_{td_{\text{CPA}}}) \in \mathbb{Z}^{11} \quad (7.13)$$

7.3.2.4 Labels

Each window carries two supervision targets, one scenario-level scalar and one per-timestep behavioral sequence:

- **Past behavior (scenario):** scalar $s \in \{0, \dots, C-1\}$ derived from the scenario folder name.
- **Future behavior (beh_label):** per-timestep integer $b_t \in \{0, \dots, C-1\}$ remapped from the simulator's 1-indexed per-timestep labels.

7.3.3 Sliding Window Segmentation

We segment each scenario CSV of length T into overlapping windows of length

$W = T_{\text{obs}}(20) + T_{\text{pred}}(5) = 25$ steps using sliding stride s :

$$\tau \in \{0, s, 2s, \dots, T - W\} \quad (7.14)$$

At stride $s = 10$, approximately 880,000 windows are produced per dataset variant from 7,998 files for noiseless and Noisy-1; Noisy-2 yields ~ 1.22 M (Table 7.26). Reducing to stride $s = 5$ doubles these counts, and $s = 1$ would yield ~ 3.5 M windows, impractical for full-dataset training on a single GPU.

7.3.4 Coordinate Conventions and Normalization

Training/Inference Coordinate Split. All model computations during training operate in *normalized relative space*: (x, y) positions in Stream A are expressed relative to the blue vessel’s position at window start τ . The generator predicts cumulative displacements relative to the *last observed* position $\mathbf{p}_{\text{last}}$. Absolute positions are reconstructed only at inference:

$$\hat{\mathbf{X}}_{\text{abs}}^{(k)} = \hat{\mathbf{X}}_{\text{rel}}^{(k)} + \mathbf{p}_{\text{last}} \quad (7.15)$$

This ensures that both the generator and the critic operate in the same coordinate scale (~ 100 m), preventing the $80\times$ scale explosion that arises when absolute offsets ($\sim 8,000$ m) are introduced inside the generator.

7.3.5 Dataset Split

Scenarios are split *by scenario ID* before window extraction to prevent data leakage. The split maintains per-behavior proportions using stratified sampling with fixed seed 42: 70% train, 15% validation, 15% test. Fixed seed allows similar data split for cross model evaluations.

7.4 Model Architecture: Shared Backbone

The MTITP backbone is shared across the three variants of the model (MTITP-WGAN, MTITP-L2, MarITGAN) and is described once in this section. It comprises four components: (1) the three-stream *Spatial-Temporal Encoder*, (2) the *Past Intent Classifier*, (3) the *Future Intent Predictor*, and (4) the intent-conditioned *Trajectory Generator*. The critic, when present, is variant-specific and is described together with each variant’s training pipeline in Section 7.5. Figure 7.1 shows the complete data flow with the critic block drawn for reference; the per-iteration training pipelines are presented separately for each variant in Section 7.5.

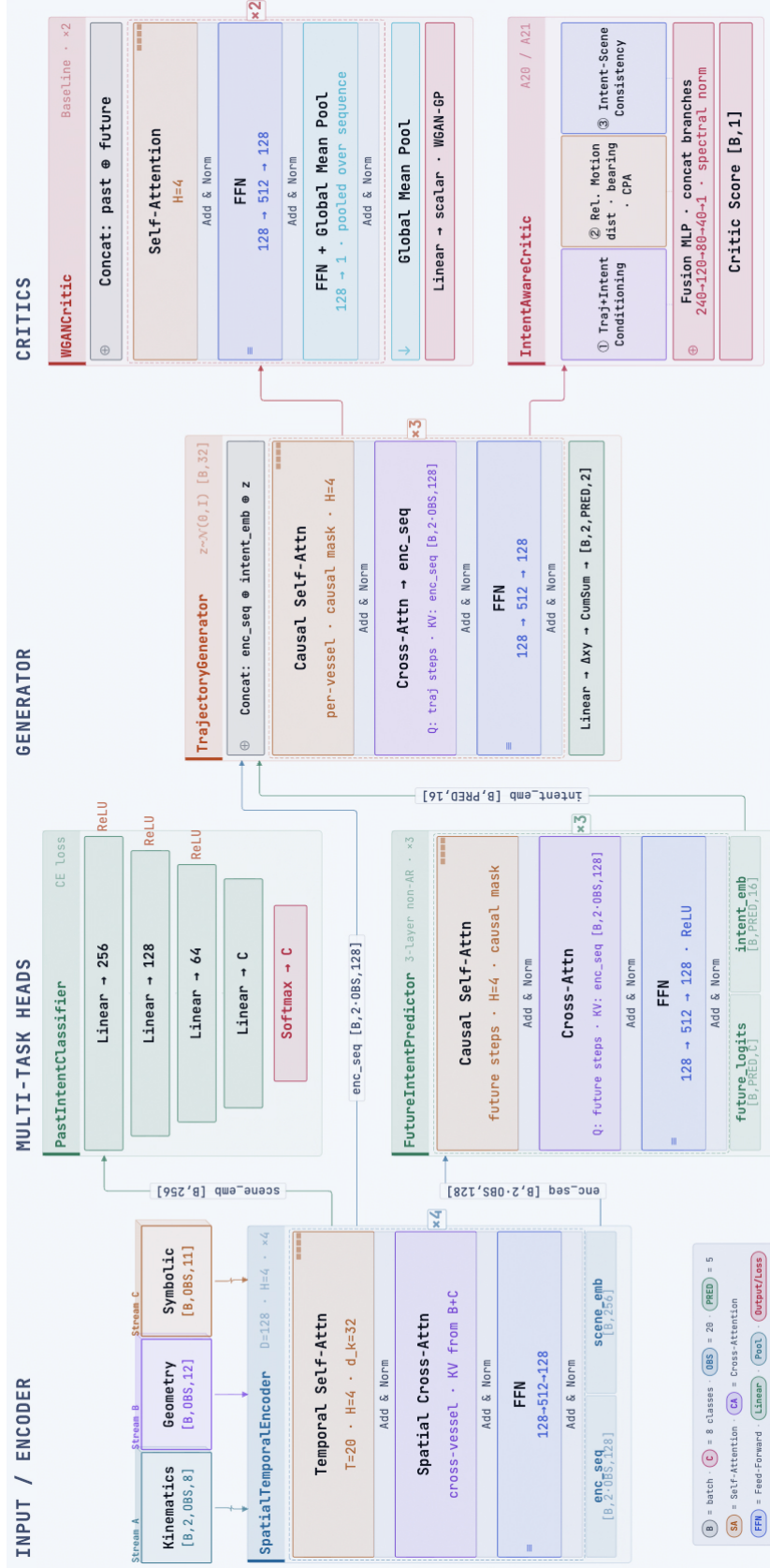


FIGURE 7.1: MTITP system architecture. The three-stream encoder (kinematics, pairwise geometry, symbolic interactions) feeds the intent heads and trajectory generator. The training-only critic is variant-specific: MTITP-WGAN (domain-agnostic), MarITGAN (three-branch IntentAwareCritic shown), MTITP-L2 (no critic).

7.4.1 Three-Stream Spatial-Temporal Encoder

The encoder (Figure 7.2) maps the three input streams to two representations used by downstream modules:

- **enc_seq** $\in \mathbb{R}^{B \times 2T_{\text{obs}} \times D_{\text{model}}}$: the full token sequence (vessels $\times$ timesteps) used as memory for decoder cross-attention.
- **scene_emb** $\in \mathbb{R}^{B \times 2D_{\text{model}}}$: a compact scene-level summary derived from per-vessel CLS tokens, used by the past intent classifier.

7.4.1.1 Stream A Input Projection

Each kinematic vector $\mathbf{A}_t^v \in \mathbb{R}^8$ is projected to model dimension $D_{\text{model}} = 128$:

$$\tilde{\mathbf{A}}_t^v = W_A \mathbf{A}_t^v + b_A \in \mathbb{R}^{D_{\text{model}}} \quad (7.16)$$

A learnable vessel identity embedding $\mathbf{e}_v \in \mathbb{R}^{D_{\text{model}}}$ is added to distinguish the two vessels:

$$\tilde{\mathbf{A}}_t^v \leftarrow \tilde{\mathbf{A}}_t^v + \mathbf{e}_v \quad (7.17)$$

Fixed sinusoidal positional encoding $\text{PE}_t \in \mathbb{R}^{D_{\text{model}}}$ is added along the time axis [107]:

$$\text{PE}_t^{(2i)} = \sin\left(\frac{t}{10000^{2i/D_{\text{model}}}}\right), \quad \text{PE}_t^{(2i+1)} = \cos\left(\frac{t}{10000^{2i/D_{\text{model}}}}\right) \quad (7.18)$$

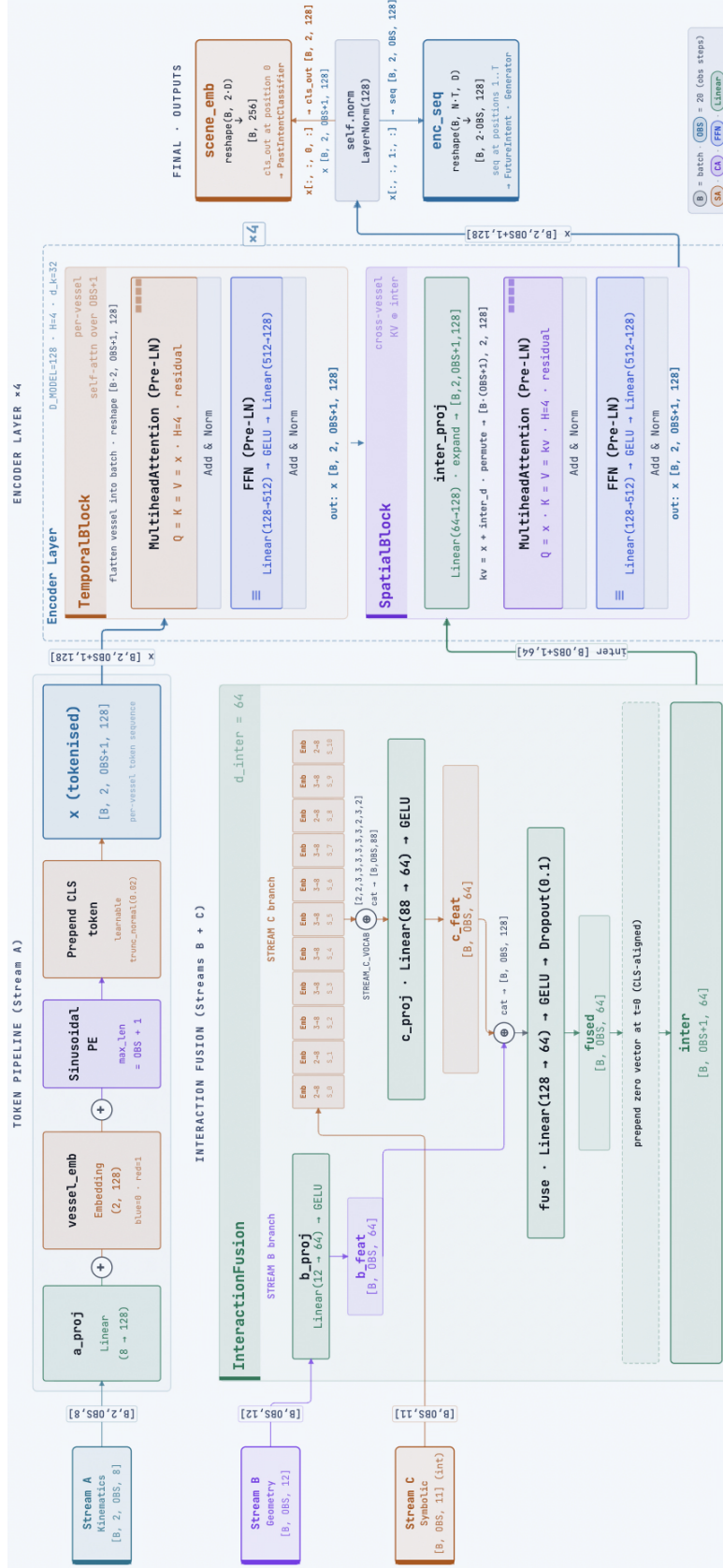


FIGURE 7.2: Three-stream Spatial-Temporal Encoder. Stream A (per-vessel kinematics) is projected and concatenated with Streams B and C through Interaction Fusion, then passed through alternating Temporal and Spatial Transformer blocks to produce `enc_seq` and `scene_emb`.

A learnable CLS token $\mathbf{c} \in \mathbb{R}^{D_{\text{model}}}$ [109] is prepended to each vessel’s sequence, yielding tokens of length $T_{\text{obs}} + 1 = 21$ per vessel.

7.4.1.2 Interaction Fusion

Stream B and Stream C are fused into a per-timestep interaction context vector

$$\mathbf{I}_t \in \mathbb{R}^{D_{\text{inter}}} \quad (D_{\text{inter}} = 64).$$

Stream B projection. The 12 continuous geometry features are projected to D_{inter} through a single linear layer with GELU activation:

$$\mathbf{I}_t^B = \text{GELU}(W_B \mathbf{B}_t + b_B) \in \mathbb{R}^{D_{\text{inter}}} \quad (7.19)$$

Stream C embedding. Each of the 11 symbolic features $c_i \in \mathbf{C}_t$ is passed through its own embedding table $\mathbf{E}_i \in \mathbb{R}^{|\mathcal{V}_i| \times D_{\text{sym}}} \quad (D_{\text{sym}} = 8)$, then concatenated and projected:

$$\mathbf{I}_t^C = \text{GELU}(W_C [\mathbf{E}_1(c_1); \dots; \mathbf{E}_{11}(c_{11})] + b_C) \in \mathbb{R}^{D_{\text{inter}}} \quad (7.20)$$

where the concatenated embedding has dimension $11 \times D_{\text{sym}} = 88$.

Fusion. The Stream B and Stream C projections are concatenated and passed through a final GELU-activated linear layer:

$$\mathbf{I}_t = \text{GELU}(W_f[\mathbf{I}_t^B; \mathbf{I}_t^C] + b_f) \in \mathbb{R}^{D_{\text{inter}}} \quad (7.21)$$

7.4.1.3 Temporal Block

Each of $N_{\text{enc}} = 4$ Temporal Blocks combines two sub-layers, multi-head self-attention (MHA) along the time axis and a position-wise feed-forward network (FFN), under a pre-LayerNorm (pre-LN) formulation that places normalization *inside* each residual branch for training stability:

$$\hat{\mathbf{h}}^v = \text{MHA}(\text{LN}(\mathbf{h}^v), \text{LN}(\mathbf{h}^v), \text{LN}(\mathbf{h}^v)) \quad (7.22)$$

$$\mathbf{h}^v \leftarrow \mathbf{h}^v + \hat{\mathbf{h}}^v \quad (7.23)$$

$$\mathbf{h}^v \leftarrow \mathbf{h}^v + \text{FFN}(\text{LN}(\mathbf{h}^v)) \quad (7.24)$$

where $\mathbf{h}^v \in \mathbb{R}^{(T_{\text{obs}}+1) \times D_{\text{model}}}$ is the token sequence for vessel v . The two vessels are processed independently by flattening the vessel index into the batch dimension. MHA uses $H = 4$ attention heads:

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad d_k = D_{\text{model}}/H \quad (7.25)$$

The MHA sub-layer mixes information *across timesteps* but is purely linear in each token’s feature channels; it cannot transform a single token’s embedding non-linearly. The FFN supplies the missing per-token non-linearity. It is applied independently and identically to every timestep (sometimes called a 1×1 pointwise MLP) and follows the standard transformer recipe of Vaswani et al. [107]: a linear projection up to a hidden dimension of $4D_{\text{model}} = 512$, a GELU activation, dropout, and a linear projection back to D_{model} :

$$\begin{aligned} \text{FFN}(\mathbf{x}) &= W_2 \text{Dropout}(\text{GELU}(W_1 \mathbf{x} + \mathbf{b}_1)) + \mathbf{b}_2, \\ W_1 &\in \mathbb{R}^{4D_{\text{model}} \times D_{\text{model}}}, \quad W_2 \in \mathbb{R}^{D_{\text{model}} \times 4D_{\text{model}}}. \end{aligned} \tag{7.26}$$

This widen-then-narrow structure can be read as a learned key–value memory: each row of W_1 acts as a feature detector and the matching column of W_2 writes the resulting non-linear update back into the residual stream. Conceptually the alternation is therefore: MHA decides *which* other timesteps each token should look at, and the FFN decides *what to do* with the gathered evidence at that token. The same FFN architecture (same widening factor, GELU activation, dropout) is reused inside the Spatial Block of Section 7.4.1.4 and the Future-Intent Decoder of Equation (7.41).

7.4.1.4 Spatial Block

The Spatial Block enables information exchange between the two vessels at each timestep. The interaction context $\mathbf{I}_t$ is projected to D_{model} and added to the key and

value tokens before cross-vessel attention [150]:

$$\mathbf{KV}_t^v = \mathbf{h}_t^v + W_I \mathbf{I}_t \quad (7.27)$$

$$\hat{\mathbf{h}}_t^v = \text{MHA}(\text{LN}(\mathbf{h}_t^v), \text{LN}(\mathbf{KV}_t^v), \text{LN}(\mathbf{KV}_t^v)) \quad (7.28)$$

$$\mathbf{h}_t^v \leftarrow \mathbf{h}_t^v + \hat{\mathbf{h}}_t^v \quad (7.29)$$

$$\mathbf{h}_t^v \leftarrow \mathbf{h}_t^v + \text{FFN}(\text{LN}(\mathbf{h}_t^v)) \quad (7.30)$$

where attention is computed between the two vessel tokens at each individual timestep t (sequence length 2).

Design Rationale. Separating temporal (within-vessel) and spatial (cross-vessel) attention into alternating blocks follows the factored attention principle [150]: temporal blocks capture the inertial dynamics of each individual vessel, while spatial blocks model interaction. The alternating structure allows each attention type to operate on representations refined by the previous block, rather than both acting on raw projections. When the two vessels are far apart, the small $\mathbf{I}_t$ contribution degrades the Spatial Block gracefully to self-attention.

7.4.1.5 Encoder Outputs

After $N_{\text{enc}} = 4$ layer pairs and a final LayerNorm, the encoder yields:

$$\mathbf{e}_{\text{scene}} = [\mathbf{h}_{\text{CLS}}^0; \mathbf{h}_{\text{CLS}}^1] \in \mathbb{R}^{B \times 2D_{\text{model}}} \quad (7.31)$$

$$\mathbf{E}_{\text{seq}} = [\mathbf{h}_{1:T_{\text{obs}}}^0; \mathbf{h}_{1:T_{\text{obs}}}^1] \in \mathbb{R}^{B \times 2T_{\text{obs}} \times D_{\text{model}}} \quad (7.32)$$

where CLS tokens (position 0) aggregate vessel-level context and the remaining T_{obs} tokens per vessel carry the full temporal resolution.

7.4.2 Past Intent Classifier

The Past Intent Classifier maps the scene embedding to behavior-class logits via a two-layer MLP with dropout:

$$\begin{aligned} \mathbf{u} &= \text{Dropout}(\text{GELU}(W_1 \mathbf{e}_{\text{scene}} + b_1)) \in \mathbb{R}^{B \times 128} \\ \ell_{\text{past}} &= W_2 \mathbf{u} + b_2 \in \mathbb{R}^{B \times C} \end{aligned} \quad (7.33)$$

where $W_1 \in \mathbb{R}^{128 \times 2D_{\text{model}}}$ and $W_2 \in \mathbb{R}^{C \times 128}$. The predicted past behavior is $\hat{s} = \arg \max_j \ell_{\text{past}}^{(j)}$.

The concatenation $\mathbf{e}_{\text{scene}} = [\mathbf{h}_{\text{CLS}}^0; \mathbf{h}_{\text{CLS}}^1]$ is used rather than mean pooling because the CLS tokens are trained specifically to aggregate classification-relevant information

(analogous to BERT’s [CLS] token [109]), while the remaining tokens are reserved for the decoder cross-attention path.

7.4.3 Future Intent Predictor

The Future Intent Predictor (Figure 7.3) forecasts per-step behavioral labels over the T_{pred} -step horizon. It applies a linear temporal pooling over `enc_seq`, followed by a two-layer MLP that maps to intent embeddings and logits:

$$\mathbf{e}_{\text{pool}} = \frac{1}{2T_{\text{obs}}} \sum_{i=1}^{2T_{\text{obs}}} (\mathbf{E}_{\text{seq}})_i \quad (7.34)$$

$$\mathbf{E}_{\text{int},t} = W_{\text{int}} \mathbf{e}_{\text{pool}} + b_{\text{int}}, \quad \forall t = 1, \dots, T_{\text{pred}} \quad (7.35)$$

$$\ell_{\text{future},t} = W_{\text{cls}} \mathbf{E}_{\text{int},t} + b_{\text{cls}} \quad (7.36)$$

where $\mathbf{e}_{\text{pool}} \in \mathbb{R}^{B \times D_{\text{model}}}$, $\mathbf{E}_{\text{int}} \in \mathbb{R}^{B \times T_{\text{pred}} \times d_{\text{int}}}$, and $\ell_{\text{future},t} \in \mathbb{R}^{B \times C}$, with $d_{\text{int}} = 16$.

7.4.4 Trajectory Generator

The Trajectory Generator (Figure 7.4) is a Transformer decoder conditioned on three signals: encoder context, predicted future intent, and stochastic noise. It predicts position displacements which are accumulated to trajectories.

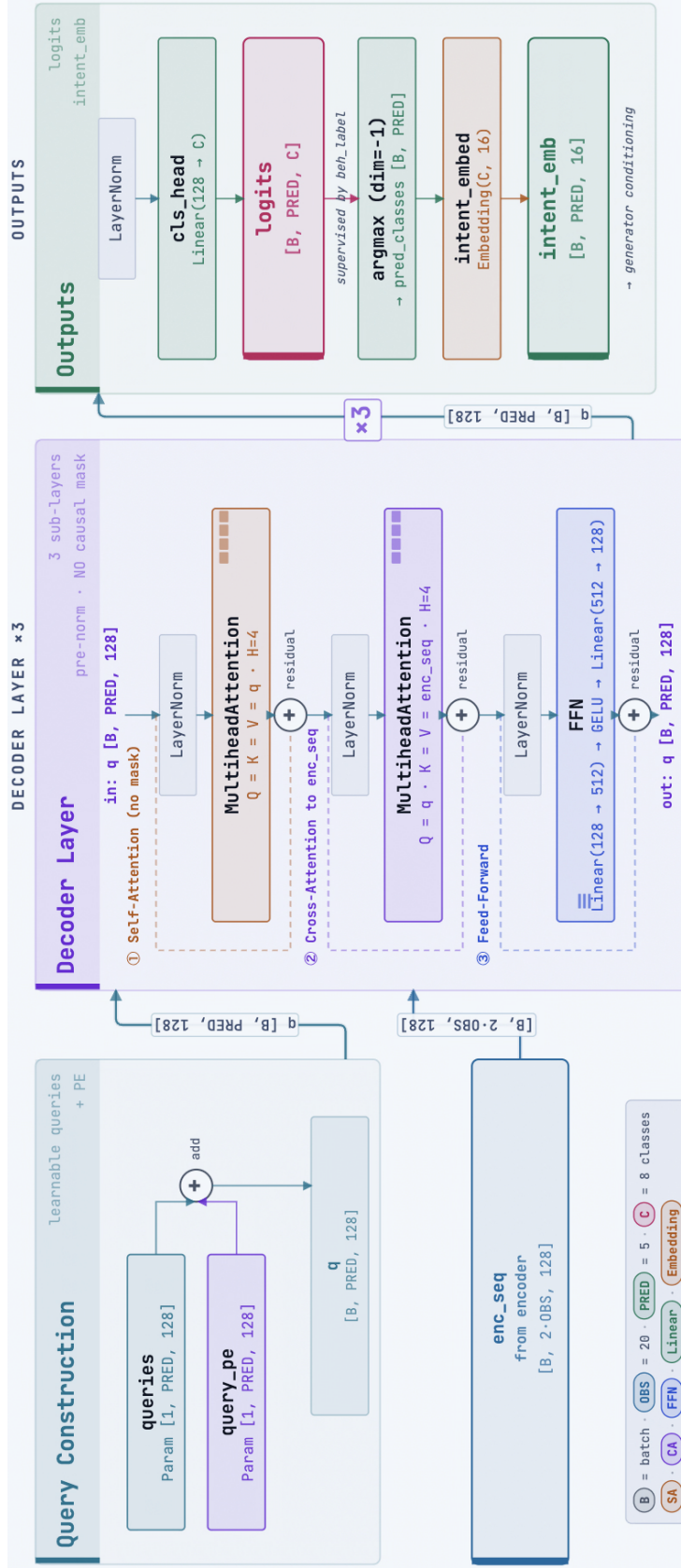


FIGURE 7.3: Future Intent Predictor. Temporal mean-pooling over the encoder sequence **enc_seq** produces $\mathbf{e}_{\text{pool}}$, which a two-layer MLP maps to per-step intent embeddings $\mathbf{E}_{\text{int},t}$ and class logits $\ell_{\text{future},t}$ over the T_{pred} -step horizon. The intent embeddings are reused as conditioning input by the Trajectory Generator and the IntentAwareCritic.

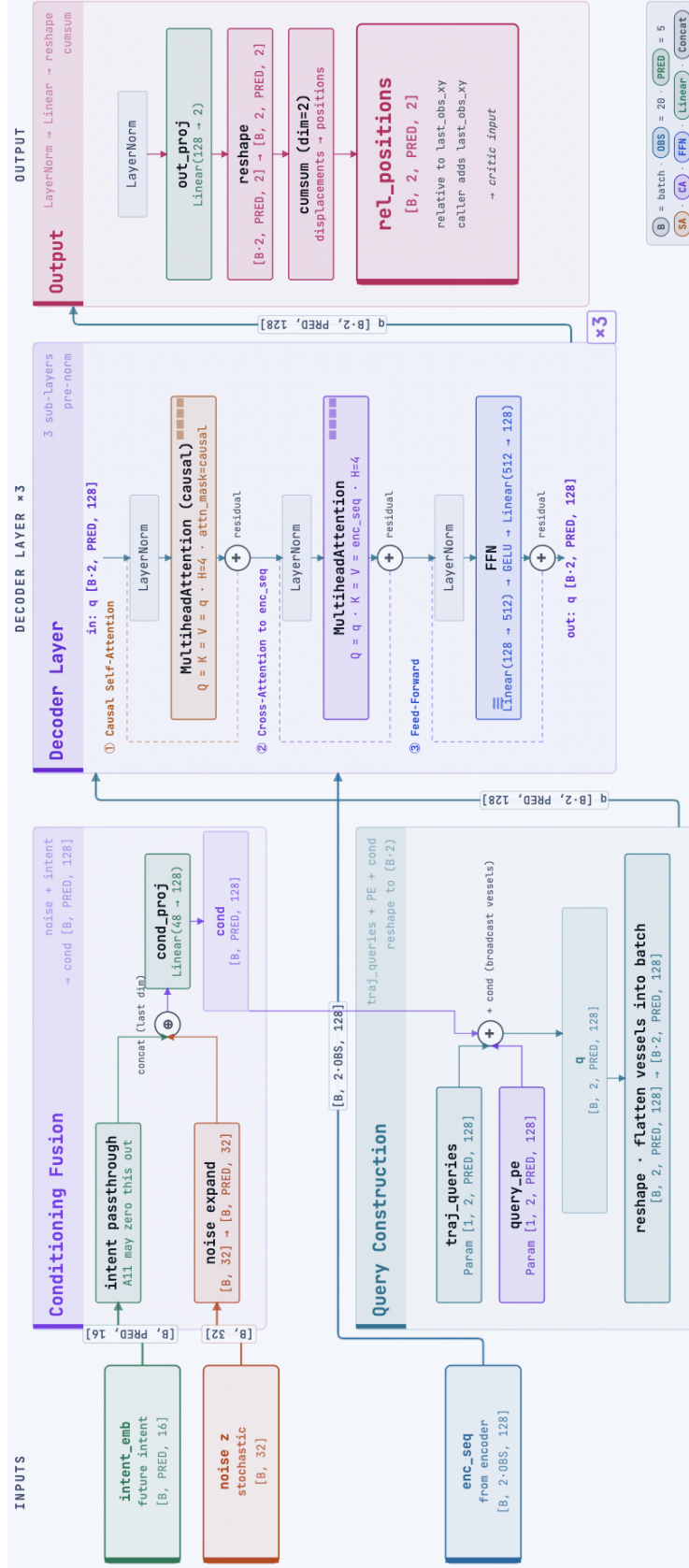


FIGURE 7.4: Trajectory Generator. Gaussian noise $\mathbf{z}$ and the per-step intent embedding $\mathbf{E}_{\text{int},t}$ are concatenated and projected to form the conditioning vector $\mathbf{c}_t$, which is added to learned trajectory queries. A three-layer Transformer decoder cross-attends to enc_seq , predicts per-step displacements, and accumulates them into the future trajectory $\hat{\mathbf{X}}$.

7.4.4.1 Conditioning

A Gaussian noise vector $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I}_{d_z})$, $d_z = 32$, is broadcast over the T_{pred} prediction steps, concatenated with the intent embedding, and projected to D_{model} :

$$\mathbf{c}_t = \text{GELU}(W_c [\mathbf{z}; \mathbf{E}_{\text{int},t}] + b_c), \quad \mathbf{c}_t \in \mathbb{R}^{B \times D_{\text{model}}} \quad (7.37)$$

7.4.4.2 Trajectory Queries

Per-vessel learned trajectory queries $\mathbf{Q} \in \mathbb{R}^{2 \times T_{\text{pred}} \times D_{\text{model}}}$ (with independent learned positional encoding $\mathbf{PE}^Q$) are augmented by the conditioning signal broadcast across the vessel dimension:

$$\mathbf{q}_t^v = \mathbf{Q}_t^v + \mathbf{PE}_t^{Q,v} + \mathbf{c}_t \quad (7.38)$$

7.4.4.3 Decoder Layers ($N_{\text{dec}} = 3$)

Each of the three decoder layers applies pre-LN causal self-attention, cross-attention to `enc_seq`, and a feed-forward network:

$$\mathbf{q} \leftarrow \mathbf{q} + \text{MHA}_{\text{causal}}(\text{LN}(\mathbf{q}), \text{LN}(\mathbf{q}), \text{LN}(\mathbf{q}); \mathbf{M}) \quad (7.39)$$

$$\mathbf{q} \leftarrow \mathbf{q} + \text{MHA}_{\text{cross}}(\text{LN}(\mathbf{q}), \text{LN}(\mathbf{E}_{\text{seq}}), \text{LN}(\mathbf{E}_{\text{seq}})) \quad (7.40)$$

$$\mathbf{q} \leftarrow \mathbf{q} + \text{FFN}(\text{LN}(\mathbf{q})) \quad (7.41)$$

The causal mask $\mathbf{M}$ is an upper-triangular Boolean matrix preventing prediction step t from attending to future steps $t' > t$, enforcing autoregressive consistency.

7.4.4.4 Displacement Prediction and Cumulative Summation

An output projection maps final decoder hidden states to 2D displacements:

$$\delta_t^v = W_{\text{out}} \mathbf{h}_t^v \in \mathbb{R}^2 \quad (7.42)$$

Positions are recovered by cumulative summation:

$$\hat{\mathbf{X}}_{\text{rel},t}^v = \sum_{\tau=1}^t \delta_\tau^v \in \mathbb{R}^2 \quad (7.43)$$

Absolute coordinates are obtained by adding the last observed position:

$$\hat{\mathbf{X}}_{\text{abs},t}^v = \hat{\mathbf{X}}_{\text{rel},t}^v + \mathbf{p}_{\text{last}}^v \quad (7.44)$$

The critic evaluates trajectories in relative space (the same scale as `real_future` labels); the final output reports in absolute meters for evaluation.

Stochasticity and Diversity. The noise vector $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I}_{32})$ is sampled independently for each of the $K = 20$ inference calls, producing K diverse trajectory hypotheses from a single encoder forward pass. The generator learns to map different noise realisations to qualitatively distinct futures conditioned on the same intent. The

signal that shapes those futures during training comes from the variant-specific critic (or, in MTITP-L2, from the L2 ground-truth target); see Section 7.5.

7.5 Training Variants

The shared backbone of Section 7.4 can be trained in three different ways. The differences among the variants are primarily based on two factors: (i) the scoring mechanism for trajectories during the joint phase, which can either employ a domain-agnostic WGAN-GP critic, have no critic at all (L2 loss), or incorporate the maritime-specific IntentAwareCritic; and (ii) the manner in which the scoring information is utilized in the generator update. Other components remain consistent across all variants, including the intent-side losses, the trajectory smoothness regularizer, the choice of optimizer, the two-phase warmup approach, and the guideline regarding detaching encoder outputs from any trajectory-related gradients. This section describes the shared components first, then walks through the three variants in turn: **MTITP-WGAN** (the baseline), **MTITP-L2** (no critic), and **MarITGAN** (intent-aware critic).

7.5.1 Shared Components

Past intent loss. Standard cross-entropy over the C behavior classes:

$$\mathcal{L}_{\text{past}} = -\frac{1}{B} \sum_{i=1}^B \log \hat{p}_{s^{(i)}}^{(i)} \quad (7.45)$$

Future intent loss. Mean cross-entropy per prediction step:

$$\mathcal{L}_{\text{future}} = -\frac{1}{BT_{\text{pred}}} \sum_{i=1}^B \sum_{t=1}^{T_{\text{pred}}} \log \hat{p}_{b^{(i,t)}}^{(i,t)} \quad (7.46)$$

The combined intent loss $\mathcal{L}_{\text{intent}} = \lambda_{\text{past}} \mathcal{L}_{\text{past}} + \lambda_{\text{future}} \mathcal{L}_{\text{future}}$ updates the encoder and both intent heads in every variant.

Trajectory smoothness loss. Physical plausibility is encouraged by penalizing the mean absolute second difference (jerk) of generated trajectories:

$$\mathcal{L}_{\text{smooth}} = \frac{1}{2B(T_{\text{pred}} - 2)} \sum_{b=1}^B \sum_{v=0}^1 \sum_{t=1}^{T_{\text{pred}}-2} \|\hat{\mathbf{X}}_{b,t+2}^v - 2\hat{\mathbf{X}}_{b,t+1}^v + \hat{\mathbf{X}}_{b,t}^v\|_1 \quad (7.47)$$

The smoothness term enters every variant’s generator loss with weight $\lambda_{\text{smooth}} = 0.1$.

Two-phase protocol. All three variants follow the same two-phase schedule. Phase 1 (epochs 1–15, the *warmup*) trains only the encoder and the intent heads under $\mathcal{L}_{\text{intent}}$, with no trajectory-side gradient. Phase 2 (epochs 16–115, the *joint phase*)

keeps running $\mathcal{L}_{\text{intent}}$ on the encoder and intent heads, then detaches `enc_seq` and `intent_emb` and runs the variant-specific trajectory training on top of those frozen embeddings. The detach step is what isolates the encoder from any pressure to fool a critic instead of discriminating intent.

Why detach encoder outputs from the trajectory-side update? After the intent update, a trajectory-side loss applied directly to the encoder could pull representations away from the intent objective, in the GAN variants, towards features that fool the critic; in MTITP-L2, towards features that are easy to regress from. Detaching `enc_seq` and `intent_emb` before any trajectory-side gradient ensures that intent supervision alone shapes the encoder, while the trajectory-side step trains only the generator (and the critic, where present).

Optimizer configuration. All parameter groups use Adam with $(\beta_1, \beta_2) = (0.5, 0.9)$, learning rate 10^{-4} , and weight decay 10^{-5} (Table 7.1). Cosine annealing with $\eta_{\min} = 10^{-6}$ is applied from epoch 16 onwards. Gradient norms are clipped to 1.0 before each update. $\beta_1 = 0.5$ follows the DCGAN/WGAN recommendation [123] to reduce momentum accumulation from early unstable gradient estimates; it is retained in MTITP-L2 for consistency, where it is harmless.

TABLE 7.1: Optimizer settings. All parameter groups use Adam with the same β values across all three training variants. The Critic group is absent in MTITP-L2.

Group	Modules	LR	(β_1, β_2)	Wt. decay
Main	Encoder + PastClassifier + FuturePredictor + Generator	10^{-4}	(0.5, 0.9)	10^{-5}
Critic	WGAN-Critic (MTITP-WGAN) or InstantAwareCritic (MarITGAN)	10^{-4}	(0.5, 0.9)	10^{-5}

7.5.2 Variant 1: MTITP-WGAN (Baseline)

MTITP-WGAN is the baseline model and the version of MTITP referred to by the chapter title and the headline numbers. It pairs the shared backbone with a domain-agnostic Wasserstein critic that scores future trajectories from past kinematics alone.

7.5.2.1 WGAN Critic

The critic scores the plausibility of a future trajectory *in context* of the observed past kinematics, taking a (past kinematics, future trajectory) pair and returning a scalar.

Input encoding. Past kinematics (x, y, v_x, v_y) and future trajectory (x, y) are each projected to D_{model} by separate linear layers, then concatenated along the time

axis per vessel:

$$\mathbf{h}_{\text{past}}^v = W_{\text{pk}} \mathbf{A}_{:4}^v \in \mathbb{R}^{T_{\text{obs}} \times D_{\text{model}}} \quad (7.48)$$

$$\mathbf{h}_{\text{fut}}^v = W_{\text{ft}} \hat{\mathbf{X}}^v \in \mathbb{R}^{T_{\text{pred}} \times D_{\text{model}}} \quad (7.49)$$

$$\mathbf{h}^v = [\mathbf{h}_{\text{past}}^v; \mathbf{h}_{\text{fut}}^v] \in \mathbb{R}^{(T_{\text{obs}}+T_{\text{pred}}) \times D_{\text{model}}} \quad (7.50)$$

Tokens from both vessels are concatenated to form a sequence of $2(T_{\text{obs}} + T_{\text{pred}}) = 50$ tokens. A learned positional embedding $\mathbf{PE}^C \in \mathbb{R}^{50 \times D_{\text{model}}}$ is added.

Architecture. Two standard Transformer encoder layers (pre-LN, $H = 4$ heads) process the 50-token sequence. A learnable CLS token aggregates the sequence; a linear head maps it to the scalar critic score:

$$\mathcal{C}(\mathbf{A}_{:4}, \hat{\mathbf{X}}) = W_{\text{out}}^C \mathbf{h}_{\text{CLS}} \in \mathbb{R} \quad (7.51)$$

The critic is unbounded (no sigmoid), as required by the Wasserstein objective. Spectral normalization is applied to all linear layers.

7.5.2.2 Losses

The critic minimises the WGAN-GP objective [119]:

$$\mathcal{L}_{\text{critic}} = \underbrace{\mathbb{E}[\mathcal{C}(\tilde{\mathbf{X}})] - \mathbb{E}[\mathcal{C}(\mathbf{X}_{\text{real}})]}_{\text{Wasserstein distance}} + \lambda_{\text{GP}} \underbrace{\mathbb{E}_{\tilde{\mathbf{X}}} \left[\left(\left\| \nabla_{\tilde{\mathbf{X}}} \mathcal{C}(\tilde{\mathbf{X}}) \right\|_2 - 1 \right)^2 \right]}_{\text{gradient penalty}} \quad (7.52)$$

where $\hat{\mathbf{X}} = \epsilon \mathbf{X}_{\text{real}} + (1 - \epsilon) \tilde{\mathbf{X}}$, $\epsilon \sim U[0, 1]$, $\tilde{\mathbf{X}}$ denotes generated trajectories, and $\lambda_{\text{GP}} = 10$.

The generator minimizes the negative critic score plus the smoothness regularizer:

$$\mathcal{L}_G^{\text{WGAN}} = \underbrace{-\mathbb{E}_{\mathbf{z}} [\mathcal{C}(\tilde{\mathbf{X}})]}_{\mathcal{L}_{\text{gen}}} + \lambda_{\text{smooth}} \mathcal{L}_{\text{smooth}}, \quad \lambda_{\text{smooth}} = 0.1 \quad (7.53)$$

7.5.2.3 Per-Iteration Pipeline

Figure 7.5 shows the three-step per-iteration data flow used by MTITP-WGAN, and Algorithm 3 states the corresponding pseudocode. The pipeline is the canonical pattern for the GAN variants; MarITGAN reuses the same shape with a richer generator loss, while MTITP-L2 omits the critic step entirely.

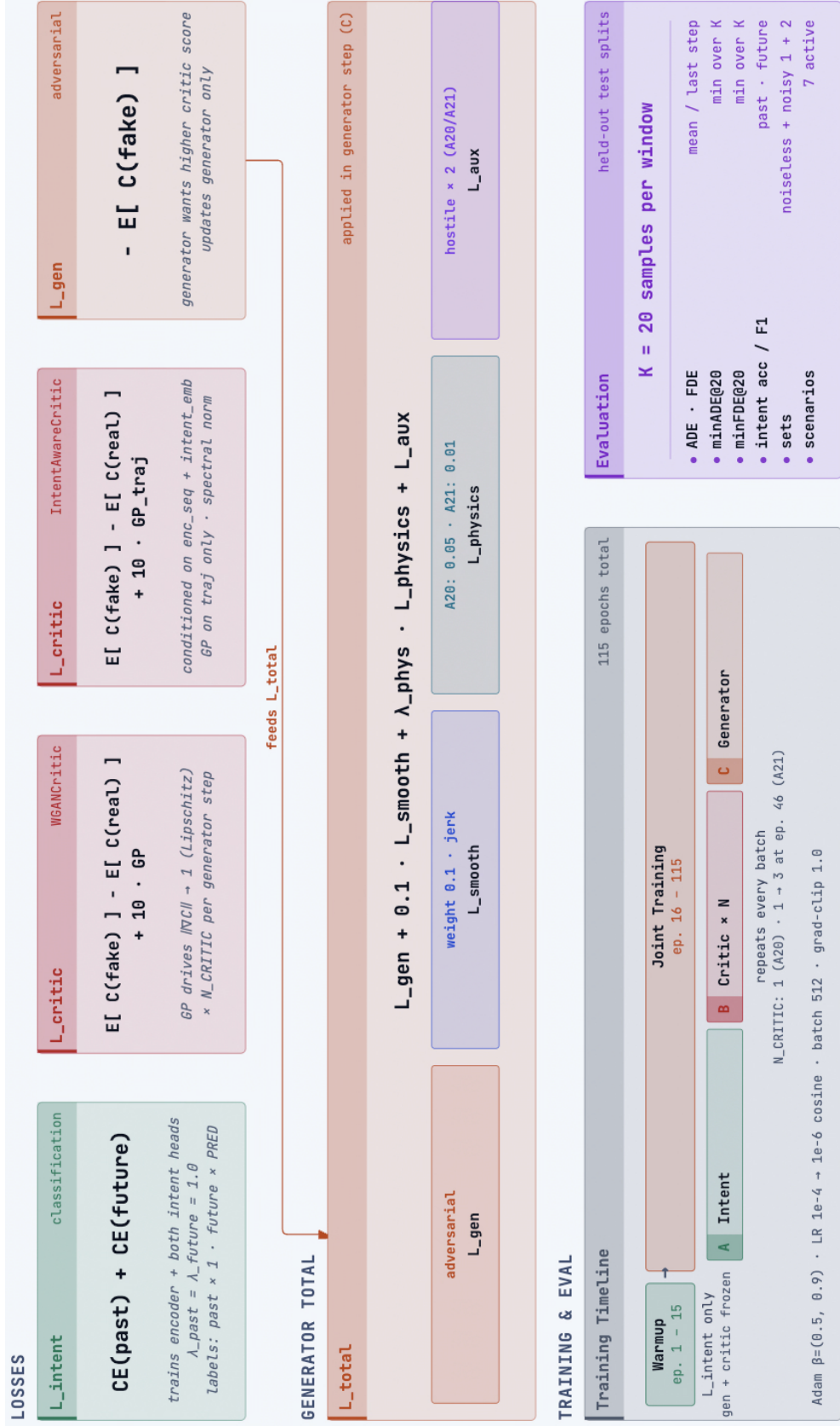


FIGURE 7.5: Per-iteration training pipeline for the GAN variants of MTTP. [A] **Intent** step updates the encoder and intent heads via $\mathcal{L}_{intent}$. After warmup, **enc_seq** and **intent_emb** are detached and reused by [B] **critic** step (N_c updates with WGAN-GP) and [C] **generator** step ($\mathcal{L}_G$ adds $\lambda_{phys}\mathcal{L}_{phys} + \mathcal{L}_{aux}$ for MarITGAN). MTTP-L2 omits step B and replaces step C with a direct L2 regression on the future trajectory. Detaching the encoder outputs prevents trajectory-side gradients from rewriting the intent representation.

Algorithm 3: MTITP-WGAN Two-Phase Training

Require: Training data $\mathcal{D}$, total epochs $E = 115$, warmup $E_w = 15$, $N_c = 1$,

$$\lambda_{\text{GP}} = 10, \lambda_{\text{smooth}} = 0.1$$

```

1: for epoch = 1 to  $E$  do

2:   for each minibatch  $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{p}_{\text{last}}, \mathbf{X}_{\text{real}}, s, b)$  do

3:     [A] Intent step: forward encoder + both intent heads

4:     Compute  $\mathcal{L}_{\text{intent}} = \lambda_{\text{past}}\mathcal{L}_{\text{past}} + \lambda_{\text{future}}\mathcal{L}_{\text{future}}$ 

5:     Backward through  $\theta_{\text{enc}}, \theta_{\text{pi}}, \theta_{\text{fi}}$ 

6:     if epoch  $> E_w$  then

7:       Detach enc_seq and intent_emb

8:       for  $n = 1$  to  $N_c$  do

9:         Sample  $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I}_{32})$ ; generate  $\tilde{\mathbf{X}}$ 

10:        [B] Critic step: compute and backprop  $\mathcal{L}_{\text{critic}}$ 

11:      end for

12:      [C] Generator step: sample  $\mathbf{z}$ ; compute and backprop  $\mathcal{L}_G^{\text{WGAN}}$ 

13:    end if

14:  end for

15:  if epoch  $> E_w$  then

16:    Cosine-anneal learning rates

17:  end if

18: end for

```

7.5.3 Variant 2: MTITP-L2

MTITP-L2 uses the same backbone with the critic removed. The trajectory generator is supervised directly against the ground-truth future, so no second network is trained and no critic step appears in the per-iteration loop.

7.5.3.1 Loss

The generator minimizes an L2 regression loss on the predicted future trajectory plus the shared smoothness term:

$$\mathcal{L}_G^{\text{L2}} = \mathcal{L}_{\text{L2}} + \lambda_{\text{smooth}} \mathcal{L}_{\text{smooth}}, \quad \mathcal{L}_{\text{L2}} = \frac{1}{2BT_{\text{pred}}} \sum_{b=1}^B \sum_{v=0}^1 \sum_{t=1}^{T_{\text{pred}}} \|\hat{\mathbf{X}}_{b,t}^v - \mathbf{X}_{b,t}^v\|_2 \quad (7.54)$$

The loss is the mean per-step Euclidean displacement error, i.e. Average Displacement Error (ADE) evaluated in the normalized relative-displacement space the generator produces; in particular, the training objective is the same quantity used as the headline trajectory metric, so the reported A13 ADE numbers are directly comparable to the value the optimizer minimizes.

7.5.3.2 Per-Iteration Pipeline

The pipeline degenerates to a two-step loop (no critic step). Phase 1 (epochs 1–15) is identical to the GAN variants: intent-only warmup. In Phase 2 (epochs 16–115), every iteration runs:

1. **Intent step.** Forward the encoder and both intent heads; backpropagate $\mathcal{L}_{\text{intent}}$ through $\theta_{\text{enc}}, \theta_{\text{pi}}, \theta_{\text{fi}}$.
2. **Generator step.** Detach `enc_seq` and `intent_emb`; sample $\mathbf{z}$; generate $\tilde{\mathbf{X}}$; backpropagate $\mathcal{L}_G^{\text{L2}}$ through θ_{gen} only.

The encoder is detached from the trajectory-side gradient in MTITP-L2 even though no adversarial pressure is present, in order to keep the comparison with the GAN variants controlled: any difference in encoder representations between the variants then comes from the intent supervision alone and not from coupling between the encoder and the generator update. The generator still receives stochastic noise $\mathbf{z}$ at inference, so MTITP-L2 still produces $K = 20$ trajectory samples; the L2 training signal does not actively reward diversity among them.

7.5.4 Variant 3: MarITGAN, Intent-Aware Interaction-Conditioned Critic

The MTITP-WGAN critic is domain-agnostic: it scores trajectory plausibility from kinematics alone, with no awareness of inter-vessel intent, relative motion geometry, or consistency between the predicted trajectory and the declared future intent class. The L2 ablation (Section 7.5.3) shows that this agnostic critic provides a weaker training signal than direct regression across all tested scenarios and horizons. **MarITGAN** (Maritime Intent-Trajectory GAN) replaces it with a purpose-built *IntentAwareCritic*

that incorporates three maritime-specific reasoning components. Two refinements are evaluated: A20 (the original MarITGAN) and A21 (*MarITGAN v2*, with a revised N_c schedule and a lighter physics constraint). Headline numbers for all three variants are reported side by side in Section 7.7; the A20-versus-A21 refinement is examined in Section 7.9.

7.5.4.1 IntentAwareCritic: Three Parallel Branches

The IntentAwareCritic (Figure 7.6) receives four inputs: past kinematics $\mathbf{A}_{:4} \in \mathbb{R}^{B \times 2 \times T_{\text{obs}} \times 4}$, future trajectory $\hat{\mathbf{X}} \in \mathbb{R}^{B \times 2 \times T_{\text{pred}} \times 2}$, encoder sequence $\mathbf{enc_seq} \in \mathbb{R}^{B \times 2 T_{\text{obs}} \times D_{\text{model}}}$, and future intent embeddings $\mathbf{e}_{\text{int}} \in \mathbb{R}^{B \times T_{\text{pred}} \times 16}$. Three parallel branches each produce a $d_{\text{branch}} = 80$ dimensional summary vector; these are concatenated and fused by an MLP to a scalar score:

$$\mathcal{C}_{\text{MarIT}}(\mathbf{A}_{:4}, \hat{\mathbf{X}}, \mathbf{enc_seq}, \mathbf{e}_{\text{int}}) = f_{\text{MLP}}([\mathbf{b}_1; \mathbf{b}_2; \mathbf{b}_3]) \in \mathbb{R} \quad (7.55)$$

Branch 1: Trajectory with intent conditioning. Future intent embeddings $\mathbf{e}_{\text{int}}$ are projected and added to the future trajectory token embeddings before a 2-layer pre-LN Transformer processes them. This branch enables the critic to penalize futures that contradict the declared intent class, such as a trajectory labeled as head-on

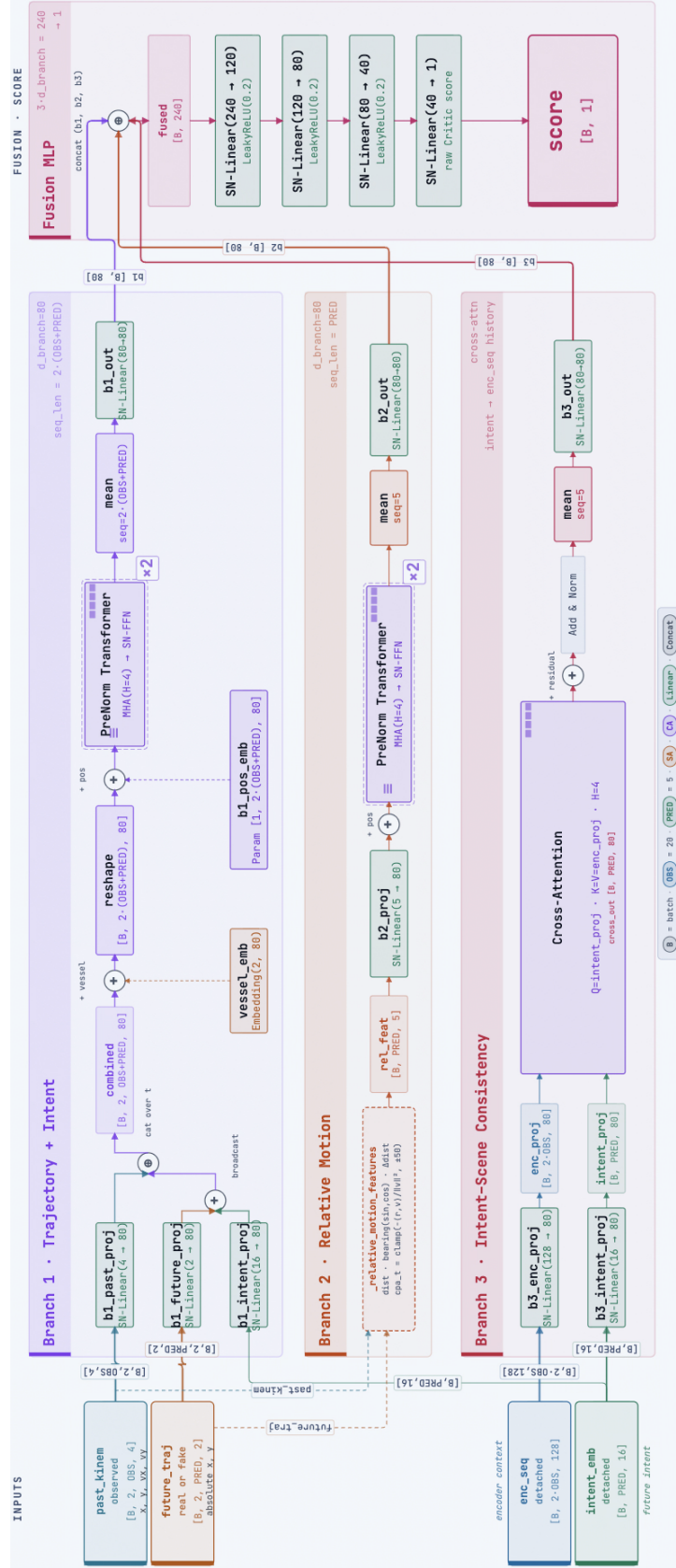


FIGURE 7.6: IntentAwareCritic used by MarITGAN. Three parallel branches reason over (1) the future trajectory with intent conditioning, (2) inter-vessel relative motion geometry computed inside the forward pass, and (3) intent-scene consistency via cross-attention between the future intent embeddings and *enc_seq*. Branch outputs are concatenated and fused by an MLP into a scalar Wasserstein score.

avoidance that never changes course:

$$\mathbf{b}_1 = \text{AvgPool}\left(\text{Transformer}_1\left(W_{\text{fut}}\hat{\mathbf{X}} + W_{\text{int}}\mathbf{e}_{\text{int}}\right)\right) \in \mathbb{R}^{d_{\text{branch}}} \quad (7.56)$$

Branch 2: Inter-vessel relative motion geometry. Five geometry features are computed *inside* the forward pass from the input tensors, requiring no additional dataloader inputs:

Feature	Description
d_t	Euclidean inter-vessel distance at each future step
$\sin \theta_t, \cos \theta_t$	Bearing angle (blue $\rightarrow$ red vessel)
Δd_t	Range rate (negative = closing)
t_{CPA}	Approximate time to closest point of approach (clamped ± 50 s)

A 2-layer pre-LN Transformer processes these geometry tokens, enabling the critic to penalize physically implausible futures such as head-on vessels that fail to diverge, herding vessels that scatter, or large discontinuities in Δd_t :

$$\mathbf{b}_2 = \text{AvgPool}(\text{Transformer}_2(W_{\text{geo}}[d_t; \sin \theta_t; \cos \theta_t; \Delta d_t; t_{\text{CPA}}])) \in \mathbb{R}^{d_{\text{branch}}} \quad (7.57)$$

Branch 3: Intent-scene consistency. Future intent embeddings cross-attend to the encoder sequence, evaluating the consistency between the declared intent and the

observed encounter history independently of the predicted trajectory shape:

$$\mathbf{b}_3 = \text{CrossAttn}(\mathbf{e}_{\text{int}}, \mathbf{enc_seq}) \in \mathbb{R}^{d_{\text{branch}}} \quad (7.58)$$

The three branch outputs are concatenated and passed through a fusion MLP ($240 \rightarrow 120 \rightarrow 80 \rightarrow 40 \rightarrow 1$). Spectral normalization is applied to all linear layers across all branches.

7.5.4.2 Losses

The critic loss retains the WGAN-GP form but restricts the gradient penalty to the interpolated future trajectory, since $\mathbf{enc_seq}$ and $\mathbf{e}_{\text{int}}$ are fixed conditioning inputs:

$$\mathcal{L}_{\text{critic}}^{\text{MarIT}} = \mathbb{E}[\mathcal{C}_{\text{MarIT}}(\hat{\mathbf{X}}_{\text{fake}})] - \mathbb{E}[\mathcal{C}_{\text{MarIT}}(\mathbf{X}_{\text{real}})] + \lambda_{\text{GP}} \mathbb{E} \left[\left(\|\nabla_{\hat{\mathbf{X}}_{\text{interp}}} \mathcal{C}_{\text{MarIT}}\|_2 - 1 \right)^2 \right] \quad (7.59)$$

Two extra terms act on the generator. A *physics loss* applies a soft ReLU penalty for trajectories that violate p99 kinematic bounds derived from the noiseless simulation data:

$$\mathcal{L}_{\text{phys}} = \langle \text{ReLU}(a_t - a_{\text{max}}) \rangle + \langle \text{ReLU}(\dot{\psi}_t - \dot{\psi}_{\text{max}}) \rangle \quad (7.60)$$

with $a_{\text{max}} = 8.0 \text{ m/s}^2$ and $\dot{\psi}_{\text{max}} = 0.35 \text{ rad/s}$ (p99 plus 20% margin), and $\langle \cdot \rangle$ denoting the mean over batch and time. The numbers are set by analyzing the datasets. A *scenario-weighted auxiliary loss* reweights the per-sample ADE to emphasize three

collision-risk classes: head-on, herd, and ram scenarios receive weight $2\times$; all others $1\times$. The full generator loss is:

$$\mathcal{L}_G^{\text{MarIT}} = -\mathbb{E}[\mathcal{C}_{\text{MarIT}}(\hat{\mathbf{X}}_{\text{fake}})] + \lambda_{\text{smooth}} \mathcal{L}_{\text{smooth}} + \lambda_{\text{phys}} \mathcal{L}_{\text{phys}} + \mathcal{L}_{\text{aux}} \quad (7.61)$$

7.5.4.3 Per-Iteration Pipeline

MarITGAN retains the three-step pipeline of Figure 7.5: intent step (A), then N_c critic steps (B) on $\mathcal{L}_{\text{critic}}^{\text{MarIT}}$, then a generator step (C) on $\mathcal{L}_G^{\text{MarIT}}$. The two refinements differ only in the value of N_c and in λ_{phys} :

- **A20 (MarITGAN).** Fixed $N_c = 1$ throughout the joint phase; $\lambda_{\text{phys}} = 0.05$.
- **A21 (MarITGAN v2).** Dynamic N_c schedule with $N_c = 1$ for the first 30 joint epochs (epochs 16–45) and $N_c = 3$ thereafter; $\lambda_{\text{phys}} = 0.01$.

The dynamic critic schedule is the principal change between A20 and A21. A constant $N_c = 3$ from the start of the joint phase allows the critic to outpace the generator early in training and produces vanishing-gradient behavior in our prototype runs. Holding $N_c = 1$ until the generator produces meaningful outputs, then increasing critic updates, maintains a balance between the two networks. The empirical effect is reported in Section 7.9.

7.6 Evaluation Metrics

7.6.1 Trajectory Metrics

Let $\hat{\mathbf{X}}^{(k)} \in \mathbb{R}^{2 \times T_{\text{pred}} \times 2}$ be the k -th sampled trajectory, $\mathbf{X}_{\text{gt}}$ the ground-truth future, and $k^* = \arg \min_k \text{ADE}(\hat{\mathbf{X}}^{(k)}, \mathbf{X}_{\text{gt}})$ the best-of- K index per sample.

$$\text{ADE} = \frac{1}{2T_{\text{pred}}} \sum_{v,t} \|\hat{\mathbf{X}}_t^{v,k^*} - \mathbf{X}_{\text{gt},t}^v\|_2, \quad (7.62)$$

$$\text{FDE} = \frac{1}{2} \sum_v \|\hat{\mathbf{X}}_{T_{\text{pred}}}^{v,k^*} - \mathbf{X}_{\text{gt},T_{\text{pred}}}^v\|_2, \quad (7.63)$$

$$\text{minADE@}K = \min_k \frac{1}{2T_{\text{pred}}} \sum_{v,t} \|\hat{\mathbf{X}}_t^{v,(k)} - \mathbf{X}_{\text{gt},t}^v\|_2, \quad (7.64)$$

$$\text{minFDE@}K = \min_k \frac{1}{2} \sum_v \|\hat{\mathbf{X}}_{T_{\text{pred}}}^{v,(k)} - \mathbf{X}_{\text{gt},T_{\text{pred}}}^v\|_2. \quad (7.65)$$

Under best-of- K selection $\text{ADE} \equiv \text{minADE@}K$ and $\text{FDE} \equiv \text{minFDE@}K$; both pairs are reported for compatibility with the deterministic and stochastic-generator literature.

7.6.2 Intent Metrics: Notation

For B test windows, let $\hat{s}^{(i)} = \arg \max_c p_\theta(s = c \mid \text{window}^{(i)})$ be the per-window scenario prediction and $\hat{b}^{(i,t)} = \arg \max_c p_\theta(b^{(i,t)} = c \mid \text{window}^{(i)})$ the per-step behavioral prediction at $t = 1, \dots, T_{\text{pred}}$, with ground truths $s^{(i)}$, $b^{(i,t)}$ and $\mathbf{1}[\cdot]$ the indicator. The

active class set $\mathcal{C}$ excludes the stationary ghost class (Section 7.3); n_c denotes the support of class c and $N = \sum_{c \in \mathcal{C}} n_c$. Per-class confusion counts are

$$\begin{aligned} \text{TP}_c &= \sum_i \mathbf{1}[s^{(i)} = c \wedge \hat{s}^{(i)} = c], \\ \text{FP}_c &= \sum_i \mathbf{1}[s^{(i)} \neq c \wedge \hat{s}^{(i)} = c], \\ \text{FN}_c &= \sum_i \mathbf{1}[s^{(i)} = c \wedge \hat{s}^{(i)} \neq c], \end{aligned} \tag{7.66}$$

with the analogous definition for the future-intent task summing over (i, t) .

7.6.3 Intent Accuracy

Accuracy captures how often the model's most confident guess is correct. Formally, top-1 accuracy is the fraction of predictions whose argmax class matches the ground-truth label, averaged over windows i for past intent and over window-step pairs (i, t) for future intent:

$$\text{past_acc} = \frac{1}{B} \sum_i \mathbf{1}[\hat{s}^{(i)} = s^{(i)}], \tag{7.67}$$

$$\text{future_acc} = \frac{1}{B T_{\text{pred}}} \sum_{i,t} \mathbf{1}[\hat{b}^{(i,t)} = b^{(i,t)}]. \tag{7.68}$$

7.6.4 Per-Class Precision, Recall, and F1

Per-class precision, recall, and F1 are computed from the confusion counts of Equation (7.66):

$$\text{Prec}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}, \quad \text{Rec}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}, \quad \text{F1}_c = \frac{2 \text{Prec}_c \text{Rec}_c}{\text{Prec}_c + \text{Rec}_c}. \quad (7.69)$$

The per-class accuracy used in the binary collapse of Section 7.7.3 satisfies $\text{Acc}_c = \text{TP}_c / (\text{TP}_c + \text{FN}_c) = \text{Rec}_c$.

7.6.5 Aggregations across Classes

Per-class F1 is reduced to a single scalar by either an unweighted (macro) or support-weighted average over the active class set:

$$\text{macroF1} = \frac{1}{|\mathcal{C}|} \sum_c \text{F1}_c, \quad \text{weightedF1} = \frac{1}{N} \sum_c n_c \text{F1}_c. \quad (7.70)$$

The same aggregations apply to Prec_c and Rec_c . Macro averages are used when class balance is the property under test; weighted averages match external baselines (Table 7.10).

7.6.6 Mean Average Precision

The Average Precision of class c is the area under its precision–recall curve $(\text{Prec}_c(r), r)$, traced by sweeping a threshold over the per-window softmax score $\hat{p}_c^{(i)} = p_\theta(s = c \mid \text{window}^{(i)})$, and the mean Average Precision is its unweighted class average:

$$\text{AP}_c = \int_0^1 \text{Prec}_c(r) \, dr, \quad \text{mAP} = \frac{1}{|\mathcal{S}|} \sum_{c \in \mathcal{S}} \text{AP}_c, \quad (7.71)$$

where $\mathcal{S} \subseteq \mathcal{C}$ is the set of classes present in the test batch. mAP is class-balanced and threshold-free, so it penalizes classifiers whose top-1 accuracy is inflated by class-prior memorization. Throughout Results and Discussion, the unqualified term “mAP” (and the column header **mAP** / **fut_mAP**) refers to *future-intent* mAP, mean Average Precision over the per-step future-intent classifier outputs. Past-intent mAP is reported only in the validation tables that explicitly distinguish past from future (Table 7.27).

7.7 Results

7.7.1 Overall Performance

Table 7.2 reports test-set trajectory and intent metrics for all three MTITP variants, aggregated across the seven active scenarios under the canonical evaluation protocol (noiseless dataset; $T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$; $K = 20$ samples).

TABLE 7.2: MTITP variants: overall test-set performance on the canonical configuration (noiseless dataset; $T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$; $K = 20$ samples; 7 active scenarios). ADE is the best-of- K minADE@20 by construction; FDE differs from minFDE@20 only at sub-decimeter precision and is reported alone. **Bold** = best per column.

Variant	ADE (m)	FDE (m)	Past Acc.	Future Acc.	Future mAP
MTITP-WGAN	3.42	5.97	99.2%	99.3%	99.4%
MTITP-L2	2.17	3.69	99.2%	99.2%	99.3%
MarITGAN v2	2.23	3.85	99.2%	99.2%	99.3%

The trajectory ranking is consistent across both displacement metrics: MTITP-L2 leads at 2.17m ADE, MarITGAN v2 follows at 2.23m, and the MTITP-WGAN baseline trails at 3.42m. Intent metrics are essentially indistinguishable across the three variants ($\sim 99.2\%$ past accuracy, $\sim 99.2\text{--}99.3\%$ future accuracy, $\sim 99.3\text{--}99.4\%$ future mAP), reflecting that the intent pathway is carried by the three-stream encoder and the multi-task heads independently of the trajectory loss. The intent scores approaching perfect accuracy reflect the structural discriminability of the scenario signatures captured by the joint Stream B/Stream C interaction representation. The L2/WGAN/MarITGAN trajectory ordering and the mechanism by which the intent-aware critic recovers near-parity with direct regression are analyzed in the ablation study (Section 7.9).

7.7.2 Per-Scenario Results

Table 7.3 reports per-scenario test ADE for all three MTITP variants alongside the MTITP-WGAN baseline’s intent metrics. Overall-level intent metrics differ by less than 0.1 percentage point across the three variants (Section 7.9, Table 7.20), so per-scenario intent is shown once. Qualitative trajectory comparisons across the three variants on representative encounters are deferred to Section 7.11.4 (Figures 7.17–7.21).

TABLE 7.3: Per-scenario test ADE (m) across MTITP variants on the canonical configuration (noiseless dataset; $T_{\text{pred}} = 5$; $K = 20$). Past/Future intent accuracy and future mAP are shown for the WGAN baseline; the L2 and MarITGAN v2 variants match within 0.1 percentage point at the overall level. **Bold** = lowest ADE per scenario.

Scenario	Diff.	WGAN ADE	L2 ADE	Mv2 ADE	Past Acc.	Future Acc.	Future mAP
benign	Easy	1.95	0.86	1.04	92.4%	99.3%	1.000
block	Med	1.98	1.19	1.41	99.7%	99.1%	0.999
cross	Med	1.93	1.02	1.16	97.7%	97.5%	0.995
overtake	Med	1.14	0.67	0.82	99.5%	99.4%	1.000
headon	Hard	1.99	0.88	0.96	99.5%	99.3%	1.000
ram	Hard	4.33	2.87	2.80	97.6%	97.4%	1.000
herd	Hard	10.63	7.66	7.40	99.9%	99.9%	1.000

The per-scenario ranking sharpens the overall picture. MTITP-L2 produces the lowest ADE on the five less-aggressive scenarios (benign, block, cross, overtake, headon), where the conditional mean is a strong predictor and direct regression has no incentive to spread mass. MarITGAN v2 surpasses L2 on the two collision-relevant scenarios that its scenario-weighted auxiliary loss upweights (ram, herd; Section 7.5.4), trading a small loss on the easier classes for a measurable gain where positional error carries direct collision risk. The MTITP-WGAN baseline trails on every scenario.

7.7.3 Hostile vs. Non-Hostile Operational View

For maritime surveillance the operationally relevant question is rarely *which* of seven scenario classes is unfolding but rather *whether* the encounter is hostile. We collapse the seven active classes into a binary hostile flag and report classification quality on each side of the split, plus trajectory error on each slice. The mapping uses domain semantics rather than statistical thresholds:

- **Non-hostile:** benign, cross, headon, overtake.
- **Hostile:** block, herd, ram.

Past intent (scenario, per-window) and future intent (`beh_label`, per-step) live in different label spaces but contain the same three hostile classes; the stationary class is excluded from both as elsewhere. *No additional binary classifiers or predictors were trained for this analysis*, the hostile flag is derived by post-hoc grouping of the

same seven-class `argmax` outputs already produced for Sections 7.7, so each model’s binary metrics are a direct re-expression of its seven-class predictions on the same test windows. The analysis is run on each model’s native test split: the three noiseless models (MTITP-WGAN, MTITP-L2 a.k.a. A13, and MarITGAN v2 a.k.a. A21) on the noiseless split, and the noisy-data ablation models (A18, A19, introduced in Section 7.9.4) on Noisy-1 and Noisy-2 respectively.

TABLE 7.5: Hostile vs. non-hostile binary classification for past intent (per-window). Per-class accuracy equals recall in the binary case. NH = benign/cross/headon/over-take; H = block/herd/ram. **Bold** = best per column across rows.

Model	Acc	Non-hostile			Hostile			macroF1
		Acc	Prec	F1	Acc	Prec	F1	
MTITP-WGAN <i>Noiseless</i>	0.998	0.997	0.997	0.997	0.998	0.998	0.998	0.997
A13 (MTITP-L2) <i>Noiseless</i>	0.997	0.998	0.995	0.996	0.997	0.999	0.998	0.997
A21 (MarITGAN v2) <i>Noiseless</i>	0.986	0.999	0.964	0.982	0.979	1.000	0.989	0.985
A18 (MTITP-L2) <i>Noisy-1</i>	0.994	0.993	0.991	0.992	0.994	0.996	0.995	0.993
A19 (MTITP-L2) <i>Noisy-2</i>	0.983	0.983	0.975	0.979	0.983	0.989	0.986	0.983

TABLE 7.6: Hostile vs. non-hostile for future intent (per-step `beh_label`). Columns as in Table 7.5. **Bold** = best per column across rows.

Model	Acc	Non-hostile			Hostile			macroF1
		Acc	Prec	F1	Acc	Prec	F1	
MTITP-WGAN <i>Noiseless</i>	0.996	0.995	0.997	0.996	0.997	0.995	0.996	0.996
A13 (MTITP-L2) <i>Noiseless</i>	0.995	0.996	0.994	0.995	0.995	0.996	0.996	0.995
A21 (MarITGAN v2) <i>Noiseless</i>	0.984	0.997	0.970	0.984	0.971	0.997	0.984	0.984
A18 (MTITP-L2) <i>Noisy-1</i>	0.992	0.991	0.992	0.991	0.992	0.991	0.992	0.992
A19 (MTITP-L2) <i>Noisy-2</i>	0.983	0.986	0.983	0.984	0.981	0.984	0.982	0.983

TABLE 7.7: Trajectory error sliced by ground-truth hostile flag (best-of- K , native test split). H = hostile, NH = non-hostile. ADE/FDE in meters. **Bold** = lowest (best) per column.

Model	NH-ADE	H-ADE	NH-FDE	H-FDE
MTITP-WGAN <i>Noiseless</i>	1.51	7.09	2.48	12.88
A13 (MTITP-L2) <i>Noiseless</i>	0.78	5.01	1.31	8.92
A21 (MarITGAN v2) <i>Noiseless</i>	0.83	4.97	1.38	8.92
A18 (MTITP-L2) <i>Noisy-1</i>	7.13	6.04	9.61	9.55
A19 (MTITP-L2) <i>Noisy-2</i>	33.07	18.83	39.87	24.92

Binary intent quality is not the bottleneck. Every model exceeds 98% on both classes for both past and future intent (Tables 7.5, 7.6). The seven-way accuracy

reported in Section 7.7 therefore translates almost losslessly into the hostile/non-hostile decision an operator actually needs. Even A19, trained on the heaviest-noise dataset, stays above 98% on both sides of the split for past and future intent. The binary collapse does not expose any failure mode that the seven-class metrics had hidden.

Two further structural patterns appear once the data is sliced this way: A21 (MarIT-GAN v2) shows a clearly asymmetric confusion structure (99.9% non-hostile recall vs. 97.9% hostile recall, with ~ 1.0 precision on both classes), and the trajectory-difficulty ordering between hostile and non-hostile classes inverts under sensor noise (A13 shows $H/NH = 6.4\times$ on noiseless data, while A19 inverts to $NH/H \approx 1.8\times$ on Noisy-2). Both patterns reflect specific design and deployment trade-offs and are interpreted in Section 7.11.

7.8 Comparison with External Baselines

The variant comparison in Section 7.7 situates MTITP against itself. This section places the model against external baselines on the same data: classical Kalman filters at matched configuration, and published deep-learning classifiers trained on the same noisy dataset.

7.8.1 Classical Trajectory Baselines

Tables 7.8 and 7.9 place MTITP in context with the classical baselines evaluated in Chapter 6, using the matched configuration ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$, Experiment B). All results use the natural test-split class distribution.

TABLE 7.8: MTITP vs. classical Kalman filter baselines at matched configuration ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$). KF metrics from Chapter 6, Experiment B. **Bold** = best per column.

Model	ADE (m)	FDE (m)	Intent
Constant Velocity	5.20	10.17	None
Standard UKF (4D)	5.40	10.41	None
UKF-CTRV (5D)	5.40	11.43	None
EKF / CTRV (5D)	5.06	10.52	None
MTITP-WGAN	3.42	5.97	99.2%
MTITP-L2	2.17	3.69	99.2%
MarITGAN v2	2.23	3.85	99.2%

TABLE 7.9: Per-scenario ADE (meters): MTITP vs. classical Kalman filters at matched configuration ($T_{\text{obs}} = 20$, $T_{\text{pred}} = 5$). KF metrics from Chapter 6, Experiment B. **Bold** = best per scenario. Stationary excluded from all evaluations.

Scenario	Diff.	CV	UKF-4D	EKF-CTRV	UKF-CTRV	MTITP
benign	Easy	0.054	0.059	0.052	1.140	1.951
overtake	Med	0.080	0.083	0.082	1.110	1.141
cross	Med	0.195	0.201	0.208	1.202	1.931
headon	Hard	0.102	0.109	0.103	1.147	1.985
block	Med	0.988	1.070	0.971	1.248	1.977
ram	Hard	3.971	3.883	6.719	7.501	4.328
herd	Hard	12.687	13.182	11.993	11.670	10.629

The aggregate gap to the best classical filter (EKF-CTRV, 5.06 m) widens substantially once the L2 and MarITGAN v2 variants enter the comparison: MTITP-L2 reduces aggregate ADE by $2.3\times$ over EKF-CTRV (2.17 m vs. 5.06 m), and MarITGAN v2 by $2.3\times$ as well (2.23 m). Per-scenario, however, the picture remains two-regime. Classical filters dominate non-hostile scenarios (benign, cross, headon, block), achieving near-zero to low single-digit ADE (0.05–1.0 m) that MTITP cannot match; MTITP pays a 1.9–2.0 m stochastic floor from its sampling-based generator on these scenarios. The advantage reverses on hostile scenarios: MTITP-WGAN already achieves the best ADE on herd (10.6 m vs. 11.7 m for UKF-CTRV) and on ram (4.3 m vs. EKF-CTRV’s 6.7 m arc failure), and the L2/MarITGAN v2 variants extend that lead further (Table 7.22).

The aggregate MTITP advantage is driven by herd, the largest class in the natural test split. MTITP additionally produces intent predictions (99.2% past accuracy, 99.4% future mAP) that no Kalman filter variant provides, enabling threat discrimination that is impossible from trajectory error alone.

Use case determines filter choice. Classical Kalman filters are unambiguously better for monitoring non-hostile traffic (near-zero ADE, no GPU required). MTITP’s advantage is concentrated on hostile scenarios (herd, ram) where multi-modal intent-conditioned generation outperforms all single-model filters. A hybrid deployment (classical filters for routine monitoring, MTITP activated on hostile intent signatures) exploits the strengths of both.

7.8.2 Published Deep-Learning Baselines

Trivial baseline and ablation comparisons identify where the headline numbers come from within MTITP itself; an external check is needed to situate them against published classifiers trained on the same `noisy_dataset_1`. Meepaganithage et al. [67] report Bi-RNN, Bi-GRU, Bi-LSTM, and Transformer classifiers under a sliding-window protocol on `Noisy_Dataset_1`, in both binary (hostile / non-hostile) and seven-way multiclass formulations. We compare against their two strongest configurations on the class-balanced split (Bi-LSTM, the strongest of the four backbones, and Transformer,

included as a comparable architectural family to MTITP’s encoder), as well as MTITP-L2, trained and evaluated on the same Noisy_Dataset_1 (A18). The two protocols differ only in how the dataset is partitioned.

TABLE 7.10: External-baseline comparison on Noisy_Dataset_1 (past-intent scenario classification). Meepaganithage et al. [67] use a class-balanced split; MTITP-L2 (A18) uses the natural distribution. Acc/Prec/Rec/F1 are weighted averages; binary-collapse rows use per-class entries from Table 7.5.

Model	Task	Acc	Prec	Rec	F1
Bi-LSTM [67]	Binary	0.903	0.899	0.903	0.901
Transformer [67]	Binary	0.870	0.871	0.870	0.865
MTITP-L2 (A18, Noisy-1)	Binary	0.994	0.994	0.994	0.994
Bi-LSTM [67]	7-class	0.880	0.895	0.880	0.879
Transformer [67]	7-class	0.740	0.767	0.740	0.733
MTITP-L2 (A18, Noisy-1)	7-class	0.984	0.975	0.969	0.972

TABLE 7.11: Per-class precision / recall / F1 for seven-way past intent on Noisy_Dataset_1. Bi-LSTM [67] (class-balanced split); MTITP-L2 (A18) (natural distribution, Table 7.23). **Bold** = higher per class per metric.

Class	Bi-LSTM [67]			MTITP-L2 (A18, Noisy-1)		
	Prec	Rec	F1	Prec	Rec	F1
Benign	0.98	0.73	0.84	0.987	0.928	0.957
Block	0.81	1.00	0.89	0.992	0.993	0.993
Cross	0.66	0.84	0.74	0.959	0.984	0.972
Head-on	0.90	0.97	0.93	0.985	0.991	0.988
Overtake	0.88	0.99	0.93	0.989	0.989	0.989
Ram	0.89	0.81	0.85	0.919	0.912	0.916
Herd	0.99	0.99	0.99	0.991	0.989	0.990
Weighted Avg	0.895	0.880	0.879	0.975	0.969	0.972

The comparison surfaces three points worth pulling out. MTITP-L2 outperforms the strongest published baseline on the same noisy data by $\sim 10\%$ on multiclass accuracy (98.4% vs. 88.0%) and $\sim 9\%$ on the binary task (99.4% vs. 90.3%). The largest gains appear on the classes where the Bi-LSTM is weakest, benign recall ($0.73 \rightarrow 0.93$), cross

(0.84 $\rightarrow$ 0.98), and ram (0.81 $\rightarrow$ 0.91), and on the binary collapse, where the Bi-LSTM’s 90.3% accuracy compresses against MTITP-L2’s >99% on both classes (Table 7.5). The published Transformer baseline outperforms Bi-LSTM by 14% on multiclass and 3% on binary, so a vanilla transformer over kinematic windows is not the architecture that defines the ceiling for this task. What carries MTITP from the published Bi-LSTM ceiling to the high-90s on the same noisy distribution is the combination of three input streams, multi-task supervision, and the encoder-cross-attention structure, not the transformer vs. recurrent backbone choice in isolation.

7.9 Ablation Study

Section 7.7 reported the headline three-way comparison between MTITP-WGAN, MTITP-L2, and MarITGAN v2 as peer variants of the same backbone. This section pivots from variant-level comparison to component-level attribution: each ablation removes or replaces exactly one design component while holding all other hyperparameters and the two-phase training procedure fixed, isolating which encoder streams, supervision heads, training objectives, and critic refinements account for the headline numbers. The ablations are organized into four groups, each defined in its own table: *architecture ablations* (Table 7.12; A3/A11/A12) that isolate structural choices, *trajectory loss ablations* (Table 7.14; A13/A15/A14/A16/A17) that compare the L2 training objective against WGAN-GP across prediction horizons, *robustness ablations* (Table 7.16; A18/A19) that train and evaluate on two progressively noisier datasets,

and *custom GAN ablations* (Table 7.18; A20/A21/A22) that introduce and refine the intent-aware, interaction-conditioned critic we term **MarITGAN** (Maritime Intent-Trajectory GAN), with A22 evaluating the refined architecture under sensor noise.

Each group addresses a specific sub-question:

1. *Does the joint interaction representation (pairwise geometry from Stream B and symbolic relational features from Stream C combined through the Interaction Fusion module) carry the COLREGs scenario signal, or is per-vessel kinematic history (Stream A) alone sufficient? (A3)*
2. *Is the explicit intent embedding in the generator the mechanism by which intent improves trajectory quality, or does the encoder’s cross-attention path already carry that signal? (A11)*
3. *Does removing intent classification supervision collapse trajectory quality, confirming that intent heads act as a regulariser for scene-discriminative encoder representations? (A12)*
4. *Does WGAN-GP adversarial training improve trajectory prediction accuracy over direct L2 regression when the generator architecture is identical, across prediction horizons $T_{pred} \in \{5, 10, 20\}$? (A13/A15/A14 vs. MTITP-WGAN/A16/A17)*

5. *Does MTITP retain intent discrimination accuracy as a complete classifier-plus-predictor under realistic sensor noise, even when the input perturbation inflates trajectory error? (RQ-MTITP2; A18, A19)*
6. *Does replacing the domain-agnostic WGAN-GP critic with a maritime-domain-specific intent-aware critic (MarITGAN) narrow the ADE gap between adversarial training and L2 regression, particularly on the three upweighted collision-risk classes (headon/herd/ram)? (A20)*
7. *Does refining the MarITGAN training schedule, dynamic N_{critic} ramping and a lighter physics constraint, close the remaining gap to L2 regression, particularly on the three upweighted classes? (A21)*
8. *Does the refined MarITGAN architecture retain its trajectory and intent advantages when trained on sensor-noisy data? (RQ-MTITP2; A22)*

TABLE 7.12: Architecture ablations: structural components removed from MTITP-WGAN.

Name	Component Removed / Changed	Design Question
A3: Stream A only	Streams B and C and the Interaction Fusion module removed; per-vessel kinematics (Stream A) only.	Does the joint interaction representation carry the intent signal?
A11: No intent cond.	Intent embedding removed from generator conditioning	Does intent conditioning guide trajectory generation?
A12: Trajectory only	Both intent classification heads removed	Does multi-task intent supervision improve trajectories?

TABLE 7.14: Trajectory loss ablations: L2 versus WGAN-GP across prediction horizons.

Name	Component Removed / Changed	Design Question
A13: L2, $T_{\text{pred}} = 5$	WGAN-GP critic by- passed; generator trained with L2 loss	Does adversarial training improve over direct regres- sion at the standard hori- zon?
A15: L2, $T_{\text{pred}} = 10$	Same as A13; prediction horizon extended to 10 steps	How does L2 degrade as the horizon grows?
A14: L2, $T_{\text{pred}} = 20$	Same as A13; prediction horizon extended to 20 steps	How does L2 degrade at the longest tested horizon?
A16: WGAN, $T_{\text{pred}} = 10$	MTITP-WGAN; predic- tion horizon extended to 10 steps	Does WGAN-GP main- tain advantage over L2 at medium horizons?
A17: WGAN, $T_{\text{pred}} = 20$	MTITP-WGAN; predic- tion horizon extended to 20 steps	Does WGAN-GP outper- form L2 at the longest hori- zon?

TABLE 7.16: Robustness ablations: training and evaluation on sensor-noisy datasets.

Name	Component Changed	Removed /	Design Question
A18: Noisy-1 data	Trained and evaluated on Noisy_Dataset_1 (moderate sensor noise)	on	Does MTITP maintain performance under moderate sensor noise?
A19: Noisy-2 data	Trained and evaluated on Noisy_Dataset_2 (high sensor noise)	on	Does intent discrimination survive at the higher noise level where trajectory accuracy degrades?

TABLE 7.18: MarITGAN ablations: intent-aware, interaction-conditioned critic and its noisy-data variant.

Name	Component Removed / Changed	Design Question
MarITGAN (A20)	WGAN-GP critic replaced with <i>IntentAwareCritic</i> (trajectory+intent, relative-motion geometry, intent-scene consistency branches); physics loss ($\lambda_{\text{phys}} = 0.05$) and scenario-weighted auxiliary L2 (headon/herd/ram $\times 2$) added	Does a domain-specific intent-aware critic narrow the ADE gap to L2 on the upweighted collision-risk classes?
MarITGAN v2 (A21)	A20 with dynamic N_{critic} (1 for joint epochs 1–30, then 3) and lighter physics weight ($\lambda_{\text{phys}} = 0.01$)	Does the refined schedule close the remaining gap to L2?
MarITGAN v2, noisy (A22)	A21 trained and evaluated on Noisy_Dataset_1 (same regime as A18)	Does refined MarITGAN retain its advantages under sensor noise?

7.9.1 Overall Ablation Results

Table 7.20 compares the architecture ablations (A3, A11, A12, A13) and both MarITGAN variants (A20, A21) at the standard $T_{\text{pred}} = 5$ horizon. Table 7.21 compares the L2 training objective against WGAN-GP across prediction horizons (A13/A15/A14 vs. MTITP-WGAN/A16/A17), using the same generator backbone in all cases.

TABLE 7.20: Architecture ablations: overall test-set metrics at $T_{\text{pred}} = 5$, noiseless dataset. ΔADE is relative to MTITP-WGAN. **Bold** marks best per column.

Model	ADE (m)	FDE (m)	ΔADE	Past Acc.	Future Acc.	Future mAP
MTITP-WGAN (baseline)	3.42	5.97	—	99.2%	99.3%	99.4%
A3: Stream A only	3.43	6.11	+0.01	96.5%	86.2%	66.2%
A11: No intent cond.	3.26	5.76	−0.16	99.2%	99.2%	99.4%
A12: Trajectory only	4.67	8.00	+1.25	—	—	—
A13: L2 bypass	2.17	3.69	−1.25	99.2%	99.2%	99.3%
A20: MarITGAN	2.40	4.47	−1.02	99.2%	99.3%	99.4%
A21: MarITGAN v2	2.23	3.85	−1.19	99.2%	99.2%	99.3%

TABLE 7.21: Trajectory loss comparison: L2 vs. WGAN-GP at $T_{\text{pred}} \in \{5, 10, 20\}$. Same generator architecture; only training loss and critic activation differ. Intent accuracy ($\sim 99\%$) is identical across configurations and omitted. **Bold** = lower ADE per horizon.

	$T_{\text{pred}} = 5$		$T_{\text{pred}} = 10$		$T_{\text{pred}} = 20$	
Scenario	L2	WGAN	L2	WGAN	L2	WGAN
benign	0.86	1.95	2.04	3.67	3.85	8.52
block	1.19	1.98	2.32	3.47	5.03	7.70
cross	1.02	1.93	2.08	3.42	4.45	6.46
overtake	0.67	1.14	1.48	2.42	3.00	4.21
headon	0.88	1.99	1.56	2.53	3.16	4.65
ram	2.87	4.33	4.47	8.75	8.39	15.33
herd	7.66	10.63	15.87	23.86	35.90	56.53
Overall	2.17	3.42	4.26	6.88	9.11	14.77

7.9.2 Per-Scenario Ablation Breakdown

Table 7.22 breaks the ablation results down by scenario, exposing which behaviors drive the aggregate ADE differences between variants.

TABLE 7.22: Per-scenario ADE (m) across ablations. **Bold** marks best (lowest) ADE per scenario on Noiseless Dataset and $T_{\text{pred}} = 5$.

Scenario	WGAN	A3	A11	A12	A13 (L2)	A20	A21 (Mv2)
benign	1.95	1.91	1.80	2.41	0.86	1.23	1.04
block	1.98	1.58	1.76	2.35	1.19	1.91	1.41
cross	1.93	2.15	1.91	2.94	1.02	1.25	1.16
overtake	1.14	1.21	1.33	1.82	0.67	1.00	0.82
headon	1.99	1.38	1.54	3.76	0.88	0.91	0.96
ram	4.33	4.72	4.46	6.47	2.87	2.90	2.80
herd	10.63	11.05	10.01	12.97	7.66	7.61	7.40
Overall	3.42	3.43	3.26	4.67	2.17	2.40	2.23

Mv2 = MarITGAN v2 (A21). A21 surpasses L2 on ram and herd; on the remaining five

scenarios A13 retains the lowest ADE.

7.9.3 Analysis

A3: The interaction representation carries the intent signal, not the trajectory signal alone. Removing the entire interaction pathway, Streams B and C and the Interaction Fusion module that combines them, leaves trajectory ADE virtually untouched (+0.01 m) while collapsing intent quality: future-intent mAP falls from 99.4% to 66.2% and future-intent accuracy from 99.3% to 86.2%. Per-vessel kinematic

history (Stream A) is sufficient for short-horizon position extrapolation regardless of scenario type, but distinguishing a crossing from a head-on encounter requires the joint interaction representation that only the fused pairwise channel provides: Stream B’s continuous pairwise geometry (inter-vessel distance, relative bearing, CPA proxies) and Stream C’s symbolic relational features (port/starboard side, CPA-trend class, heading-change class) are combined into a per-step interaction context that no per-vessel kinematic signal alone can recover.

A11: Intent conditioning is already implicit in the encoder. Removing the explicit intent embedding from generator conditioning produces a slight *improvement* in ADE (-0.16 m). The encoder’s `enc_seq`, delivered to the generator via cross-attention, already carries intent-sensitive information; the explicit intent embedding adds a redundant and slightly noisy conditioning signal on top of it. Since intent heads remain fully active in A11, this result does not diminish the case for intent conditioning, it identifies *where* that conditioning actually operates.

A12: Multi-task supervision builds a useful encoder. Removing both intent heads results in the sharpest degradation in the ablation study: $+1.25$ m ADE and $+2.03$ m FDE. Without classification supervision, the encoder is trained solely on the adversarial trajectory signal, which is a weaker, less discriminative objective. The GAN loss does not force the encoder to learn which scenario is unfolding; the intent

heads do. Intent classification regularizes the encoder toward scene-discriminative features, which benefit trajectory generation as a side effect.

A13/A15/A14/A16/A17: L2 training outperforms WGAN-GP at every horizon. An important distinction must be drawn before interpreting these results.

All five configurations share the identical generator architecture. The L2 configurations (A13/A15/A14) are MTITP-L2 instances, where the critic is removed and the generator is trained with direct ground-truth supervision. Without the critic, there is no minimax game and the noise vector $\mathbf{z}$ receives no gradient pressure to produce diverse samples; the generator simply learns to ignore $\mathbf{z}$ and predict the conditional mean. The MTITP-WGAN/A16/A17 configurations activate the critic and train the generator with the Wasserstein adversarial loss, making them intent-conditioned WGAN-GPs as described.

L2 outperforms WGAN-GP at every tested horizon by a consistent $\sim 37\%$ ADE margin. At $T_{\text{pred}} = 5$: A13 (L2: 2.17 m) vs. MTITP-WGAN (3.42 m). At $T_{\text{pred}} = 10$: A15 (L2: 4.26 m) vs. A16 (WGAN: 6.88 m). At $T_{\text{pred}} = 20$: A14 (L2: 9.11 m) vs. A17 (WGAN: 14.77 m). Activating the WGAN-GP critic does not recover any distributional advantage even as prediction uncertainty grows with the horizon.

Intent accuracy is unaffected by the trajectory training objective. At $T_{\text{pred}} = 5$, both A13 and MTITP-WGAN achieve 99.2% past accuracy and 99.3–99.4% future mAP. The same parity holds at $T_{\text{pred}} = 10$ ($\sim 99.1\%$) and $T_{\text{pred}} = 20$ (~ 99.1 – 99.3%).

The multi-task architecture (three-stream encoding, symbolic feature integration, joint intent supervision) is the primary determinant of intent quality; the generator’s training objective is irrelevant to it.

Both objectives degrade with horizon at similar rates. L2 grows $\times 2.0$ at $T_{\text{pred}} = 10$ and $\times 4.2$ at $T_{\text{pred}} = 20$ relative to $T_{\text{pred}} = 5$; on herd: $7.66 \text{ m} \rightarrow 15.87 \text{ m} \rightarrow 35.90 \text{ m}$. WGAN-GP degrades at a comparable rate ($\times 2.0$ and $\times 4.3$), confirming that adversarial training does not provide better long-horizon trajectory coverage with this generator architecture.

The conditional WGAN-GP generator architecture is valuable for its multi-modal sampling. When intent conditioning already strongly constrains the generator’s output space, direct ground-truth supervision (L2) produces lower ADE than adversarial training by a consistent $\sim 37\%$ margin across all tested horizons ($T_{\text{pred}} \in \{5, 10, 20\}$). The GAN architecture matters; the adversarial training objective is not the source of trajectory quality. To minimize ADE as a deployment metric, the L2 configuration is the recommended choice; the WGAN-GP configuration remains valuable when multimodal coverage and sample diversity are required.

A20: Domain-specific critic (MarITGAN) closes the ADE gap on the three upweighted classes. The domain-agnostic WGAN-GP critic is the root cause of the L2/GAN gap identified above: it cannot distinguish a trajectory that is geometrically plausible from one that is intent-consistent and physically valid for the

specific maritime encounter underway. A20 replaces this critic with a purpose-built *IntentAwareCritic* operating on three parallel branches. Branch 1 injects future intent embeddings directly into trajectory token representations, forcing the critic to detect trajectories that contradict the declared intent class. Branch 2 computes five inter-vessel geometry features (inter-vessel distance, bearing, range-rate, and approximate CPA time) inside the critic’s forward pass, enabling detection of physically absurd futures (e.g. head-on vessels that fail to diverge, herding vessels that scatter). Branch 3 performs cross-attention between future intent embeddings and the encoder sequence, verifying that the declared intent is plausible given the observed encounter history. An auxiliary scenario-weighted L2 term ($\times 2$ for the three upweighted classes headon, herd, ram, where positional error carries direct collision risk) and a physics loss penalizing excess acceleration and turn-rate complete the training signal. This configuration is termed **MarITGAN** (Maritime Intent-Trajectory GAN).

MarITGAN achieves 2.40 m overall ADE: a 30% improvement over the baseline WGAN-GP (3.42 m) and within 0.23 m of the L2 upper bound (2.17 m). The gain is concentrated where the design targets it. On the three upweighted classes the gap to L2 collapses: headon (0.91 m vs. 0.88 m), ram (2.90 m vs. 2.87 m), and herd (7.61 m vs. 7.66 m, MarITGAN actually lower). On the four classes the auxiliary loss does not target (benign, block, cross, overtake) MarITGAN sits between L2 and WGAN-GP, reflecting the scenario-weighting design. Intent accuracy is unchanged (99.2% past, 99.4% future mAP), confirming that modifying the critic has no effect on the intent pathway.

This pinpoints the source of the L2/GAN gap. The baseline critic was domain-agnostic and therefore provided a weaker training signal than direct ground-truth supervision. Once the critic carries the same domain knowledge (inter-vessel geometry, intent-trajectory consistency, physical plausibility bounds), the adversarial objective recovers near-parity with L2 on the upweighted classes the auxiliary loss prioritizes, while the generator keeps its multi-modal sampling. The residual 0.23 m gap motivates the A21 refinement.

A21: Refined MarITGAN training schedule closes the gap to L2. A20 revealed two sources of training instability: gradient conflict between the physics penalty and the intent loss in early joint training (occasionally causing NaN spikes), and a critic trained at $N_{\text{critic}} = 1$ throughout, insufficient update frequency for the more complex *IntentAwareCritic* to keep pace with the generator once the encoder has stabilized. A21 addresses both: the physics weight is reduced from $\lambda_{\text{phys}} = 0.05$ to 0.01 (reducing gradient conflict while retaining the physical plausibility constraint), and a dynamic schedule ramps N_{critic} from 1 (joint epochs 1–30) to 3 (joint epochs 31–100), allowing the critic to strengthen its training signal once the encoder representations have converged. The scenario-weighted auxiliary loss is retained from A20.

A21 achieves 2.23 m overall ADE, which is a further 7% improvement over A20 (2.40 m) and within **0.06 m** of the L2 upper bound (A13: 2.17 m). The per-scenario breakdown sharpens the finding: A21 *surpasses L2* on ram (2.80 m vs. 2.87 m) and herd (7.40 m vs. 7.66 m), two of the three upweighted classes the auxiliary loss targets. On the

remaining five scenarios, A13 (L2) retains its advantage, consistent with the scenario-weighting design: the elevated penalty applies only to the three upweighted classes (headon/herd/ram). FDE improves markedly over A20 across all scenarios (3.85 m vs. 4.47 m overall), indicating that the stabilized critic tightens the endpoint distribution in addition to reducing average displacement. Intent accuracy is essentially unchanged (99.2% past and future), confirming that the training schedule change does not affect the intent pathway.

The 0.23 m residual gap between A20 and L2 therefore came from training instability and undersized critic updates, not from any fundamental limit of the intent-aware adversarial objective. Once those two issues are corrected, the MarITGAN v2 critic delivers a training signal that matches or surpasses direct L2 supervision on the upweighted classes the auxiliary loss prioritizes, and the generator keeps its multi-modal sampling.

7.9.4 Robustness to Sensor Noise (A18, A19, and A22)

Ablations A18 and A19 train and evaluate MTITP (L2 objective, identical architecture to A13) on two progressively noisier versions of the same scenario collection. *Noisy_Dataset_1* (A18) adds moderate simulated AIS/GPS positional and velocity jitter; *Noisy_Dataset_2* (A19) applies a higher noise level. A22 trains and evaluates the refined MarITGAN (A21 architecture) on *Noisy_Dataset_1*, allowing a direct comparison of the intent-aware adversarial objective against L2 regression under sensor noise.

All datasets are fully self-contained train/val/test splits, so each model is trained and evaluated on the same noise regime. We first characterise the visual signature of the noise on the raw input the model sees; the quantitative tables (7.23, 7.24, 7.25) follow.

7.9.4.1 Dataset Noise Characterisation

The pattern of asymmetric per-scenario degradation reported in the tables below is most easily understood by inspection of the raw input. For each behavior and each dataset we plot full-length, start-to-finish vessel trajectories taken directly from the CSV files, in raw latitude and longitude. For every (scenario, dataset) pair we picked the encounter at the 50th percentile of minimum vessel-pair distance, which favors a representative interaction (not the closest near-miss or the farthest disengagement) and keeps the relative difficulty consistent across columns. Noiseless_Dataset and Noisy_Dataset_1 share the same file naming, so when an encounter is selected from the noiseless side, the same encounter (with sensor noise applied) is shown in the Noisy_Dataset_1 column. Noisy_Dataset_2 was generated independently and uses a different ID range, so the third column shows a structurally similar but not identical encounter from the heavier-noise distribution.

Visual signature of noise across datasets. Across all seven behaviors, the Noisy_Dataset_1 column preserves the macro-shape of the corresponding noiseless trajectory: the start and end positions are very close to those in the noiseless column, and the overall path silhouette is recognizable. The mid-track is perturbed by localized

positional jitter on the order of tens of meters, consistent with the simulated AIS/GPS noise model used to generate the dataset. The `Noisy_Dataset_2` column shows a different encounter from a heavier-noise distribution. The path is no longer a perturbation of the noiseless trace, and the noise envelope is large enough to be comparable in magnitude to the inter-vessel separation that defines the scenario class.

Non-hostile behaviors. The non-hostile encounters (benign, cross, headon, overtake) are characterized by long, near-linear or gently curving paths spanning hundreds to thousands of meters. The signal that distinguishes one non-hostile behavior from another is contained in small differences in relative bearing or relative course between the two vessels. On `Noisy_Dataset_1`, the positional jitter is comparable in magnitude to those small differences, so the per-window features the model uses (relative position, relative heading, change of bearing direction) become noisy and less reliable. The model sees a trajectory that is approximately linear in shape but inconsistent in detail, and the result is the order-of-magnitude increase in ADE reported in Table 7.23. On `Noisy_Dataset_2`, the perturbations are large enough that the within-window relative geometry is effectively erased, which is consistent with the much larger increases reported for all four non-hostile behaviors in Table 7.24.

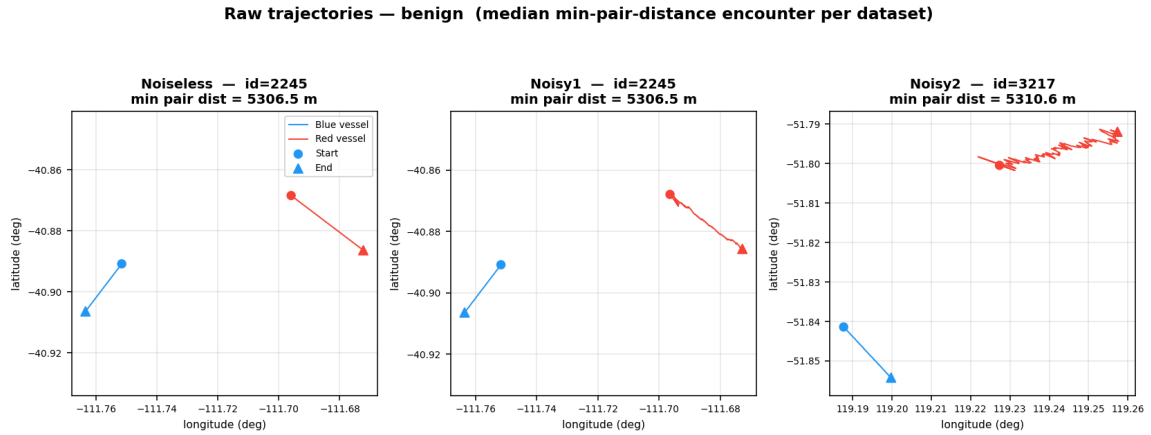


FIGURE 7.7: Benign encounter across the three datasets. The two vessels travel in roughly parallel directions with no engagement. The minimum vessel-pair distance is large (about 5,300 m), and the heading offset that separates benign from cross is small. The Noisy_Dataset_1 column shows how the simulated AIS/GPS jitter perturbs the path while preserving the start and end positions. The marker ● denotes the start position and ▲ the end position.

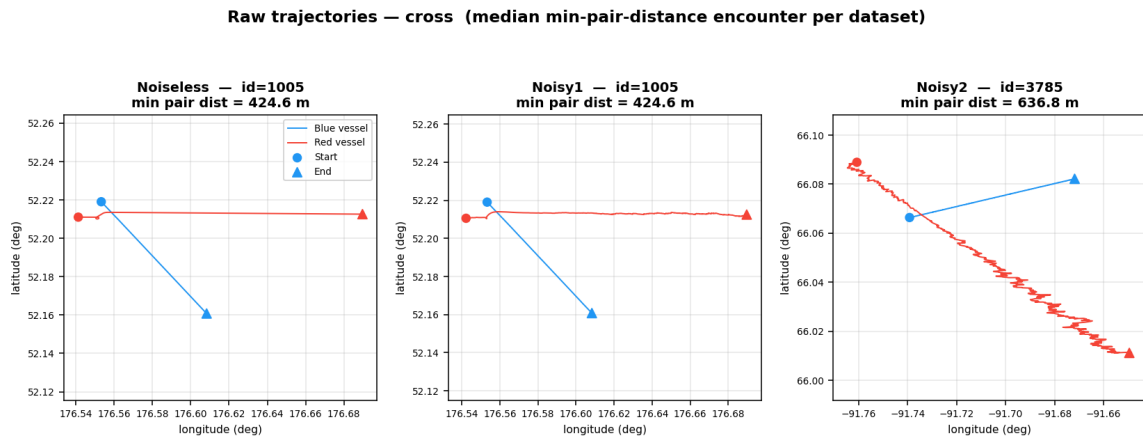


FIGURE 7.8: Cross encounter across the three datasets. The two trajectories meet at an oblique angle. The Noisy_Dataset_1 column retains the macro crossing geometry, but the precise crossing point and timing, which the model has to regress, are blurred by the mid-track jitter. The Noisy_Dataset_2 column shows a structurally similar but independently sampled encounter at higher noise.

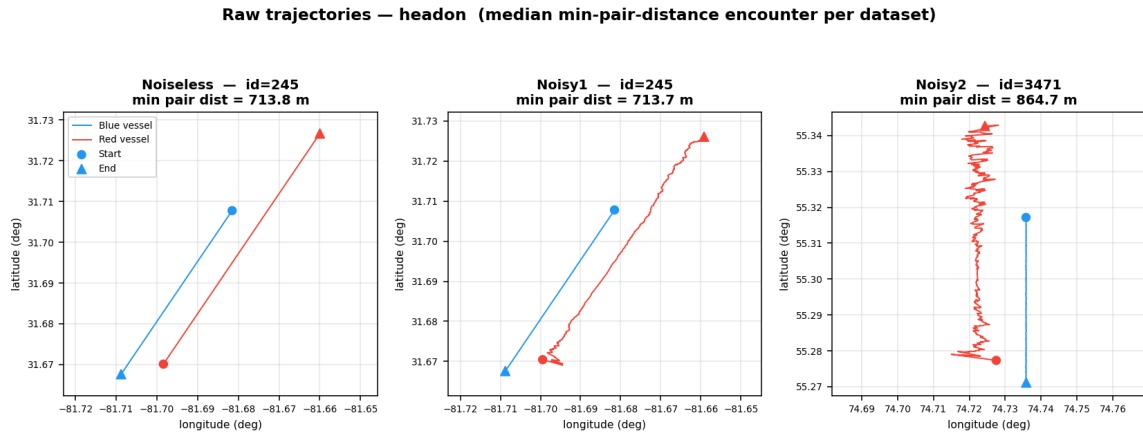


FIGURE 7.9: Headon encounter across the three datasets. The two vessels approach on near-reciprocal headings, and the discriminative signal is a small lateral offset that determines the passing side. The Noisy_Dataset_1 jitter is comparable in magnitude to that offset, which is consistent with the increase in headon ADE from 0.88 m on the clean baseline to 7.14 m on Noisy_Dataset_1.

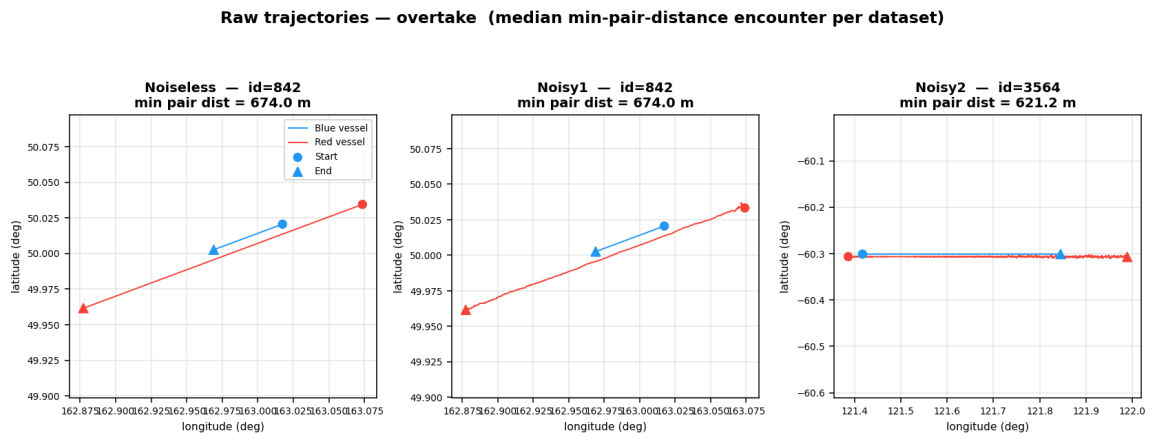


FIGURE 7.10: Overtake encounter across the three datasets. A faster vessel passes a slower one travelling in the same general direction. The defining feature is a small relative-velocity vector, which is the kind of low-magnitude kinematic cue that does not survive the Noisy_Dataset_1 perturbation and is replaced by a different sample geometry under Noisy_Dataset_2.

Hostile behaviors. The hostile encounters (block, herd, ram) are characterized by sharp, short-duration features: a heading reversal in ram, repeated cut-ins in herd, and an abrupt cross-the-bow maneuver in block. These features are large in displacement

and concentrated in time, so the same Noisy_Dataset_1 jitter that obscures the small relative-bearing signal in non-hostile behaviors is small relative to the displacement of the hostile maneuver itself. The model can still recover the maneuver from the noisy input, and the result is the much milder degradation seen in Table 7.23. The herd improvement (7.66 m on the clean baseline, 5.96 m on Noisy_Dataset_1) is consistent with the moderate noise acting as a regularizer on a genuinely multi-modal future: on the clean baseline the generator has more freedom to produce diverse but inaccurate trajectories, while the noisier input pushes it toward the dominant mode of the distribution.

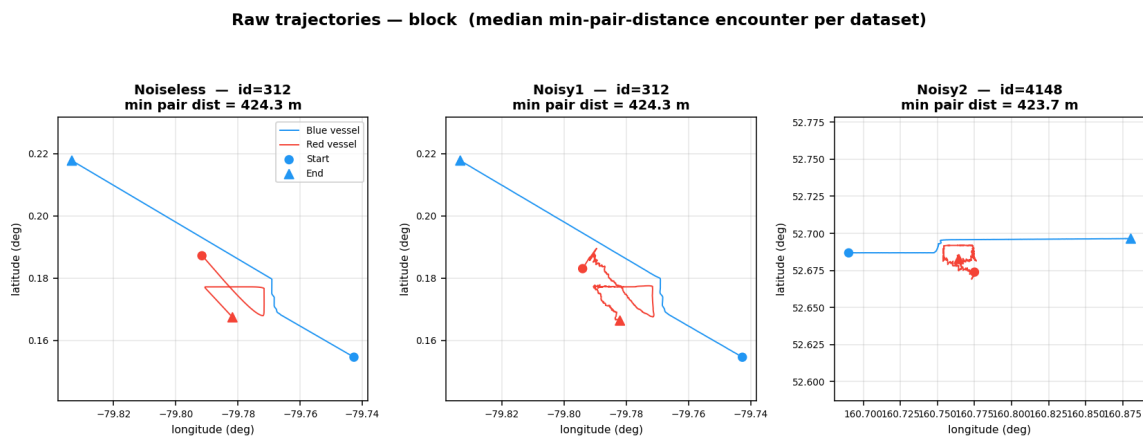


FIGURE 7.11: Block encounter across the three datasets. One vessel cuts ahead of another to obstruct its path. The cut-in is a sharp geometric feature visible in all three columns, and the Noisy_Dataset_1 column perturbs the mid-track without erasing the L-shape of the maneuver. Block ADE increases from 1.19 m on the clean baseline to 6.09 m on Noisy_Dataset_1, which is much less than the order-of-magnitude jumps reported for the non-hostile behaviors.

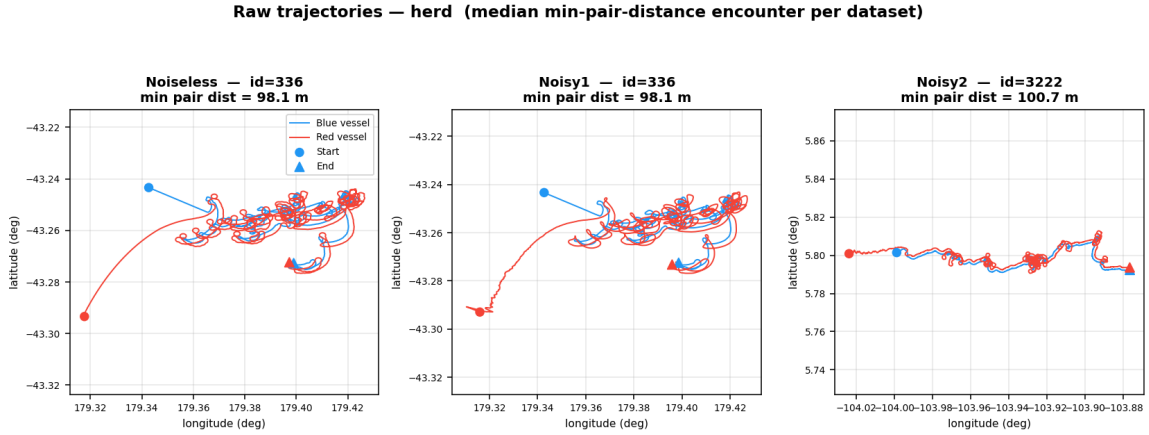


FIGURE 7.12: Herd encounter across the three datasets. One vessel repeatedly maneuvers to corral or steer the other, producing a high-frequency oscillating geometry that is large compared to the noise envelope. Herd is the only behavior for which Noisy_Dataset_1 actually improves ADE (7.66 m on the clean baseline, 5.96 m on Noisy_Dataset_1). We attribute this to the noise acting as a regularizer on a genuinely multi-modal future.

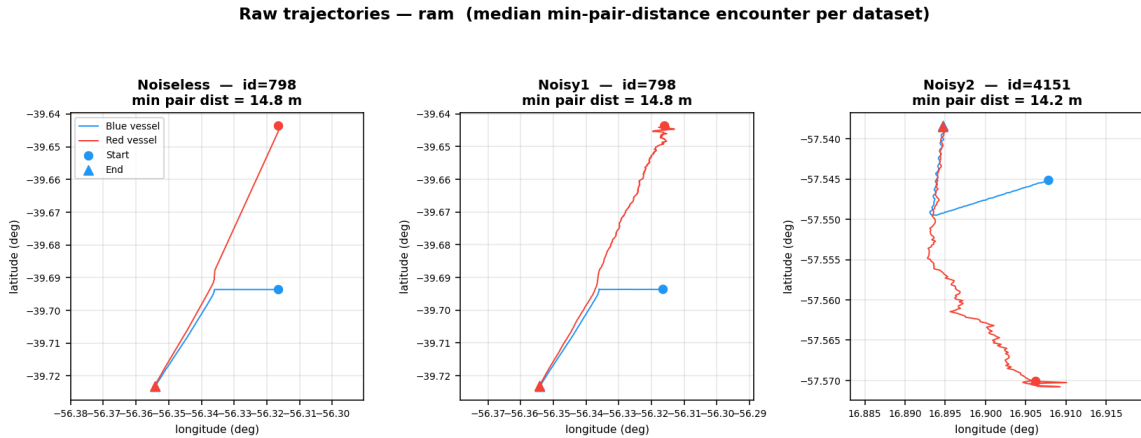


FIGURE 7.13: Ram encounter across the three datasets. A sharp adversarial closure brings the two vessels to within about 15 m, often with a heading reversal at the terminal phase. The displacement of the maneuver is much larger than the noise envelope, which is consistent with ram ADE only doubling on Noisy_Dataset_1 (2.87 m on the clean baseline, 6.26 m on Noisy_Dataset_1) where the smoother behaviors increase by an order of magnitude.

Implications for the quantitative results. The visual evidence is consistent

with the asymmetric per-scenario degradation reported in the tables below. The

asymmetry is not a property of the model, since the same architecture is used for all behaviors. It is a property of the input: smooth, low-curvature paths place the inter-class signal at the same magnitude as the `Noisy_Dataset_1` measurement noise, while sharp adversarial paths place it well above. `Noisy_Dataset_2` follows the same logic at a higher noise level: every behavior fails when the perturbation magnitude is comparable to the inter-vessel baseline. This observation supports the deployment guidance from the classical-baseline comparison (Section 7.8.1): under low signal-to-noise sensing, the classical baselines (CV or the Kalman-filter variants) or a denoising preprocessing step should be used for non-hostile traffic, and MTITP should be activated when an aggressive intent signature is detected.

7.9.4.2 Quantitative Ablation Results

Tables 7.23, 7.24, and 7.25 report per-scenario trajectory and intent metrics for A18, A19, and A22 against the clean A13 baseline.

TABLE 7.23: Robustness ablation: MTITP-L2 on moderate noise (A18, Noisy_Dataset_1) vs. clean baseline (A13). ADE/FDE in meters; Past Acc. and Future Acc. are scenario and per-step intent accuracy. **Bold** = better per metric per row.

Scenario	A13: clean data				A18: Noisy_Dataset_1			
	ADE	FDE	Past Acc.	Future Acc.	ADE	FDE	Past Acc.	Future Acc.
benign	0.86	1.42	94.5%	100.0%	13.15	17.93	92.8%	99.4%
block	1.19	2.04	99.6%	99.0%	6.09	8.44	99.3%	98.3%
cross	1.02	1.74	97.4%	97.8%	6.66	9.03	98.4%	98.1%
headon	0.88	1.48	99.6%	99.4%	7.14	9.57	99.1%	98.8%
overtake	0.67	1.11	99.6%	99.6%	6.39	8.58	98.9%	98.9%
ram	2.87	4.25	96.3%	95.5%	6.26	8.63	91.2%	90.5%
herd	7.66	13.82	99.9%	99.8%	5.96	10.43	98.9%	99.0%
Overall	2.17	3.69	99.2%	99.2%	7.38	10.37	98.4%	98.3%

TABLE 7.24: Robustness ablation: MTITP-L2 on high noise (A19, Noisy_Dataset_2) vs. clean baseline (A13). ADE/FDE in meters; Past Acc. and Future Acc. are scenario and per-step intent accuracy. **Bold** = better per metric per row.

Scenario	A13: clean data				A19: Noisy_Dataset_2			
	ADE	FDE	Past Acc.	Future Acc.	ADE	FDE	Past Acc.	Future Acc.
benign	0.86	1.42	94.5%	100.0%	57.19	72.54	86.7%	98.8%
block	1.19	2.04	99.6%	99.0%	27.09	32.73	98.5%	98.4%
cross	1.02	1.74	97.4%	97.8%	30.46	36.82	94.8%	95.2%
headon	0.88	1.48	99.6%	99.4%	32.49	38.98	97.8%	98.5%
overtake	0.67	1.11	99.6%	99.6%	31.84	37.95	99.3%	99.1%
ram	2.87	4.25	96.3%	95.5%	20.12	24.08	79.6%	79.7%
herd	7.66	13.82	99.9%	99.8%	12.25	18.98	98.9%	98.8%
Overall	2.17	3.69	99.2%	99.2%	30.20	37.44	97.2%	97.5%

A18: MTITP retains intent discrimination under moderate sensor noise.

Trained and evaluated on Noisy_Dataset_1, A18 achieves 7.38 m overall ADE and 98.4% past intent accuracy. The intent drop relative to clean data (99.2% $\rightarrow$ 98.4%) is under one percentage point overall. Per-scenario intent is resilient on five of seven classes: block (99.6% $\rightarrow$ 99.3%), cross (97.4% $\rightarrow$ 98.4%), headon (99.6% $\rightarrow$ 99.1%), overtake (99.6% $\rightarrow$ 98.9%), and herd (99.9% $\rightarrow$ 98.9%) all stay within $\sim$ 1 pp of the clean baseline. The two largest drops occur at benign (94.5% $\rightarrow$ 92.8%) and ram

(96.3% $\rightarrow$ 91.2%), both scenarios where positional jitter is large relative to the motion scale. Future intent accuracy follows the same pattern, declining by $<1\%$ on five classes and by 0.5–1.2% on benign and ram.

The major finding is the model’s overall behavior under noise rather than the contribution of any single stream. MTITP keeps its scenario decision surface essentially intact (under one percentage point of intent accuracy lost) even when the input has been perturbed enough to inflate trajectory ADE on the non-hostile classes by $5\text{--}15\times$ (Section 7.9.4.1). The three-stream encoder, the Interaction Fusion module that combines pairwise geometry with discrete relational state, and the multi-task heads degrade gracefully together: the encoder still recovers the COLREGs scenario class, the future-intent head still tracks the per-step intent signal, and the trajectory generator continues to produce scenario-appropriate motion, and it simply does so at a wider spatial spread because the input itself is wider. A18 is therefore evidence that MTITP holds up as a complete classifier-plus-predictor under realistic sensor noise, with the intent pathway essentially unchanged and the trajectory pathway tracking the input’s noise envelope rather than collapsing.

The trajectory results exhibit a scenario-dependent inversion: benign ADE climbs sharply (0.86 m $\rightarrow$ 13.15 m) while herd ADE falls (7.66 m $\rightarrow$ 5.96 m). Benign vessels travel near-straight at low speed, and their true 5-step displacement is only a few meters. So, positional jitter dwarfs the true motion signal and dominates ADE even when the model predicts the correct motion direction. The other low-displacement

scenarios (block, cross, headon, overtake) experience 5–10 $\times$ ADE inflation by the same mechanism. Herd vessels execute large-scale lateral maneuvers (>50 m) where the noise is negligible relative to the true displacement; increased input uncertainty may additionally smooth high-frequency prediction noise that inflated herd ADE on the clean set. RAM ADE grows from 2.87 m to 6.26 m, which is a smaller proportional increase than in benign cases.

A19: High noise: trajectory quality degrades proportionally, intent is remarkably stable. Trained and evaluated on Noisy_Dataset_2, A19 exhibits a 4 $\times$ increase in Average Displacement Error (ADE) compared to A18, with values of 30.20 m versus 7.38 m overall, consistent with the noise-magnitude scaling expected when positional jitter dominates the signal. The scenario-dependent inversion observed in A18 sharpens: benign reaches 57.19 m and ram 20.12 m, while herd remains the lowest-error scenario at 12.25 m. The hierarchy is the same SNR-driven ordering reported for A18, shifted upward in absolute magnitude.

The striking result is the intent robustness under this elevated noise. Overall, past accuracy drops only from 99.2% (clean) to 97.2% (A19), a 2% loss despite the 14 $\times$ increase in overall ADE. Scenario-level intent tells the same story: overtake (99.6% $\rightarrow$ 99.3%), headon (99.6% $\rightarrow$ 97.8%), herd (99.9% $\rightarrow$ 98.9%), and block (99.6% $\rightarrow$ 98.5%) remain essentially unaffected. The two largest degradations are ram (96.3% $\rightarrow$ 79.6%) and cross (97.4% $\rightarrow$ 94.8%); precisely the scenarios whose discriminative COLREGs signatures (tight angular geometry for cross; extreme approach rate for ram) are

most susceptible to bearing and range-rate corruption under high positional noise. Future intent accuracy tracks past accuracy closely, confirming that the encoder’s scene representation remains reliable even when trajectory generation fails.

The separation of intent robustness from trajectory robustness is the core finding of the A18/A19 ablation pair. Noise degrades position-based prediction much more rapidly than it degrades semantic scene understanding, because Stream C operates on categorical relational features that are more robust to continuous jitter. For maritime surveillance applications, this means MTITP can continue to provide reliable intent classification, a crucial safety-critical output, even when sensor noise renders precise trajectory forecasts unreliable.

A22: MarITGANv2 under sensor noise: trajectory parity with L2, intent advantage retained. Table 7.25 compares A22 (MarITGAN v2 on Noisy_Dataset_1) against A18 (L2 on Noisy_Dataset_1), isolating the effect of the training objective under identical noise conditions.

TABLE 7.25: Adversarial vs. L2 under moderate noise (Noisy_Dataset_1): A22 (MarITGAN v2) vs. A18 (L2). ADE/FDE in meters; Past Acc. and Future Acc. as fractions. **Bold** = better per metric per row; ties unbolded.

Scenario	A18: L2				A22: MarITGAN v2			
	ADE	FDE	Past Acc.	Future Acc.	ADE	FDE	Past Acc.	Future Acc.
benign	13.15	17.93	92.8%	99.4%	13.26	18.06	92.3%	99.5%
block	6.09	8.44	99.3%	98.3%	6.23	8.72	99.4%	98.5%
cross	6.66	9.03	98.4%	98.1%	6.84	9.31	98.4%	98.2%
headon	7.14	9.57	99.1%	98.8%	7.17	9.53	98.9%	98.8%
overtake	6.39	8.58	98.9%	98.9%	6.43	8.61	99.3%	99.1%
ram	6.26	8.63	91.2%	90.5%	6.39	8.81	92.9%	92.1%
herd	5.96	10.43	98.9%	99.0%	6.00	10.56	98.8%	99.1%
Overall	7.38	10.37	98.4%	98.3%	7.48	10.51	98.5%	98.5%

A22 achieves 7.48m overall ADE compared to A18’s 7.38m, a difference of 0.10m (1.4%), within the margin expected from training stochasticity on noisy data. Intent accuracy is marginally better under A22: past accuracy is 98.5% vs. 98.4%, and future accuracy is 98.5% vs. 98.3%. The per-scenario pattern from A18 is reproduced faithfully: benign ADE is dominated by positional jitter (13.26m), while herd (6.00m) retains the best ADE among hostile scenarios due to the noise-reduction effect of large-displacement trajectories. Ram is the scenario with the largest noise-induced

past-accuracy drop under both objectives (91.2% under A18 and 92.9% under A22), consistent with ram’s sensitivity to range-rate corruption; under A22 the drop is the smaller of the two.

The key finding is that MarITGAN v2 does not degrade under noise relative to L2: across all seven scenarios, A22 is within 0.40 m of A18 in ADE, and at the overall level the gap (0.10 m) is negligible. Intent accuracy under A22 matches or slightly exceeds A18, confirming that the scenario-weighted auxiliary loss provides a stable training signal even when the input distribution is corrupted. The intent-aware adversarial objective is fully compatible with noisy training distributions.

7.10 Robustness and Sanity Checks

The headline numbers above are high enough to invite skepticism, and the obvious worry is that they reflect data leakage, overfitting to the test distribution, or some other numerically degenerate behavior rather than genuine model capability. Five independent sanity checks address that concern in turn: split integrity, majority-class baselines, t-SNE of scene embeddings, cross-dataset generalization, and McNemar’s test for statistical significance. All checks are implemented in a standalone evaluation notebook that is fully reproducible from the saved checkpoints and normalization statistics.

7.10.1 Split Integrity

The most damaging form of data leakage in a sliding-window dataset is *encounter-level leakage*: windows from the same vessel encounter appearing in both the training and test sets. Because windows overlap by $T_{\text{obs}} + T_{\text{pred}} - \text{stride}$ timesteps, a naïve per-window split would allow the model to memorize individual encounters.

MTITP uses a stratified encounter-level split: all windows belonging to the same encounter (identified by CSV file stem) are assigned to the same split before any random selection. The split is deterministic (seed 42) and stratified by scenario class to preserve class balance. The *stationary* scenario is excluded from all splits. It is a reserved ghost class and never appears in any training, validation, or test sample.

TABLE 7.26: Dataset split after excluding stationary (stride $s = 10$, seed 42). 70/15/15 split at the *encounter* level, so window counts deviate from these ratios.

Dataset	Total windows	Train	Val	Test
Noiseless	879,730	610,462	132,654	136,614
Noisy-1 (A18/A22)	880,261	614,389	133,430	132,442
Noisy-2 (A19)	1,215,483	849,212	180,955	185,316

We verified the split integrity by reconstructing the train, validation, and test sets from the saved windows and checking for overlap at two levels of granularity: (i) *encounter-level*, no encounter ID appears in more than one split; and (ii) *window-level*, no `(encounter_id, window_start)` pair appears in more than one split. Both checks

returned zero overlapping entries across all three datasets (Table 7.26). Stride was confirmed to be $s = 10$ throughout. Re-running the split function with the same seed and data produces bit-identical partitions, confirming full reproducibility.

7.10.2 Majority-Class Baselines

A necessary sanity check for any classification result is that the model substantially outperforms a trivially-defined baseline. Two trivial predictors are evaluated:

- **Majority-class predictor:** always predicts the most frequent class in the training set.
- **Uniform random predictor:** assigns each of the 7 active classes with equal probability ($1/7 \approx 14.3\%$).

Table 7.27 summarizes the comparison. For past intent, the majority class is *herd* (36.6% of noiseless test windows); all three noiseless MTITP variants achieve 99.2% past accuracy, a **62.6 pp** improvement over this ceiling. For future intent, the dominant future class (*beh_label* 0) accounts for 40.0% of all predicted timesteps; MTITP’s 99.2–99.3% future accuracy represents a **59.2–59.3 pp** margin. The same ordering holds on the noisy datasets: majority-class accuracy is essentially unchanged by noise level, but MTITP’s advantage narrows only slightly (~ 2 pp), consistent with the noise robustness reported in Section 7.9.4.

TABLE 7.27: Intent accuracy: trivial baselines vs. trained models (noiseless test set). Majority-class always predicts the most frequent training class; random samples uniformly over 7 active classes.

Predictor	Past intent (scenario)			Future intent (beh_label)		
	Acc	F1	mAP	Acc	F1	mAP
Majority-class (herd / class-0)	0.366	0.074	—	0.400	0.080	—
Uniform random (1/7)	0.143	0.143	—	0.143	0.143	—
MTITP-WGAN (baseline)	0.992	0.983	0.994	0.993	0.972	0.994
A13 (MTITP-L2)	0.992	0.983	0.993	0.992	0.970	0.993
A21 MarITGAN v2	0.992	0.982	0.993	0.992	0.972	0.993

7.10.3 t-SNE of Encoder Representations

The encoder produces two outputs that feed different downstream heads, and we project both to check that each downstream consumer sees a class-discriminative input. The scene embedding $\mathbf{e}_{\text{scene}} = [\mathbf{c}_{\text{blue}} || \mathbf{c}_{\text{red}}] \in \mathbb{R}^{2D_{\text{model}}}$ is the only input to the past-intent classifier, while the full encoder sequence $\mathbf{E}_{\text{seq}} \in \mathbb{R}^{2T_{\text{obs}} \times D_{\text{model}}}$ is cross-attended by both the future-intent predictor and the trajectory generator. We extracted 4,000 embeddings from each model’s native test split and applied t-SNE (perplexity 30, random initialisation) after standardising each embedding dimension to zero mean and unit variance. For $\mathbf{E}_{\text{seq}}$ the per-timestep representation is collapsed to a single

vector by mean-pooling over the T_{obs} time tokens per vessel and concatenating the two pooled vectors, so the projection is dimensionally comparable to $\mathbf{e}_{\text{scene}}$.

Past-intent input ($\mathbf{e}_{\text{scene}}$). Figure 7.14 shows the projection for all five models.

Three observations stand out:

1. **MarITGAN v2 (A21) produces the tightest clusters** among all noiseless models. The scenario-weighted auxiliary loss directly reinforces discriminability of the three upweighted classes: headon, herd, and ram form compact, well-separated islands, with a notably larger margin from the benign cluster than in the baseline.
2. **MTITP-WGAN and A13 (MTITP-L2) produce qualitatively similar structure**, confirming that the encoder representations are driven by the multi-task intent supervision signal and are largely invariant to whether the trajectory objective is adversarial or L2.
3. **Noisy-data models (A18, A19) retain recognizable cluster geometry** despite operating on a degraded input distribution, corroborating the intent robustness finding of Section 7.9.4: the encoder’s scene representation remains organized even when positional accuracy degrades.

t-SNE of Scene Embeddings by Model

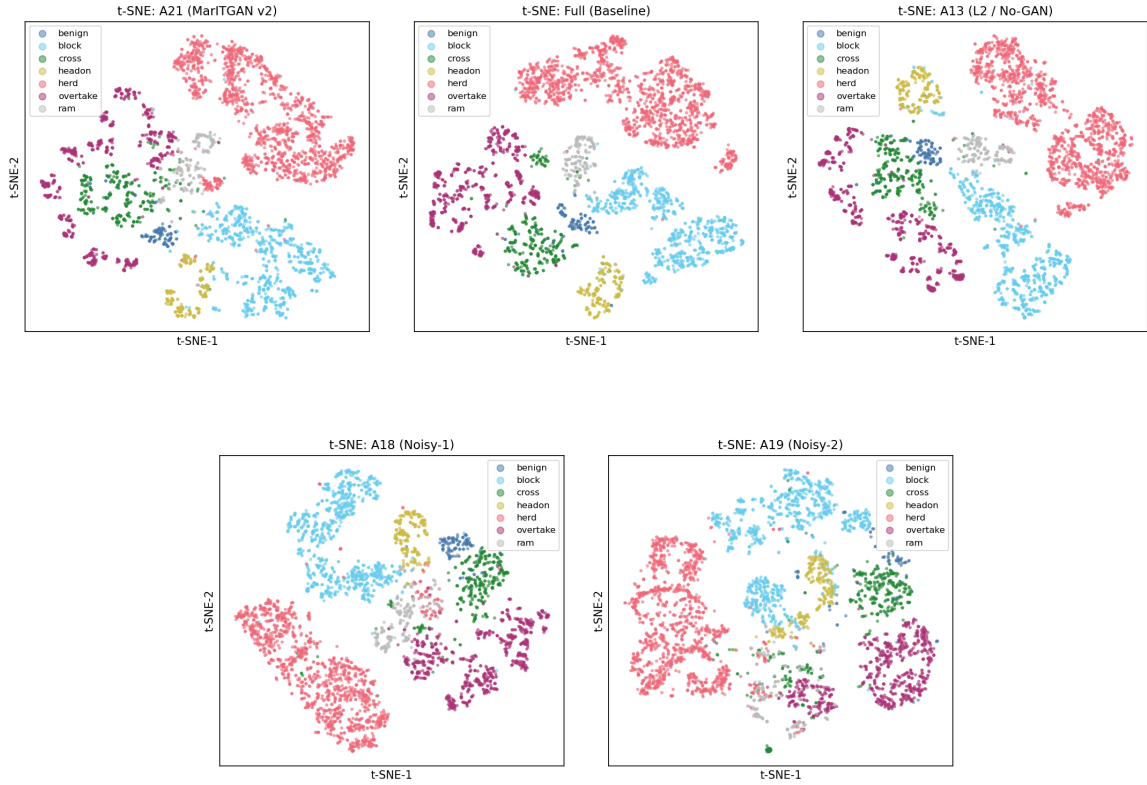


FIGURE 7.14: t-SNE projections of scene embeddings $\mathbf{e}_{\text{scene}} \in \mathbb{R}^{2D_{\text{model}}}$ (4,000 test samples per model; one point per window, colored by scenario). MarITGAN v2 (A21, top-left) shows tighter clustering of the three upweighted classes (headon/herd/ram) from scenario-weighted training. Noisy-data models (A18, A19) retain seven-way separability despite corrupted inputs.

Future-intent and trajectory input ($\mathbf{E}_{\text{seq}}$). Figure 7.15 shows the analogous projection for the encoder sequence. Across all five models the seven scenario classes form well-separated, internally compact clusters, often visibly tighter than under $\mathbf{e}_{\text{scene}}$. The cross-attention path that the future-intent predictor and the trajectory generator both consume therefore receives the same discriminative signal that the past-intent classifier benefits from, not a diluted version of it. This rules out an

alternative explanation for the strong future-intent and trajectory metrics in which the multi-task supervision sharpens only the CLS slot while leaving the per-step encodings ambiguous: the structure that drives past-intent accuracy is reproduced at the per-timestep level, and the noisy-data models again retain recognizable seven-way separability. In short, the trajectory and future-intent decoders are conditioned on a representation that is class-organized at every step they cross-attend to.

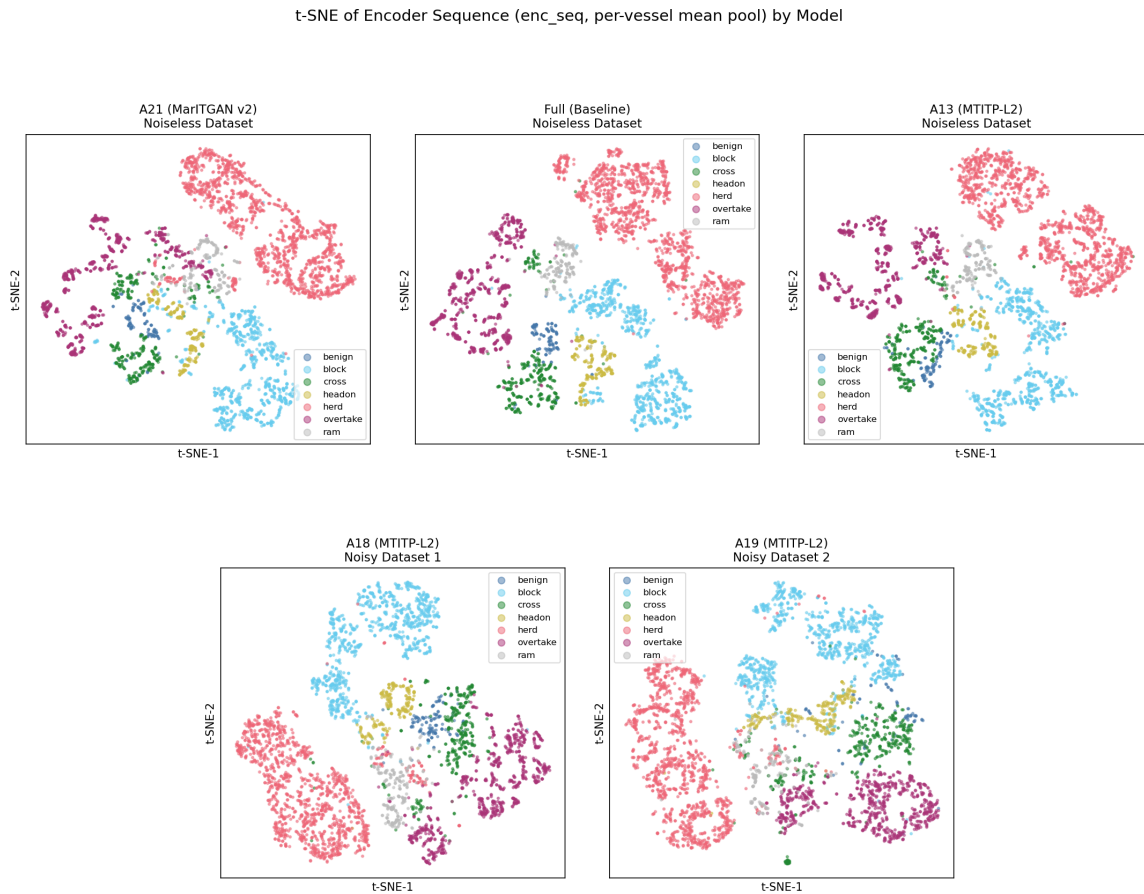


FIGURE 7.15: t-SNE projections of the encoder sequence $\mathbf{E}_{\text{seq}}$ (mean-pooled over time per vessel and concatenated; 4,000 test samples per model, colored by scenario). $\mathbf{E}_{\text{seq}}$ is the input the future-intent predictor and the trajectory generator cross-attend to. All five models produce well-separated clusters with class-internal compactness comparable to or tighter than $\mathbf{e}_{\text{scene}}$, indicating that the discriminative signal is present at every timestep the decoders consume.

7.10.4 Cross-Dataset Generalization

To test whether each model’s performance generalizes beyond its training distribution, every checkpoint was evaluated on all three datasets (noiseless, Noisy-1, Noisy-2) using the model’s own normalization statistics. Table 7.28 shows ADE and past intent accuracy for this 5×3 evaluation grid. Diagonal cells (native dataset, shown in bold) measure in-distribution performance; off-diagonal cells measure out-of-distribution behavior.

TABLE 7.28: Cross-dataset generalization: each model evaluated on all three datasets using its own normalization. **Bold** = native (in-distribution) result. ADE in meters; Past-Acc and Future-Acc as fractions. A22 omitted (not cross-evaluated). Diagonal cells are produced by the validation-notebook pipeline, which differs from the headline-results pipeline (Table 7.20) in its normalization file and in explicit filtering of stationary-vessel windows; native values may therefore differ from the corresponding rows of Table 7.20 by a small amount (e.g., A21 native ADE 2.18 vs. headline 2.23). Both pipelines load the same `ckpt_best_ade.pt` checkpoint per model.

Model	Noiseless			Noisy-1			Noisy-2		
	ADE	Past Acc.	Future Acc.	ADE	Past Acc.	Future Acc.	ADE	Past Acc.	Future Acc.
MTITP-WGAN <i>Noiseless</i>	3.42	0.992	0.993	11.40	0.592	0.700	42.88	0.501	0.567
A13 (MTITP-L2) <i>Noiseless</i>	2.17	0.992	0.992	10.41	0.694	0.756	40.57	0.514	0.639
A21 (MarITGAN v2) <i>Noiseless</i>	2.18	0.974	0.975	10.54	0.604	0.721	40.53	0.525	0.731
A18 (MTITP-L2) <i>Noisy-1</i>	3.88	0.969	0.974	7.38	0.984	0.983	38.30	0.582	0.797
A19 (MTITP-L2) <i>Noisy-2</i>	6.14	0.946	0.956	11.10	0.933	0.938	30.20	0.972	0.975

Three properties stand out. First, **graceful ADE degradation under matched training**: each noisy-trained model achieves the lowest ADE on its native distribution (A18 reaches 7.38 m on Noisy-1; A19 reaches 30.20 m on Noisy-2), while noiseless-trained models degrade sharply when sensor noise is introduced (MTITP-WGAN ADE rises from 3.42 m on noiseless to 42.88 m on Noisy-2). Second, **noisy-model transfer**: A18, trained on Noisy-1, retains 96.9% past accuracy on the clean test set, training on a harder distribution acts as an implicit regularizer for intent recognition, though trajectory error (3.88 m) is slightly worse than baselines trained on clean data. A19, trained on the heaviest-noise distribution, shows the broadest cross-dataset robustness: it is the only model that holds past accuracy above 93% on every dataset, at the cost of higher ADE everywhere. Third, **MarITGAN generalization parity**: A21 follows the same degradation pattern as its L2 counterpart A13 across all three datasets, confirming that the intent-aware adversarial objective does not introduce brittleness to domain shift.

7.10.5 McNemar’s Test for Statistical Significance

Accuracy differences between models trained on the same dataset may be due to chance rather than genuine capability improvement. McNemar’s test [151] provides a formal pairwise significance test. Given models A and B on the same N test samples, let b be the count where A is correct and B wrong, and c the reverse. The test statistic with continuity correction is:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}, \tag{7.72}$$

distributed as χ^2_1 under the null hypothesis of equal error rates.

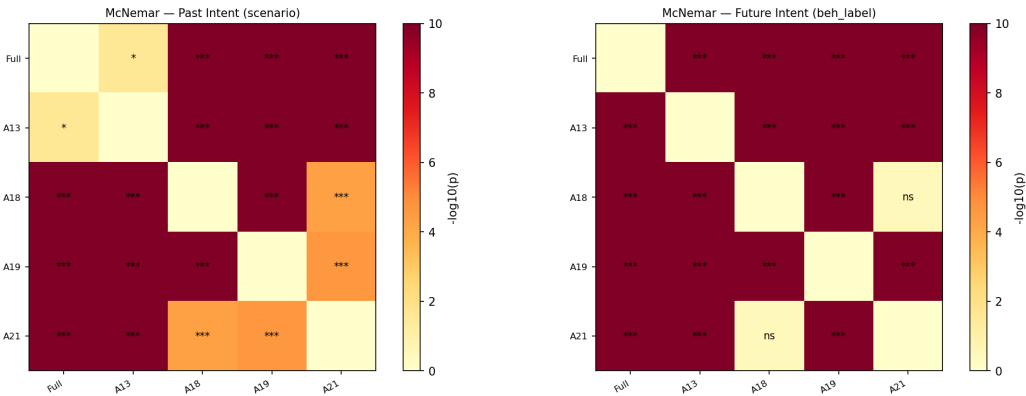


FIGURE 7.16: McNemar’s test $-\log_{10}(p)$ for past (left) and future (right) intent. Rows/cols: MTITP-WGAN, A13, A18, A19, A21. Annotations: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, ns not significant. Cross-dataset pairs (A18/A19 vs. noiseless models) violate the paired-sample assumption and are shown for reference.

Table 7.29 summarizes the pairwise results for the three noiseless-trained models (MTITP-WGAN, A13, A21), where the paired-sample assumption holds exactly.

TABLE 7.29: McNemar’s test for noiseless-trained models (MTITP-WGAN, A13, A21) on the shared 136,614-window noiseless test set. ns = not significant; *** = $p < 0.001$.

Model A	Model B	Past intent	Future intent
MTITP-WGAN	A13 (L2)	ns	***
MTITP-WGAN	A21 (MarITGAN v2)	ns	***
A13 (L2)	A21 (MarITGAN v2)	ns	***

Past intent. All three noiseless models are statistically indistinguishable in scenario classification ($p > 0.05$, Table 7.29). The encoder’s scenario-discrimination ability is robust to changes in the trajectory loss formulation: whether the trajectory signal comes from a WGAN-GP critic, an L2 regression, or the MarITGAN v2 critic makes no measurable difference for past intent.

Future intent. All three pairwise comparisons are significant at $p < 0.001$. The trajectory loss formulation *does* affect the future intent pathway: the WGAN-GP and MarITGAN critics indirectly regularize the intent embedding head through the generator’s conditioning path, and A21’s scenario-weighted auxiliary loss additionally reinforces correct classification of the three upweighted future behaviors (head-on, herd, ram).

Cross-dataset pairs. Comparisons involving A18 or A19 versus noiseless models are shown in Figure 7.16 for completeness but are not used as evidence of differential capability: the test windows differ between datasets, so the paired-sample assumption of Equation (7.72) does not hold.

7.11 Discussion

The headline metrics in Section 7.7, the variant comparisons against external baselines in Section 7.8, the ablation evidence in Section 7.9, and the robustness checks in Section 7.10 together produce a consistent picture of the model. This section consolidates that evidence into a unified reading: which design choices drive the observed performance, what the per-scenario behavior reveals, and what the operational implications and caveats are.

7.11.1 Intent and Trajectory as Separable Pathways

Intent accuracy and trajectory accuracy move independently. Herd reaches 99.9% past intent accuracy alongside 10.63 m ADE: the model identifies what is happening with near-certainty while the generator places future positions with much wider spread. The two are measuring different things. The three-stream encoder, guided by Stream C's symbolic features (port/starboard relationship, CPA class, heading-change category), reads the interaction signatures that distinguish scenario classes, and those signatures are present in the past window before the future has been resolved. The generator's spread on herd is the model expressing future uncertainty that the data itself contains, the herding vessel's next move depends on an adversarial reaction that has not yet occurred. A 10.63 m herd ADE means the future is genuinely under-constrained, not that the model has misread the encounter.

The ablations formalize this decoupling. Removing Streams B and C (A3) reduces future-intent mAP from 99.4% to 66.2% with negligible change in ADE. Removing the intent heads (A12) drops past accuracy to near chance and inflates ADE by 1.25 m. Switching the trajectory signal from the WGAN-GP critic to direct L2 regression (A13/A15/A14, the MTITP-L2 variant) preserves intent accuracy to within a tenth of a percentage point at every horizon tested. The McNemar comparison (Table 7.29) tells the same story from a different angle: past intent is statistically indistinguishable across MTITP-WGAN, MTITP-L2 (A13), and MarITGAN v2 (A21) even though the three variants use different trajectory losses. The encoder’s scenario-discrimination ability comes from the symbolic relational streams and the multi-task supervision, not from the trajectory objective.

7.11.2 Where Trajectory Quality Comes From

Turning to the trajectory pathway, MTITP-L2 outperforms MTITP-WGAN by a consistent $\sim 37\%$ ADE margin across the three tested horizons, and removing the explicit intent embedding from the generator (A11) actually yields a small ADE improvement. The intent conditioning that benefits trajectory generation therefore enters through the encoder’s cross-attention path, not through the additional embedding handed to the decoder. Read together, these two results pin the WGAN-GP critic itself as the limiting factor. The conditional generator architecture remains worth keeping because

L2 regression collapses to the conditional mean and forfeits multimodal sampling; what does not earn its keep is the original adversarial objective.

That diagnosis motivated the MarITGAN design. Replacing the generic critic with an intent-aware, interaction-conditioned one (A20) shrinks the gap to MTITP-L2 from 1.25 m to 0.23 m, and the v2 schedule (A21) reduces it further to 0.06 m overall while surpassing MTITP-L2 on two of the three upweighted classes, ram (2.80 m vs. 2.87 m) and herd (7.40 m vs. 7.66 m). McNemar’s test registers a statistically significant future-intent difference for every pair involving A21 ($p < 0.001$), so the scenario-weighted auxiliary loss is propagating into the future intent head rather than touching only the trajectory output. The t-SNE projection in Figure 7.14 shows the same effect geometrically: the encoder keeps the `scene_emb` and `enc_emb` of similar behaviors together in the same cluster, and the upweighted-class clusters under A21 are visibly tighter than under MTITP-WGAN. The L2-versus-GAN gap is a critic problem, not a fundamental limit of adversarial training.

7.11.3 Scenario-Specific Analysis

The scenarios analyzed below are grouped by the binary intent partition used elsewhere in this chapter: *non-hostile* (benign, cross, head-on, overtake) and *hostile* (block, herd, ram). The four scenarios that exhibit distinctive predictive behavior (overtake, head-on, ram, and herd) are discussed individually under their respective groups;

benign, cross, and block produce broadly similar near-linear kinematics within their groups and are not separately profiled.

Non-hostile encounters

Overtake (ADE 1.14 m). Overtaking scenarios produce quasi-linear relative trajectories in the local frame. The pursuing vessel maintains a consistent bearing offset, and 5-step-ahead positions vary little across plausible futures. With so little spread to capture, all 20 samples cluster close to the ground truth.

Head-on (ADE 1.99 m). The head-on classification as “hard” captures the collision risk, not the predictive difficulty. Two vessels on near-opposite headings have tightly constrained near-term kinematics that they will be close, and physics limits how fast that geometry can change over 5 steps. The generator exploits this kinematic constraint effectively.

Hostile encounters

Ram (ADE 4.33 m). The ramming trajectory unfolds in two phases: an approach that looks linear, then a sharp terminal acceleration and heading change at close range. The 5-step horizon sometimes captures the approach phase and sometimes the turn, and those two futures are spatially very different. The generator learns the

approach geometry reliably but cannot resolve from kinematics alone which phase a given window falls in.

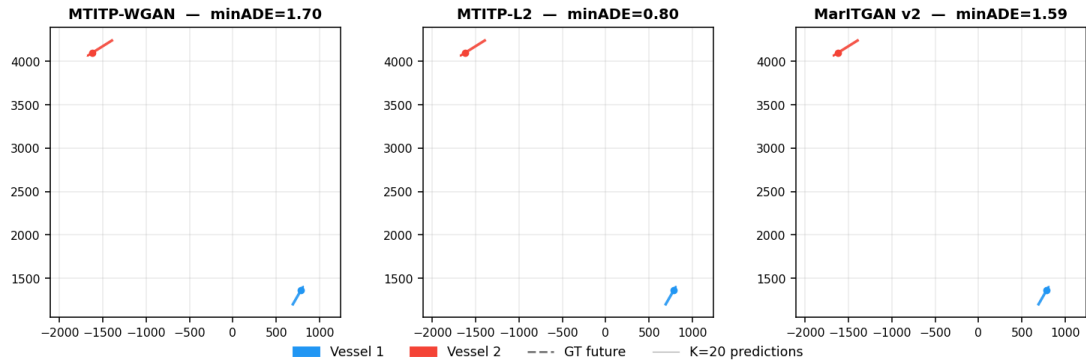
Herd (ADE 10.63 m). Herd is the genuinely hard case. The herding vessel adjusts its approach angle in response to the target’s evasion, producing a future that depends on an adversarial reaction that has not yet happened. With $K = 20$ samples the best-of- K trajectory still sits 10.63 m from the ground truth on average, so the sample set does not cover the realised future even when we keep its closest member. Closing this gap requires conditioning on the *target’s* anticipated response, which lies outside the current model’s scope.

7.11.4 Qualitative Trajectory Comparison

Figures 7.17–7.21 compare predicted trajectories produced by the three MTITP variants (*MTITP-WGAN*, the WGAN-GP baseline; *MTITP-L2*, no adversarial loss; and *MarITGAN v2*, the intent-aware critic) on the noiseless test set. Each panel plots the observed track (solid), the ground-truth future (dashed), and the variant’s 20 sampled trajectories (faint fan). For the per-scenario comparison (Figures 7.17–7.18), the same encounter is shown across the three columns to enable a direct visual comparison; encounters are selected from the closest-30% by minimum vessel-pair distance and ranked by mean $\text{minADE}@K$ across the three variants, so each scenario row is a representative close encounter rather than a degenerate easy case. Two

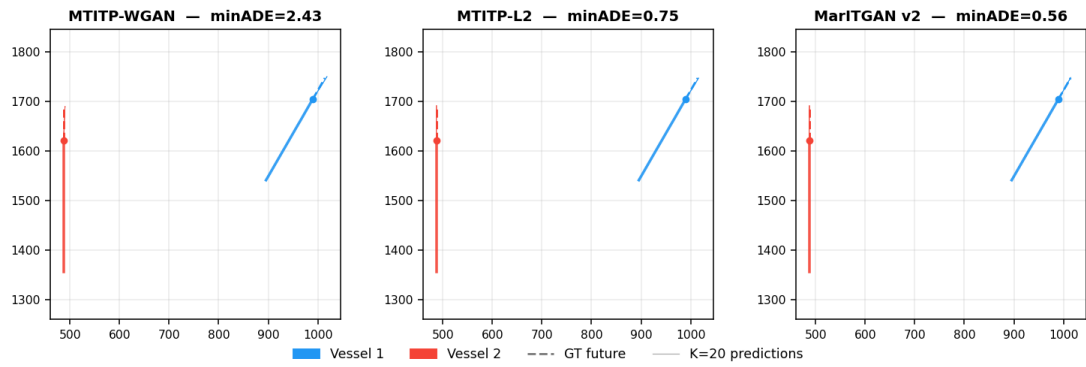
encounter groupings follow the binary intent partition used elsewhere in this chapter: *non-hostile* encounters (benign, cross, head-on, overtake) where COLREGs-compliant or routine kinematics dominate, and *hostile* encounters (block, herd, ram) where one vessel exhibits aggressive intent toward the other.

Trajectory Forecast — benign (noiseless test set)



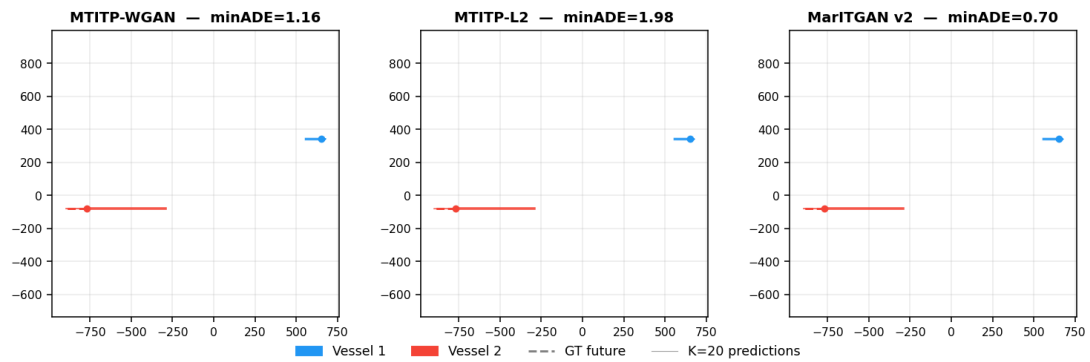
(A) Benign

Trajectory Forecast — cross (noiseless test set)



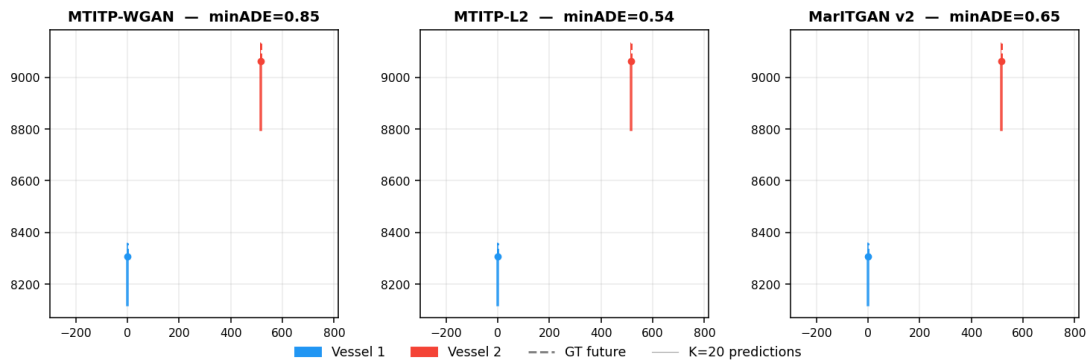
(B) Cross

Trajectory Forecast — headon (noiseless test set)



(c) Head-on

Trajectory Forecast — overtake (noiseless test set)



(D) Overtake

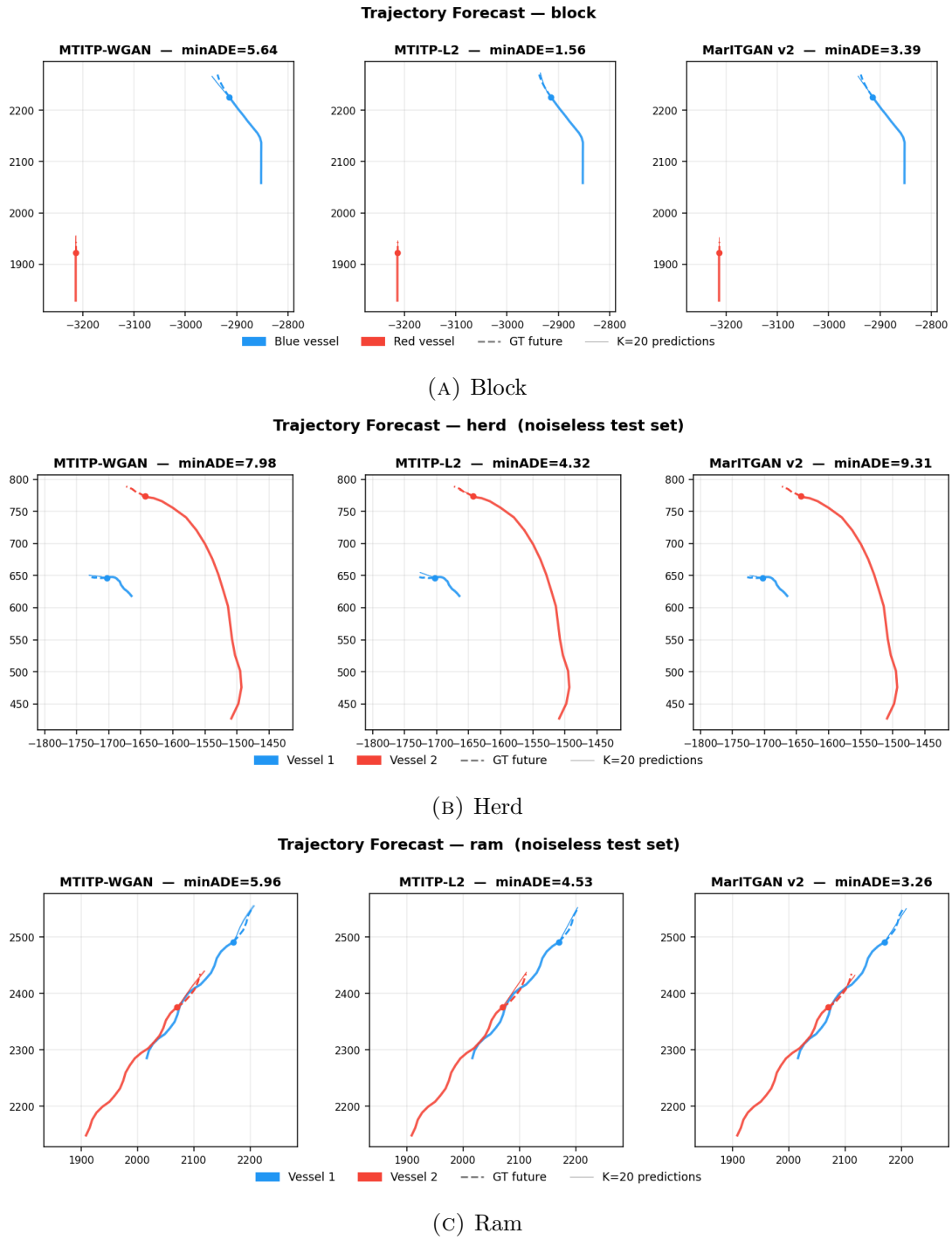


FIGURE 7.18: Per-scenario trajectory forecasts on *hostile* encounters. Columns: MTITP-WGAN, MTITP-L2, MarITGAN v2. Same encounter shown across columns. The wider spread in the sample reflects genuine multimodal uncertainty about the future rather than poor calibration.

Cross-variant comparison. On non-hostile encounters (Figure 7.17), all three variants concentrate samples tightly around the ground-truth track. The kinematic prior is strong enough that adversarial supervision adds little. MTITP-L2 produces a slightly tighter fan than the GAN variants on these scenarios, consistent with its lower marginal ADE on non-hostile slices and its bias toward the conditional mean. On hostile encounters (Figure 7.18), the picture inverts: MTITP-L2’s fan collapses around a single mode that often misses the ground truth, while MTITP-WGAN and especially MarITGAN v2 produce broader sample distributions that cover the actual future direction even when individual samples deviate. The wider fan is what we want here; the future is genuinely multi-modal and a single point estimate cannot represent it. MarITGAN v2 also delivers the lowest per-scenario ADE on both ram (2.80 m) and herd (7.40 m) in Table 7.22, so the broader sample spread on these scenarios is not paid for by worse mean accuracy.

7.11.4.1 Best, Median, and Worst Cases on Close Encounters

To complement the median-case view above, Figures 7.19–7.21 stratify three close-encounter scenarios (head-on, herd, and ram) into best, median, and worst cases (rows) and contrast the three MTITP variants (columns) on each. Encounters are filtered to the closest 30% by minimum vessel-pair distance, the regime where prediction errors carry operational consequence, and ranked by mean minADE@K across the variants. Two of these scenarios (herd, ram) belong to the hostile group; head-on is included as

the closest-quarter non-hostile case, where COLREGs-defined bow-to-bow geometry produces small errors on average but occasional large deviations on the worst-case encounters. This view exposes the failure modes that are averaged out in aggregate metrics: even on the worst-case herd encounter, MarITGAN v2 retains at least one sample within a plausible distance of the ground truth, whereas MTITP-L2 produces a confident point estimate that misses entirely.

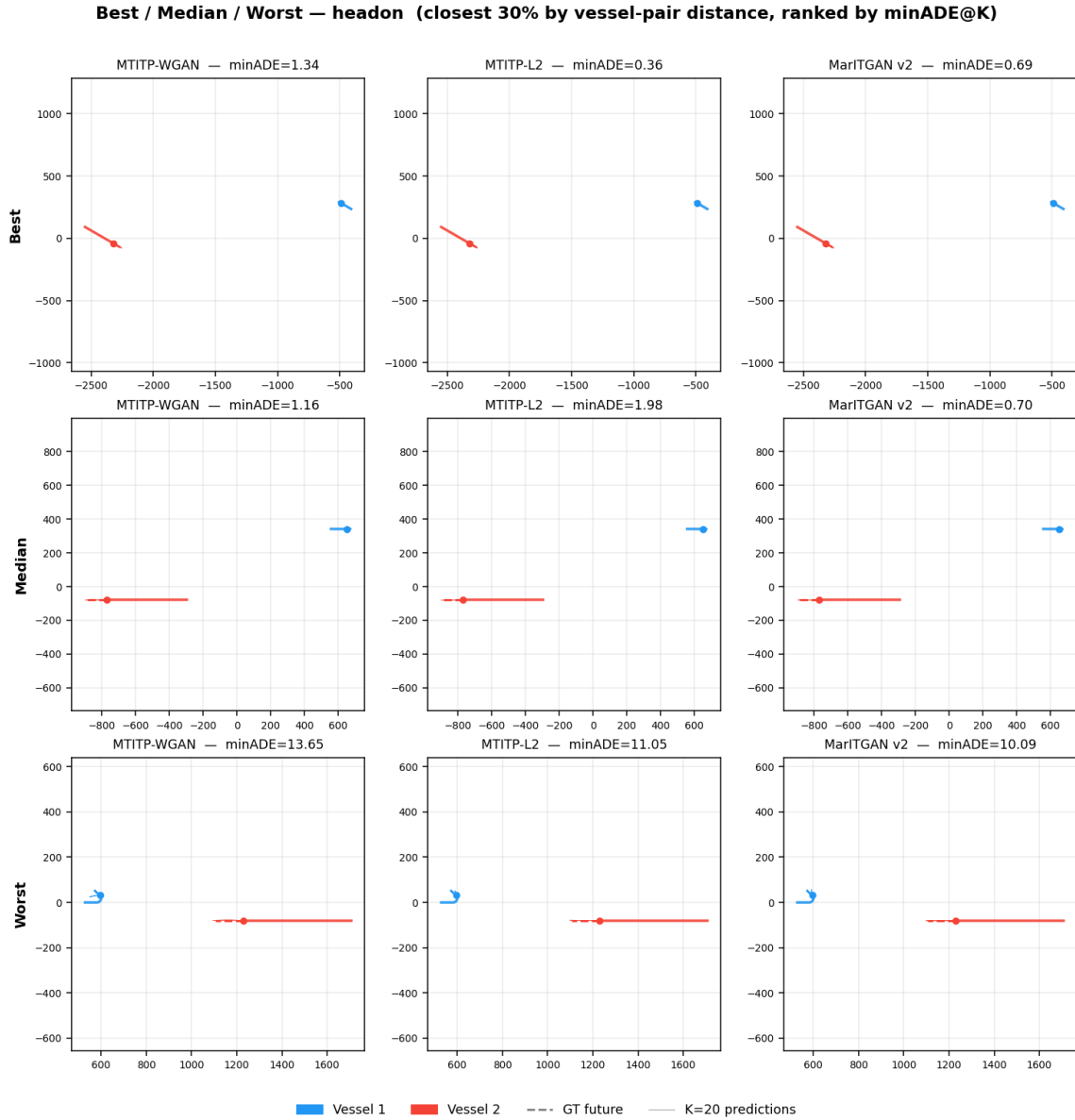


FIGURE 7.19: Best, median, and worst cases on the *head-on* test slice (rows) across the three MTITP variants (columns). Same encounter shown across each row.

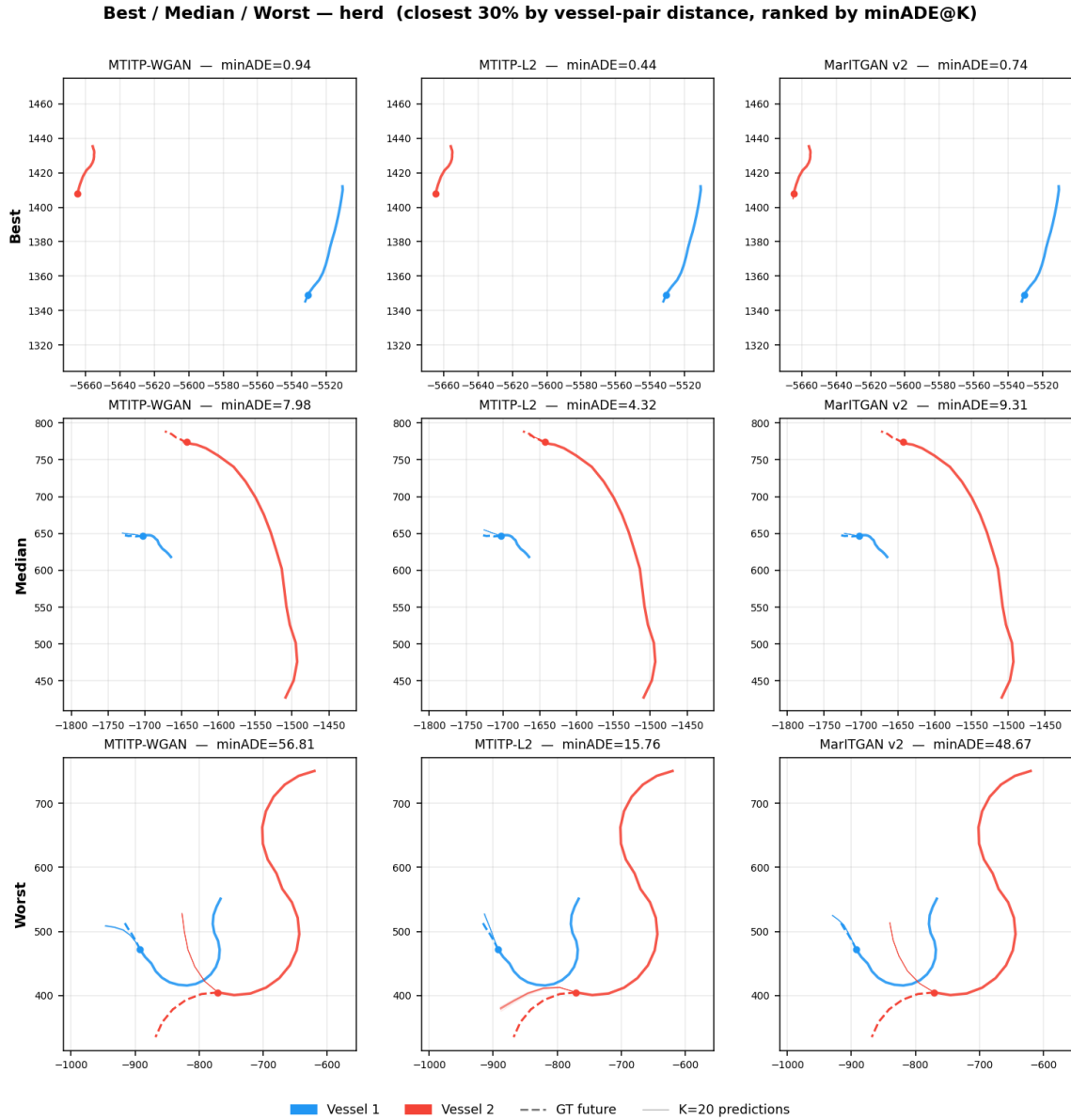


FIGURE 7.20: Best, median, and worst cases on the *herd* test slice (rows) across the three MTITP variants (columns). Same encounter shown across each row.

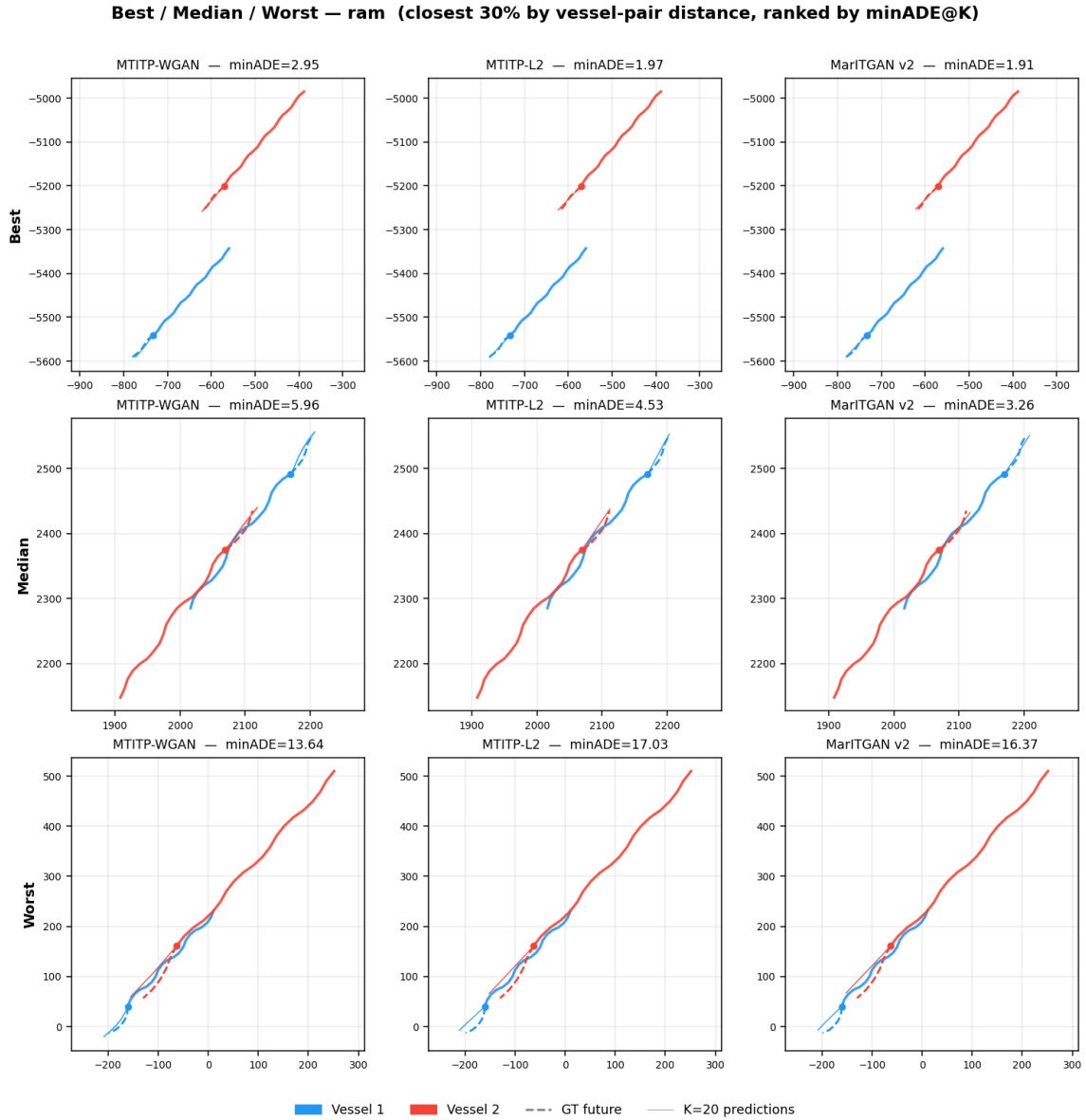


FIGURE 7.21: Best, median, and worst cases on the *ram* test slice (rows) across the three MTITP variants (columns). Same encounter shown across each row.

7.11.5 Robustness and Operational Implications

The robustness experiments separate the intent and trajectory pathways under sensor noise. A18 loses less than a percentage point of intent accuracy on Noisy-1 despite a

$3.4\times$ inflation in ADE, and A19 reproduces the pattern at higher magnitude ($14\times$ ADE inflation, intent at 97.2%). The mechanism is the model as a whole: the three-stream encoder, the Interaction Fusion module, and the multi-task heads degrade gracefully together, so the trajectory pathway tracks the noise envelope of the input while the intent pathway holds its scenario decision surface essentially intact. A22 confirms that MarITGAN inherits this property without paying any trajectory penalty over L2 (the two variants are within 0.10 m on Noisy-1), and the cross-dataset evaluation (Table 7.28) shows noisy-trained models transferring back to clean data with intent accuracy comparable to the noiseless baseline. The intent pathway is robust across input distributions.

The hostile/non-hostile binary collapse (Section 7.7.3) brings two findings to the surface that the seven-class metrics did not. A21 (MarITGAN v2) is the only model with a clearly asymmetric confusion structure: non-hostile recall 99.9% (almost no hostile alarms on benign traffic), but hostile recall 97.9%, 1,817 hostile windows are predicted as non-hostile out of 87,391. Both classes still reach ~ 1.0 precision, so when A21 says “hostile” it is almost always right; the cost is that a small fraction of hostile encounters get classified as non-hostile. The other models distribute errors more evenly across the two classes. This is the operational signature of the scenario-weighted auxiliary loss: by penalizing hostile misclassification more heavily during training, A21 learned a decision boundary biased toward the non-hostile half-plane in ambiguous regions. For surveillance, this trades a modest false-negative rate for very low false-positive alarms, a trade-off whose value depends on the application.

The trajectory-difficulty ordering also inverts under sensor noise. On the noiseless test split, hostile trajectories are intrinsically harder to predict: A13 reports H-ADE 5.01 m versus NH-ADE 0.78 m (a $6.4\times$ gap), and A21 mirrors this almost exactly (4.97 m vs. 0.83 m). The cause is kinematic: ram and herd involve large, abrupt maneuvers that the model cannot anticipate from observation alone, while benign and overtake produce nearly straight tracks. Under sensor noise, the ordering inverts: A18 reports H-ADE 6.04 m vs. NH-ADE 7.13 m, and A19 reports 18.83 m vs. 33.07 m. The reason is signal-to-noise: large-displacement hostile maneuvers dominate the noise floor more easily than slow, near-stationary non-hostile traffic. A deployed system should therefore expect hostile-class trajectory error to be the bottleneck on clean tracks, but the non-hostile slice to be the bottleneck once positional jitter is significant. Combined with the hybrid-deployment guidance from the classical-baseline comparison (Section 7.8), this argues for a tiered architecture: a Kalman-filter or CV propagator for routine non-hostile traffic on clean tracks, MTITP for hostile/ambiguous encounters on clean tracks, and a denoising stage or noise-robust variant (A18-style training) once positional jitter is significant.

7.11.6 Validation Summary and Caveats

The five validation checks were included precisely because the headline metrics are high enough to invite skepticism, and each one rules out a specific alternative explanation. The encounter-level split with zero overlap at both encounter and window granularity

(Table 7.26) eliminates data leakage. The 62% gap to the majority-class predictor on past intent and the 59% gap on future intent (Table 7.27) are too large to be explained by class-prior memorization. The t-SNE projection shows that the model leans on a learned discriminative representation rather than a surface shortcut that would dissolve under noise. The cross-dataset evaluation (Table 7.28) shows no model is exploiting a single-distribution artifact: noisy-trained checkpoints transfer back to clean data at intent accuracy comparable to the noiseless baseline. McNemar’s test formalizes the paired comparisons that the ablation tables make descriptively. None of these checks is novel on its own; taken together, they justify treating the ablation conclusions as substantive.

Several caveats apply to the conclusions above. Each ablation was run once owing to compute constraints. Prototype runs indicate single-seed variance on the noiseless test set of approximately ± 0.05 m ADE, which is comfortably below the L2-versus-WGAN gap (1.25 m) and the A20-to-L2 gap (0.23 m). The per-scenario margins by which A21 surpasses L2 fall on either side of this noise floor: ram (0.07 m) sits within run-to-run variance and is reported as parity with L2 rather than a strict improvement, while herd (0.26 m) exceeds the variance by roughly five-fold and is treated as a substantive gain. The retained multimodal sampling is the more durable advantage in either case. All experiments are conducted on simulated data: it reproduces COLREGs geometry and supports controlled noise injection, but it does not replicate AIS artifacts such as message dropouts or transponder mode switches, so validation on real AIS data remains an open item. The model also assumes a two-vessel encounter; both Stream B

and Stream C scale as $O(V^2)$ and would require a graph-based reformulation to extend to harbor-scale scenes. Finally, the cross-dataset McNemar cells involving noisy and noiseless models are shown in Figure 7.16 for completeness only, as they violate the paired-sample assumption and are not used as evidence in the analyses above.

7.12 Conclusion

The argument of this dissertation is that intent recognition and trajectory prediction are better solved together than apart, and MTITP makes that argument concrete. A single shared three-stream encoder feeds the intent heads and an intent-conditioned generator that, in one pass, classifies the encounter, forecasts per-step future behavior, and produces a multi-modal trajectory distribution. On the seven COLREGs-derived scenarios, this yields 99.2% past intent accuracy, 99.4% future intent mAP, and a $2.3\times$ improvement in aggregate ADE over the best classical baseline (MTITP-L2 at 2.17 m vs. EKF-CTRV at 5.06 m, the strongest of the Kalman-filter variants); a combination of outputs that no single baseline in the comparison provides.

The ablation study attributes this performance to two largely distinct pathways. Intent is carried by the joint interaction representation (continuous pairwise geometry from Stream B combined with symbolic relational features from Stream C through the Interaction Fusion module) and reinforced by the multi-task supervision heads, so that switching the trajectory loss among WGAN-GP, L2 regression, and the MarITGAN

v2 critic moves intent accuracy by less than a tenth of a percentage point. Trajectory quality, by contrast, depends on the loss. The original WGAN-GP critic is the limiting factor; replacing it with the intent-aware MarITGAN v2 critic closes the gap to L2 regression to within 0.06 m overall and surpasses L2 on two of the three upweighted classes (ram, herd), while keeping the multi-modal sampling that L2 forfeits.

The five validation checks then rule out the obvious alternative explanations. Encounter-level split integrity rules out leakage; the 60+% gap to majority-class predictors rules out class-prior memorization; t-SNE geometry rules out surface-feature shortcuts; cross-dataset transfer rules out single-distribution overfitting; and McNemar’s test formalizes the paired comparisons. Layered on top of the seven-class metrics, the hostile/non-hostile binary collapse gives the operational reading. Every model clears 98% on both sides of the split. MarITGAN v2 alone shows an asymmetric confusion structure with 99.9% non-hostile recall and 97.9% hostile recall, trading a low false-negative rate for very low false-positive alarms. The trajectory-difficulty ordering also inverts under sensor noise: hostile maneuvers dominate error on clean tracks, and slow non-hostile traffic dominates once positional jitter becomes significant.

Two scope limits temper these findings. All experiments run on simulated data by HII that reproduces COLREGs geometry but not real AIS artifacts such as message dropouts and transponder mode switches. The model also assumes a two-vessel encounter, with Stream B and Stream C scaling as $O(V^2)$, so harbor- or fleet-scale scenes need a graph-based reformulation. The simulator-to-deployment gap

is the greater concern and a future task to cross-validate the intent pathway using independently collected maritime encounter data. Three further directions remain open beyond that: a graph-based architecture for multi-vessel scenes, flow-based or diffusion generators for tighter long-horizon sample coverage, and online intent adaptation as encounters evolve.

Chapter 8

Conclusion

The work in this dissertation concerns maritime intent recognition under three persistent obstacles: a scarcity of labeled encounter data, sensor noise, and the ambiguous interaction geometry of multi-vessel encounters. A distinction maintained throughout is that intent classification, trajectory prediction, and intent prediction are three separate problems rather than one. Intent classification assigns a behavior label to a past observation window; trajectory prediction forecasts future positions; intent prediction forecasts future intent at every step of the prediction horizon. Chapters 4 and 5 address intent classification, Chapter 6 addresses trajectory prediction, and Chapter 7 introduces intent prediction, where it appears alongside the other two as outputs of a single model.

8.1 Summary of Contributions

Chapter 3: NavySim. Chapter 3 introduces NavySim, a Unity-based multi-vessel simulator with scriptable adversarial and benign scenarios, COLREGs-compliant Velocity Obstacle planners, per-timestep behavior labels, threat and vulnerability heatmaps, and a generalized intent recognition pipeline that supports both Hidden Markov Models and the deep-learning classifiers developed in later chapters. Seven scenario classes are produced under controlled noise injection and reproducible seeds. The encounter data generated by NavySim underlies every subsequent chapter.

Chapter 4: Early Intent Classification. Chapter 4 develops a deep-learning pipeline that classifies vessel behavior over a long observation window (the first 500 frames of each scenario) using simulator-derived feature subsets. The LSTM and FCDL classifiers reach 97% accuracy on the seven-class behavior task and 99% on the binary hostile versus non-hostile collapse, providing the standalone classifier baseline against which later models are measured.

Chapter 5: CPFI. The contribution of Chapter 5 is Categorical Permutation Feature Importance (CPFI), a model-agnostic attribution method that corrects the bias of standard permutation importance against categorical and constant predictors. CPFI is evaluated alongside the Transformer Feature Importance Score (TFIS), developed by a collaborator. Class-conditional CPFI rankings reveal behavior-specific predictors

(S_SIDE, S_DREL_BEAR, abs_dheading) that global rankings obscure, and classifiers trained on the resulting subsets retain accuracy under noise injection.

Chapter 6: Classical Trajectory Baselines. Chapter 6 evaluates six Bayesian filters (CV, UKF-4D, UKF-5D-CV, asymmetric UKF-6D, EKF-CTRV, UKF-CTRV) across four observation-and-prediction configurations. Three structural findings emerge. The linear filters (CV, UKF-4D, UKF-5D-CV) are essentially interchangeable at every horizon. EKF-CTRV performs best at very short horizons but loses ground as the constant- ω arc compounds. UKF-CTRV diverges once the unobserved turn-rate covariance grows past the branching threshold. Hostile scenarios (block, ram, herd) produce the largest errors across all six filters, and no filter falls below 11.6 m ADE on herd. The matched-configuration baselines from this evaluation feed directly into the comparisons of the next chapter.

Chapter 7: MTITP. Chapter 7 develops a shared three-stream encoder that fuses per-vessel kinematics (Stream A), pairwise geometry (Stream B), and symbolic COLREGs features (Stream C), and feeds three heads: a past-intent classifier, a per-step future-intent predictor, and an intent-conditioned multi-modal trajectory generator. A two-phase training protocol, in which encoder warmup precedes joint training, avoids the encoder collapse observed when classification and adversarial objectives are imposed simultaneously. Three variants of the model share this backbone and differ only in the trajectory loss: MTITP-WGAN uses a Wasserstein critic with

gradient penalty, MTITP-L2 uses an L2 regression objective, and MarITGAN uses a purpose-built IntentAwareCritic. The joint model attains 99.2% past-intent accuracy, 99.4% future-intent mAP, and a $2.3\times$ aggregate-ADE improvement over the best classical baseline (MTITP-L2 at 2.17 m versus EKF-CTRV at 5.06 m at the matched horizon). The accompanying ablation isolates two pathways within the model. Intent quality is carried by Streams B and C together with the multi-task heads, and the choice of trajectory loss moves intent by less than 0.1 percentage point. Trajectory quality, in contrast, is loss-driven: MarITGAN v2 narrows the WGAN-versus-L2 gap to within 0.06 m overall and surpasses L2 on the ram and herd scenarios while preserving multi-modal sampling. Five validation checks (split integrity, majority-class baselines, t-SNE, cross-dataset transfer, McNemar’s test) discount the obvious alternative explanations of the result.

8.2 Answers to Dissertation Research Questions

RQ1. The Bayesian filter family reaches a structural ceiling on hostile scenarios that no covariance-propagation refinement removes (Chapter 6). No parametric motion model can represent the deliberate course changes that define adversarial intent, and any successor model must therefore carry scenario context, admit multi-modal futures, and produce intent and trajectory together from the same observation window. The MTITP architecture is one such successor.

RQ2. The joint MTITP model improves on both the standalone classifiers of Chapters 4 and 5 and the classical trajectory predictors of Chapter 6 on the same dataset, by the margins reported in the contribution summary. The improvement is concentrated in the hostile scenarios. Future-intent prediction has no prior baseline within the dissertation, since the task itself is introduced in Chapter 7.

8.3 Limitations

Several limitations qualify the results above. The most consequential is the gap between simulation and real deployment. Although NavySim reproduces COLREGs geometry and controlled noise injection, it does not model real AIS artifacts such as message dropouts and transponder-mode switches, and validation on real AIS encounters remains open. A second limitation is the two-vessel assumption built into Streams B and C, whose features scale as $O(V^2)$; harbor-scale and fleet-scale scenes will require a graph-based reformulation. The evaluation is also single-seed: prototype runs place the per-seed variance near ± 0.05 m ADE, comfortably below the L2-versus-WGAN gap of 1.25 m but on the order of the ram margin of 0.07 m. The classical-baseline numbers, in turn, reflect heuristic noise-tuning and should be read as upper bounds on what per-scenario tuning could achieve, although the cross-model comparison in Chapter 7 preserves the configuration on both sides. Finally, CPFI has so far been validated only on the maritime dataset, and its ranking properties have not been tested on other categorical-heavy sequential domains.

8.4 Future Work

The most immediate next step is the validation of the MTITP intent pathway on real AIS data, extending the cross-dataset robustness already shown in Chapter 7. A natural scaling step is a graph-based reformulation of Streams B and C that treats vessels as nodes and pairwise interactions as edges, allowing the encoder to accept scenes of variable size. On the generative side, the encoder-loss decoupling demonstrated by MarITGAN suggests that alternative trajectory generators, such as normalizing-flow or diffusion models, could replace the WGAN-GP critic without disturbing the rest of the model. The IntentAwareCritic also opens an online adaptation regime in which the previous window’s intent distribution becomes a prior for the next, and the critic scores trajectory and intent consistency at each step. Beyond the maritime setting, a comparative study of CPFI against standard PFI on non-maritime sequential datasets, together with an extension of NavySim toward multi-modal sensor synthesis (radar, EO/IR, SAR), would clarify how far the present framework generalizes.

8.5 Closing Remarks

The position taken in this dissertation is that vessel intent and vessel trajectory are best treated as joint outputs of a single model rather than as separate problems. The standalone classifiers of Chapters 4 and 5 produce intent labels but no forecasts; the classical filters of Chapter 6 produce trajectory forecasts but no scenario context.

Chapter 7 brings the two together within a shared backbone, and the supporting contributions (NavySim, CPFI and TFIS, the classical-baseline study) provide the data, the attribution machinery, and the reference points that make this joint formulation interpretable rather than incidental.

Appendix A

Supplementary Material for

Chapter 7: MTITP

This appendix collects implementation notes, hyperparameter listings, and feature-index tables that support Chapter 7 (MTITP). The content is reference material extracted from the chapter to keep the main narrative focused on results and analysis.

A.1 Implementation Notes

Coordinate scale consistency. During early development the generator returned absolute positions by adding $\mathbf{p}_{\text{last}} \approx 8,000$ m offsets internally, while the critic evaluated `real_future` labels in relative space (~ 100 m). The $\sim 80\times$ scale difference drove the Wasserstein distance to -2.5×10^6 within three epochs. We keep all generator and critic

computations in relative space and apply the absolute reconstruction of Eq. (7.44) only at inference. For any GAN operating on geographic coordinate data, this separation is required for the loss to be well posed, not an optional refinement.

Training stability. A handful of configuration choices made the difference between stable training and divergence across all MTITP variants. The 15-epoch intent warmup is non-negotiable: without it, adversarial gradients arrive before the encoder has formed useful representations and joint training never recovers. The critic-update ratio matters in much the same way. Setting $N_c = 3$ from the start of the joint phase lets the critic outpace the generator and produce vanishing gradients; the MarITGAN v2 schedule (A21) sidesteps that by holding $N_c = 1$ for the first 30 joint epochs and stepping up to $N_c = 3$ once the generator has caught up. Window stride is the main lever on wall-clock cost: stride 1 yields a dataset of roughly 3.5 M windows and epoch times above four hours on a single GPU, while stride 10 produces roughly 880 K windows per dataset variant with no measurable hit to final metrics. The coordinate-scale issue above also warrants an explicit pre-flight check: confirming that $\|\mathbf{real_future}\|_\infty$ and $\|\mathbf{fake_future}\|_\infty$ agree within a factor of two before the first epoch catches the failure mode early.

A.2 Hyperparameter Summary

TABLE A.1: MTITP data and windowing hyperparameters.

Parameter	Value	Description
T_{obs}	20	Observation window length (steps at 1 Hz)
T_{pred}	5	Prediction horizon (steps)
Δt	1.0 s	Timestep interval
Stride s	10	Sliding window stride

TABLE A.2: MTITP model architecture hyperparameters.

Parameter	Value	Description
D_{model}	128	Token dimension
D_{inter}	64	Interaction fusion dimension
D_{sym}	8	Per-symbol embedding dimension
H	4	Attention heads (all MHA layers)
N_{enc}	4	Temporal+Spatial block pairs
N_{dec}	3	Generator decoder layers
d_z	32	Noise vector dimension
d_{int}	16	Intent embedding dimension
Dropout	0.1	Applied in all FFN and MHA layers

TABLE A.3: MTITP training and optimisation hyperparameters.

Parameter	Value	Description
λ_{GP}	10.0	WGAN-GP gradient penalty coefficient
λ_{smooth}	0.1	Jerk penalty coefficient
$\lambda_{\text{past/future}}$	1.0/1.0	Intent loss weights
N_c	1	Critic updates per generator step
Warmup epochs E_w	15	Intent-only training phase
Total epochs E	115	Total training epochs
Batch size	512	Mini-batch size
LR (main / critic)	10^{-4}	Adam learning rate
(β_1, β_2)	(0.5, 0.9)	Adam momentum parameters
Weight decay	10^{-5}	L2 regularisation
Gradient clip norm	1.0	Applied before each optimiser step
K	20	Trajectory samples at inference

A.3 Feature Dimensions

TABLE A.4: Stream A per-vessel kinematic features (8 per vessel per timestep).

Idx	Symbol	Description	Unit
0	x	Easting (relative to window origin)	m
1	y	Northing (relative to window origin)	m
2	v_x	Eastward velocity	m/s
3	v_y	Northward velocity	m/s
4	$\sin \theta$	Sine of heading (CW from North)	—
5	$\cos \theta$	Cosine of heading	—
6	σ	Speed	m/s
7	$\dot{\sigma}$	Longitudinal acceleration	m/s ²

TABLE A.5: Stream B pairwise geometry features (12 per timestep).

Idx	Symbol	Description	Unit
0	d	Inter-vessel distance	m
1	α^{br}	Absolute bearing, blue $\rightarrow$ red	deg
2	α^{rb}	Absolute bearing, red $\rightarrow$ blue	deg
3	ρ^{br}	Relative bearing, blue $\rightarrow$ red	deg
4	ρ^{rb}	Relative bearing, red $\rightarrow$ blue	deg
5	$\Delta\psi$	Absolute heading difference	deg
6	x_r	Relative position: easting	m
7	y_r	Relative position: northing	m
8	Δv_x	Velocity difference: easting	m/s
9	Δv_y	Velocity difference: northing	m/s
10	t_{CPA}	Time to closest point of approach (clipped)	s
11	d_{CPA}	Distance at CPA (clipped)	m

Bibliography

- [1] Kashif Talpur, Raza Hasan, Ismet Gocer, Shakeel Ahmad, and Zakirul Bhuiyan. Ai in maritime security: applications, challenges, future directions, and key data sources. *Information*, 16(8):658, 2025.
- [2] Ying Yang, Yang Liu, Guorong Li, Zekun Zhang, and Yanbin Liu. Harnessing the power of machine learning for ais data-driven maritime research: A comprehensive review. *Transportation research part E: logistics and transportation review*, 183:103426, 2024.
- [3] Zhiye Xie, Enmei Tu, Xianping Fu, Guoliang Yuan, and Yi Han. Ais data-driven maritime monitoring based on transformer: A comprehensive review. In *2025 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2025.
- [4] Huanhuan Li, Hang Jiao, and Zaili Yang. Ship trajectory prediction based on machine learning and deep learning: A systematic review and methods analysis. *Engineering Applications of Artificial Intelligence*, 126:107062, 2023.

- [5] Muhammad Adeel Rishi, Jennifer Creamer, and Kathy Sexton-Radek. Maritime industry safety risks: fatigue and poor sleep. *International Maritime Health*, 75(4):267–268, 2024.
- [6] Md Abu Sayed, Korben DiArchangel, Mayamin Hamid Raha, Parvaneh Aliniya, Monica Nicolescu, Mircea Nicolescu, and Sushil Louis. Navysim: A multi-vessel simulation and analysis engine for naval domains. In *2024 IEEE Conference on Games (CoG)*, pages 1–8, 2024.
- [7] Sasha Azad and Chris Martens. Little computer people: A survey and taxonomy of simulated models of social interaction. *Proceedings of the ACM on Human-Computer Interaction*, 5(CHI PLAY):1–30, 2021.
- [8] Nattawat Chavanit, Sukawit Bualoy, and Samart Moodleah. Naval wargame prototyping: Multiplayer real-time strategy game simulation using unreal engine. In *2023 15th International Conference on Information Technology and Electrical Engineering (ICITEE)*, pages 1–6. IEEE, 2023.
- [9] Chunliu Wang, Yutong Xu, and Daihui Zhang. Multi-modal perception and fusion for maritime autonomy: A survey. *Preprints*, October 2025. doi: 10.20944/preprints202510.0977.v1. URL <https://doi.org/10.20944/preprints202510.0977.v1>.
- [10] Jonathan Becktor, Frederik ET Schöller, Evangelos Boukas, Mogens Blanke, and Lazaros Nalpantidis. Bolstering maritime object detection with synthetic data. *IFAC-PapersOnLine*, 55(31):64–69, 2022.

- [11] Loukas Ilias, Giannis Tsapelas, Panagiotis Kapsalis, Vasilis Michalakopoulos, Giorgos Kormpakis, Spiros Mouzakis, and Dimitris Askounis. Leveraging extreme scale analytics, ai and digital twins for maritime digitalization: the vesselai architecture. *Frontiers in Big Data*, 6:1220348, 2023.
- [12] Daniel Menges and Adil Rasheed. Digital twin for autonomous surface vessels: Enabler for safe maritime navigation. *arXiv preprint arXiv:2411.03465*, 2024.
- [13] Nuwan Sri Madusanka, Yijie Fan, Shaolong Yang, and Xianbo Xiang. Digital twin in the maritime domain: A review and emerging trends. *Journal of marine science and engineering*, 11(5):1021, 2023.
- [14] Wei Wang, Han Zhang, Yang Li, Zhenyu Zhang, Xiangfeng Luo, and Shaorong Xie. Usvs-sim: A general simulation platform for unmanned surface vessels autonomous learning. *Concurrency and Computation: Practice and Experience*, 34(3):e6567, 2022.
- [15] Georgios Grigoropoulos, Giannis Spiliopoulos, Ilias Chamatidis, Manolis Kaliorakis, Alexandros Troupiotis-Kapeliaris, Marios Vondas, Evangelia Filippou, Eva Chondrodima, Nikos Pelekis, Yannis Theodoridis, Dimitris Zissis, and Konstantina Bereta. Gmsa: A digital twin application for maritime route and event forecasting. In *Proceedings of the 27th International Conference on Extending Database Technology (EDBT)*, pages 798–801. OpenProceedings.org, 2024. doi: 10.48786/edbt.2024.73.

- [16] Eunkyu Lee, Junaid Khan, Umar Zaman, Jaebin Ku, Sanha Kim, and Kyungsup Kim. Synthetic maritime traffic generation system for performance verification of maritime autonomous surface ships. *Applied Sciences*, 14(3):1176, 2024.
- [17] Hyobin Park, Jinwook Jung, Minseok Seo, Hyunsoo Choi, Deukjae Cho, Sekil Park, and Dong-Geol Choi. Ais-llm: A unified framework for maritime trajectory prediction, anomaly detection, and collision risk assessment with explainable forecasting. *arXiv preprint arXiv:2508.07668*, 2025.
- [18] Chaewon Kim, Seonghun Hong, Jeonghong Park, Jinwoo Choi, and Hye-Jin Kim. Generation of navigation database using ais data for remote situational awareness of coastal vessels. *Applied Ocean Research*, 154:104401, 2025.
- [19] Md Abu Sayed, Korben DiArchangel, Parvaneh Aliniya, Casey Brenner, Mayamin Hamid Raha, Monica Nicolescu, Mircea Nicolescu, and Sushil Louis. Navysim 2.0: Enhanced multi-vessel simulation and analysis engine for advanced naval research. *IEEE Transactions on Games*, 2026. Submitted.
- [20] Zhuoyue Zhang and Haitong Xu. Explainable ai for maritime autonomous surface ships (mass): Adaptive interfaces and trustworthy human-ai collaboration. *arXiv preprint arXiv:2509.15959*, 2025.
- [21] Jianping Hua, Zixiang Xiong, James Lowey, Edward Suh, and Edward R. Dougherty. Optimal number of features as a function of sample size for various

- classification rules. *Bioinformatics*, 21(8):1509–1515, 11 2004. ISSN 1367-4803. doi: 10.1093/bioinformatics/bti171. URL <https://doi.org/10.1093/bioinformatics/bti171>.
- [22] Amina Benkessirat and Nadjia Benblidia. Fundamentals of feature selection: An overview and comparison. In *2019 IEEE/ACS 16th International Conference on Computer Systems and Applications (AICCSA)*, pages 1–6, 2019. doi: 10.1109/AICCSA47632.2019.9035281.
- [23] Mingshun Yuan, Zijiang Yang, and Guoli Ji. Partial maximum correlation information: A new feature selection method for microarray data classification. *Neurocomputing*, 323:231–243, 2019.
- [24] Jie Xue, Peijie Yang, Qianbing Li, Yuanming Song, PHAJM van Gelder, Eleonora Papadimitriou, and Hao Hu. Machine learning in maritime safety for autonomous shipping: A bibliometric review and future trends. *Journal of Marine Science and Engineering*, 13(4):746, 2025.
- [25] Helen L Smith, Patrick J Biggs, Nigel P French, Adam NH Smith, and Jonathan C Marshall. Out of (the) bag—encoding categorical predictors impacts out-of-bag samples. *PeerJ Computer Science*, 10:e2445, 2024.
- [26] Hyunseung Hwang, Andrew Bell, Joao Fonseca, Venetia Pliatsika, Julia Stoyanovich, and Steven Euijong Whang. Shap-based explanations are sensitive to feature representation. In *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency*, pages 1588–1601, 2025.

- [27] Xuanxiang Huang and Joao Marques-Silva. On the failings of shapley values for explainability. *International Journal of Approximate Reasoning*, 171:109112, 2024.
- [28] Xin Cheng, Jintao Wang, Xinjun Chen, and Fan Zhang. Attention-enhanced and integrated deep learning approach for fishing vessel classification based on multiple features. *Scientific Reports*, 15(1):8642, 2025.
- [29] Zhizheng Wu, Shengzheng Wang, Leyao Li, and Yongfeng Suo. An interpretable ship risk model based on machine learning and shap interpretation technique. *Ocean Engineering*, 335:121686, 2025.
- [30] Jonatan Katz Nielsen. Machine learning-based classification of vessel types in straits using ais tracks. *arXiv preprint arXiv:2509.18109*, 2025.
- [31] Nanyu Chen, Anran Yang, Hui Wu, Luo Chen, Wei Xiong, and Ning Jing. Semint: an llm-empowered long-term vessel trajectory prediction framework. *International Journal of Geographical Information Science*, 39(9):1938–1972, 2025.
- [32] Feilong Huang, Zhuoran Liu, Xiaohe Li, Fangli Mou, Pengfei Li, and Zide Fan. Vessel trajectory prediction based on ais data: dual-path spatial-temporal attention network with multi-attribute information. *Journal of Marine Science and Engineering*, 12(11):2031, 2024.

- [33] Wanrong Wu, Dongmei Yan, Jun Yan, and Xiaowei Wang. Ship behavior prediction and anomaly detection using lstm-dcross model based on ais and remote sensing data. *International Journal of Digital Earth*, 18(1):2515251, 2025.
- [34] Qianqian Chen, Changshi Xiao, Yuanqiao Wen, Mengwei Tao, and Wenqiang Zhan. Ship intention prediction at intersections based on vision and bayesian framework. *Journal of Marine Science and Engineering*, 10(5):639, 2022.
- [35] Xiao Han, Yang Zhou, Jianjun Weng, Lijia Chen, and Kang Liu. Research on fishing vessel recognition based on vessel behavior characteristics from ais data. *Frontiers in Marine Science*, 12:1547658, 2025.
- [36] Fangcheng Han, Yang Liu, Hao Tian, Jianchao Li, and Yongjun Tian. A comprehensive framework incorporating deep learning for analyzing fishing vessel activity using automatic identification system data. *ICES Journal of Marine Science*, 82(2):fsae166, 2025.
- [37] Gil-ho Shin and Hyun Yang. Vessel trajectory prediction in harbors: A deep learning approach with maritime-based data preprocessing and berthing side integration. *Ocean Engineering*, 316:119908, 2025.
- [38] Blake Cole and Gabriel Schamberg. Unscented kalman filter for long-distance vessel tracking in geodetic coordinates. *Applied Ocean Research*, 124:103205, 2022.

- [39] Xiaoyou He, Yu Su, and Yuhe Qiu. An improved unscented kalman filter for maneuvering target tracking. In *Journal of Physics: Conference Series*, volume 2216, page 012010. IOP Publishing, 2022.
- [40] Wei Liu, Xingdi Wang, Yuan Hu, Panting Ma, Jiansen Zhao, Bing Han, and Shengzheng Wang. An improved center difference kalman filtering method for navigation and positioning of unmanned surface vessels. *Ocean Engineering*, 324:120696, 2025.
- [41] Jiaxuan YANG, Baiguo CHEN, and Lingqi MA. Ship trajectory tracking based on imm-sckf algorithm. *Chinese Journal of Ship Research*, 17(4):12–23, 2022.
- [42] Xiaohua Li, Bo Lu, Yuxing Li, Xiaofeng Lu, and Haiyan Jin. Adaptive interacting multiple model for underwater maneuvering target tracking with one-step randomly delayed measurements. *Ocean engineering*, 280:114933, 2023.
- [43] Sebastian Dingler. State estimation with the interacting multiple model (imm) method. *arXiv preprint arXiv:2207.04875*, 2022.
- [44] Lin Ma, Hao Cao, and Guo-You Shi. Ship behavior pattern recognition method based on hybrid graph neural networks. *Frontiers in Marine Science*, 12:1605216, 2025.
- [45] Pengyue Wang, Mingyang Pan, Zongying Liu, Shaoxi Li, Yuanlong Chen, and Yang Wei. Ship trajectory prediction in complex waterways based on transformer

- and social variational autoencoder (socialvae). *Journal of Marine Science and Engineering*, 12(12):2233, 2024.
- [46] Junhao Jiang, Yi Zuo, and Zhiyuan Li. Multi-modal graph convolutional network for vessel trajectory prediction based on cooperative intention enhance using conditional variational autoencoder. *Reliability Engineering & System Safety*, page 111885, 2025.
- [47] Chengfeng Jia, Jie Ma, Xin Yang, and Xiaolei Lv. Ragan: A generative adversarial network for risk-aware trajectory prediction in multi-ship encounter situations. *Ocean Engineering*, 289:116188, 2023.
- [48] Pengfei Chen, Fengkai Yang, Junmin Mou, Linying Chen, and Mengxia Li. Regional ship behavior and trajectory prediction for maritime traffic management: A social generative adversarial network approach. *Ocean Engineering*, 299:117186, 2024.
- [49] Hang Jiao, Huanhuan Li, Jasmine Siu Lee Lam, Xiaowei Gao, and Zaili Yang. Multi-factor influence-based ship trajectory prediction analysis via deep learning. *Journal of Marine Engineering & Technology*, pages 1–19, 2025.
- [50] Duong Nguyen and Ronan Fablet. A transformer network with sparse augmented data representation and cross entropy loss for ais-based vessel trajectory prediction. *IEEE Access*, 12:21596–21609, 2024.

- [51] Hang Yuan, Kezhong Liu, Xiaolie Wu, Yuerong Yu, Xuri Xin, and Weiqiang Wang. Gatransformer: A vessel trajectory prediction method based on attention algorithm in complex navigable waters. *Ocean Engineering*, 326:120902, 2025.
- [52] Junhao Jiang, Yi Zuo, Yang Xiao, Wenjun Zhang, and Tieshan Li. Stmgf-net: a spatiotemporal multi-graph fusion network for vessel trajectory forecasting in intelligent maritime navigation. *IEEE Transactions on Intelligent Transportation Systems*, 25(12):21367–21379, 2024.
- [53] Fan Yang, Chunlin He, Yi Liu, Anping Zeng, and Longhe Hu. Vessel trajectory prediction based on automatic identification system data: multi-gated attention encoder decoder network. *Journal of Marine Science and Engineering*, 12(10):1695, 2024.
- [54] Wentao Wang, Wei Xiong, Xue Ouyang, and Luo Chen. Tptrans: Vessel trajectory prediction model based on transformer using ais data. *ISPRS International Journal of Geo-Information*, 13(11):400, 2024.
- [55] Jiao Liu, Kaige Zhu, Yaqing Shu, Yuanqiang Zhang, and Pengjun Zheng. Trajectory prediction model of a ship crossing under a bridge based on multi-head attention mechanism and wasserstein gan with gradient penalty. *Ocean Engineering*, 334:121551, 2025.
- [56] Joseph Tafataona Mtetwa, Kingsley A Ogudo, and Sameerchand Pudaruth. Adaptive gradient penalty for wasserstein gans: Theory and applications. *Mathematics*, 13(16):2651, 2025.

- [57] Lei Xie, Tao Guo, Jiliang Chang, Chengpeng Wan, Xinyuan Hu, Yang Yang, and Changkui Ou. A novel model for ship trajectory anomaly detection based on gaussian mixture variational autoencoder. *IEEE Transactions on Vehicular Technology*, 72(11):13826–13835, 2023.
- [58] Jiawei Hou, Hongzhu Zhou, Manel Grifoll, Yusheng Zhou, Jiao Liu, Yun Ye, and Pengjun Zheng. A transformer–vae approach for detecting ship trajectory anomalies in cross-sea bridge areas. *Journal of marine science and engineering*, 13(5):849, 2025.
- [59] Chiyu Jiang, Andre Cornman, Cheolho Park, Benjamin Sapp, Yin Zhou, Dragomir Anguelov, et al. Motiondiffuser: Controllable multi-agent motion prediction using diffusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9644–9653, 2023.
- [60] Changlin Li, Yanglei Gan, Tian Lan, Yuxiang Cai, Xueyi Liu, Run Lin, and Qiao Liu. Diffutraj: A stochastic vessel trajectory prediction approach via guided diffusion process. *arXiv preprint arXiv:2410.09550*, 2024.
- [61] Jin Wenzhe, Tang Haina, and Zhang Xudong. Stgdpm: Vessel trajectory prediction with spatio-temporal graph diffusion probabilistic model. *arXiv preprint arXiv:2503.08065*, 2025.
- [62] Farzeen Munir and Tomasz Piotr Kucner. Context-aware multi-task learning for pedestrian intent and trajectory prediction. *Transportation Research Part C: Emerging Technologies*, 178:105203, 2025.

- [63] Neha Sharma, Chhavi Dhiman, and Sreedevi Indu. Predicting pedestrian intentions with multimodal intentformer: A co-learning approach. *Pattern Recognition*, 161:111205, 2025.
- [64] Fenyu Jiang, Huandong Wang, and Yong Li. Vesnet: a vessel network for jointly learning route pattern and future trajectory. *ACM Transactions on Intelligent Systems and Technology*, 15(2):1–25, 2024.
- [65] Yuegao Wu, Weiqiang Liao, Wanneng Yu, Guangmiao Zeng, Yifan Shang, and Xin Dong. Multi-task learning for ship trajectory prediction and motion planning via node relationship modeling. *IEEE Transactions on Intelligent Transportation Systems*, 26(9):13051–13064, 2025. doi: 10.1109/TITS.2025.3573811.
- [66] Yuyol Shin, Namwoo Kim, Hyeyeong Lee, Soh Young In, Mark Hansen, and Yoonjin Yoon. Deep learning framework for vessel trajectory prediction using auxiliary tasks and convolutional networks. *Engineering Applications of Artificial Intelligence*, 132:107936, 2024.
- [67] Ayesh Meepaganithage, Md Abu Sayed, Mircea Nicolescu, and Monica Nicolescu. Proactive maritime threat prediction: Vessel intent classification with lstms and transformers using a sliding window approach. In *2025 IEEE 21st International Conference on Automation Science and Engineering (CASE)*, pages 3261–3267, 2025. doi: 10.1109/CASE58245.2025.11164112.

- [68] Vasilios D Tsoukalas, Dimitrios A Papachristos, Nikolaos K Tsoumas, and Elisabeth C Mattheu. Marine engineers' training: educational assessment for an engine room simulator. *WMU Journal of Maritime Affairs*, 7:429–448, 2008.
- [69] Karina Hjelmervik, Salman Nazir, and Andreas Myhrvold. Simulator training for maritime complex tasks: an experimental study. *WMU Journal of Maritime Affairs*, 17(1):17–30, 2018.
- [70] Vladimir M Petrović. Artificial intelligence and virtual worlds—toward human-level ai agents. *IEEE Access*, 6:39976–39988, 2018.
- [71] Vladimir M Petrović and Branko D Kovačević. Avilab—gamified virtual educational tool for introduction to agent theory fundamentals. *Electronics*, 11(3):344, 2022.
- [72] Michal Jakob, Ondrej Vanek, Ondrej Hrstka, and Michal Pechoucek. Agents vs. pirates: multi-agent simulation and optimization to fight maritime piracy. In *AAMAS*, pages 37–44, Spain, 2012. AAMAS 2012.
- [73] Jiabao Cao, Jinfeng Dou, Jilong Liu, Xuanning Wei, and Zhongwen Guo. Multi-agent deep reinforcement learning framework strategized by unmanned aerial vehicles for multi-vessel full communication connection. *Remote Sensing*, 15(16):4059, 2023.
- [74] Helmut Hilgert and Michael Baldauf. A common risk model for the assessment of encounter situations on board ships. *Ocean Dynamics*, 49(4):531–542, 1997.

- [75] Jun Min Mou, Cees van der Tak, and Han Ligteringen. Study on collision avoidance in busy waterways by using ais data. *Ocean Engineering*, 37(5): 483–490, 2010. ISSN 0029-8018. doi: <https://doi.org/10.1016/j.oceaneng.2010.01.012>. URL <https://www.sciencedirect.com/science/article/pii/S002980181000020X>.
- [76] Logan Carlson, Dalton Navalta, Monica Nicolescu, Mircea Nicolescu, and Gail Woodward. Early classification of intent for maritime domains using multinomial hidden markov models. *Frontiers in Artificial Intelligence*, 4:702153, 2021. doi: 10.3389/frai.2021.702153. URL <https://www.frontiersin.org/articles/10.3389/frai.2021.702153/full>.
- [77] Wei Zhang, Kezhong Liu, Jie Ma, and Yaqing Shu. Research into vessel behaviour pattern recognition in the maritime domain. *Ocean Engineering*, 232:109154, 2021. doi: 10.1016/j.oceaneng.2021.109154.
- [78] Zhiyuan Wang, Lei Zhou, Wei He, Jinyu Lei, and Chengqiang Yu. A two-stage hybrid learning framework for ship encounter recognition based on ais data. *Ocean Engineering*, 356:125192, 2026. ISSN 0029-8018. doi: <https://doi.org/10.1016/j.oceaneng.2026.125192>. URL <https://www.sciencedirect.com/science/article/pii/S0029801826010267>.
- [79] Ayesh Meepaganithage, Mircea Nicolescu, and Monica Nicolescu. Hostile vessel behavior prediction using lstms. In *Proceedings of the 2024 7th International Conference on Machine Vision and Applications*, pages 48–53, 2024.

- [80] Md Abu Sayed, Md Azizul Hakim, Ayesh Meepaganithage, Monica Nicolescu, and Mircea Nicolescu. Feature-aware deep learning for maritime intent recognition. In *2025 IEEE International Conference on Future Machine Learning and Data Science (FMLDS)*, pages 338–344. IEEE, 2025.
- [81] Kun Xu, Jimmy Ba, Jamie Kiros, and Kyunghyun Cho. Show, attend and tell: Neural image caption generation with visual attention. *International Journal of Computer Vision*, 123(1):55–79, 2019.
- [82] Hwan Kim, Mingyu Choi, Sekil Park, and Sungsu Lim. Vessel trajectory classification via transfer learning with deep convolutional neural networks. *PLoS One*, 19(8):e0308934, 2024.
- [83] Tao Guo, Lei Xie, and Bing Wu. Memory and skip variational autoencoder: a novel approach for ship trajectory anomaly detection. *Journal of Marine Science and Technology*, 30(3):466–479, 2025.
- [84] Francesco Quinzan, Ashkan Soleymani, Patrick Jaillet, Cristian R Rojas, and Stefan Bauer. Drcfs: Doubly robust causal feature selection. In *International Conference on Machine Learning*, pages 28468–28491. PMLR, 2023.
- [85] Ian Connick Covert, Wei Qiu, Mingyu Lu, Na Yoon Kim, Nathan J White, and Su-In Lee. Learning to maximize mutual information for dynamic feature selection. In *International Conference on Machine Learning*, pages 6424–6447. PMLR, 2023.

- [86] Aoran Wang, Hongyang Yang, Feng Mao, Zongzhang Zhang, Yang Yu, and Xiaoyang Liu. Anti-drifting feature selection via deep reinforcement learning (student abstract). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 16356–16357, 2023.
- [87] I-Lun Huang, Man-Chun Lee, Chung-Yuan Nieh, and Juan-Chen Huang. Ship classification based on ais data and machine learning methods. *Electronics*, 13(1):98, 2023.
- [88] Leo Breiman. Random forests. *Machine learning*, 45:5–32, 2001.
- [89] Aaron Fisher, Cynthia Rudin, and Francesca Dominici. All models are wrong, but many are useful: Learning a variable’s importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research*, 20(177):1–81, 2019.
- [90] Raisa Abedin Disha and Sajjad Waheed. Performance analysis of machine learning models for intrusion detection system using gini impurity-based weighted random forest (giwrf) feature selection technique. *Cybersecurity*, 5(1):1, 2022.
- [91] Firuz Kamalov, Fadi Thabtah, and Ho Hon Leung. Feature selection in imbalanced data. *Annals of Data Science*, 10(6):1527–1541, 2023.
- [92] Md. Azizul Hakim, Md. Zahid Hasan, Md. Mahabur Alam, Md. Mehadi Hasan, and Mohammad Nurul Huda. An efficient modified bagging method for early prediction of brain stroke. In *2019 International Conference on Computer*,

- Communication, Chemical, Materials and Electronic Engineering (IC4ME2)*, pages 1–4, Bangladesh, 2019. IEEE. doi: 10.1109/IC4ME247184.2019.9036700.
- [93] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30:4765–4774, 2017.
- [94] Alexander Mott, Daniel Zoran, Mike Chrzanowski, Steven Cavallo, Simon Osindero, and Nando de Freitas. Towards interpretable reinforcement learning using attention augmented agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 12329–12338, 2019.
- [95] Andrei Margeloiu, Nikola Simidjievski, Pietro Lio, and Mateja Jamnik. Weight predictor network with feature selection for small sample tabular biomedical data. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 9081–9089, 2023.
- [96] Wael Hassanieh and Abdallah Chehade. Selective deep autoencoder for unsupervised feature selection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 12322–12330, 2024.
- [97] Miao Zhang, Chris Ding, Ya Zhang, and Feiping Nie. Feature selection at the discrete limit. In *Proceedings of the AAAI conference on artificial intelligence*, volume 28, 2014.

- [98] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 03 1960. ISSN 0021-9223. doi: 10.1115/1.3662552. URL <https://doi.org/10.1115/1.3662552>.
- [99] Arthur Gelb et al. *Applied optimal estimation*. MIT press, 1974.
- [100] Simon J Julier and Jeffrey K Uhlmann. New extension of the kalman filter to nonlinear systems. In *Signal processing, sensor fusion, and target recognition VI*, volume 3068, pages 182–193. Spie, 1997.
- [101] Lokukaluge P Perera, Paulo Oliveira, and C Guedes Soares. Maritime traffic monitoring based on vessel detection, tracking, state estimation, and trajectory prediction. *IEEE Transactions on Intelligent Transportation Systems*, 13(3): 1188–1200, 2012.
- [102] H.A.P. Blom and Y. Bar-Shalom. The interacting multiple model algorithm for systems with markovian switching coefficients. *IEEE Transactions on Automatic Control*, 33(8):780–783, 1988. doi: 10.1109/9.1299.
- [103] Dirk Helbing and Peter Molnar. Social force model for pedestrian dynamics. *Physical review E*, 51(5):4282, 1995.
- [104] Alexandre Alahi, Kratarth Goel, Vignesh Ramanathan, Alexandre Robicquet, Li Fei-Fei, and Silvio Savarese. Social lstm: Human trajectory prediction in crowded spaces. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 961–971, 2016.

- [105] Agrim Gupta, Justin Johnson, Li Fei-Fei, Silvio Savarese, and Alexandre Alahi. Social gan: Socially acceptable trajectories with generative adversarial networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2255–2264, 2018.
- [106] Tim Salzmann, Boris Ivanovic, Punarjay Chakravarty, and Marco Pavone. Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data. In *European conference on computer vision*, pages 683–700. Springer, 2020.
- [107] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [108] Ye Yuan, Xinshuo Weng, Yanglan Ou, and Kris M Kitani. Agentformer: Agent-aware transformers for socio-temporal multi-agent forecasting. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9813–9823, 2021.
- [109] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

- [110] Ryan Wen Liu, Weixin Zheng, and Maohan Liang. Spatio-temporal multi-graph transformer network for joint prediction of multiple vessel trajectories. *Engineering Applications of Artificial Intelligence*, 129:107625, 2024.
- [111] Siwen Wang, Ying Li, Hu Xing, and Zhaoyi Zhang. Vessel trajectory prediction based on spatio-temporal graph convolutional network for complex and crowded sea areas. *Ocean Engineering*, 298:117232, 2024.
- [112] Zhiwen Zhang, Zipei Fan, Zewu Lv, Xuan Song, and Ryosuke Shibasaki. Long-term vessel trajectory imputation with physics-guided diffusion probabilistic model. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*, pages 4398–4407, 2024.
- [113] Rich Caruana. Multitask learning. *Machine learning*, 28(1):41–75, 1997.
- [114] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7482–7491, 2018.
- [115] Lukas Kirchdorfer, Cathrin Elich, Simon Kutsche, Heiner Stuckenschmidt, Lukas Schott, and Jan M Köhler. Analytical uncertainty-based loss weighting in multi-task learning. In *DAGM German Conference on Pattern Recognition*, pages 344–361. Springer, 2024.

- [116] Lukas Kirhdorfer, Tobias Sesterhenn, Christian Bartelt, Heiner Stuckenschmidt, Lukas Schott, and Jan M Köhler. Investigating uncertainty weighting for multi-task learning: Insights and analytical alternative: L. kirhdorfer et al. *International Journal of Computer Vision*, 134(1):8, 2026.
- [117] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, volume 27, 2014.
- [118] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *International conference on machine learning*, pages 214–223. Pmlr, 2017.
- [119] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron C Courville. Improved training of wasserstein gans. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [120] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations*, 2018.
- [121] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. *arXiv preprint arXiv:1411.1784*, 2014.

- [122] Augustus Odena, Christopher Olah, and Jonathon Shlens. Conditional image synthesis with auxiliary classifier gans. In *International Conference on Machine Learning*, pages 2642–2651, 2017.
- [123] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In *International Conference on Learning Representations*, 2016.
- [124] Cristóbal Esteban, Stephanie L Hyland, and Gunnar Rätsch. Real-valued (medical) time series generation with recurrent conditional gans. *arXiv preprint arXiv:1706.02633*, 2017.
- [125] Jinsung Yoon, Daniel Jarrett, and Mihaela van der Schaar. Time-series generative adversarial networks. In *Advances in Neural Information Processing Systems*, volume 32, pages 5508–5518, 2019.
- [126] Maayan Frid-Adar, Eyal Klang, Michal Amitai, Jacob Goldberger, and Hayit Greenspan. Synthetic data augmentation using gan for improved liver lesion classification. *IEEE Transactions on Medical Imaging*, 38(3):682–693, 2018.
- [127] David Bau, Jun-Yan Zhu, Hendrik Strobelt, Bolei Zhou, Joshua B Tenenbaum, William T Freeman, and Antonio Torralba. Gan dissection: Visualizing and understanding generative adversarial networks. In *International Conference on Learning Representations*, 2019.

- [128] R Anuja and J Annrose. End-to-end deep learning for smart maritime threat detection: an ae-cnn-lstm-based approach. *Scientific Reports*, 15(1):36316, 2025.
- [129] Mayamin Hamid Raha, Md Abu Sayed, Sergiu Dascalu, Monica Nicolescu, and Mircea Nicolescu. Keep sailing: An investigation of effective navigation controls and subconscious learning in simulated maritime environment. In *International Conference on Information Technology-New Generations*, pages 313–322. Springer, 2024.
- [130] Korben DiArchangel. Comparing potential fields and velocity obstacles for rules of the road compliant ship driving. Master’s thesis, University of Nevada, Reno, 2024.
- [131] Casey Brenner. Real-time allocation of mobile resources in a naval defense strategy. Master’s thesis, University of Nevada, Reno, 2024.
- [132] Sushil Louis. NavySimulation. <https://github.com/sushillouis/NavySimulation>. URL <https://github.com/sushillouis/NavySimulation>.
- [133] Paolo Fiorini and Zvi Shiller. Motion planning in dynamic environments using velocity obstacles. *The International Journal of Robotics Research*, 17(7):760–772, 1998. doi: 10.1177/027836499801700706. URL <https://doi.org/10.1177/027836499801700706>.
- [134] US Coast Guard. *Navigation Rules*. 2012.

- [135] Md Abu Sayed, Mayamin Hamid Raha, Parvaneh Aliniya, Monica Nicolescu, and Mircea Nicolescu. Threatmap: A framework for enhancing security awareness and decision-making for naval agents. In *Proceedings of the 26th International Conference on Harbor, Maritime and Multimodal Logistic Modeling & Simulation (HMS 2024)*, page 007. CAL-TEK, 2024. doi: 10.46354/i3m.2024.hms.007.
- [136] Md Azizul Hakim, Md Abu Sayed, Ayesh Meepaganithage, Tyler J Becker, Monica Nicolescu, and Mircea Nicolescu. Early classification of intentions for maritime domains using deep learning models. In *2025 IEEE 21st International Conference on Automation Science and Engineering (CASE)*, pages 1171–1178. IEEE, 2025.
- [137] Mayamin Hamid Raha, Md Abu Sayed, Monica Nicolescu, Mircea Nicolescu, and Sushil Louis. Ram: Resource allocation for multi-agent maritime environment. In *International Conference on Informatics in Control, Automation and Robotics*, pages 69–95. Springer, 2023.
- [138] Mayamin Hamid Raha, Md Abu Sayed, Monica Nicolescu, Mircea Nicolescu, and Sushil Louis. Maritime dynamic resource allocation and risk minimization using visual analytics and elitist multi-objective optimization. In *Proceedings of the 20th International Conference on Informatics in Control, Automation and Robotics - Volume 1: ICINCO*, pages 54–63. SciTePress, 2023. ISBN 978-989-758-670-5. doi: 10.5220/0012190100003543.

- [139] Sukhpal Kaur, Himanshu Aggarwal, and Rinkle Rani. Hyper-parameter optimization of deep learning model for prediction of parkinson's disease. *Machine Vision and Applications*, 31(5):32, 2020. doi: 10.1007/s00138-020-01078-1.
- [140] Wennan Wang, Jiadong Lu, Linkai Zhu, Shugeng Dai, and Shiyang Song. A multimodal deep learning approach for hurricane tack forecast based on encoder-decoder framework. *Pattern Analysis and Applications*, 27(4):122, 2024. doi: 10.1007/s10044-024-01344-2.
- [141] Weichao Xu, Shun Liu, Xu Sun, Siyang Liu, and Yun Zhang. A fast algorithm for unbiased estimation of variance of auc based on dynamic programming. *IEEE Access*, 4:9553–9560, 2016. doi: 10.1109/ACCESS.2016.2628102.
- [142] Bryan Auslander, Kalyan Moy Gupta, and David W Aha. Maritime threat detection using plan recognition. In *2012 IEEE Conference on Technologies for Homeland Security (HST)*, pages 249–254. IEEE, 2012.
- [143] Feihong Qiao, Yunqiao Xi, Jian Liu, and Junping Zhang. Intent recognition of maritime group targets based on biconvlstm-attention. In *Signal Processing, Sensor/Information Fusion, and Target Recognition XXXIII*, volume 13057, pages 405–409. SPIE, 2024.
- [144] Jie Ma, Chengfeng Jia, Yaqing Shu, Kezhong Liu, Yu Zhang, and Yulong Hu. Intent prediction of vessels in intersection waterway based on learning vessel motion patterns with early observations. *Ocean Engineering*, 232:109154, 2021.

- [145] Qianqian Chen, Changshi Xiao, Yuanqiao Wen, Mengwei Tao, and Wenqiang Zhan. Ship intention prediction at intersections based on vision and bayesian framework. *Journal of Marine Science and Engineering*, 10(5):639, 2022.
- [146] Christoph Molnar. *Interpretable machine learning*. Lulu. com, 2020.
- [147] Ihor Vasylytsov and Wooseok Chang. Efficient softmax approximation for deep neural networks with attention mechanism. *arXiv preprint arXiv:2111.10770*, 2021.
- [148] Eric A. Wan and Rudolph van der Merwe. The unscented kalman filter for nonlinear estimation. In *Proceedings of the IEEE Adaptive Systems for Signal Processing, Communications, and Control Symposium (AS-SPCC)*, pages 153–158. IEEE, 2000.
- [149] Rudolph van der Merwe. *Sigma-Point Kalman Filters for Probabilistic Inference in Dynamic State-Space Models*. PhD thesis, Oregon Health & Science University, 2004.
- [150] Jonathan Ho, Nal Kalchbrenner, Dirk Weissenborn, and Tim Salimans. Axial attention in multidimensional transformers. *arXiv preprint arXiv:1912.12180*, 2019.
- [151] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.